

ENGINEERING MODELING AND ANALYSIS WITH PYTHON



Carl Greco
Arkansas Tech University

Engineering Modeling and Analysis with Python

This text is disseminated via the Open Education Resource (OER) LibreTexts Project (<https://LibreTexts.org>) and like the thousands of other texts available within this powerful platform, it is freely available for reading, printing, and "consuming."

The LibreTexts mission is to bring together students, faculty, and scholars in a collaborative effort to provide an accessible, and comprehensive platform that empowers our community to develop, curate, adapt, and adopt openly licensed resources and technologies; through these efforts we can reduce the financial burden born from traditional educational resource costs, ensuring education is more accessible for students and communities worldwide.

Most, but not all, pages in the library have licenses that may allow individuals to make changes, save, and print this book. Carefully consult the applicable license(s) before pursuing such effects. Instructors can adopt existing LibreTexts texts or Remix them to quickly build course-specific resources to meet the needs of their students. Unlike traditional textbooks, LibreTexts' web based origins allow powerful integration of advanced features and new technologies to support learning.



LibreTexts is the adaptable, user-friendly non-profit open education resource platform that educators trust for creating, customizing, and sharing accessible, interactive textbooks, adaptive homework, and ancillary materials. We collaborate with individuals and organizations to champion open education initiatives, support institutional publishing programs, drive curriculum development projects, and more.

The LibreTexts libraries are Powered by [NICE CXone Expert](#) and was supported by the Department of Education Open Textbook Pilot Project, the California Education Learning Lab, the UC Davis Office of the Provost, the UC Davis Library, the California State University Affordable Learning Solutions Program, and Merlot. This material is based upon work supported by the National Science Foundation under Grant No. 1246120, 1525057, and 1413739.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation nor the US Department of Education.

Have questions or comments? For information about adoptions or adaptations contact info@LibreTexts.org or visit our main website at <https://LibreTexts.org>.

This text was compiled on 09/15/2025

TABLE OF CONTENTS

About the Authors

Licensing

About the Book

Licensing

1: Introduction to Engineering Modeling and Analysis with Python

- 1.1: Introduction

2: Variables, Expressions, and Statements

- 2.1: Values and Types
- 2.2: Variables
- 2.3: Variable names and Keywords
- 2.4: Statements
- 2.5: Operators and Operands
- 2.6: Expressions
- 2.7: Order of Operations
- 2.8: Modulus Operator
- 2.9: String Operations
- 2.10: Asking the user for input
- 2.11: Comments
- 2.12: Choosing Mnemonic Variable Names
- 2.13: Debugging
- 2.E: Variables, Expressions, and Statements (Exercises)
- 2.G: Glossary

3: Conditional Execution

- 3.1: Boolean Expressions
- 3.2: Logical Operators
- 3.3: Conditional Execution
- 3.4: Alternative Execution
- 3.5: Chained Conditionals
- 3.6: Nested Conditionals
- 3.7: Catching exceptions Using Try and Except
- 3.8: Short-Circuit Evaluation of Logical Expressions
- 3.9: Debugging
- 3.E: Conditional Execution (Exercises)
- 3.G: Conditional Execution (Glossary)

4: Functions

- 4.1: Function Calls
- 4.2: Built-in Functions
- 4.3: Type Conversion Functions
- 4.4: Random Numbers

- 4.5: Math Functions
- 4.6: Adding New Functions
- 4.7: Definitions and Uses
- 4.8: Flow of Execution
- 4.9: Parameters and Arguments
- 4.10: Fruitful functions and void functions
- 4.11: Why functions?
- 4.12: Debugging
- 4.E: Functions (Exercises)
- 4.G: Functions (Glossary)

5: Iterations

- 5.1: Updating Variables
- 5.2: The while Statement
- 5.3: Infinite Loops
- 5.4: "Infinite loops" and break
- 5.5: Finishing iterations with continue
- 5.6: Definite loops using for
- 5.7: Loop patterns
- 5.8: Counting and Summing Loops
- 5.9: Maximum and Minimum Loops
- 5.10: Debugging
- 5.E: Iterations (Exercises)
- 5.G: Iterations (Glossary)

6: Strings

- 6.1: A string is a sequence
- 6.2: Getting the length of a string using len
- 6.3: Traversal through a string with a loop
- 6.4: String Slices
- 6.5: Strings are immutable
- 6.6: Looping and Counting
- 6.7: The in operator
- 6.8: String Comparison
- 6.9: String Methods
- 6.10: Parsing strings
- 6.11: Format operator
- 6.12: Debugging
- 6.E: Strings (Exercises)
- 6.G: Strings (Glossary)

7: Files

- 7.1: Persistence
- 7.2: Opening Files
- 7.3: Text files and Lines
- 7.4: Reading Files
- 7.5: Searching through a File
- 7.6: Letting the user choose the file name
- 7.7: Using try, except, and open
- 7.8: Writing Files
- 7.9: Debugging

- 7.E: Files (Exercises)
- 7.G: Files (Glossary)

8: Lists

- 8.1: A list is a sequence
- 8.2: Lists are mutable
- 8.3: Traversing a List
- 8.4: List operations
- 8.5: List Slices
- 8.6: List Methods
- 8.7: Deleting Elements
- 8.8: Lists and Functions
- 8.9: Lists and Strings
- 8.10: Parsing lines
- 8.11: Objects and Values
- 8.12: Aliasing
- 8.13: List arguments
- 8.14: Debugging
- 8.E: Lists (Exercises)
- 8.G: Lists (Glossary)

9: Dictionaries

- 9.1: Dictionaries
- 9.2: Dictionary as a Set of Counters
- 9.3: Dictionaries and Files
- 9.4: Looping and Dictionaries
- 9.5: Advanced Text Parsing
- 9.6: Debugging
- 9.E: Dictionaries (Exercises)
- 9.G: Dictionaries (Glossary)

10: Tuples

- 10.1: Tuples are Immutable
- 10.2: Comparing Tuples
- 10.3: Tuple Assignment
- 10.4: Dictionaries and Tuples
- 10.5: Multiple assignment with dictionaries
- 10.6: The most common words
- 10.7: Using Tuples as Keys in Dictionaries
- 10.8: Sequences- strings, lists, and tuples - Oh My!
- 10.9: Debugging
- 10.E: Tuples (Exercises)
- 10.G: Tuples (Glossary)

11: Libraries

- 11.1: NumPy - Numerical Python
- 11.2: SciPy - Scientific Computing in Python
- 11.3: Pandas - Data Manipulation and Analysis
- 11.4: Matplotlib — Data Visualization in Python
- 11.5: Scikit-learn (sklearn) - Supervised and Unsupervised Learning

12: Model Building and Regression

- 12.1: Functions for Model Building and Regression
- 12.2: Linear Regression
- 12.3: Optimization - Parameter Estimation for Nonlinear Functions
- 12.4: Summary

13: Statistics, Probability, and Interpolation

- 13.1: Statistics and Histograms
- 13.2: Normal Distribution
- 13.3: Random Number Generation
- 13.4: Interpolation
- 13.5: Summary

14: Linear Algebra Equations

- 14.1: Matrix Methods for Linear Equations
- 14.2: Well-Determined Fully Specified System
- 14.3: Underdetermined Systems
- 14.4: Overdetermined Systems
- 14.5: A General Solution
- 14.6: Summary

15: Numerical Methods for Calculus and Differential Equations

- 15.1: Numerical Integration
- 15.2: Numerical Differentiation
- 15.3: First-Order Differential Equations
- 15.4: Higher-Order Differential Equations
- 15.5: Summary

16: Regular Expressions

- 16.1: Regular Expressions
- 16.2: Character matching in regular expressions
- 16.3: Extracting data using regular expressions
- 16.4: Combining searching and extracting
- 16.5: Escape Character
- 16.6: Bonus section for Unix / Linux users
- 16.7: Debugging
- 16.E: Regular Expressions (Exercises)
- 16.G: Regular Expressions (Glossary)
- 16.S: Regular Expressions (Summary)

17: Object-Oriented Programming

- 17.1: Managing Larger Programs
- 17.2: Getting Started
- 17.3: Using Objects
- 17.4: Starting with Programs
- 17.5: Subdividing a Problem - Encapsulation
- 17.6: Our First Python Object
- 17.7: Classes as Types
- 17.8: Many Instances

- [17.9: Object Lifecycle](#)
- [17.10: Inheritance](#)
- [17.G: Object-Oriented Programming \(Glossary\)](#)
- [17.S: Object-Oriented Programming \(Summary\)](#)

[Index](#)

[Glossary](#)

[Detailed Licensing](#)

[Detailed Licensing](#)

About the Authors

	Carl Greco Dr. Greco is a Professor of Electrical and Computer Engineering with research interest in biomedical signal processing. He teaches courses in digital systems, signals and systems, communications and biomedical signal processing, and is an enthusiastic supporter of Open Educational Resources.
	Zahra Zamanipour Dr. Zahra Zamanipour has a PhD in Electrical Engineering and has taught engineering courses for seven years. She currently works on Thermoelectric generators and its applications. Her research interest is energy harvesting systems, thermoelectric generators, solar cell systems and machine learning applications in electronics.
	Thomas M. Dodds Thomas Dodds is an Instructor in Electrical Engineering. He has bachelor degrees in Electrical and Computer Engineering and a master's degree in Electrical Engineering.

Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

About the Book

1: Engineering Modeling and Analysis with Python: A Practical Introduction

This hands-on textbook provides a practical and accessible introduction to the fundamental concepts of engineering modeling and analysis, leveraging the power of the Python programming language. Before diving into complex engineering applications, the book offers a concise yet comprehensive introduction to the Python language itself. Readers will learn the basics of Python syntax, data types (like numbers, strings, lists, and dictionaries), control flow (conditionals and loops), functions, and how to effectively use its extensive libraries. This foundational understanding ensures that even those new to programming can quickly become proficient in Python's core functionalities.

Designed for undergraduate students and practicing engineers alike, the book bridges the gap between theoretical principles and real-world application. Through a series of clear explanations, illustrative examples, and engaging exercises, readers will learn to formulate engineering problems, develop mathematical models, implement numerical solutions using Python libraries such as NumPy, SciPy, Pandas, Scikit-learn, and Matplotlib, and critically analyze the results. Topics covered include data analysis, curve fitting, optimization, linear algebra, and first as well as higher order ordinary differential equations, all presented within the context of practical engineering scenarios. By the end of this book, readers will possess the essential skills to confidently tackle a wide range of engineering challenges using computational tools.

Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

CHAPTER OVERVIEW

1: Introduction to Engineering Modeling and Analysis with Python

The Essence of Engineering Modeling

Engineering is fundamentally about design, innovation, and problem-solving. From bridges that span vast rivers to microchips powering our devices, every engineering marvel begins with an idea, which then undergoes rigorous analysis and refinement. At the heart of this process lies **engineering modeling**.

Modeling, in the engineering context, is the art and science of creating simplified representations of complex real-world systems, processes, or phenomena. These representations, often expressed through mathematical equations, diagrams, or computational algorithms, allow engineers to:

- **Understand:** Gain deeper insights into how a system behaves under various conditions.
- **Predict:** Forecast future performance or outcomes without having to build and test physical prototypes, which can be costly and time-consuming.
- **Optimize:** Find the best possible design parameters or operating conditions to achieve desired goals (e.g., maximize efficiency, minimize cost, enhance safety).
- **Communicate:** Clearly articulate complex ideas and findings to colleagues, stakeholders, and clients.
- **Troubleshoot:** Diagnose issues in existing systems by simulating different scenarios.

Models can range from simple hand calculations for a beam's deflection to sophisticated finite element analysis (FEA) simulations of an aircraft wing, or complex computational fluid dynamics (CFD) models of airflow around a vehicle. Regardless of their complexity, all models share the goal of abstracting reality to make it amenable to analysis.

The Power of Analysis in Engineering

Once a model is constructed, it becomes a tool for **engineering analysis**. Analysis involves applying mathematical, scientific, and computational techniques to extract meaningful information from the model. This could mean:

- Solving equations to find critical values.
- Running simulations to observe dynamic behavior over time.
- Performing statistical tests to quantify uncertainties.
- Visualizing data to identify trends and patterns.

The insights gleaned from analysis directly inform design decisions, validate hypotheses, and ultimately lead to more robust, efficient, and reliable engineering solutions. Without rigorous analysis, even the most innovative designs remain untested and potentially flawed.

Python for Engineering

Historically, engineers relied on specialized commercial software packages or compiled languages like Fortran and C++ for demanding computational tasks. While these tools remain vital, Python has emerged as an incredibly powerful and versatile alternative, rapidly gaining traction across all engineering disciplines. Its rise can be attributed to several key advantages:

- **Simplicity and Readability:** Python's clear, intuitive syntax makes it easy to learn, write, and understand, significantly reducing development time compared to more verbose languages. It is an interpretive language similar to MATLAB® where each line is executed individually unlike C or C++ requiring the file to be compiled and linked prior to execution.
- **Vast Ecosystem of Libraries:** This is perhaps Python's greatest strength for engineering. Libraries like NumPy, SciPy, Matplotlib, Pandas, SymPy, and many others provide highly optimized functionalities for numerical computation, scientific computing, data analysis, visualization, and symbolic mathematics.
- **Interoperability:** Python can easily integrate with other languages (e.g., C, C++, Fortran) and interact with commercial engineering software, allowing engineers to leverage existing codebases and tools.
- **Open Source and Community Support:** Being open-source, Python and its libraries are freely available. A massive, active global community contributes to its development, provides extensive documentation, and offers support.

- **Rapid Prototyping:** Its ease of use and extensive libraries enable engineers to quickly prototype ideas, test algorithms, and build custom analysis tools.
- **Data Handling and Visualization:** Python excels at handling large datasets and offers powerful visualization capabilities, which are crucial for interpreting simulation results and experimental data.
- **Machine Learning and AI Integration:** As machine learning and artificial intelligence become increasingly relevant in engineering (e.g., predictive maintenance, optimized control systems, generative design), Python's leading role in these fields makes it an indispensable tool.

What You Will Learn in This Book

This book aims to provide a practical introduction to applying Python for engineering modeling and analysis. We will cover fundamental concepts and equip you with the skills to tackle a wide range of engineering problems. Specifically, you will learn to:

- Set up your Python environment for scientific and engineering computing.
- Master essential Python libraries like NumPy for numerical operations, SciPy for engineering analysis and design, and Matplotlib for data visualization.
- Develop mathematical models for various engineering systems.
- Implement numerical methods for solving engineering equations (e.g., root finding, integration, ordinary differential equations).
- Perform data analysis, manipulation, and statistical inference on engineering data.
- Visualize complex engineering data effectively.
- Apply Python to solve practical problems in areas such as mechanics, thermodynamics, electrical circuits, and more.

By the end of this book, you will not only be proficient in using Python as a powerful engineering tool but also gain a deeper appreciation for the principles of modeling and analysis that underpin all successful engineering endeavors. Let's begin our journey into the exciting world where engineering meets computation!

1.1: Introduction

This page titled [1: Introduction to Engineering Modeling and Analysis with Python](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by .

1.1: Introduction

Introduction to Engineering Modeling and Analysis in Python

In today's data-driven world, engineers across all disciplines increasingly rely on computational tools to understand, design, and optimize complex systems. **Engineering modeling and analysis** refers to the process of creating mathematical or computational representations of physical systems, processes, or phenomena to predict their behavior, evaluate performance, and make informed design decisions. Python has emerged as a leading language for these tasks, offering a powerful, flexible, and accessible environment for engineers.

What is Engineering Modeling and Analysis?

At its core, engineering modeling involves translating real-world problems into a structured, quantifiable form. This can range from simple algebraic equations to complex systems of differential equations, finite element models, or statistical representations. Analysis then involves solving these models, interpreting the results, and drawing conclusions that inform engineering practice.

Key aspects include:

- **Simulation:** Running models to observe system behavior under various conditions.
- **Optimization:** Finding the best design parameters to achieve specific objectives (e.g., minimize cost, maximize efficiency).
- **Data Analysis:** Processing, visualizing, and extracting insights from experimental or simulated data.
- **Control Systems Design:** Developing algorithms to manage and regulate system performance.
- **Statistical Analysis:** Quantifying uncertainty, performing hypothesis testing, and building predictive models.

Why Python for Engineering?

Python's rise in engineering is due to several compelling advantages:

1. **Readability and Ease of Use:** Python's clear syntax makes it relatively easy to learn and write, allowing engineers to focus more on the problem at hand rather than intricate coding details.
2. **Extensive Ecosystem of Libraries:** A rich collection of specialized libraries provides pre-built functionalities for almost any engineering task, from numerical computation to data visualization and machine learning.
3. **Versatility:** Python is not just for numerical analysis; it can be used for scripting, web development, data science, and more, making it a versatile tool in an engineer's toolkit.
4. **Open Source and Community Support:** Being open-source, Python and its libraries are freely available. A large and active community contributes to its development, provides support, and shares resources.
5. **Interoperability:** Python can easily interface with other languages (like C/C++ or Fortran) and software, allowing engineers to leverage existing codebases or specialized tools.

Key Python Libraries for Engineering

Several libraries form the backbone of Python for scientific and engineering computing:

- **NumPy:** The fundamental package for numerical computation in Python. It provides powerful N-dimensional array objects and functions for linear algebra, Fourier transforms, and random number generation. It's the basis for many other scientific libraries.
- **SciPy:** Built on NumPy, SciPy offers a collection of algorithms and functions for scientific and technical computing. This includes modules for optimization, interpolation, signal processing, linear algebra, sparse matrices, statistics, and more.
- **Matplotlib:** A comprehensive library for creating static, animated, and interactive visualizations in Python. It's essential for plotting data, visualizing model outputs, and creating publication-quality figures.
- **Pandas:** A powerful and flexible open-source data analysis and manipulation tool, providing data structures like DataFrames for efficient handling of tabular data. It's crucial for data preparation, cleaning, and exploration.
- **SymPy:** A library for symbolic mathematics. It allows you to perform algebraic manipulations, differentiation, integration, and solve equations symbolically, which is invaluable for deriving analytical solutions or simplifying complex expressions.
- **Scikit-learn:** While often associated with machine learning, its tools for regression, classification, clustering, and dimensionality reduction are highly relevant for building predictive models from engineering data.
- **Other Specialized Libraries:** Depending on the engineering discipline, there are many other libraries, such as **OpenCV** for computer vision, **Pillow** for image processing, **Pyomo** for optimization modeling, **Keras/TensorFlow/PyTorch** for deep

learning, and domain-specific libraries for fields like fluid dynamics (e.g., **Pytorch-CFD**), structural analysis, or circuit simulation.

The Engineering Workflow in Python

A typical engineering modeling and analysis workflow using Python might involve:

1. **Data Acquisition and Loading:** Using Pandas to load data from various sources (CSV, Excel, databases).
2. **Data Preprocessing:** Cleaning, transforming, and preparing data using Pandas and NumPy.
3. **Model Formulation:** Defining mathematical models using NumPy arrays, SciPy functions, or even SymPy for symbolic manipulation.
4. **Simulation/Solution:** Solving differential equations with **scipy.integrate**, performing optimization with **scipy.optimize**, or running statistical analyses with SciPy/Scikit-learn.
5. **Visualization:** Plotting results, trends, and comparisons using Matplotlib.
6. **Interpretation and Reporting:** Analyzing the outputs, drawing engineering conclusions, and potentially generating reports or interactive dashboards.

By combining these powerful libraries, engineers can tackle complex problems, automate tedious tasks, and gain deeper insights into the systems they work with, significantly enhancing efficiency and innovation in their respective fields.

This page titled [1.1: Introduction](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by .

CHAPTER OVERVIEW

2: Variables, Expressions, and Statements

- [2.1: Values and Types](#)
- [2.2: Variables](#)
- [2.3: Variable names and Keywords](#)
- [2.4: Statements](#)
- [2.5: Operators and Operands](#)
- [2.6: Expressions](#)
- [2.7: Order of Operations](#)
- [2.8: Modulus Operator](#)
- [2.9: String Operations](#)
- [2.10: Asking the user for input](#)
- [2.11: Comments](#)
- [2.12: Choosing Mnemonic Variable Names](#)
- [2.13: Debugging](#)
- [2.E: Variables, Expressions, and Statements \(Exercises\)](#)
- [2.G: Glossary](#)

This page titled [2: Variables, Expressions, and Statements](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

2.1: Values and Types

A *value* is one of the basic things a program works with, like a letter or a number. The values we have seen so far are `1` , `2` , and `"Hello, World!"`

These values belong to different *types*: `2` is an integer, and `"Hello, World!"` is a *string*, so called because it contains a "string" of letters. You (and the interpreter) can identify strings because they are enclosed in quotation marks.

The `print` statement also works for integers. We use the `python` command to start the interpreter.

```
python
>>> print(4)
4
```

If you are not sure what type a value has, the interpreter can tell you.

```
>>> type('Hello, World!')
<class 'str'>
>>> type(17)
<class 'int'>
```

Not surprisingly, strings belong to the type `str` and integers belong to the type `int` . Less obviously, numbers with a decimal point belong to a type called `float` , because these numbers are represented in a format called *floating point*.

```
>>> type(3.2)
<class 'float'>
```

What about values like `"17"` and `"3.2"`? They look like numbers, but they are in quotation marks like strings.

```
>>> type('17')
<class 'str'>
>>> type('3.2')
<class 'str'>
```

They're strings.

When you type a large integer, you might be tempted to use commas between groups of three digits, as in `1,000,000` . This is not a legal integer in Python, but it is legal:

```
>>> print(1,000,000)
1 0 0
```

Well, that's not what we expected at all! Python interprets `1,000,000` as a comma-separated sequence of integers, which it prints with spaces between.

This is the first example we have seen of a semantic error: the code runs without producing an error message, but it doesn't do the "right" thing.

This page titled [2.1: Values and Types](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.1: Values and Types](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.2: Variables

One of the most powerful features of a programming language is the ability to manipulate *variables*. A variable is a name that refers to a value.

An *assignment statement* creates new variables and gives them values:

```
>>> message = 'And now for something completely different'
>>> n = 17
>>> pi = 3.1415926535897931
```

This example makes three assignments. The first assigns a string to a new variable named `message` ; the second assigns the integer `17` to `n` ; the third assigns the (approximate) value of π to `pi` .

To display the value of a variable, you can use a print statement:

```
>>> print(n)
17
>>> print(pi)
3.141592653589793
```

The type of a variable is the type of the value it refers to.

```
>>> type(message)
<class 'str'>
>>> type(n)
<class 'int'>
>>> type(pi)
<class 'float'>
```

This page titled [2.2: Variables](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.2: Variables](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.3: Variable names and Keywords

Programmers generally choose names for their variables that are meaningful and document what the variable is used for.

Variable names can be arbitrarily long. They can contain both letters and numbers, but they cannot start with a number. It is legal to use uppercase letters, but it is a good idea to begin variable names with a lowercase letter (you'll see why later).

The underscore character (`_`) can appear in a name. It is often used in names with multiple words, such as `my_name` or `airspeed_of_unladen_swallow` . Variable names can start with an underscore character, but we generally avoid doing this unless we are writing library code for others to use.

If you give a variable an illegal name, you get a syntax error:

```
>>> 76trombones = 'big parade'
SyntaxError: invalid syntax
>>> more@ = 1000000
SyntaxError: invalid syntax
>>> class = 'Advanced Theoretical Zymurgy'
SyntaxError: invalid syntax
```

`76trombones` is illegal because it begins with a number. `more@` is illegal because it contains an illegal character, `@` . But what's wrong with `class` ?

It turns out that `class` is one of Python's *keywords*. The interpreter uses keywords to recognize the structure of the program, and they cannot be used as variable names.

Python reserves 33 keywords:

<code>and</code>	<code>del</code>	<code>from</code>	<code>None</code>	<code>True</code>
<code>as</code>	<code>elif</code>	<code>global</code>	<code>nonlocal</code>	<code>try</code>
<code>assert</code>	<code>else</code>	<code>if</code>	<code>not</code>	<code>while</code>
<code>break</code>	<code>except</code>	<code>import</code>	<code>or</code>	<code>with</code>
<code>class</code>	<code>False</code>	<code>in</code>	<code>pass</code>	<code>yield</code>
<code>continue</code>	<code>finally</code>	<code>is</code>	<code>raise</code>	
<code>def</code>	<code>for</code>	<code>lambda</code>	<code>return</code>	

You might want to keep this list handy. If the interpreter complains about one of your variable names and you don't know why, see if it is on this list.

This page titled [2.3: Variable names and Keywords](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.3: Variable names and Keywords](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.4: Statements

A *statement* is a unit of code that the Python interpreter can execute. We have seen two kinds of statements: expression statement with "print" and assignment.

When you type a statement in interactive mode, the interpreter executes it and displays the result, if there is one.

A script usually contains a sequence of statements. If there is more than one statement, the results appear one at a time as the statements execute.

For the script below, notice that the assignment statement (in this case $x = 2$) produces no output.

This page titled [2.4: Statements](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.4: Statements](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.5: Operators and Operands

Operators are special symbols that represent computations like addition and multiplication. The values the operator is applied to are called *operands*.

The operators `+`, `-`, `*`, `/`, and `**` perform addition, subtraction, multiplication, division, and exponentiation, as in the following examples:

```
20+32    hour-1    hour*60+minute    minute/60    5**2    (5+9)*(15-7)
```

There has been a change in the division operator between Python 2.x and Python 3.x. In Python 3.x, the result of this division is a floating point result:

```
>>> minute = 59
>>> minute/60
0.9833333333333333
```

The division operator in Python 2.0 would divide two integers and truncate the result to an integer:

```
>>> minute = 59
>>> minute/60
0
```

To obtain the same answer in Python 3.0 use floored (`//` integer) division.

```
>>> minute = 59
>>> minute//60
0
```

In Python 3.0 integer division functions much more as you would expect if you entered the expression on a calculator.

This page titled [2.5: Operators and Operands](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **2.5: Operators and Operands** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.6: Expressions

An *expression* is a combination of values, variables, and operators. A value all by itself is considered an expression, and so is a variable, so the following are all legal expressions (assuming that the variable `x` has been assigned a value):

```
17
x
x + 17
```

If you type an expression in interactive mode, the interpreter *evaluates* it and displays the result:

```
>>> 1 + 1
2
```

But in a script, an expression all by itself doesn't do anything! This is a common source of confusion for beginners.

Exercise 1: Type the following statements in the Python interpreter to see what they do:

```
5
x = 5
x + 1
```

This page titled [2.6: Expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.6: Expressions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.7: Order of Operations

When more than one operator appears in an expression, the order of evaluation depends on the *rules of precedence*. For mathematical operators, Python follows mathematical convention. The acronym *PEMDAS* is a useful way to remember the rules:

- Parentheses have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, $2 * (3 - 1)$ is 4, and $(1 + 1) ** (5 - 2)$ is 8. You can also use parentheses to make an expression easier to read, as in `(minute * 100) / 60`, even if it doesn't change the result.
- Exponentiation has the next highest precedence, so $2 ** 1 + 1$ is 3, not 4, and $3 * 1 ** 3$ is 3, not 27.
- Multiplication and Division have the same precedence, which is higher than Addition and Subtraction, which also have the same precedence. So $2 * 3 - 1$ is 5, not 4, and $6 + 4 / 2$ is 8.0, not 5.
- Operators with the same precedence are evaluated from left to right. So the expression $5 - 3 - 1$ is 1, not 3, because the $5 - 3$ happens first and then 1 is subtracted from 2 .

When in doubt, always put parentheses in your expressions to make sure the computations are performed in the order you intend.

This page titled [2.7: Order of Operations](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.7: Order of Operations](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.8: Modulus Operator

The *modulus operator* works on integers and yields the remainder when the first operand is divided by the second. In Python, the modulus operator is a percent sign (`%`). The syntax is the same as for other operators:

So 7 divided by 3 is 2 with 1 left over.

The modulus operator turns out to be surprisingly useful. For example, you can check whether one number is divisible by another: if `x % y` is zero, then `x` is divisible by `y` .

You can also extract the right-most digit or digits from a number. For example, `x % 10` yields the right-most digit of `x` (in base 10). Similarly, `x % 100` yields the last two digits.

This page titled [2.8: Modulus Operator](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.8: Modulus Operator](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.9: String Operations

The `+` operator works with strings, but it is not addition in the mathematical sense. Instead it performs *concatenation*, which means joining the strings by linking them end to end. For example:

The output of this program is `100150` .

This page titled [2.9: String Operations](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.9: String Operations](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

2.10: Asking the user for input

Sometimes we would like to take the value for a variable from the user via their keyboard. Python provides a built-in function called `input` that gets input from the keyboard¹. When this function is called, the program stops and waits for the user to type something. When the user presses `Return` or `Enter`, the program resumes and `input` returns what the user typed as a string.

```
>>> inp = input()
Some silly stuff
>>> print(inp)
Some silly stuff
```

Before getting input from the user, it is a good idea to print a prompt telling the user what to input. You can pass a string to `input` to be displayed to the user before pausing for input:

The sequence `\n` at the end of the prompt represents a *newline*, which is a special character that causes a line break. That's why the user's input appears below the prompt.

If you expect the user to type an integer, you can try to convert the return value to `int` using the `int()` function:

But if the user types something other than a string of digits, you get an error:

```
>>> speed = input(prompt)
What...is the airspeed velocity of an unladen swallow?
What do you mean, an African or a European swallow?
>>> int(speed)
ValueError: invalid literal for int() with base 10:
```

We will see how to handle this kind of error later.

This page titled [2.10: Asking the user for input](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.10: Asking the user for input](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.11: Comments

As programs get bigger and more complicated, they get more difficult to read. Formal languages are dense, and it is often difficult to look at a piece of code and figure out what it is doing, or why.

For this reason, it is a good idea to add notes to your programs to explain in natural language what the program is doing. These notes are called *comments*, and in Python they start with the `#` symbol:

```
# compute the percentage of the hour that has elapsed
percentage = (minute * 100) / 60
```

In this case, the comment appears on a line by itself. You can also put comments at the end of a line:

```
percentage = (minute * 100) / 60      # percentage of an hour
```

Everything from the `\#` to the end of the line is ignored; it has no effect on the program.

Comments are most useful when they document non-obvious features of the code. It is reasonable to assume that the reader can figure out *what* the code does; it is much more useful to explain *why*.

This comment is redundant with the code and useless:

```
v = 5      # assign 5 to v
```

This comment contains useful information that is not in the code:

```
v = 5      # velocity in meters/second.
```

Good variable names can reduce the need for comments, but long names can make complex expressions hard to read, so there is a trade-off.

This page titled [2.11: Comments](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.11: Comments](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.12: Choosing Mnemonic Variable Names

As long as you follow the simple rules of variable naming, and avoid reserved words, you have a lot of choice when you name your variables. In the beginning, this choice can be confusing both when you read a program and when you write your own programs. For example, the following three programs are identical in terms of what they accomplish, but very different when you read them and try to understand them.

```
a = 35.0
b = 12.50
c = a * b
print(c)
```

run restart restart & run all

```
hours = 35.0
rate = 12.50
pay = hours * rate
print(pay)
```

run restart restart & run all

```
x1q3z9ahd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ahd * x1q3z9afd
print(x1q3p9afd)
```

run restart restart & run all

The Python interpreter sees all three of these programs as *exactly the same* but humans see and understand these programs quite differently. Humans will most quickly understand the *intent* of the second program because the programmer has chosen variable names that reflect their intent regarding what data will be stored in each variable.

We call these wisely chosen variable names "mnemonic variable names". The word *mnemonic*² means "memory aid". We choose mnemonic variable names to help us remember why we created the variable in the first place.

While this all sounds great, and it is a very good idea to use mnemonic variable names, mnemonic variable names can get in the way of a beginning programmer's ability to parse and understand code. This is because beginning programmers have not yet memorized the reserved words (there are only 33 of them) and sometimes variables with names that are too descriptive start to look like part of the language and not just well-chosen variable names.

Take a quick look at the following Python sample code which loops through some data. We will cover loops soon, but for now try to just puzzle through what this means:

```
for word in words:
    print(word)
```

What is happening here? Which of the tokens (for, word, in, etc.) are reserved words and which are just variable names? Does Python understand at a fundamental level the notion of words? Beginning programmers have trouble separating what parts of the code *must* be the same as this example and what parts of the code are simply choices made by the programmer.

The following code is equivalent to the above code:

```
for slice in pizza:
    print(slice)
```

PYTHON SESSION: READY

It is easier for the beginning programmer to look at this code and know which parts are reserved words defined by Python and which parts are simply variable names chosen by the programmer. It is pretty clear that Python has no fundamental understanding of pizza and slices and the fact that a pizza consists of a set of one or more slices.

But if our program is truly about reading data and looking for words in the data, `pizza` and `slice` are very un-mnemonic variable names. Choosing them as variable names distracts from the meaning of the program.

After a pretty short period of time, you will know the most common reserved words and you will start to see the reserved words jumping out at you:

```
for word in words:  
    print word
```

The parts of the code that are defined by Python (`for` , `in` , `print` , and `:`) are in bold and the programmer-chosen variables (`word` and `words`) are not in bold. Many text editors are aware of Python syntax and will color reserved words differently to give you clues to keep your variables and reserved words separate. After a while you will begin to read Python and quickly determine what is a variable and what is a reserved word.

This page titled [2.12: Choosing Mnemonic Variable Names](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.12: Choosing Mnemonic Variable Names](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

2.13: Debugging

At this point, the syntax error you are most likely to make is an illegal variable name, like `class` and `yield`, which are keywords, or `odd~job` and `US$`, which contain illegal characters.

If you put a space in a variable name, Python thinks it is two operands without an operator:

```
>>> bad name = 5
SyntaxError: invalid syntax
```

```
>>> month = 09
      File "<stdin>", line 1
        month = 09
                ^
SyntaxError: invalid token
```

For syntax errors, the error messages don't help much. The most common messages are `SyntaxError: invalid syntax` and `SyntaxError: invalid token`, neither of which is very informative.

The runtime error you are most likely to make is a "use before def;" that is, trying to use a variable before you have assigned a value. This can happen if you spell a variable name wrong:

```
>>> principal = 327.68
>>> interest = principle * rate
NameError: name 'principle' is not defined
```

Variables names are case sensitive, so `LaTeX` is not the same as `latex`.

At this point, the most likely cause of a semantic error is the order of operations. For example, to evaluate $1/2\pi$, you might be tempted to write

```
>>> 1.0 / 2.0 * pi
```

But the division happens first, so you would get $\pi/2$, which is not the same thing! There is no way for Python to know what you meant to write, so in this case you don't get an error message; you just get the wrong answer.

This page titled [2.13: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.13: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

2.E: Variables, Expressions, and Statements (Exercises)

Exercise 2: Write a program that uses `input` to prompt a user for their name and then welcomes them.

```
Enter your name: Chuck
Hello Chuck
```

```
%%python3
#Use this space to write and run your code
```

run restart restart & run all

Exercise 3: Write a program to prompt the user for hours and rate per hour to compute gross pay.

```
Enter Hours: 35
Enter Rate: 2.75
Pay: 96.25
```

We won't worry about making sure our pay has exactly two digits after the decimal place for now. If you want, you can play with the built-in Python `round` function to properly round the resulting pay to two decimal places.

```
%%python3
#Use this space to write and run your code
#Save this code for a later exercise!
```

Exercise 4: Assume that we execute the following assignment statements:

```
width = 17
height = 12.0
```

For each of the following expressions, write the value of the expression and the type (of the value of the expression).

1. `width//2`
2. `width/2.0`
3. `height/3`
4. `1 + 2 * 5`

```
%%python3
#Use this space to check your answers
```

Exercise 5: Write a program which prompts the user for a Celsius temperature, convert the temperature to Fahrenheit, and print out the converted temperature.

```
%%python3
#Use this space to write and run your code
```

PYTHON SESSION: READY

1. In Python 2.0, this function was named `raw_input` ↩

2. See <http://en.Wikipedia.org/wiki/Mnemonic> for an extended description of the word "mnemonic".↩

This page titled [2.E: Variables, Expressions, and Statements \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [2.E: Variables, Expressions, and Statements \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

2.G: Glossary

assignment

A statement that assigns a value to a variable.

concatenate

To join two operands end to end.

comment

Information in a program that is meant for other programmers (or anyone reading the source code) and has no effect on the execution of the program.

evaluate

To simplify an expression by performing the operations in order to yield a single value.

expression

A combination of variables, operators, and values that represents a single result value.

floating point

A type that represents numbers with fractional parts.

integer

A type that represents whole numbers.

keyword

A reserved word that is used by the compiler to parse a program; you cannot use keywords like `if` , `def` , and `while` as variable names.

mnemonic

A memory aid. We often give variables mnemonic names to help us remember what is stored in the variable.

modulus operator

An operator, denoted with a percent sign (`%`), that works on integers and yields the remainder when one number is divided by another.

operand

One of the values on which an operator operates.

operator

A special symbol that represents a simple computation like addition, multiplication, or string concatenation.

rules of precedence

The set of rules governing the order in which expressions involving multiple operators and operands are evaluated.

statement

A section of code that represents a command or action. So far, the statements we have seen are assignments and print expression statement.

string

A type that represents sequences of characters.

type

A category of values. The types we have seen so far are integers (type `int`), floating-point numbers (type `float`), and strings (type `str`).

value

One of the basic units of data, like a number or string, that a program manipulates.

variable

A name that refers to a value.

This page titled [2.G: Glossary](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

3: Conditional Execution

- [3.1: Boolean Expressions](#)
- [3.2: Logical Operators](#)
- [3.3: Conditional Execution](#)
- [3.4: Alternative Execution](#)
- [3.5: Chained Conditionals](#)
- [3.6: Nested Conditionals](#)
- [3.7: Catching exceptions Using Try and Except](#)
- [3.8: Short-Circuit Evaluation of Logical Expressions](#)
- [3.9: Debugging](#)
- [3.E: Conditional Execution \(Exercises\)](#)
- [3.G: Conditional Execution \(Glossary\)](#)

This page titled [3: Conditional Execution](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

3.1: Boolean Expressions

A *boolean expression* is an expression that is either true or false. The following examples use the operator `==`, which compares two operands and produces `True` if they are equal and `False` otherwise:

```
>>> 5 == 5
True
>>> 5 == 6
False
{}
```

`True` and `False` are special values that belong to the class `bool`; they are not strings:

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

The `==` operator is one of the *comparison operators*; the others are:

<code>x != y</code>	<code># x is not equal to y</code>
<code>x > y</code>	<code># x is greater than y</code>
<code>x < y</code>	<code># x is less than y</code>
<code>x >= y</code>	<code># x is greater than or equal to y</code>
<code>x <= y</code>	<code># x is less than or equal to y</code>
<code>x is y</code>	<code># x is the same as y</code>
<code>x is not y</code>	<code># x is not the same as y</code>

Although these operations are probably familiar to you, the Python symbols are different from the mathematical symbols for the same operations. A common error is to use a single equal sign (`=`) instead of a double equal sign (`==`). Remember that `=` is an assignment operator and `==` is a comparison operator. There is no such thing as `=<` or `=>`.

This page titled [3.1: Boolean Expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [3.1: Boolean Expressions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.2: Logical Operators

There are three *logical operators*: `and` , `or` , and `not` . The semantics (meaning) of these operators is similar to their meaning in English. For example,

```
x > 0 and x < 10
```

is true only if `x` is greater than 0 *and* less than 10.

```
n%2 == 0 or n%3 == 0
```

 is true if *either* of the conditions is true, that is, if the number is divisible by 2 *or* 3.

Finally, the `not` operator negates a boolean expression, so `not (x > y)` is true if `x > y` is false; that is, if `x` is less than or equal to `y` .

Strictly speaking, the operands of the logical operators should be boolean expressions, but Python is not very strict. Any nonzero number is interpreted as "true."

```
>>> 17 and True
True
```

This flexibility can be useful, but there are some subtleties to it that might be confusing. You might want to avoid it until you are sure you know what you are doing.

This page titled [3.2: Logical Operators](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

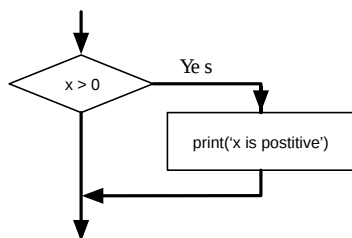
- [3.2: Logical Operators](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.3: Conditional Execution

In order to write useful programs, we almost always need the ability to check conditions and change the behavior of the program accordingly. *Conditional statements* give us this ability. The simplest form is the `if` statement:

```
if x > 0 :  
    print('x is positive')
```

The boolean expression after the `if` statement is called the *condition*. We end the `if` statement with a colon character (`:`) and the line(s) after the if statement are indented.



If Logic

If the logical condition is true, then the indented statement gets executed. If the logical condition is false, the indented statement is skipped.

`if` statements have the same structure as function definitions or `for` loops¹. The statement consists of a header line that ends with the colon character (`:`) followed by an indented block. Statements like this are called *compound statements* because they stretch across more than one line.

There is no limit on the number of statements that can appear in the body, but there must be at least one. Occasionally, it is useful to have a body with no statements (usually as a place holder for code you haven't written yet). In that case, you can use the `pass` statement, which does nothing.

```
if x < 0 :  
    pass          # need to handle negative values!
```

If you enter an `if` statement in the Python interpreter, the prompt will change from three chevrons to three dots to indicate you are in the middle of a block of statements, as shown below:

```
>>> x = 3  
>>> if x < 10:  
...     print('Small')  
...  
Small  
>>>
```

This page titled [3.3: Conditional Execution](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

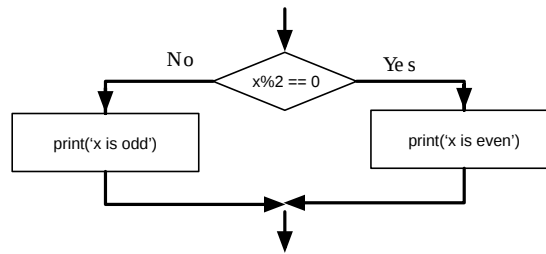
- [3.3: Conditional Execution](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.4: Alternative Execution

A second form of the `if` statement is *alternative execution*, in which there are two possibilities and the condition determines which one gets executed. The syntax looks like this:

```
if x%2 == 0 :  
    print('x is even')  
else :  
    print('x is odd')
```

If the remainder when `x` is divided by 2 is 0, then we know that `x` is even, and the program displays a message to that effect. If the condition is false, the second set of statements is executed.



If-Then-Else Logic

Since the condition must either be true or false, exactly one of the alternatives will be executed. The alternatives are called *branches*, because they are branches in the flow of execution.

This page titled [3.4: Alternative Execution](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

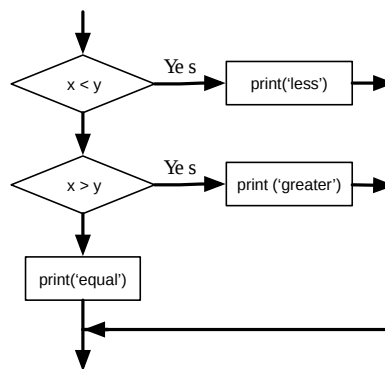
- [3.4: Alternative Execution](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.5: Chained Conditionals

Sometimes there are more than two possibilities and we need more than two branches. One way to express a computation like that is a *chained conditional*:

```
if x < y:
    print('x is less than y')
elif x > y:
    print('x is greater than y')
else:
    print('x and y are equal')
```

`elif` is an abbreviation of "else if." Again, exactly one branch will be executed.



If-Then-ElseIf Logic

There is no limit on the number of `elif` statements. If there is an `else` clause, it has to be at the end, but there doesn't have to be one.

Each condition is checked in order. If the first is false, the next is checked, and so on. If one of them is true, the corresponding branch executes, and the statement ends. Even if more than one condition is true, only the first true branch executes.

This page titled [3.5: Chained Conditionals](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

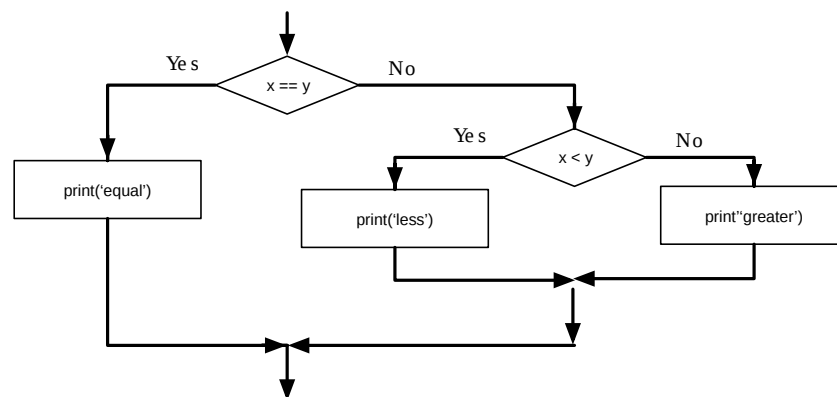
- [3.5: Chained Conditionals](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.6: Nested Conditionals

One conditional can also be nested within another. We could have written the three-branch example like this:

```
if x == y:
    print('x and y are equal')
else:
    if x < y:
        print('x is less than y')
    else:
        print('x is greater than y')
```

The outer conditional contains two branches. The first branch contains a simple statement. The second branch contains another `if` statement, which has two branches of its own. Those two branches are both simple statements, although they could have been conditional statements as well.



Nested If Statements

Although the indentation of the statements makes the structure apparent, *nested conditionals* become difficult to read very quickly. In general, it is a good idea to avoid them when you can.

Logical operators often provide a way to simplify nested conditional statements. For example, we can rewrite the following code using a single conditional:

```
if 0 < x:
    if x < 10:
        print('x is a positive single-digit number.')
```

The `print` statement is executed only if we make it past both conditionals, so we can get the same effect with the `and` operator:

```
if 0 < x and x < 10:
    print('x is a positive single-digit number.')
```

This page titled [3.6: Nested Conditionals](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [3.6: Nested Conditionals](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.7: Catching exceptions Using Try and Except

Earlier we saw a code segment where we used the `input` and `int` functions to read and parse an integer number entered by the user. We also saw how treacherous doing this could be:

```
>>> prompt = "What...is the airspeed velocity of an unladen swallow?\n"
>>> speed = input(prompt)
What...is the airspeed velocity of an unladen swallow?
What do you mean, an African or a European swallow?
>>> int(speed)
ValueError: invalid literal for int() with base 10:
>>>
```

When we are executing these statements in the Python interpreter, we get a new prompt from the interpreter, think "oops", and move on to our next statement.

However if you place this code in a Python script and this error occurs, your script immediately stops in its tracks with a traceback. It does not execute the following statement.

Here is a sample program to convert a Fahrenheit temperature to a Celsius temperature:

If we execute this code and give it invalid input, it simply fails with an unfriendly error message:

```
python fahren.py
Enter Fahrenheit Temperature:72
22.22222222222222
```

```
python fahren.py
Enter Fahrenheit Temperature:fred
Traceback (most recent call last):
  File "fahren.py", line 2, in <module>
    fahr = float(inp)
ValueError: could not convert string to float: 'fred'
```

There is a conditional execution structure built into Python to handle these types of expected and unexpected errors called "try / except". The idea of `try` and `except` is that you know that some sequence of instruction(s) may have a problem and you want to add some statements to be executed if an error occurs. These extra statements (the `except` block) are ignored if there is no error.

You can think of the `try` and `except` feature in Python as an "insurance policy" on a sequence of statements.

We can rewrite our temperature converter as follows:

Python starts by executing the sequence of statements in the `try` block. If all goes well, it skips the `except` block and proceeds. If an exception occurs in the `try` block, Python jumps out of the `try` block and executes the sequence of statements in the `except` block.

```
python fahren2.py
Enter Fahrenheit Temperature:72
22.22222222222222
```

```
python fahren2.py
Enter Fahrenheit Temperature:fred
PYTHON SESSION: READY
```

Handling an exception with a `try` statement is called *catching* an exception. In this example, the `except` clause prints an error message. In general, catching an exception gives you a chance to fix the problem, or try again, or at least end the program gracefully.

This page titled [3.7: Catching exceptions Using Try and Except](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [3.7: Catching exceptions Using Try and Except](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

3.8: Short-Circuit Evaluation of Logical Expressions

When Python is processing a logical expression such as `x >= 2 and (x/y) > 2`, it evaluates the expression from left to right. Because of the definition of `and`, if `x` is less than 2, the expression `x >= 2` is `False` and so the whole expression is `False` regardless of whether `(x/y) > 2` evaluates to `True` or `False`.

When Python detects that there is nothing to be gained by evaluating the rest of a logical expression, it stops its evaluation and does not do the computations in the rest of the logical expression. When the evaluation of a logical expression stops because the overall value is already known, it is called *short-circuiting* the evaluation.

While this may seem like a fine point, the short-circuit behavior leads to a clever technique called the *guardian pattern*. Consider the following code sequence in the Python interpreter:

```
>>> x = 6
>>> y = 2
>>> x >= 2 and (x/y) > 2
True
>>> x = 1
>>> y = 0
>>> x >= 2 and (x/y) > 2
False
>>> x = 6
>>> y = 0
>>> x >= 2 and (x/y) > 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>
```

The third calculation failed because Python was evaluating `(x/y)` and `y` was zero, which causes a runtime error. But the second example did *not* fail because the first part of the expression `x >= 2` evaluated to `False` so the `(x/y)` was not ever executed due to the *short-circuit* rule and there was no error.

We can construct the logical expression to strategically place a *guard* evaluation just before the evaluation that might cause an error as follows:

```
>>> x = 1
>>> y = 0
>>> x >= 2 and y != 0 and (x/y) > 2
False
>>> x = 6
>>> y = 0
>>> x >= 2 and y != 0 and (x/y) > 2
False
>>> x >= 2 and (x/y) > 2 and y != 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>>
```

In the first logical expression, `x >= 2` is `False` so the evaluation stops at the `and`. In the second logical expression, `x >= 2` is `True` but `y != 0` is `False` so we never reach `(x/y)`.

In the third logical expression, the `y != 0` is *after* the `(x/y)` calculation so the expression fails with an error.

In the second expression, we say that $y \neq 0$ acts as a *guard* to insure that we only execute (x/y) if y is non-zero.

This page titled [3.8: Short-Circuit Evaluation of Logical Expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **3.8: Short-Circuit Evaluation of Logical Expressions** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.9: Debugging

The traceback Python displays when an error occurs contains a lot of information, but it can be overwhelming. The most useful parts are usually:

- What kind of error it was, and
- Where it occurred.

Syntax errors are usually easy to find, but there are a few gotchas. Whitespace errors can be tricky because spaces and tabs are invisible and we are used to ignoring them.

```
>>> x = 5
>>> y = 6
      File "<stdin>", line 1
        y = 6
        ^
IndentationError: unexpected indent
```

In this example, the problem is that the second line is indented by one space. But the error message points to `y`, which is misleading. In general, error messages indicate where the problem was discovered, but the actual error might be earlier in the code, sometimes on a previous line.

In general, error messages tell you where the problem was discovered, but that is often not where it was caused.

This page titled [3.9: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **3.9: Debugging** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

3.E: Conditional Execution (Exercises)

Exercise 1: Rewrite your pay computation to give the employee 1.5 times the hourly rate for hours worked above 40 hours.

```
Enter Hours: 45
Enter Rate: 10
Pay: 475.0
```

%%python3

#Use this space to write and run your code

run

restart

restart & run all

Exercise 2: Rewrite your pay program using `try` and `except` so that your program handles non-numeric input gracefully by printing a message and exiting the program. The following shows two executions of the program:

```
Enter Hours: 20
Enter Rate: nine
Error, please enter numeric input
```

```
Enter Hours: forty
Error, please enter numeric input
```

%%python3

#Use this space to write and run your code

run

restart

restart & run all

Exercise 3: Write a program to prompt for a score between 0.0 and 1.0. If the score is out of range, print an error message. If the score is between 0.0 and 1.0, print a grade using the following table:

Score	Grade
≥ 0.9	A
≥ 0.8	B
≥ 0.7	C
≥ 0.6	D
< 0.6	F

```
Enter score: 0.95
A
```

```
Enter score: perfect
Bad score
```

```
Enter score: 10.0
Bad score
```

```
Enter score: 0.75
```

PYTHON SESSION: READY

Enter score: 0.5
F

```
%%python3
```

#Use this space to write and run your code

run

restart

restart & run all

Run the program repeatedly as shown above to test the various different values for input.

1. We will learn about functions in Chapter 4 and loops in Chapter 5.↵

This page titled [3.E: Conditional Execution \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [3.E: Conditional Execution \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

3.G: Conditional Execution (Glossary)

body

The sequence of statements within a compound statement.

boolean expression

An expression whose value is either `True` or `False` .

branch

One of the alternative sequences of statements in a conditional statement.

chained conditional

A conditional statement with a series of alternative branches.

comparison operator

One of the operators that compares its operands: `==` , `!=` , `>` , `<` , `>=` , and `<=` .

conditional statement

A statement that controls the flow of execution depending on some condition.

condition

The boolean expression in a conditional statement that determines which branch is executed.

compound statement

A statement that consists of a header and a body. The header ends with a colon (:). The body is indented relative to the header.

guardian pattern

Where we construct a logical expression with additional comparisons to take advantage of the short-circuit behavior.

logical operator

One of the operators that combines boolean expressions: `and` , `or` , and `not` .

nested conditional

A conditional statement that appears in one of the branches of another conditional statement.

traceback

A list of the functions that are executing, printed when an exception occurs.

short circuit

When Python is part-way through evaluating a logical expression and stops the evaluation because Python knows the final value for the expression without needing to evaluate the rest of the expression.

This page titled [3.G: Conditional Execution \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

4: Functions

- [4.1: Function Calls](#)
- [4.2: Built-in Functions](#)
- [4.3: Type Conversion Functions](#)
- [4.4: Random Numbers](#)
- [4.5: Math Functions](#)
- [4.6: Adding New Functions](#)
- [4.7: Definitions and Uses](#)
- [4.8: Flow of Execution](#)
- [4.9: Parameters and Arguments](#)
- [4.10: Fruitful functions and void functions](#)
- [4.11: Why functions?](#)
- [4.12: Debugging](#)
- [4.E: Functions \(Exercises\)](#)
- [4.G: Functions \(Glossary\)](#)

This page titled [4: Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.1: Function Calls

In the context of programming, a *function* is a named sequence of statements that performs a computation. When you define a function, you specify the name and the sequence of statements. Later, you can "call" the function by name. We have already seen one example of a *function call*:

```
>>> type(32)
<class 'int'>
```

The name of the function is `type`. The expression in parentheses is called the *argument* of the function. The argument is a value or variable that we are passing into the function as input to the function. The result, for the `type` function, is the type of the argument.

It is common to say that a function "takes" an argument and "returns" a result. The result is called the *return value*.

This page titled [4.1: Function Calls](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.1: Function Calls](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.2: Built-in Functions

Python provides a number of important built-in functions that we can use without needing to provide the function definition. The creators of Python wrote a set of functions to solve common problems and included them in Python for us to use.

The `max` and `min` functions give us the largest and smallest values in a list, respectively:

```
>>> max('Hello world')
'w'
>>> min('Hello world')
' '
```

The `max` function tells us the "largest character" in the string (which turns out to be the letter "w") and the `min` function shows us the smallest character (which turns out to be a space).

Another very common built-in function is the `len` function which tells us how many items are in its argument. If the argument to `len` is a string, it returns the number of characters in the string.

```
>>> len('Hello world')
11
>>>
```

These functions are not limited to looking at strings. They can operate on any set of values, as we will see in later chapters.

You should treat the names of built-in functions as reserved words (i.e., avoid using "max" as a variable name).

This page titled [4.2: Built-in Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.2: Built-in Functions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.3: Type Conversion Functions

Python also provides built-in functions that convert values from one type to another. The `int` function takes any value and converts it to an integer, if it can, or complains otherwise:

```
>>> int('32')
32
>>> int('Hello')
ValueError: invalid literal for int() with base 10: 'Hello'
```

`int` can convert floating-point values to integers, but it doesn't round off; it chops off the fraction part:

```
>>> int(3.99999)
3
>>> int(-2.3)
-2
```

`float` converts integers and strings to floating-point numbers:

```
>>> float(32)
32.0
>>> float('3.14159')
3.14159
```

Finally, `str` converts its argument to a string:

```
>>> str(32)
'32'
>>> str(3.14159)
'3.14159'
```

This page titled [4.3: Type Conversion Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.3: Type Conversion Functions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.4: Random Numbers

Given the same inputs, most computer programs generate the same outputs every time, so they are said to be *deterministic*. Determinism is usually a good thing, since we expect the same calculation to yield the same result. For some applications, though, we want the computer to be unpredictable. Games are an obvious example, but there are more.

Making a program truly nondeterministic turns out to be not so easy, but there are ways to make it at least seem nondeterministic. One of them is to use *algorithms* that generate *pseudorandom* numbers. Pseudorandom numbers are not truly random because they are generated by a deterministic computation, but just by looking at the numbers it is all but impossible to distinguish them from random.

The `random` module provides functions that generate pseudorandom numbers (which I will simply call "random" from here on).

The function `random` returns a random float between 0.0 and 1.0 (including 0.0 but not 1.0). Each time you call `random`, you get the next number in a long series. To see a sample, run this loop:

```
import random

for i in range(10):
    x = random.random()
    print(x)
```

This program produces the following list of 10 random numbers between 0.0 and up to but not including 1.0.

```
0.11132867921152356
0.5950949227890241
0.04820265884996877
0.841003109276478
0.997914947094958
0.04842330803368111
0.7416295948208405
0.510535245390327
0.27447040171978143
0.028511805472785867
```

Exercise 1: Run the program on your system and see what numbers you get. Run the program more than once and see what numbers you get.

The `random` function is only one of many functions that handle random numbers. The function `randint` takes the parameters `low` and `high`, and returns an integer between `low` and `high` (including both).

```
>>> random.randint(5, 10)
5
>>> random.randint(5, 10)
9
```

To choose an element from a sequence at random, you can use `choice`:

```
>>> t = [1, 2, 3]
>>> random.choice(t)
2
```

PYTHON SESSION: READY

The `random` module also provides functions to generate random values from continuous distributions including Gaussian, exponential, gamma, and a few more.

This page titled [4.4: Random Numbers](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.4: Random Numbers](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

4.5: Math Functions

Python has a `math` module that provides most of the familiar mathematical functions. Before we can use the module, we have to import it:

```
>>> import math
```

This statement creates a *module object* named `math`. If you print the module object, you get some information about it:

```
>>> print(math)
<module 'math' (built-in)>
```

The module object contains the functions and variables defined in the module. To access one of the functions, you have to specify the name of the module and the name of the function, separated by a dot (also known as a period). This format is called *dot notation*.

```
>>> ratio = signal_power / noise_power
>>> decibels = 10 * math.log10(ratio)

>>> radians = 0.7
>>> height = math.sin(radians)
```

The first example computes the logarithm base 10 of the signal-to-noise ratio. The `math` module also provides a function called `log` that computes logarithms base e .

The second example finds the sine of `radians`. The name of the variable is a hint that `sin` and the other trigonometric functions (`cos`, `tan`, etc.) take arguments in radians. To convert from degrees to radians, divide by 360 and multiply by 2π :

```
>>> degrees = 45
>>> radians = degrees / 360.0 * 2 * math.pi
>>> math.sin(radians)
0.7071067811865476
```

The expression `math.pi` gets the variable `pi` from the `math` module. The value of this variable is an approximation of π , accurate to about 15 digits.

If you know your trigonometry, you can check the previous result by comparing it to the square root of two divided by two:

```
>>> math.sqrt(2) / 2.0
0.7071067811865476
```

This page titled [4.5: Math Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **4.5: Math Functions** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.6: Adding New Functions

So far, we have only been using the functions that come with Python, but it is also possible to add new functions. A *function definition* specifies the name of a new function and the sequence of statements that execute when the function is called. Once we define a function, we can reuse the function over and over throughout our program.

Here is an example:

```
def print_lyrics():
    print("I'm a lumberjack, and I'm okay.")
    print('I sleep all night and I work all day.')
```

`def` is a keyword that indicates that this is a function definition. The name of the function is `print_lyrics`. The rules for function names are the same as for variable names: letters, numbers and some punctuation marks are legal, but the first character can't be a number. You can't use a keyword as the name of a function, and you should avoid having a variable and a function with the same name.

The empty parentheses after the name indicate that this function doesn't take any arguments. Later we will build functions that take arguments as their inputs.

The first line of the function definition is called the *header*; the rest is called the *body*. The header has to end with a colon and the body has to be indented. By convention, the indentation is always four spaces. The body can contain any number of statements.

The strings in the print statements are enclosed in quotes. Single quotes and double quotes do the same thing; most people use single quotes except in cases like this where a single quote (which is also an apostrophe) appears in the string.

If you type a function definition in interactive mode, the interpreter prints ellipses (...) to let you know that the definition isn't complete:

```
>>> def print_lyrics():
...     print("I'm a lumberjack, and I'm okay.")
...     print('I sleep all night and I work all day.')
... 
```

To end the function, you have to enter an empty line (this is not necessary in a script).

Defining a function creates a variable with the same name.

```
>>> print(print_lyrics)
<function print_lyrics at 0xb7e99e9c>
>>> print(type(print_lyrics))
<class 'function'>
```

The value of `print_lyrics` is a *function object*, which has type "function".

The syntax for calling the new function is the same as for built-in functions:

```
>>> print_lyrics()
I'm a lumberjack, and I'm okay.
I sleep all night and I work all day.
```

Once you have defined a function, you can use it inside another function. For example, to repeat the previous refrain, we could write a function called `repeat_lyrics`:

```
def repeat_lyrics():  
    print_lyrics()  
    print_lyrics()
```

And then call `repeat_lyrics` :

```
>>> repeat_lyrics()  
I'm a lumberjack, and I'm okay.  
I sleep all night and I work all day.  
I'm a lumberjack, and I'm okay.  
I sleep all night and I work all day.
```

But that's not really how the song goes.

This page titled [4.6: Adding New Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.6: Adding New Functions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.7: Definitions and Uses

Pulling together the code fragments from the previous section, the whole program looks like this:

This program contains two function definitions: `print_lyrics` and `repeat_lyrics`. Function definitions get executed just like other statements, but the effect is to create function objects. The statements inside the function do not get executed until the function is called, and the function definition generates no output.

As you might expect, you have to create a function before you can execute it. In other words, the function definition has to be executed before the first time it is called.

Exercise 2: Edit the code block and move the last line of this program to the top, so the function call appears before the definitions. Run the program and see what error message you get.

Exercise 3: Move the function call back to the bottom and move the definition of `print_lyrics` after the definition of `repeat_lyrics`. What happens when you run this program?

This page titled [4.7: Definitions and Uses](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.7: Definitions and Uses](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

4.8: Flow of Execution

In order to ensure that a function is defined before its first use, you have to know the order in which statements are executed, which is called the *flow of execution*.

Execution always begins at the first statement of the program. Statements are executed one at a time, in order from top to bottom.

Function *definitions* do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.

A function call is like a detour in the flow of execution. Instead of going to the next statement, the flow jumps to the body of the function, executes all the statements there, and then comes back to pick up where it left off.

That sounds simple enough, until you remember that one function can call another. While in the middle of one function, the program might have to execute the statements in another function. But while executing that new function, the program might have to execute yet another function!

Fortunately, Python is good at keeping track of where it is, so each time a function completes, the program picks up where it left off in the function that called it. When it gets to the end of the program, it terminates.

What's the moral of this sordid tale? When you read a program, you don't always want to read from top to bottom. Sometimes it makes more sense if you follow the flow of execution.

This page titled [4.8: Flow of Execution](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.8: Flow of Execution](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.9: Parameters and Arguments

Some of the built-in functions we have seen require arguments. For example, when you call `math.sin` you pass a number as an argument. Some functions take more than one argument: `math.pow` takes two, the base and the exponent.

Inside the function, the arguments are assigned to variables called *parameters*. Here is an example of a user-defined function that takes an argument:

```
def print_twice(bruce):  
    print(bruce)  
    print(bruce)
```

This function assigns the argument to a parameter named `bruce`. When the function is called, it prints the value of the parameter (whatever it is) twice.

This function works with any value that can be printed.

```
>>> print_twice('Spam')  
Spam  
Spam  
>>> print_twice(17)  
17  
17  
>>> import math  
>>> print_twice(math.pi)  
3.141592653589793  
3.141592653589793
```

The same rules of composition that apply to built-in functions also apply to user-defined functions, so we can use any kind of expression as an argument for `print_twice`:

```
>>> print_twice('Spam '*4)  
Spam Spam Spam Spam  
Spam Spam Spam Spam  
>>> print_twice(math.cos(math.pi))  
-1.0  
-1.0
```

The argument is evaluated before the function is called, so in the examples the expressions `Spam '*4` and `math.cos(math.pi)` are only evaluated once.

You can also use a variable as an argument:

```
>>> michael = 'Eric, the half a bee.'  
>>> print_twice(michael)  
Eric, the half a bee.  
Eric, the half a bee.
```

The name of the variable we pass as an argument (`michael`) has nothing to do with the name of the parameter (`bruce`). It doesn't matter what the value was called back home (in the caller); here in `print_twice`, we call everybody `bruce`.

This page titled [4.9: Parameters and Arguments](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.9: Parameters and Arguments](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.10: Fruitful functions and void functions

Some of the functions we are using, such as the math functions, yield results; for lack of a better name, I call them *fruitful functions*. Other functions, like `print_twice`, perform an action but don't return a value. They are called *void functions*.

When you call a fruitful function, you almost always want to do something with the result; for example, you might assign it to a variable or use it as part of an expression:

```
x = math.cos(radians)
golden = (math.sqrt(5) + 1) / 2
```

When you call a function in interactive mode, Python displays the result:

```
>>> math.sqrt(5)
2.23606797749979
```

But in a script, if you call a fruitful function and do not store the result of the function in a variable, the return value vanishes into the mist!

```
math.sqrt(5)
```

This script computes the square root of 5, but since it doesn't store the result in a variable or display the result, it is not very useful.

Void functions might display something on the screen or have some other effect, but they don't have a return value. If you try to assign the result to a variable, you get a special value called `None`.

```
>>> result = print_twice('Bing')
Bing
Bing
>>> print(result)
None
```

The value `None` is not the same as the string "None". It is a special value that has its own type:

```
>>> print(type(None))
<class 'NoneType'>
```

To return a result from a function, we use the `return` statement in our function. For example, we could make a very simple function called `addtwo` that adds two numbers together and returns a result.

When this script executes, the `print` statement will print out "8" because the `addtwo` function was called with 3 and 5 as arguments. Within the function, the parameters `a` and `b` were 3 and 5 respectively. The function computed the sum of the two numbers and placed it in the local function variable named `added`. Then it used the `return` statement to send the computed value back to the calling code as the function result, which was assigned to the variable `x` and printed out.

This page titled [4.10: Fruitful functions and void functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.10: Fruitful functions and void functions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

4.11: Why functions?

It may not be clear why it is worth the trouble to divide a program into functions. There are several reasons:

- Creating a new function gives you an opportunity to name a group of statements, which makes your program easier to read, understand, and debug.
- Functions can make a program smaller by eliminating repetitive code. Later, if you make a change, you only have to make it in one place.
- Dividing a long program into functions allows you to debug the parts one at a time and then assemble them into a working whole.
- Well-designed functions are often useful for many programs. Once you write and debug one, you can reuse it.

Throughout the rest of the book, often we will use a function definition to explain a concept. Part of the skill of creating and using functions is to have a function properly capture an idea such as "find the smallest value in a list of values". Later we will show you code that finds the smallest in a list of values and we will present it to you as a function named `min` which takes a list of values as its argument and returns the smallest value in the list.

This page titled [4.11: Why functions?](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **4.11: Why functions?** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.12: Debugging

If you are using a text editor to write your scripts, you might run into problems with spaces and tabs. The best way to avoid these problems is to use spaces exclusively (no tabs). Most text editors that know about Python do this by default, but some don't.

Tabs and spaces are usually invisible, which makes them hard to debug, so try to find an editor that manages indentation for you.

Also, don't forget to save your program before you run it. Some development environments do this automatically, but some don't. In that case, the program you are looking at in the text editor is not the same as the program you are running.

Debugging can take a long time if you keep running the same incorrect program over and over!

Make sure that the code you are looking at is the code you are running. If you're not sure, put something like `print("hello")` at the beginning of the program and run it again. If you don't see `hello`, you're not running the right program!

This page titled [4.12: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.12: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.E: Functions (Exercises)

Exercise 4: What is the purpose of the "def" keyword in Python?

- a) It is slang that means "the following code is really cool"
- b) It indicates the start of a function
- c) It indicates that the following indented section of code is to be stored for later
- d) b and c are both true
- e) None of the above

Exercise 5: What will the following Python program print out?

```
def fred():  
    print("Zap")  
  
def jane():  
    print("ABC")  
  
jane()  
fred()  
jane()
```

- a) Zap ABC jane fred jane
- b) Zap ABC Zap
- c) ABC Zap jane
- d) ABC Zap ABC
- e) Zap Zap Zap

Exercise 6: Rewrite your pay computation with time-and-a-half for overtime and create a function called `compute_pay` which takes two parameters (`hours` and `rate`).

```
Enter Hours: 45  
Enter Rate: 10  
Pay: 475.0
```

Exercise 7: Rewrite the grade program from the previous chapter using a function called `compute_grade` that takes a score as its parameter and returns a grade as a string.

Score	Grade
> 0.9	A
> 0.8	B
> 0.7	C
> 0.6	D
<= 0.6	F

Program Execution:

```
Enter score: 0.95  
A
```

```
Enter score: perfect  
Bad score
```

Enter score: 10.0
Bad score

Enter score: 0.75
C

Enter score: 0.5
F

Run the program repeatedly to test the various different values for input.

This page titled [4.E: Functions \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [4.E: Functions \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

4.G: Functions (Glossary)

algorithm

A general process for solving a category of problems.

argument

A value provided to a function when the function is called. This value is assigned to the corresponding parameter in the function.

body

The sequence of statements inside a function definition.

composition

Using an expression as part of a larger expression, or a statement as part of a larger statement.

deterministic

Pertaining to a program that does the same thing each time it runs, given the same inputs.

dot notation

The syntax for calling a function in another module by specifying the module name followed by a dot (period) and the function name.

flow of execution

The order in which statements are executed during a program run.

fruitful function

A function that returns a value.

function

A named sequence of statements that performs some useful operation. Functions may or may not take arguments and may or may not produce a result.

function call

A statement that executes a function. It consists of the function name followed by an argument list.

function definition

A statement that creates a new function, specifying its name, parameters, and the statements it executes.

function object

A value created by a function definition. The name of the function is a variable that refers to a function object.

header

The first line of a function definition.

import statement

A statement that reads a module file and creates a module object.

module object

A value created by an `import` statement that provides access to the data and code defined in a module.

parameter

A name used inside a function to refer to the value passed as an argument.

pseudorandom

Pertaining to a sequence of numbers that appear to be random, but are generated by a deterministic program.

return value

The result of a function. If a function call is used as an expression, the return value is the value of the expression.

void function

A function that does not return a value.

This page titled [4.G: Functions \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

5: Iterations

- [5.1: Updating Variables](#)
- [5.2: The while Statement](#)
- [5.3: Infinite Loops](#)
- [5.4: "Infinite loops" and break](#)
- [5.5: Finishing iterations with continue](#)
- [5.6: Definite loops using for](#)
- [5.7: Loop patterns](#)
- [5.8: Counting and Summing Loops](#)
- [5.9: Maximum and Minimum Loops](#)
- [5.10: Debugging](#)
- [5.E: Iterations \(Exercises\)](#)
- [5.G: Iterations \(Glossary\)](#)

This page titled [5: Iterations](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.1: Updating Variables

A common pattern in assignment statements is an assignment statement that updates a variable, where the new value of the variable depends on the old.

```
x = x + 1
```

This means "get the current value of `x`, add 1, and then update `x` with the new value."

If you try to update a variable that doesn't exist, you get an error, because Python evaluates the right side before it assigns a value to `x`:

```
>>> x = x + 1
NameError: name 'x' is not defined
```

Before you can update a variable, you have to *initialize* it, usually with a simple assignment:

```
>>> x = 0
>>> x = x + 1
```

Updating a variable by adding 1 is called an *increment*; subtracting 1 is called a *decrement*.

This page titled [5.1: Updating Variables](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.1: Updating Variables](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.2: The while Statement

Computers are often used to automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well and people do poorly. Because iteration is so common, Python provides several language features to make it easier.

One form of iteration in Python is the `while` statement. Here is a simple program that counts down from five and then says "Blastoff!".

You can almost read the `while` statement as if it were English. It means, "While `n` is greater than 0, display the value of `n` and then reduce the value of `n` by 1. When you get to 0, exit the `while` statement and display the word `Blastoff!` "

More formally, here is the flow of execution for a `while` statement:

1. Evaluate the condition, yielding `True` or `False` .
2. If the condition is false, exit the `while` statement and continue execution at the next statement.
3. If the condition is true, execute the body and then go back to step 1.

This type of flow is called a *loop* because the third step loops back around to the top. We call each time we execute the body of the loop an *iteration*. For the above loop, we would say, "It had five iterations", which means that the body of the loop was executed five times.

The body of the loop should change the value of one or more variables so that eventually the condition becomes false and the loop terminates. We call the variable that changes each time the loop executes and controls when the loop finishes the *iteration variable*. If there is no iteration variable, the loop will repeat forever, resulting in an *infinite loop*.

This page titled [5.2: The while Statement](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.2: The while Statement](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.3: Infinite Loops

An endless source of amusement for programmers is the observation that the directions on shampoo, "Lather, rinse, repeat," are an infinite loop because there is no *iteration variable* telling you how many times to execute the loop.

In the case of `countdown`, we can prove that the loop terminates because we know that the value of `n` is finite, and we can see that the value of `n` gets smaller each time through the loop, so eventually we have to get to 0. Other times a loop is obviously infinite because it has no iteration variable at all.

This page titled [5.3: Infinite Loops](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.3: Infinite Loops](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.4: "Infinite loops" and break

Sometimes you don't know it's time to end a loop until you get half way through the body. In that case you can write an infinite loop on purpose and then use the `break` statement to jump out of the loop.

This loop is obviously an *infinite loop* because the logical expression on the `while` statement is simply the logical constant `True` :

```
n = 10
while True:
    print(n, end=' ')
    n = n - 1
print('Done!')
```

If you make the mistake and run this code, you will learn quickly how to stop a runaway Python process on your system or find where the power-off button is on your computer. This program will run forever or until your battery runs out because the logical expression at the top of the loop is always true by virtue of the fact that the expression is the constant value `True` .

While this is a dysfunctional infinite loop, we can still use this pattern to build useful loops as long as we carefully add code to the body of the loop to explicitly exit the loop using `break` when we have reached the exit condition.

For example, suppose you want to take input from the user until they type `done` . You could write:

The loop condition is `True` , which is always true, so the loop runs repeatedly until it hits the break statement.

Each time through, it prompts the user with an angle bracket. If the user types `done` , the `break` statement exits the loop. Otherwise the program echoes whatever the user types and goes back to the top of the loop. Here's a sample run:

```
> hello there
hello there
> finished
finished
> done
Done!
```

This way of writing `while` loops is common because you can check the condition anywhere in the loop (not just at the top) and you can express the stop condition affirmatively ("stop when this happens") rather than negatively ("keep going until that happens.").

This page titled 5.4: "Infinite loops" and break is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- 5.4: "Infinite loops" and break by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.5: Finishing iterations with continue

Sometimes you are in an iteration of a loop and want to finish the current iteration and immediately jump to the next iteration. In that case you can use the `continue` statement to skip to the next iteration without finishing the body of the loop for the current iteration.

Here is an example of a loop that copies its input until the user types "done", but treats lines that start with the hash character as lines not to be printed (kind of like Python comments).

Here is a sample run of this new program with `continue` added.

```
> hello there
hello there
> # don't print this
> print this!
print this!
> done
Done!
```

All the lines are printed except the one that starts with the hash sign because when the `continue` is executed, it ends the current iteration and jumps back to the `while` statement to start the next iteration, thus skipping the `print` statement.

This page titled [5.5: Finishing iterations with continue](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.5: Finishing iterations with continue](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.6: Definite loops using for

Sometimes we want to loop through a *set* of things such as a list of words, the lines in a file, or a list of numbers. When we have a list of things to loop through, we can construct a *definite* loop using a `for` statement. We call the `while` statement an *indefinite* loop because it simply loops until some condition becomes `False`, whereas the `for` loop is looping through a known set of items so it runs through as many iterations as there are items in the set.

The syntax of a `for` loop is similar to the `while` loop in that there is a `for` statement and a loop body:

In Python terms, the variable `friends` is a list¹ of three strings and the `for` loop goes through the list and executes the body once for each of the three strings in the list resulting in this output:

```
Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally
Done!
```

Translating this `for` loop to English is not as direct as the `while`, but if you think of `friends` as a *set*, it goes like this: "Run the statements in the body of the `for` loop once for each friend *in* the set named `friends`."

Looking at the `for` loop, `for` and `in` are reserved Python keywords, and `friend` and `friends` are variables.

```
for friend in friends:
    print('Happy New Year:', friend)
```

In particular, `friend` is the *iteration variable* for the `for` loop. The variable `friend` changes for each iteration of the loop and controls when the `for` loop completes. The *iteration variable* steps successively through the three strings stored in the `friends` variable.

This page titled [5.6: Definite loops using for](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.6: Definite loops using for](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.7: Loop patterns

Often we use a `for` or `while` loop to go through a list of items or the contents of a file and we are looking for something such as the largest or smallest value of the data we scan through.

These loops are generally constructed by:

- Initializing one or more variables before the loop starts
- Performing some computation on each item in the loop body, possibly changing the variables in the body of the loop
- Looking at the resulting variables when the loop completes

We will use a list of numbers to demonstrate the concepts and construction of these loop patterns.

This page titled [5.7: Loop patterns](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.7: Loop patterns](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.8: Counting and Summing Loops

For example, to count the number of items in a list, we would write the following `for` loop:

We set the variable `count` to zero before the loop starts, then we write a `for` loop to run through the list of numbers. Our *iteration* variable is named `itervar` and while we do not use `itervar` in the loop, it does control the loop and cause the loop body to be executed once for each of the values in the list.

In the body of the loop, we add 1 to the current value of `count` for each of the values in the list. While the loop is executing, the value of `count` is the number of values we have seen "so far".

Once the loop completes, the value of `count` is the total number of items. The total number "falls in our lap" at the end of the loop. We construct the loop so that we have what we want when the loop finishes.

Another similar loop that computes the total of a set of numbers is as follows:

In this loop we *do* use the *iteration variable*. Instead of simply adding one to the `count` as in the previous loop, we add the actual number (3, 41, 12, etc.) to the running total during each loop iteration. If you think about the variable `total`, it contains the "running total of the values so far". So before the loop starts `total` is zero because we have not yet seen any values, during the loop `total` is the running total, and at the end of the loop `total` is the overall total of all the values in the list.

As the loop executes, `total` accumulates the sum of the elements; a variable used this way is sometimes called an *accumulator*.

Neither the counting loop nor the summing loop are particularly useful in practice because there are built-in functions `len()` and `sum()` that compute the number of items in a list and the total of the items in the list respectively.

This page titled [5.8: Counting and Summing Loops](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.8: Counting and Summing Loops](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.9: Maximum and Minimum Loops

To find the largest value in a list or sequence, we construct the following loop:

When the program executes, the output is as follows:

```
Before: None
Loop: 3 3
Loop: 41 41
Loop: 12 41
Loop: 9 41
Loop: 74 74
Loop: 15 74
Largest: 74
```

The variable `largest` is best thought of as the "largest value we have seen so far". Before the loop, we set `largest` to the constant `None`. `None` is a special constant value which we can store in a variable to mark the variable as "empty".

Before the loop starts, the largest value we have seen so far is `None` since we have not yet seen any values. While the loop is executing, if `largest` is `None` then we take the first value we see as the largest so far. You can see in the first iteration when the value of `itervar` is 3, since `largest` is `None`, we immediately set `largest` to be 3.

After the first iteration, `largest` is no longer `None`, so the second part of the compound logical expression that checks `itervar > largest` triggers only when we see a value that is larger than the "largest so far". When we see a new "even larger" value we take that new value for `largest`. You can see in the program output that `largest` progresses from 3 to 41 to 74.

At the end of the loop, we have scanned all of the values and the variable `largest` now does contain the largest value in the list.

To compute the smallest number, the code is very similar with one small change:

Again, `smallest` is the "smallest so far" before, during, and after the loop executes. When the loop has completed, `smallest` contains the minimum value in the list.

Again as in counting and summing, the built-in functions `max()` and `min()` make writing these exact loops unnecessary.

The following is a simple version of the Python built-in `min()` function:

```
def min(values):
    smallest = None
    for value in values:
        if smallest is None or value < smallest:
            smallest = value
    return smallest
```

In the function version of the smallest code, we removed all of the `print` statements so as to be equivalent to the `min` function which is already built in to Python.

This page titled [5.9: Maximum and Minimum Loops](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.9: Maximum and Minimum Loops](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

5.10: Debugging

As you start writing bigger programs, you might find yourself spending more time debugging. More code means more chances to make an error and more places for bugs to hide. One way to cut your debugging time is "debugging by bisection." For example, if there are 100 lines in your program and you check them one at a time, it would take 100 steps.

Instead, try to break the problem in half. Look at the middle of the program, or near it, for an intermediate value you can check. Add a `print` statement (or something else that has a verifiable effect) and run the program.

If the mid-point check is incorrect, the problem must be in the first half of the program. If it is correct, the problem is in the second half.

Every time you perform a check like this, you halve the number of lines you have to search. After six steps (which is much less than 100), you would be down to one or two lines of code, at least in theory.

In practice it is not always clear what the "middle of the program" is and not always possible to check it. It doesn't make sense to count lines and find the exact midpoint. Instead, think about places in the program where there might be errors and places where it is easy to put a check. Then choose a spot where you think the chances are about the same that the bug is before or after the check.

This page titled [5.10: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **5.10: Debugging** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.E: Iterations (Exercises)

Exercise 1: Write a program which repeatedly reads numbers until the user enters "done". Once "done" is entered, print out the total, count, and average of the numbers. If the user enters anything other than a number, detect their mistake using `try` and `except` and print an error message and skip to the next number.

```
Enter a number: 4
Enter a number: 5
Enter a number: bad data
Invalid input
Enter a number: 7
Enter a number: done
16 3 5.333333333333333
```

Exercise 2: Write another program that prompts for a list of numbers as above and at the end prints out both the maximum and minimum of the numbers instead of the average.

1. We will examine lists in more detail in a later chapter.↩
-

This page titled [5.E: Iterations \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [5.E: Iterations \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

5.G: Iterations (Glossary)

accumulator

A variable used in a loop to add up or accumulate a result.

counter

A variable used in a loop to count the number of times something happened. We initialize a counter to zero and then increment the counter each time we want to "count" something.

decrement

An update that decreases the value of a variable.

initialize

An assignment that gives an initial value to a variable that will be updated.

increment

An update that increases the value of a variable (often by one).

infinite loop

A loop in which the terminating condition is never satisfied or for which there is no terminating condition.

iteration

Repeated execution of a set of statements using either a function that calls itself or a loop.

This page titled [5.G: Iterations \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

6: Strings

- [6.1: A string is a sequence](#)
- [6.2: Getting the length of a string using len](#)
- [6.3: Traversal through a string with a loop](#)
- [6.4: String Slices](#)
- [6.5: Strings are immutable](#)
- [6.6: Looping and Counting](#)
- [6.7: The in operator](#)
- [6.8: String Comparison](#)
- [6.9: String Methods](#)
- [6.10: Parsing strings](#)
- [6.11: Format operator](#)
- [6.12: Debugging](#)
- [6.E: Strings \(Exercises\)](#)
- [6.G: Strings \(Glossary\)](#)

This page titled [6: Strings](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

6.1: A string is a sequence

A string is a *sequence* of characters. You can access the characters one at a time with the bracket operator:

```
>>> fruit = 'banana'
>>> letter = fruit[1]
```

The second statement extracts the character at index position 1 from the `fruit` variable and assigns it to the `letter` variable.

The expression in brackets is called an *index*. The index indicates which character in the sequence you want (hence the name).

But you might not get what you expect:

```
>>> print(letter)
a
```

For most people, the first letter of "banana" is `b`, not `a`. But in Python, the index is an offset from the beginning of the string, and the offset of the first letter is zero.

```
>>> letter = fruit[0]
>>> print(letter)
b
```

So `b` is the 0th letter ("zero-eth") of "banana", `a` is the 1th letter ("one-eth"), and `n` is the 2th ("two-eth") letter.

b	a	n	a	n	a
[0]	[1]	[2]	[3]	[4]	[5]

String Indexes

You can use any expression, including variables and operators, as an index, but the value of the index has to be an integer. Otherwise you get:

```
>>> letter = fruit[1.5]
TypeError: string indices must be integers
```

This page titled [6.1: A string is a sequence](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **6.1: A string is a sequence** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.2: Getting the length of a string using len

`len` is a built-in function that returns the number of characters in a string:

```
>>> fruit = 'banana'
>>> len(fruit)
6
```

To get the last letter of a string, you might be tempted to try something like this:

```
>>> length = len(fruit)
>>> last = fruit[length]
IndexError: string index out of range
```

The reason for the `IndexError` is that there is no letter in `'banana'` with the index 6. Since we started counting at zero, the six letters are numbered 0 to 5. To get the last character, you have to subtract 1 from `length` :

```
>>> last = fruit[length-1]
>>> print(last)
a
```

Alternatively, you can use negative indices, which count backward from the end of the string. The expression `fruit[-1]` yields the last letter, `fruit[-2]` yields the second to last, and so on.

This page titled [6.2: Getting the length of a string using len](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.2: Getting the length of a string using len](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.3: Traversal through a string with a loop

A lot of computations involve processing a string one character at a time. Often they start at the beginning, select each character in turn, do something to it, and continue until the end. This pattern of processing is called a *traversal*. One way to write a traversal is with a `while` loop:

```
index = 0
while index < len(fruit):
    letter = fruit[index]
    print(letter)
    index = index + 1
```

This loop traverses the string and displays each letter on a line by itself. The loop condition is `index < len(fruit)`, so when `index` is equal to the length of the string, the condition is false, and the body of the loop is not executed. The last character accessed is the one with the index `len(fruit)-1`, which is the last character in the string.

Exercise 1: Write a `while` loop that starts at the last character in the string and works its way backwards to the first character in the string, printing each letter on a separate line, except backwards.

Another way to write a traversal is with a `for` loop:

```
for char in fruit:
    print(char)
```

Each time through the loop, the next character in the string is assigned to the variable `char`. The loop continues until no characters are left.

This page titled [6.3: Traversal through a string with a loop](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **6.3: Traversal through a string with a loop** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.4: String Slices

A segment of a string is called a *slice*. Selecting a slice is similar to selecting a character:

```
>>> s = 'Monty Python'
>>> print(s[0:5])
Monty
>>> print(s[6:12])
Python
```

The operator returns the part of the string from the "n-eth" character to the "m-eth" character, including the first but excluding the last.

If you omit the first index (before the colon), the slice starts at the beginning of the string. If you omit the second index, the slice goes to the end of the string:

```
>>> fruit = 'banana'
>>> fruit[:3]
'ban'
>>> fruit[3:]
'ana'
```

If the first index is greater than or equal to the second the result is an *empty string*, represented by two quotation marks:

```
>>> fruit = 'banana'
>>> fruit[3:3]
''
```

An empty string contains no characters and has length 0, but other than that, it is the same as any other string.

Exercise 2: Given that `fruit` is a string, what does `fruit[:]` mean?

This page titled [6.4: String Slices](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **6.4: String Slices** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.5: Strings are immutable

It is tempting to use the operator on the left side of an assignment, with the intention of changing a character in a string. For example:

```
>>> greeting = 'Hello, world!'
>>> greeting[0] = 'J'
TypeError: 'str' object does not support item assignment
```

The "object" in this case is the string and the "item" is the character you tried to assign. For now, an *object* is the same thing as a value, but we will refine that definition later. An *item* is one of the values in a sequence.

The reason for the error is that strings are *immutable*, which means you can't change an existing string. The best you can do is create a new string that is a variation on the original:

```
>>> greeting = 'Hello, world!'
>>> new_greeting = 'J' + greeting[1:]
>>> print(new_greeting)
Jello, world!
```

This example concatenates a new first letter onto a slice of `greeting`. It has no effect on the original string.

This page titled [6.5: Strings are immutable](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.5: Strings are immutable](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.6: Looping and Counting

The following program counts the number of times the letter `a` appears in a string:

This program demonstrates another pattern of computation called a *counter*. The variable `count` is initialized to 0 and then incremented each time an `a` is found. When the loop exits, `count` contains the result: the total number of `a`'s.

Exercise 3:

Encapsulate this code in a function named `count`, and generalize it so that it accepts the string and the letter as arguments.

This page titled [6.6: Looping and Counting](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.6: Looping and Counting](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

6.7: The in operator

The word `in` is a boolean operator that takes two strings and returns `True` if the first appears as a substring in the second:

```
>>> 'a' in 'banana'
True
>>> 'seed' in 'banana'
False
```

This page titled [6.7: The in operator](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.7: The in operator](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.8: String Comparison

The comparison operators work on strings. To see if two strings are equal:

```
if word == 'banana':  
    print('All right, bananas.')
```

Other comparison operations are useful for putting words in alphabetical order:

```
if word < 'banana':  
    print('Your word,' + word + ', comes before banana.')
```

```
elif word > 'banana':  
    print('Your word,' + word + ', comes after banana.')
```

```
else:  
    print('All right, bananas.')
```

Python does not handle uppercase and lowercase letters the same way that people do. All the uppercase letters come before all the lowercase letters, so:

```
Your word, Pineapple, comes before banana.
```

A common way to address this problem is to convert strings to a standard format, such as all lowercase, before performing the comparison. Keep that in mind in case you have to defend yourself against a man armed with a Pineapple.

This page titled [6.8: String Comparison](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.8: String Comparison](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.9: String Methods

Strings are an example of Python *objects*. An object contains both data (the actual string itself) and *methods*, which are effectively functions that are built into the object and are available to any *instance* of the object.

Python has a function called `dir` which lists the methods available for an object. The `type` function shows the type of an object and the `dir` function shows the available methods.

```
>>> stuff = 'Hello world'
>>> type(stuff)
<class 'str'>
>>> dir(stuff)
['capitalize', 'casefold', 'center', 'count', 'encode',
'endswith', 'expandtabs', 'find', 'format', 'format_map',
'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit',
'identifier', 'islower', 'isnumeric', 'isprintable',
'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower',
'lstrip', 'maketrans', 'partition', 'replace', 'rfind',
'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',
'split', 'splitlines', 'startswith', 'strip', 'swapcase',
'title', 'translate', 'upper', 'zfill']
>>> help(str.capitalize)
Help on method_descriptor:

capitalize(...)
    S.capitalize() -> str

    Return a capitalized version of S, i.e. make the first character
    have upper case and the rest lower case.

>>>
```

While the `dir` function lists the methods, and you can use `help` to get some simple documentation on a method, a better source of documentation for string methods would be <https://docs.python.org/3.5/library/string-methods>.

Calling a *method* is similar to calling a function (it takes arguments and returns a value) but the syntax is different. We call a method by appending the method name to the variable name using the period as a delimiter.

For example, the method `upper` takes a string and returns a new string with all uppercase letters:

Instead of the function syntax `upper(word)`, it uses the method syntax `word.upper()`.

```
>>> word = 'banana'
>>> new_word = word.upper()
>>> print(new_word)
BANANA
```

This form of dot notation specifies the name of the method, `upper`, and the name of the string to apply the method to, `word`. The empty parentheses indicate that this method takes no argument.

A method call is called an *invocation*; in this case, we would say that we are invoking `upper` on the `word`.

For example, there is a string method named `find` that searches for the position of one string within another:

```
>>> word = 'banana'
>>> index = word.find('a')
>>> print(index)
1
```

In this example, we invoke `find` on `word` and pass the letter we are looking for as a parameter.

The `find` method can find substrings as well as characters:

```
>>> word.find('na')
2
```

It can take as a second argument the index where it should start:

```
>>> word.find('na', 3)
4
```

One common task is to remove white space (spaces, tabs, or newlines) from the beginning and end of a string using the `strip` method:

```
>>> line = ' Here we go '
>>> line.strip()
'Here we go'
```

Some methods such as `startswith` return boolean values.

```
>>> line = 'Have a nice day'
>>> line.startswith('Have')
True
>>> line.startswith('h')
False
```

You will note that `startswith` requires case to match, so sometimes we take a line and map it all to lowercase before we do any checking using the `lower` method.

```
>>> line = 'Have a nice day'
>>> line.startswith('h')
False
>>> line.lower()
'have a nice day'
>>> line.lower().startswith('h')
True
```

In the last example, the method `lower` is called and then we use `startswith` to see if the resulting lowercase string starts with the letter "h". As long as we are careful with the order, we can make multiple method calls in a single expression.

Exercise 4:

There is a string method called `count` that is similar to the function in the previous exercise. Read the documentation of this method at <https://docs.python.org/3.5/library/string-methods> and write an invocation that counts the number of times the letter `a` occurs in "banana".

This page titled [6.9: String Methods](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- 6.9: String Methods by Chuck Severance is licensed CC BY-NC-SA 4.0. Original source: <https://www.py4e.com/html3>.

6.10: Parsing strings

Often, we want to look into a string and find a substring. For example if we were presented a series of lines formatted as follows:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

and we wanted to pull out only the second half of the address (i.e., `uct.ac.za`) from each line, we can do this by using the `find` method and string slicing.

First, we will find the position of the at-sign in the string. Then we will find the position of the first space *after* the at-sign. And then we will use string slicing to extract the portion of the string which we are looking for.

```
>>> data = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
>>> atpos = data.find('@')
>>> print(atpos)
21
>>> sppos = data.find(' ', atpos)
>>> print(sppos)
31
>>> host = data[atpos+1:sppos]
>>> print(host)
uct.ac.za
>>>
```

We use a version of the `find` method which allows us to specify a position in the string where we want `find` to start looking. When we slice, we extract the characters from "one beyond the at-sign through up to *but not including* the space character".

The documentation for the `find` method is available at

<https://docs.python.org/3.5/library/string-methods>.

This page titled [6.10: Parsing strings](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.10: Parsing strings](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.11: Format operator

The *format operator*, `%` allows us to construct strings, replacing parts of the strings with the data stored in variables. When applied to integers, `%` is the modulus operator. But when the first operand is a string, `%` is the format operator.

The first operand is the *format string*, which contains one or more *format sequences* that specify how the second operand is formatted. The result is a string.

For example, the format sequence `"%d"` means that the second operand should be formatted as an integer (`d` stands for "decimal");

```
>>> camels = 42
>>> '%d' % camels
'42'
```

The result is the string `"42"`, which is not to be confused with the integer value `42`.

A format sequence can appear anywhere in the string, so you can embed a value in a sentence:

```
>>> camels = 42
>>> 'I have spotted %d camels.' % camels
'I have spotted 42 camels.'
```

If there is more than one format sequence in the string, the second argument has to be a tuple¹. Each format sequence is matched with an element of the tuple, in order.

The following example uses `"%d"` to format an integer, `"%g"` to format a floating-point number (don't ask why), and `"%s"` to format a string:

```
>>> 'In %d years I have spotted %g %s.' % (3, 0.1, 'camels')
'In 3 years I have spotted 0.1 camels.'
```

The number of elements in the tuple must match the number of format sequences in the string. The types of the elements also must match the format sequences:

```
>>> '%d %d %d' % (1, 2)
TypeError: not enough arguments for format string
>>> '%d' % 'dollars'
TypeError: %d format: a number is required, not str
```

In the first example, there aren't enough elements; in the second, the element is the wrong type.

The format operator is powerful, but it can be difficult to use. You can read more about it at

<https://docs.python.org/3.5/library/...ing-formatting>.

This page titled [6.11: Format operator](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **6.11: Format operator** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.12: Debugging

A skill that you should cultivate as you program is always asking yourself, "What could go wrong here?" or alternatively, "What crazy thing might our user do to crash our (seemingly) perfect program?"

For example, look at the program which we used to demonstrate the `while` loop in the chapter on iteration:

Look what happens when the user enters an empty line of input:

```
> hello there
hello there
> # don't print this
> print this!
print this!
>
Traceback (most recent call last):
  File "copytildone.py", line 3, in <module>
    if line[0] == '#':
IndexError: string index out of range
```

The code works fine until it is presented an empty line. Then there is no zero-th character, so we get a traceback. There are two solutions to this to make line three "safe" even if the line is empty.

One possibility is to simply use the `startswith` method which returns `False` if the string is empty.

```
if line.startswith('#):
```

Another way is to safely write the `if` statement using the *guardian* pattern and make sure the second logical expression is evaluated only where there is at least one character in the string.:

```
if len(line) > 0 and line[0] == '#):
```

This page titled [6.12: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.12: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.E: Strings (Exercises)

Exercise 5: Take the following Python code that stores a string:

```
str = 'X-DSPAM-Confidence: 0.8475 '
```

Use `find` and string slicing to extract the portion of the string after the colon character and then use the `float` function to convert the extracted string into a floating point number.

Exercise 6:

Read the documentation of the string methods at

<https://docs.python.org/3.5/library/string-methods>

You might want to experiment with some of them to make sure you understand how they work. `strip` and `replace` are particularly useful.

The documentation uses a syntax that might be confusing. For example, in `find(sub[, start[, end]])`, the brackets indicate optional arguments. So `sub` is required, but `start` is optional, and if you include `start`, then `end` is optional.

1. A tuple is a sequence of comma-separated values inside a pair of parenthesis. We will cover tuples in Chapter 10↩

This page titled [6.E: Strings \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [6.E: Strings \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

6.G: Strings (Glossary)

counter

A variable used to count something, usually initialized to zero and then incremented.

empty string

A string with no characters and length 0, represented by two quotation marks.

format operator

An operator, `%`, that takes a format string and a tuple and generates a string that includes the elements of the tuple formatted as specified by the format string.

format sequence

A sequence of characters in a format string, like `%d`, that specifies how a value should be formatted.

format string

A string, used with the format operator, that contains format sequences.

flag

A boolean variable used to indicate whether a condition is true or false.

invocation

A statement that calls a method.

immutable

The property of a sequence whose items cannot be assigned.

index

An integer value used to select an item in a sequence, such as a character in a string.

item

One of the values in a sequence.

method

A function that is associated with an object and called using dot notation.

object

Something a variable can refer to. For now, you can use "object" and "value" interchangeably.

search

A pattern of traversal that stops when it finds what it is looking for.

sequence

An ordered set; that is, a set of values where each value is identified by an integer index.

slice

A part of a string specified by a range of indices.

traverse

To iterate through the items in a sequence, performing a similar operation on each.

This page titled [6.G: Strings \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

7: Files

In this chapter, we start to work with *Secondary Memory* (or files). Secondary memory is not erased when the power is turned off. Or in the case of a USB flash drive, the data we write from our programs can be removed from the system and transported to another system.

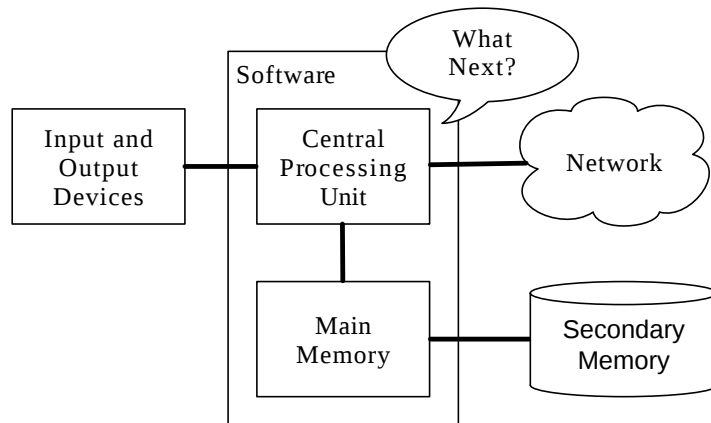
- [7.1: Persistence](#)
- [7.2: Opening Files](#)
- [7.3: Text files and Lines](#)
- [7.4: Reading Files](#)
- [7.5: Searching through a File](#)
- [7.6: Letting the user choose the file name](#)
- [7.7: Using try, except, and open](#)
- [7.8: Writing Files](#)
- [7.9: Debugging](#)
- [7.E: Files \(Exercises\)](#)
- [7.G: Files \(Glossary\)](#)

This page titled [7: Files](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

7.1: Persistence

So far, we have learned how to write programs and communicate our intentions to the *Central Processing Unit* using conditional execution, functions, and iterations. We have learned how to create and use data structures in the *Main Memory*. The CPU and memory are where our software works and runs. It is where all of the "thinking" happens.

But if you recall from our hardware architecture discussions, once the power is turned off, anything stored in either the CPU or main memory is erased. So up to now, our programs have just been transient fun exercises to learn Python.



Secondary Memory

In this chapter, we start to work with *Secondary Memory* (or files). Secondary memory is not erased when the power is turned off. Or in the case of a USB flash drive, the data we write from our programs can be removed from the system and transported to another system.

We will primarily focus on reading and writing text files such as those we create in a text editor. Later we will see how to work with database files which are binary files, specifically designed to be read and written through database software.

This page titled [7.1: Persistence](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

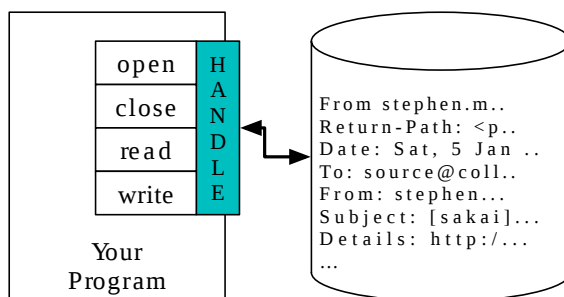
- [7.1: Persistence](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.2: Opening Files

When we want to read or write a file (say on your hard drive), we first must *open* the file. Opening the file communicates with your operating system, which knows where the data for each file is stored. When you open a file, you are asking the operating system to find the file by name and make sure the file exists. In this example, we open the file `mbox.txt`, which should be stored in the same folder that you are in when you start Python. You can download this file from www.py4e.com/code3/mbox.txt

```
>>> fhand = open('mbox.txt')
>>> print(fhand)
<_io.TextIOWrapper name='mbox.txt' mode='r' encoding='cp1252'>
```

If the `open` is successful, the operating system returns us a *file handle*. The file handle is not the actual data contained in the file, but instead it is a "handle" that we can use to read the data. You are given a handle if the requested file exists and you have the proper permissions to read the file.



A File Handle

If the file does not exist, `open` will fail with a traceback and you will not get a handle to access the contents of the file:

```
>>> fhand = open('stuff.txt')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'stuff.txt'
```

Later we will use `try` and `except` to deal more gracefully with the situation where we attempt to open a file that does not exist.

This page titled [7.2: Opening Files](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.2: Opening Files](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.3: Text files and Lines

A text file can be thought of as a sequence of lines, much like a Python string can be thought of as a sequence of characters. For example, this is a sample of a text file which records mail activity from various individuals in an open source project development team:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: source.sakaiproject.org/views...=rev&rev=39772
...
```

The entire file of mail interactions is available from

www.py4e.com/code3/mbox.txt

and a shortened version of the file is available from

www.py4e.com/code3/mbox-short.txt

These files are in a standard format for a file containing multiple mail messages. The lines which start with "From " separate the messages and the lines which start with "From:" are part of the messages. For more information about the mbox format, see en.Wikipedia.org/wiki/Mbox.

To break the file into lines, there is a special character that represents the "end of the line" called the *newline* character.

In Python, we represent the *newline* character as a backslash-n in string constants. Even though this looks like two characters, it is actually a single character. When we look at the variable by entering "stuff" in the interpreter, it shows us the `\n` in the string, but when we use `print` to show the string, we see the string broken into two lines by the newline character.

```
>>> stuff = 'Hello\nWorld!'
>>> stuff
'Hello\nWorld!'
>>> print(stuff)
Hello
World!
>>> stuff = 'X\nY'
>>> print(stuff)
X
Y
>>> len(stuff)
3
```

You can also see that the length of the string `X\nY` is *three* characters because the newline character is a single character.

So when we look at the lines in a file, we need to *imagine* that there is a special invisible character called the newline at the end of each line that marks the end of the line.

So the newline character separates the characters in the file into lines.

This page titled [7.3: Text files and Lines](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.3: Text files and Lines](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.4: Reading Files

While the *file handle* does not contain the data for the file, it is quite easy to construct a `for` loop to read through and count each of the lines in a file:

```
fhand = open('mbox-short.txt')
count = 0
for line in fhand:
    count = count + 1
print('Line Count:', count)

# Code: http://www.py4e.com/code3/open.py
```

We can use the file handle as the sequence in our `for` loop. Our `for` loop simply counts the number of lines in the file and prints them out. The rough translation of the `for` loop into English is, "for each line in the file represented by the file handle, add one to the `count` variable."

The reason that the `open` function does not read the entire file is that the file might be quite large with many gigabytes of data. The `open` statement takes the same amount of time regardless of the size of the file. The `for` loop actually causes the data to be read from the file.

When the file is read using a `for` loop in this manner, Python takes care of splitting the data in the file into separate lines using the newline character. Python reads each line through the newline and includes the newline as the last character in the `line` variable for each iteration of the `for` loop.

Because the `for` loop reads the data one line at a time, it can efficiently read and count the lines in very large files without running out of main memory to store the data. The above program can count the lines in any size file using very little memory since each line is read, counted, and then discarded.

If you know the file is relatively small compared to the size of your main memory, you can read the whole file into one string using the `read` method on the file handle.

```
>>> fhand = open('mbox-short.txt')
>>> inp = fhand.read()
>>> print(len(inp))
94626
>>> print(inp[:20])
From stephen.marquar
```

In this example, the entire contents (all 94,626 characters) of the file `mbox-short.txt` are read directly into the variable `inp`. We use string slicing to print out the first 20 characters of the string data stored in `inp`.

When the file is read in this manner, all the characters including all of the lines and newline characters are one big string in the variable `inp`. Remember that this form of the `open` function should only be used if the file data will fit comfortably in the main memory of your computer.

If the file is too large to fit in main memory, you should write your program to read the file in chunks using a `for` or `while` loop.

This page titled [7.4: Reading Files](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.4: Reading Files](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.5: Searching through a File

When you are searching through data in a file, it is a very common pattern to read through a file, ignoring most of the lines and only processing lines which meet a particular condition. We can combine the pattern for reading a file with string methods to build simple search mechanisms.

For example, if we wanted to read a file and only print out lines which started with the prefix "From:", we could use the string method *startswith* to select only those lines with the desired prefix:

```
fhand = open('mbox-short.txt')
count = 0
for line in fhand:
    if line.startswith('From:'):
        print(line)

# Code: http://www.py4e.com/code3/search1.py
```

When this program runs, we get the following output:

```
From: stephen.marquard@uct.ac.za

From: louis@media.berkeley.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu
...
```

The output looks great since the only lines we are seeing are those which start with "From:", but why are we seeing the extra blank lines? This is due to that invisible *newline* character. Each of the lines ends with a newline, so the `print` statement prints the string in the variable *line* which includes a newline and then `print` adds *another* newline, resulting in the double spacing effect we see.

We could use line slicing to print all but the last character, but a simpler approach is to use the *rstrip* method which strips whitespace from the right side of a string as follows:

```
fhand = open('mbox-short.txt')
for line in fhand:
    line = line.rstrip()
    if line.startswith('From:'):
        print(line)

# Code: http://www.py4e.com/code3/search2.py
```

When this program runs, we get the following output:

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
...
```

As your file processing programs get more complicated, you may want to structure your search loops using `continue`. The basic idea of the search loop is that you are looking for "interesting" lines and effectively skipping "uninteresting" lines. And then when we find an interesting line, we do something with that line.

We can structure the loop to follow the pattern of skipping uninteresting lines as follows:

```
fhand = open('mbox-short.txt')
for line in fhand:
    line = line.rstrip()
    # Skip 'uninteresting lines'
    if not line.startswith('From:'):
        continue
    # Process our 'interesting' line
    print(line)

# Code: http://www.py4e.com/code3/search3.py
```

The output of the program is the same. In English, the uninteresting lines are those which do not start with "From:", which we skip using `continue`. For the "interesting" lines (i.e., those that start with "From:") we perform the processing on those lines.

We can use the `find` string method to simulate a text editor search that finds lines where the search string is anywhere in the line. Since `find` looks for an occurrence of a string within another string and either returns the position of the string or -1 if the string was not found, we can write the following loop to show lines which contain the string "@uct.ac.za" (i.e., they come from the University of Cape Town in South Africa):

```
fhand = open('mbox-short.txt')
for line in fhand:
    line = line.rstrip()
    if line.find('@uct.ac.za') == -1: continue
    print(line)

# Code: http://www.py4e.com/code3/search4.py
```

Which produces the following output:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
X-Authentication-Warning: set sender to stephen.marquard@uct.ac.za using -f
From: stephen.marquard@uct.ac.za
Author: stephen.marquard@uct.ac.za
From david.horwitz@uct.ac.za Fri Jan  4 07:02:32 2008
X-Authentication-Warning: set sender to david.horwitz@uct.ac.za using -f
From: david.horwitz@uct.ac.za
Author: david.horwitz@uct.ac.za
...
```

This page titled [7.5: Searching through a File](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.5: Searching through a File](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.6: Letting the user choose the file name

We really do not want to have to edit our Python code every time we want to process a different file. It would be more usable to ask the user to enter the file name string each time the program runs so they can use our program on different files without changing the Python code.

This is quite simple to do by reading the file name from the user using `input` as follows:

```
fname = input('Enter the file name: ')
fhand = open(fname)
count = 0
for line in fhand:
    if line.startswith('Subject:'):
        count = count + 1
print('There were', count, 'subject lines in', fname)

# Code: http://www.py4e.com/code3/search6.py
```

We read the file name from the user and place it in a variable named `fname` and open that file. Now we can run the program repeatedly on different files.

```
python search6.py
Enter the file name: mbox.txt
There were 1797 subject lines in mbox.txt

python search6.py
Enter the file name: mbox-short.txt
There were 27 subject lines in mbox-short.txt
```

Before peeking at the next section, take a look at the above program and ask yourself, "What could go possibly wrong here?" or "What might our friendly user do that would cause our nice little program to ungracefully exit with a traceback, making us look not-so-cool in the eyes of our users?"

This page titled [7.6: Letting the user choose the file name](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **7.6: Letting the user choose the file name** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.7: Using try, except, and open

I told you not to peek. This is your last chance.

What if our user types something that is not a file name?

```
python search6.py
Enter the file name: missing.txt
Traceback (most recent call last):
  File "search6.py", line 2, in <module>
    fhand = open(fname)
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'

python search6.py
Enter the file name: na na boo boo
Traceback (most recent call last):
  File "search6.py", line 2, in <module>
    fhand = open(fname)
FileNotFoundError: [Errno 2] No such file or directory: 'na na boo boo'
```

Do not laugh. Users will eventually do every possible thing they can do to break your programs, either on purpose or with malicious intent. As a matter of fact, an important part of any software development team is a person or group called *Quality Assurance* (or QA for short) whose very job it is to do the craziest things possible in an attempt to break the software that the programmer has created.

The QA team is responsible for finding the flaws in programs before we have delivered the program to the end users who may be purchasing the software or paying our salary to write the software. So the QA team is the programmer's best friend.

So now that we see the flaw in the program, we can elegantly fix it using the `try / except` structure. We need to assume that the `open` call might fail and add recovery code when the `open` fails as follows:

```
fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()
count = 0
for line in fhand:
    if line.startswith('Subject:'):
        count = count + 1
print('There were', count, 'subject lines in', fname)

# Code: http://www.py4e.com/code3/search7.py
```

The `exit` function terminates the program. It is a function that we call that never returns. Now when our user (or QA team) types in silliness or bad file names, we "catch" them and recover gracefully:

```
python search7.py
Enter the file name: mbox.txt
There were 1797 subject lines in mbox.txt

python search7.py
Enter the file name: na na boo boo
File cannot be opened: na na boo boo
```

Protecting the `open` call is a good example of the proper use of `try` and `except` in a Python program. We use the term "Pythonic" when we are doing something the "Python way". We might say that the above example is the Pythonic way to open a file.

Once you become more skilled in Python, you can engage in repartee with other Python programmers to decide which of two equivalent solutions to a problem is "more Pythonic". The goal to be "more Pythonic" captures the notion that programming is part engineering and part art. We are not always interested in just making something work, we also want our solution to be elegant and to be appreciated as elegant by our peers.

This page titled [7.7: Using try, except, and open](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.7: Using try, except, and open](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.8: Writing Files

To write a file, you have to open it with mode "w" as a second parameter:

```
>>> fout = open('output.txt', 'w')
>>> print(fout)
<_io.TextIOWrapper name='output.txt' mode='w' encoding='cp1252'>
```

If the file already exists, opening it in write mode clears out the old data and starts fresh, so be careful! If the file doesn't exist, a new one is created.

The `write` method of the file handle object puts data into the file, returning the number of characters written. The default write mode is text for writing (and reading) strings.

```
>>> line1 = "This here's the wattle,\n"
>>> fout.write(line1)
24
```

Again, the file object keeps track of where it is, so if you call `write` again, it adds the new data to the end.

We must make sure to manage the ends of lines as we write to the file by explicitly inserting the newline character when we want to end a line. The `print` statement automatically appends a newline, but the `write` method does not add the newline automatically.

```
>>> line2 = 'the emblem of our land.\n'
>>> fout.write(line2)
24
```

When you are done writing, you have to close the file to make sure that the last bit of data is physically written to the disk so it will not be lost if the power goes off.

```
>>> fout.close()
```

We could close the files which we open for read as well, but we can be a little sloppy if we are only opening a few files since Python makes sure that all open files are closed when the program ends. When we are writing files, we want to explicitly close the files so as to leave nothing to chance.

This page titled [7.8: Writing Files](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **7.8: Writing Files** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.9: Debugging

When you are reading and writing files, you might run into problems with whitespace. These errors can be hard to debug because spaces, tabs, and newlines are normally invisible:

```
>>> s = '1 2\t 3\n 4'
>>> print(s)
1 2 3
 4
```

The built-in function `repr` can help. It takes any object as an argument and returns a string representation of the object. For strings, it represents whitespace characters with backslash sequences:

```
>>> print(repr(s))
'1 2\t 3\n 4'
```

This can be helpful for debugging.

One other problem you might run into is that different systems use different characters to indicate the end of a line. Some systems use a newline, represented `\n`. Others use a return character, represented `\r`. Some use both. If you move files between different systems, these inconsistencies might cause problems.

For most systems, there are applications to convert from one format to another. You can find them (and read more about this issue) at [Wikipedia.org/wiki/Newline](https://en.wikipedia.org/wiki/Newline). Or, of course, you could write one yourself.

This page titled [7.9: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.9: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.E: Files (Exercises)

Exercise 1: Write a program to read through a file and print the contents of the file (line by line) all in upper case. Executing the program will look as follows:

```
python shout.py
Enter a file name: mbox-short.txt
FROM STEPHEN.MARQUARD@UCT.AC.ZA SAT JAN  5 09:14:16 2008
RETURN-PATH: <POSTMASTER@COLLAB.SAKAIPROJECT.ORG>
RECEIVED: FROM MURDER (MAIL.UMICH.EDU [141.211.14.90])
          BY FRANKENSTEIN.MAIL.UMICH.EDU (CYRUS V2.3.8) WITH LMTPA;
          SAT, 05 JAN 2008 09:14:16 -0500
```

You can download the file from

www.py4e.com/code3/mbox-short.txt

Exercise 2: Write a program to prompt for a file name, and then read through the file and look for lines of the form:

```
X-DSPAM-Confidence: 0.8475
```

When you encounter a line that starts with "X-DSPAM-Confidence:" pull apart the line to extract the floating-point number on the line. Count these lines and then compute the total of the spam confidence values from these lines. When you reach the end of the file, print out the average spam confidence.

```
Enter the file name: mbox.txt
Average spam confidence: 0.894128046745

Enter the file name: mbox-short.txt
Average spam confidence: 0.750718518519
```

Test your file on the `mbox.txt` and `mbox-short.txt` files.

Exercise 3: Sometimes when programmers get bored or want to have a bit of fun, they add a harmless *Easter Egg* to their program. Modify the program that prompts the user for the file name so that it prints a funny message when the user types in the exact file name "na na boo boo". The program should behave normally for all other files which exist and don't exist. Here is a sample execution of the program:

```
python egg.py
Enter the file name: mbox.txt
There were 1797 subject lines in mbox.txt

python egg.py
Enter the file name: missing.tyxt
File cannot be opened: missing.tyxt

python egg.py
Enter the file name: na na boo boo
NA NA BOO BOO TO YOU - You have been punk'd!
```

We are not encouraging you to put Easter Eggs in your programs; this is just an exercise.

This page titled [7.E: Files \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [7.E: Files \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

7.G: Files (Glossary)

catch

To prevent an exception from terminating a program using the `try` and `except` statements.

newline

A special character used in files and strings to indicate the end of a line.

Pythonic

A technique that works elegantly in Python. "Using try and except is the *Pythonic* way to recover from missing files".

Quality Assurance

A person or team focused on insuring the overall quality of a software product. QA is often involved in testing a product and identifying problems before the product is released.

text file

A sequence of characters stored in permanent storage like a hard drive.

This page titled [7.G: Files \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

8: Lists

- [8.1: A list is a sequence](#)
- [8.2: Lists are mutable](#)
- [8.3: Traversing a List](#)
- [8.4: List operations](#)
- [8.5: List Slices](#)
- [8.6: List Methods](#)
- [8.7: Deleting Elements](#)
- [8.8: Lists and Functions](#)
- [8.9: Lists and Strings](#)
- [8.10: Parsing lines](#)
- [8.11: Objects and Values](#)
- [8.12: Aliasing](#)
- [8.13: List arguments](#)
- [8.14: Debugging](#)
- [8.E: Lists \(Exercises\)](#)
- [8.G: Lists \(Glossary\)](#)

This page titled [8: Lists](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

8.1: A list is a sequence

Like a string, a *list* is a sequence of values. In a string, the values are characters; in a list, they can be any type. The values in list are called *elements* or sometimes *items*.

There are several ways to create a new list; the simplest is to enclose the elements in square brackets (`[` and `]`):

```
[10, 20, 30, 40]
['crunchy frog', 'ram bladder', 'lark vomit']
```

The first example is a list of four integers. The second is a list of three strings. The elements of a list don't have to be the same type. The following list contains a string, a float, an integer, and (lo!) another list:

```
['spam', 2.0, 5, [10, 20]]
```

A list within another list is *nested*.

A list that contains no elements is called an empty list; you can create one with empty brackets, `[]` .

As you might expect, you can assign list values to variables:

```
>>> cheeses = ['Cheddar', 'Edam', 'Gouda']
>>> numbers = [17, 123]
>>> empty = []
>>> print(cheeses, numbers, empty)
['Cheddar', 'Edam', 'Gouda'] [17, 123] []
```

This page titled [8.1: A list is a sequence](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.1: A list is a sequence](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.2: Lists are mutable

The syntax for accessing the elements of a list is the same as for accessing the characters of a string: the bracket operator. The expression inside the brackets specifies the index. Remember that the indices start at 0:

```
>>> print(cheeses[0])
Cheddar
```

Unlike strings, lists are mutable because you can change the order of items in a list or reassign an item in a list. When the bracket operator appears on the left side of an assignment, it identifies the element of the list that will be assigned.

```
>>> numbers = [17, 123]
>>> numbers[1] = 5
>>> print(numbers)
[17, 5]
```

The one-eth element of `numbers`, which used to be 123, is now 5.

You can think of a list as a relationship between indices and elements. This relationship is called a *mapping*; each index "maps to" one of the elements.

List indices work the same way as string indices:

- Any integer expression can be used as an index.
- If you try to read or write an element that does not exist, you get an `IndexError`.
- If an index has a negative value, it counts backward from the end of the list.

The `in` operator also works on lists.

```
>>> cheeses = ['Cheddar', 'Edam', 'Gouda']
>>> 'Edam' in cheeses
True
>>> 'Brie' in cheeses
False
```

This page titled [8.2: Lists are mutable](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.2: Lists are mutable](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.3: Traversing a List

The most common way to traverse the elements of a list is with a `for` loop. The syntax is the same as for strings:

This works well if you only need to read the elements of the list. But if you want to write or update the elements, you need the indices. A common way to do that is to combine the functions `range` and `len` :

This loop traverses the list and updates each element. `len` returns the number of elements in the list. `range` returns a list of indices from 0 to $n - 1$, where n is the length of the list. Each time through the loop, `i` gets the index of the next element. The assignment statement in the body uses `i` to read the old value of the element and to assign the new value.

A `for` loop over an empty list never executes the body:

```
for x in empty:
    print('This never happens.')
```

Although a list can contain another list, the nested list still counts as a single element. The length of this list is four:

```
['spam', 1, ['Brie', 'Roquefort', 'Pol le Veq'], [1, 2, 3]]
```

This page titled [8.3: Traversing a List](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.3: Traversing a List](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.4: List operations

The `+` operator concatenates lists:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print(c)
[1, 2, 3, 4, 5, 6]
```

Similarly, the `*` operator repeats a list a given number of times:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats four times. The second example repeats the list three times.

This page titled [8.4: List operations](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.4: List operations](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.5: List Slices

The slice operator also works on lists:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3]
['b', 'c']
>>> t[:4]
['a', 'b', 'c', 'd']
>>> t[3:]
['d', 'e', 'f']
```

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. So if you omit both, the slice is a copy of the whole list.

```
>>> t[:]
['a', 'b', 'c', 'd', 'e', 'f']
```

Since lists are mutable, it is often useful to make a copy before performing operations that fold, spindle, or mutilate lists.

A slice operator on the left side of an assignment can update multiple elements:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> t[1:3] = ['x', 'y']
>>> print(t)
['a', 'x', 'y', 'd', 'e', 'f']
```

This page titled [8.5: List Slices](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **8.5: List Slices** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.6: List Methods

Python provides methods that operate on lists. For example, `append` adds a new element to the end of a list:

```
>>> t = ['a', 'b', 'c']
>>> t.append('d')
>>> print(t)
['a', 'b', 'c', 'd']
```

`extend` takes a list as an argument and appends all of the elements:

```
>>> t1 = ['a', 'b', 'c']
>>> t2 = ['d', 'e']
>>> t1.extend(t2)
>>> print(t1)
['a', 'b', 'c', 'd', 'e']
```

This example leaves `t2` unmodified.

`sort` arranges the elements of the list from low to high:

```
>>> t = ['d', 'c', 'e', 'b', 'a']
>>> t.sort()
>>> print(t)
['a', 'b', 'c', 'd', 'e']
```

Most list methods are void; they modify the list and return `None`. If you accidentally write `t = t.sort()`, you will be disappointed with the result.

This page titled [8.6: List Methods](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- **8.6: List Methods** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.7: Deleting Elements

There are several ways to delete elements from a list. If you know the index of the element you want, you can use `pop` :

```
>>> t = ['a', 'b', 'c']
>>> x = t.pop(1)
>>> print(t)
['a', 'c']
>>> print(x)
b
```

`pop` modifies the list and returns the element that was removed. If you don't provide an index, it deletes and returns the last element.

If you don't need the removed value, you can use the `del` operator:

```
>>> t = ['a', 'b', 'c']
>>> del t[1]
>>> print(t)
['a', 'c']
```

If you know the element you want to remove (but not the index), you can use `remove` :

```
>>> t = ['a', 'b', 'c']
>>> t.remove('b')
>>> print(t)
['a', 'c']
```

The return value from `remove` is `None` .

To remove more than one element, you can use `del` with a slice index:

```
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del t[1:5]
>>> print(t)
['a', 'f']
```

As usual, the slice selects all the elements up to, but not including, the second index.

This page titled [8.7: Deleting Elements](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.7: Deleting Elements](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.8: Lists and Functions

There are a number of built-in functions that can be used on lists that allow you to quickly look through a list without writing your own loops:

```
>>> nums = [3, 41, 12, 9, 74, 15]
>>> print(len(nums))
6
>>> print(max(nums))
74
>>> print(min(nums))
3
>>> print(sum(nums))
154
>>> print(sum(nums)/len(nums))
25
```

The `sum()` function only works when the list elements are numbers. The other functions (`max()`, `len()`, etc.) work with lists of strings and other types that can be comparable.

We could rewrite an earlier program that computed the average of a list of numbers entered by the user using a list.

First, the program to compute an average without a list:

In this program, we have `count` and `total` variables to keep the number and running total of the user's numbers as we repeatedly prompt the user for a number.

We could simply remember each number as the user entered it and use built-in functions to compute the sum and count at the end.

We make an empty list before the loop starts, and then each time we have a number, we append it to the list. At the end of the program, we simply compute the sum of the numbers in the list and divide it by the count of the numbers in the list to come up with the average.

This page titled [8.8: Lists and Functions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.8: Lists and Functions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.9: Lists and Strings

A string is a sequence of characters and a list is a sequence of values, but a list of characters is not the same as a string. To convert from a string to a list of characters, you can use `list` :

```
>>> s = 'spam'
>>> t = list(s)
>>> print(t)
['s', 'p', 'a', 'm']
```

Because `list` is the name of a built-in function, you should avoid using it as a variable name. I also avoid the letter `l` because it looks too much like the number `1` . So that's why I use `t` .

The `list` function breaks a string into individual letters. If you want to break a string into words, you can use the `split` method:

```
>>> s = 'pining for the fjords'
>>> t = s.split()
>>> print(t)
['pining', 'for', 'the', 'fjords']
>>> print(t[2])
the
```

Once you have used `split` to break the string into a list of words, you can use the index operator (square bracket) to look at a particular word in the list.

You can call `split` with an optional argument called a *delimiter* that specifies which characters to use as word boundaries. The following example uses a hyphen as a delimiter:

```
>>> s = 'spam-spam-spam'
>>> delimiter = '-'
>>> s.split(delimiter)
['spam', 'spam', 'spam']
```

`join` is the inverse of `split` . It takes a list of strings and concatenates the elements. `join` is a string method, so you have to invoke it on the delimiter and pass the list as a parameter:

```
>>> t = ['pining', 'for', 'the', 'fjords']
>>> delimiter = ' '
>>> delimiter.join(t)
'pining for the fjords'
```

In this case the delimiter is a space character, so `join` puts a space between words. To concatenate strings without spaces, you can use the empty string, `''`, as a delimiter.

This page titled [8.9: Lists and Strings](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **8.9: Lists and Strings** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.10: Parsing lines

Usually when we are reading a file we want to do something to the lines other than just printing the whole line. Often we want to find the "interesting lines" and then *parse* the line to find some interesting *part* of the line. What if we wanted to print out the day of the week from those lines that start with "From "?

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

The `split` method is very effective when faced with this kind of problem. We can write a small program that looks for lines where the line starts with "From ", `split` those lines, and then print out the third word in the line:

```
fhand = open('mbox-short.txt')
for line in fhand:
    line = line.rstrip()
    if not line.startswith('From '): continue
    words = line.split()
    print(words[2])

# Code: http://www.py4e.com/code3/search5.py
```

Here we also use the contracted form of the `if` statement where we put the `continue` on the same line as the `if`. This contracted form of the `if` functions the same as if the `continue` were on the next line and indented.

The program produces the following output:

```
Sat
Fri
Fri
Fri
...
```

Later, we will learn increasingly sophisticated techniques for picking the lines to work on and how we pull those lines apart to find the exact bit of information we are looking for.

This page titled [8.10: Parsing lines](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.10: Parsing lines](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.11: Objects and Values

If we execute these assignment statements:

```
a = 'banana'
b = 'banana'
```

we know that `a` and `b` both refer to a string, but we don't know whether they refer to the *same* string. There are two possible states:

`a` → 'banana' `a` → 'banana'
`b` → 'banana' `b` → 'banana'

Variables and Objects

In one case, `a` and `b` refer to two different objects that have the same value. In the second case, they refer to the same object. To check whether two variables refer to the same object, you can use the `is` operator.

```
>>> a = 'banana'
>>> b = 'banana'
>>> a is b
True
```

In this example, Python only created one string object, and both `a` and `b` refer to it.

But when you create two lists, you get two objects:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
```

In this case we would say that the two lists are *equivalent*, because they have the same elements, but not *identical*, because they are not the same object. If two objects are identical, they are also equivalent, but if they are equivalent, they are not necessarily identical.

Until now, we have been using "object" and "value" interchangeably, but it is more precise to say that an object has a value. If you execute `a = [1, 2, 3]`, `a` refers to a list object whose value is a particular sequence of elements. If another list has the same elements, we would say it has the same value.

This page titled [8.11: Objects and Values](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **8.11: Objects and Values** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.12: Aliasing

If `a` refers to an object and you assign `b = a`, then both variables refer to the same object:

```
>>> a = [1, 2, 3]
>>> b = a
>>> b is a
True
```

The association of a variable with an object is called a *reference*. In this example, there are two references to the same object.

An object with more than one reference has more than one name, so we say that the object is *aliased*.

If the aliased object is mutable, changes made with one alias affect the other:

```
>>> b[0] = 17
>>> print(a)
[17, 2, 3]
```

Although this behavior can be useful, it is error-prone. In general, it is safer to avoid aliasing when you are working with mutable objects.

For immutable objects like strings, aliasing is not as much of a problem. In this example:

```
a = 'banana'
b = 'banana'
```

it almost never makes a difference whether `a` and `b` refer to the same string or not.

This page titled [8.12: Aliasing](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.12: Aliasing](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.13: List arguments

When you pass a list to a function, the function gets a reference to the list. If the function modifies a list parameter, the caller sees the change. For example, `delete_head` removes the first element from a list:

```
def delete_head(t):  
    del t[0]
```

Here's how it is used:

```
>>> letters = ['a', 'b', 'c']  
>>> delete_head(letters)  
>>> print(letters)  
['b', 'c']
```

The parameter `t` and the variable `letters` are aliases for the same object.

It is important to distinguish between operations that modify lists and operations that create new lists. For example, the `append` method modifies a list, but the `+` operator creates a new list:

```
>>> t1 = [1, 2]  
>>> t2 = t1.append(3)  
>>> print(t1)  
[1, 2, 3]  
>>> print(t2)  
None  
  
>>> t3 = t1 + [3]  
>>> print(t3)  
[1, 2, 3]  
>>> t2 is t3  
False
```

This difference is important when you write functions that are supposed to modify lists. For example, this function *does not* delete the head of a list:

```
def bad_delete_head(t):  
    t = t[1:]          # WRONG!
```

The slice operator creates a new list and the assignment makes `t` refer to it, but none of that has any effect on the list that was passed as an argument.

An alternative is to write a function that creates and returns a new list. For example, `tail` returns all but the first element of a list:

```
def tail(t):  
    return t[1:]
```

This function leaves the original list unmodified. Here's how it is used:

```
>>> letters = ['a', 'b', 'c']
>>> rest = tail(letters)
>>> print(rest)
['b', 'c']
```

Exercise 1:

Write a function called `chop` that takes a list and modifies it, removing the first and last elements, and returns `None` .

Then write a function called `middle` that takes a list and returns a new list that contains all but the first and last elements.

This page titled [8.13: List arguments](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.13: List arguments](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.14: Debugging

Careless use of lists (and other mutable objects) can lead to long hours of debugging. Here are some common pitfalls and ways to avoid them:

1. Don't forget that most list methods modify the argument and return `None`. This is the opposite of the string methods, which return a new string and leave the original alone.

If you are used to writing string code like this:

```
word = word.strip()
```

It is tempting to write list code like this: `~ { .python } t = t.sort() # WRONG! ~`

Because `sort` returns `None`, the next operation you perform with `t` is likely to fail.

Before using list methods and operators, you should read the documentation carefully and then test them in interactive mode. The methods and operators that lists share with other sequences (like strings) are documented at <https://docs.python.org/3.5/library/...nce-operations>. The methods and operators that only apply to mutable sequences are documented at <https://docs.python.org/3.5/library/...sequence-types>.

2. Pick an idiom and stick with it.

Part of the problem with lists is that there are too many ways to do things. For example, to remove an element from a list, you can use `pop`, `remove`, `del`, or even a slice assignment.

To add an element, you can use the `append` method or the `+` operator. But don't forget that these are right:

```
t.append(x)
t = t + [x]
```

And these are wrong:

```
t.append([x])      # WRONG!
t = t.append(x)     # WRONG!
t + [x]            # WRONG!
t = t + x           # WRONG!
```

Try out each of these examples in interactive mode to make sure you understand what they do. Notice that only the last one causes a runtime error; the other three are legal, but they do the wrong thing.

3. Make copies to avoid aliasing.

If you want to use a method like `sort` that modifies the argument, but you need to keep the original list as well, you can make a copy.

```
orig = t[:]
t.sort()
```

In this example you could also use the built-in function `sorted`, which returns a new, sorted list and leaves the original alone. But in that case you should avoid using `sorted` as a variable name!

4. Lists, `split`, and files

When we read and parse files, there are many opportunities to encounter input that can crash our program so it is a good idea to revisit the *guardian* pattern when it comes writing programs that read through a file and look for a "needle in the haystack".

Let's revisit our program that is looking for the day of the week on the from lines of our file:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

Since we are breaking this line into words, we could dispense with the use of `startswith` and simply look at the first word of the line to determine if we are interested in the line at all. We can use `continue` to skip lines that don't have "From" as the first word as follows:

```
fhand = open('mbox-short.txt')
for line in fhand:
    words = line.split()
    if words[0] != 'From' : continue
    print(words[2])
```

This looks much simpler and we don't even need to do the `rstrip` to remove the newline at the end of the file. But is it better?

```
python search8.py
Sat
Traceback (most recent call last):
  File "search8.py", line 5, in <module>
    if words[0] != 'From' : continue
IndexError: list index out of range
```

It kind of works and we see the day from the first line (Sat), but then the program fails with a traceback error. What went wrong? What messed-up data caused our elegant, clever, and very Pythonic program to fail?

You could stare at it for a long time and puzzle through it or ask someone for help, but the quicker and smarter approach is to add a `print` statement. The best place to add the print statement is right before the line where the program failed and print out the data that seems to be causing the failure.

Now this approach may generate a lot of lines of output, but at least you will immediately have some clue as to the problem at hand. So we add a print of the variable `words` right before line five. We even add a prefix "Debug:" to the line so we can keep our regular output separate from our debug output.

```
for line in fhand:
    words = line.split()
    print('Debug:', words)
    if words[0] != 'From' : continue
    print(words[2])
```

When we run the program, a lot of output scrolls off the screen but at the end, we see our debug output and the traceback so we know what happened just before the traceback.

```
Debug: ['X-DSPAM-Confidence:', '0.8475']
Debug: ['X-DSPAM-Probability:', '0.0000']
Debug: []
Traceback (most recent call last):
  File "search9.py", line 6, in <module>
    if words[0] != 'From' : continue
IndexError: list index out of range
```

Each debug line is printing the list of words which we get when we `split` the line into words. When the program fails, the list of words is empty `[]`. If we open the file in a text editor and look at the file, at that point it looks as follows:

```
X-DSPAM-Result: Innocent
X-DSPAM-Processed: Sat Jan 5 09:14:16 2008
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
```

Details: <http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772>

The error occurs when our program encounters a blank line! Of course there are "zero words" on a blank line. Why didn't we think of that when we were writing the code? When the code looks for the first word (`word[0]`) to check to see if it matches "From", we get an "index out of range" error.

This of course is the perfect place to add some *guardian* code to avoid checking the first word if the first word is not there. There are many ways to protect this code; we will choose to check the number of words we have before we look at the first word:

```
fhand = open('mbox-short.txt')
count = 0
for line in fhand:
    words = line.split()
    # print 'Debug:', words
    if len(words) == 0 : continue
    if words[0] != 'From' : continue
    print(words[2])
```

First we commented out the debug print statement instead of removing it, in case our modification fails and we need to debug again. Then we added a guardian statement that checks to see if we have zero words, and if so, we use `continue` to skip to the next line in the file.

We can think of the two `continue` statements as helping us refine the set of lines which are "interesting" to us and which we want to process some more. A line which has no words is "uninteresting" to us so we skip to the next line. A line which does not have "From" as its first word is uninteresting to us so we skip it.

The program as modified runs successfully, so perhaps it is correct. Our guardian statement does make sure that the `words[0]` will never fail, but perhaps it is not enough. When we are programming, we must always be thinking, "What might go wrong?"

Exercise 2: Figure out which line of the above program is still not properly guarded. See if you can construct a text file which causes the program to fail and then modify the program so that the line is properly guarded and test it to make sure it handles your new text file.

Exercise 3: Rewrite the guardian code in the above example without two `if` statements. Instead, use a compound logical expression using the `and` logical operator with a single `if` statement.

This page titled [8.14: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.14: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.E: Lists (Exercises)

Exercise 4: Download a copy of the file from www.py4e.com/code3/romeo.txt

Write a program to open the file `romeo.txt` and read it line by line. For each line, split the line into a list of words using the `split` function.

For each word, check to see if the word is already in a list. If the word is not in the list, add it to the list.

When the program completes, sort and print the resulting words in alphabetical order.

```
Enter file: romeo.txt
['Arise', 'But', 'It', 'Juliet', 'Who', 'already',
'and', 'breaks', 'east', 'envious', 'fair', 'grief',
'is', 'kill', 'light', 'moon', 'pale', 'sick', 'soft',
'sun', 'the', 'through', 'what', 'window',
'with', 'yonder']
```

Exercise 5: Write a program to read through the mail box data and when you find line that starts with "From", you will split the line into words using the `split` function. We are interested in who sent the message, which is the second word on the From line.

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

You will parse the From line and print out the second word for each From line, then you will also count the number of From (not From:) lines and print out a count at the end.

This is a good sample output with a few lines removed:

```
python fromcount.py
Enter a file name: mbox-short.txt
stephen.marquard@uct.ac.za
louis@media.berkeley.edu
zqian@umich.edu

[...some output removed...]

ray@media.berkeley.edu
cwen@iupui.edu
cwen@iupui.edu
cwen@iupui.edu
There were 27 lines in the file with From as the first word
```

Exercise 6: Rewrite the program that prompts the user for a list of numbers and prints out the maximum and minimum of the numbers at the end when the user enters "done". Write the program to store the numbers the user enters in a list and use the `max()` and `min()` functions to compute the maximum and minimum numbers after the loop completes.

```
Enter a number: 6
Enter a number: 2
Enter a number: 9
Enter a number: 3
Enter a number: 5
Enter a number: done
Maximum: 9.0
Minimum: 2.0
```

This page titled [8.E: Lists \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [8.E: Lists \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

8.G: Lists (Glossary)

aliasing

A circumstance where two or more variables refer to the same object.

delimiter

A character or string used to indicate where a string should be split.

element

One of the values in a list (or other sequence); also called items.

equivalent

Having the same value.

index

An integer value that indicates an element in a list.

identical

Being the same object (which implies equivalence).

list

A sequence of values.

list traversal

The sequential accessing of each element in a list.

nested list

A list that is an element of another list.

object

Something a variable can refer to. An object has a type and a value.

reference

The association between a variable and its value.

This page titled [8.G: Lists \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

9: Dictionaries

[9.1: Dictionaries](#)

[9.2: Dictionary as a Set of Counters](#)

[9.3: Dictionaries and Files](#)

[9.4: Looping and Dictionaries](#)

[9.5: Advanced Text Parsing](#)

[9.6: Debugging](#)

[9.E: Dictionaries \(Exercises\)](#)

[9.G: Dictionaries \(Glossary\)](#)

This page titled [9: Dictionaries](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

9.2: Dictionary as a Set of Counters

Suppose you are given a string and you want to count how many times each letter appears. There are several ways you could do it:

1. You could create 26 variables, one for each letter of the alphabet. Then you could traverse the string and, for each character, increment the corresponding counter, probably using a chained conditional.
2. You could create a list with 26 elements. Then you could convert each character to a number (using the built-in function `ord`), use the number as an index into the list, and increment the appropriate counter.
3. You could create a dictionary with characters as keys and counters as the corresponding values. The first time you see a character, you would add an item to the dictionary. After that you would increment the value of an existing item.

Each of these options performs the same computation, but each of them implements that computation in a different way.

An *implementation* is a way of performing a computation; some implementations are better than others. For example, an advantage of the dictionary implementation is that we don't have to know ahead of time which letters appear in the string and we only have to make room for the letters that do appear.

Here is what the code might look like:

We are effectively computing a *histogram*, which is a statistical term for a set of counters (or frequencies).

The `for` loop traverses the string. Each time through the loop, if the character `c` is not in the dictionary, we create a new item with key `c` and the initial value 1 (since we have seen this letter once). If `c` is already in the dictionary we increment `d[c]`.

Here's the output of the program:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

The histogram indicates that the letters `'a'` and `"b"` appear once; `"o"` appears twice, and so on.

Dictionaries have a method called `get` that takes a key and a default value. If the key appears in the dictionary, `get` returns the corresponding value; otherwise it returns the default value. For example:

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

We can use `get` to write our histogram loop more concisely. Because the `get` method automatically handles the case where a key is not in a dictionary, we can reduce four lines down to one and eliminate the `if` statement.

The use of the `get` method to simplify this counting loop ends up being a very commonly used "idiom" in Python and we will use it many times in the rest of the book. So you should take a moment and compare the loop using the `if` statement and `in` operator with the loop using the `get` method. They do exactly the same thing, but one is more succinct.

This page titled [9.2: Dictionary as a Set of Counters](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **9.2: Dictionary as a Set of Counters** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

9.3: Dictionaries and Files

One of the common uses of a dictionary is to count the occurrence of words in a file with some written text. Let's start with a very simple file of words taken from the text of *Romeo and Juliet*.

For the first set of examples, we will use a shortened and simplified version of the text with no punctuation. Later we will work with the text of the scene with punctuation included.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

We will write a Python program to read through the lines of the file, break each line into a list of words, and then loop through each of the words in the line and count each word using a dictionary.

You will see that we have two `for` loops. The outer loop is reading the lines of the file and the inner loop is iterating through each of the words on that particular line. This is an example of a pattern called *nested loops* because one of the loops is the *outer* loop and the other loop is the *inner* loop.

Because the inner loop executes all of its iterations each time the outer loop makes a single iteration, we think of the inner loop as iterating "more quickly" and the outer loop as iterating more slowly.

The combination of the two nested loops ensures that we will count every word on every line of the input file.

```
fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)

# Code: http://www.py4e.com/code3/count1.py
```

When we run the program, we see a raw dump of all of the counts in unsorted hash order. (the `romeo.txt` file is available at www.py4e.com/code3/romeo.txt)

```
python count1.py
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1,
'is': 3, 'through': 1, 'pale': 1, 'yonder': 1,
'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1,
'window': 1, 'sick': 1, 'east': 1, 'breaks': 1,
'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1,
'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}
```

It is a bit inconvenient to look through the dictionary to find the most common words and their counts, so we need to add some more Python code to get us the output that will be more helpful.

This page titled [9.3: Dictionaries and Files](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [9.3: Dictionaries and Files](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

9.5: Advanced Text Parsing

In the above example using the file `romeo.txt`, we made the file as simple as possible by removing all punctuation by hand. The actual text has lots of punctuation, as shown below.

```
But, soft! what light through yonder window breaks?  
It is the east, and Juliet is the sun.  
Arise, fair sun, and kill the envious moon,  
Who is already sick and pale with grief,
```

Since the Python `split` function looks for spaces and treats words as tokens separated by spaces, we would treat the words "soft!" and "soft" as *different* words and create a separate dictionary entry for each word.

Also since the file has capitalization, we would treat "who" and "Who" as different words with different counts.

We can solve both these problems by using the string methods `lower`, `punctuation`, and `translate`. The `translate` is the most subtle of the methods. Here is the documentation for `translate`:

```
line.translate(str.maketrans(fromstr, tostr, deletestr))
```

Replace the characters in `fromstr` with the character in the same position in `tostr` and delete all characters that are in `deletestr`. The `fromstr` and `tostr` can be empty strings and the `deletestr` parameter can be omitted.

We will not specify the `table` but we will use the `deletechars` parameter to delete all of the punctuation. We will even let Python tell us the list of characters that it considers "punctuation":

```
>>> import string  
>>> string.punctuation  
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

The parameters used by `translate` were different in Python 2.0.

We make the following modifications to our program:

```
import string

fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    line = line.rstrip()
    line = line.translate(line.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)

# Code: http://www.py4e.com/code3/count2.py
```

Part of learning the "Art of Python" or "Thinking Pythonically" is realizing that Python often has built-in capabilities for many common data analysis problems. Over time, you will see enough example code and read enough of the documentation to know where to look to see if someone has already written something that makes your job much easier.

The following is an abbreviated version of the output:

```
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

Looking through this output is still unwieldy and we can use Python to give us exactly what we are looking for, but to do so, we need to learn about Python *tuples*. We will pick up this example once we learn about tuples.

This page titled [9.5: Advanced Text Parsing](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [9.5: Advanced Text Parsing](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

9.6: Debugging

As you work with bigger datasets it can become unwieldy to debug by printing and checking data by hand. Here are some suggestions for debugging large datasets:

Scale down the input

If possible, reduce the size of the dataset. For example if the program reads a text file, start with just the first 10 lines, or with the smallest example you can find. You can either edit the files themselves, or (better) modify the program so it reads only the first n lines.

If there is an error, you can reduce n to the smallest value that manifests the error, and then increase it gradually as you find and correct errors.

Check summaries and types

Instead of printing and checking the entire dataset, consider printing summaries of the data: for example, the number of items in a dictionary or the total of a list of numbers.

A common cause of runtime errors is a value that is not the right type. For debugging this kind of error, it is often enough to print the type of a value.

Write self-checks

Sometimes you can write code to check for errors automatically. For example, if you are computing the average of a list of numbers, you could check that the result is not greater than the largest element in the list or less than the smallest. This is called a "sanity check" because it detects results that are "completely illogical".

Another kind of check compares the results of two different computations to see if they are consistent. This is called a "consistency check".

Pretty print the output

Formatting debugging output can make it easier to spot an error.

Again, time you spend building scaffolding can reduce the time you spend debugging.

This page titled [9.6: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [9.6: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

9.E: Dictionaries (Exercises)

Exercise 2: Write a program that categorizes each mail message by which day of the week the commit was done. To do this look for lines that start with "From", then look for the third word and keep a running count of each of the days of the week. At the end of the program print out the contents of your dictionary (order does not matter).

Sample Line:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

Sample Execution:

```
python dow.py
Enter a file name: mbox-short.txt
{'Fri': 20, 'Thu': 6, 'Sat': 1}
```

Exercise 3: Write a program to read through a mail log, build a histogram using a dictionary to count how many messages have come from each email address, and print the dictionary.

```
Enter file name: mbox-short.txt
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3,
'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1,
'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3,
'david.horowitz@uct.ac.za': 4, 'wagnermr@iupui.edu': 1,
'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2,
'ray@media.berkeley.edu': 1}
```

Exercise 4: Add code to the above program to figure out who has the most messages in the file.

After all the data has been read and the dictionary has been created, look through the dictionary using a maximum loop (see Section [maximumloop]) to find who has the most messages and print how many messages the person has.

```
Enter a file name: mbox-short.txt
cwen@iupui.edu 5

Enter a file name: mbox.txt
zqian@umich.edu 195
```

Exercise 5: This program records the domain name (instead of the address) where the message was sent from instead of who the mail came from (i.e., the whole email address). At the end of the program, print out the contents of your dictionary.

```
python schoolcount.py
Enter a file name: mbox-short.txt
{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7,
'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}
```

This page titled [9.E: Dictionaries \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [9.E: Dictionaries \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

9.G: Dictionaries (Glossary)

dictionary

A mapping from a set of keys to their corresponding values.

hashtable

The algorithm used to implement Python dictionaries.

hash function

A function used by a hashtable to compute the location for a key.

histogram

A set of counters.

implementation

A way of performing a computation.

item

Another name for a key-value pair.

key

An object that appears in a dictionary as the first part of a key-value pair.

key-value pair

The representation of the mapping from a key to a value.

lookup

A dictionary operation that takes a key and finds the corresponding value.

nested loops

When there are one or more loops "inside" of another loop. The inner loop runs to completion each time the outer loop runs once.

value

An object that appears in a dictionary as the second part of a key-value pair. This is more specific than our previous use of the word "value".

This page titled [9.G: Dictionaries \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

10: Tuples

- 10.1: Tuples are Immutable
- 10.2: Comparing Tuples
- 10.3: Tuple Assignment
- 10.4: Dictionaries and Tuples
- 10.5: Multiple assignment with dictionaries
- 10.6: The most common words
- 10.7: Using Tuples as Keys in Dictionaries
- 10.8: Sequences- strings, lists, and tuples - Oh My!
- 10.9: Debugging
- 10.E: Tuples (Exercises)
- 10.G: Tuples (Glossary)

This page titled [10: Tuples](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

10.1: Tuples are Immutable

A tuple¹ is a sequence of values much like a list. The values stored in a tuple can be any type, and they are indexed by integers. The important difference is that tuples are *immutable*. Tuples are also *comparable* and *hashable* so we can sort lists of them and use tuples as key values in Python dictionaries.

Syntactically, a tuple is a comma-separated list of values:

```
>>> t = 'a', 'b', 'c', 'd', 'e'
```

Although it is not necessary, it is common to enclose tuples in parentheses to help us quickly identify tuples when we look at Python code:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

To create a tuple with a single element, you have to include the final comma:

```
>>> t1 = ('a',)
>>> type(t1)
<type 'tuple'>
```

Without the comma Python treats `('a')` as an expression with a string in parentheses that evaluates to a string:

```
>>> t2 = ('a')
>>> type(t2)
<type 'str'>
```

Another way to construct a tuple is the built-in function `tuple`. With no argument, it creates an empty tuple:

```
>>> t = tuple()
>>> print(t)
()
```

If the argument is a sequence (string, list, or tuple), the result of the call to `tuple` is a tuple with the elements of the sequence:

```
>>> t = tuple('lupins')
>>> print(t)
('l', 'u', 'p', 'i', 'n', 's')
```

Because `tuple` is the name of a constructor, you should avoid using it as a variable name.

Most list operators also work on tuples. The bracket operator indexes an element:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[0])
'a'
```

And the slice operator selects a range of elements.

```
>>> print(t[1:3])
('b', 'c')
```

But if you try to modify one of the elements of the tuple, you get an error:

```
>>> t[0] = 'A'  
TypeError: object doesn't support item assignment
```

You can't modify the elements of a tuple, but you can replace one tuple with another:

```
>>> t = ('A',) + t[1:]  
>>> print(t)  
( 'A', 'b', 'c', 'd', 'e')
```

This page titled [10.1: Tuples are Immutable](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.1: Tuples are Immutable](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.2: Comparing Tuples

The comparison operators work with tuples and other sequences. Python starts by comparing the first element from each sequence. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are really big).

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
```

The `sort` function works the same way. It sorts primarily by first element, but in the case of a tie, it sorts by second element, and so on.

This feature lends itself to a pattern called *DSU* for

Decorate

a sequence by building a list of tuples with one or more sort keys preceding the elements from the sequence,

Sort

the list of tuples using the Python built-in `sort`, and

Undecorate

by extracting the sorted elements of the sequence.

[DSU]

For example, suppose you have a list of words and you want to sort them from longest to shortest:

The first loop builds a list of tuples, where each tuple is a word preceded by its length.

`sort` compares the first element, length, first, and only considers the second element to break ties. The keyword argument `reverse=True` tells `sort` to go in decreasing order.

The second loop traverses the list of tuples and builds a list of words in descending order of length. The four-character words are sorted in *reverse* alphabetical order, so "what" appears before "soft" in the following list.

The output of the program is as follows:

```
['yonder', 'window', 'breaks', 'light', 'what',
 'soft', 'but', 'in']
```

Of course the line loses much of its poetic impact when turned into a Python list and sorted in descending word length order.

This page titled [10.2: Comparing Tuples](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [via source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.2: Comparing Tuples](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.3: Tuple Assignment

One of the unique syntactic features of the Python language is the ability to have a tuple on the left side of an assignment statement. This allows you to assign more than one variable at a time when the left side is a sequence.

In this example we have a two-element list (which is a sequence) and assign the first and second elements of the sequence to the variables `x` and `y` in a single statement.

```
>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x
'have'
>>> y
'fun'
>>>
```

It is not magic, Python *roughly* translates the tuple assignment syntax to be the following:²

```
>>> m = [ 'have', 'fun' ]
>>> x = m[0]
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
>>>
```

Stylistically when we use a tuple on the left side of the assignment statement, we omit the parentheses, but the following is an equally valid syntax:

```
>>> m = [ 'have', 'fun' ]
>>> (x, y) = m
>>> x
'have'
>>> y
'fun'
>>>
```

A particularly clever application of tuple assignment allows us to *swap* the values of two variables in a single statement:

```
>>> a, b = b, a
```

Both sides of this statement are tuples, but the left side is a tuple of variables; the right side is a tuple of expressions. Each value on the right side is assigned to its respective variable on the left side. All the expressions on the right side are evaluated before any of the assignments.

The number of variables on the left and the number of values on the right must be the same:

```
>>> a, b = 1, 2, 3
ValueError: too many values to unpack
```

More generally, the right side can be any kind of sequence (string, list, or tuple). For example, to split an email address into a user name and a domain, you could write:

```
>>> addr = 'monty@python.org'
>>> uname, domain = addr.split('@')
```

The return value from `split` is a list with two elements; the first element is assigned to `uname` , the second to `domain` .

```
>>> print(uname)
monty
>>> print(domain)
python.org
```

This page titled [10.3: Tuple Assignment](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.3: Tuple Assignment](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.4: Dictionaries and Tuples

Dictionaries have a method called `items` that returns a list of tuples, where each tuple is a key-value pair:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]
```

As you should expect from a dictionary, the items are in no particular order.

However, since the list of tuples is a list, and tuples are comparable, we can now sort the list of tuples. Converting a dictionary to a list of tuples is a way for us to output the contents of a dictionary sorted by key:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

The new list is sorted in ascending alphabetical order by the key value.

This page titled [10.4: Dictionaries and Tuples](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.4: Dictionaries and Tuples](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.5: Multiple assignment with dictionaries

Combining `items`, tuple assignment, and `for`, you can see a nice code pattern for traversing the keys and values of a dictionary in a single loop:

This loop has two *iteration variables* because `items` returns a list of tuples and `key, val` is a tuple assignment that successively iterates through each of the key-value pairs in the dictionary.

For each iteration through the loop, both `key` and `value` are advanced to the next key-value pair in the dictionary (still in hash order).

The output of this loop is:

```
10 a
22 c
1 b
```

Again, it is in hash key order (i.e., no particular order).

If we combine these two techniques, we can print out the contents of a dictionary sorted by the *value* stored in each key-value pair.

To do this, we first make a list of tuples where each tuple is `(value, key)`. The `items` method would give us a list of `(key, value)` tuples, but this time we want to sort by value, not key. Once we have constructed the list with the value-key tuples, it is a simple matter to sort the list in reverse order and print out the new, sorted list.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) )
...
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
>>>
```

By carefully constructing the list of tuples to have the value as the first element of each tuple, we can sort the list of tuples and get our dictionary contents sorted by value.

This page titled [10.5: Multiple assignment with dictionaries](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.5: Multiple assignment with dictionaries](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.6: The most common words

Coming back to our running example of the text from *Romeo and Juliet* Act 2, Scene 2, we can augment our program to use this technique to print the ten most common words in the text as follows:

```
import string
fhand = open('romeo-full.txt')
counts = dict()
for line in fhand:
    line = line.translate(str.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

# Sort the dictionary by value
lst = list()
for key, val in list(counts.items()):
    lst.append((val, key))

lst.sort(reverse=True)

for key, val in lst[:10]:
    print(key, val)

# Code: http://www.py4e.com/code3/count3.py
```

The first part of the program which reads the file and computes the dictionary that maps each word to the count of words in the document is unchanged. But instead of simply printing out `counts` and ending the program, we construct a list of `(val, key)` tuples and then sort the list in reverse order.

Since the value is first, it will be used for the comparisons. If there is more than one tuple with the same value, it will look at the second element (the key), so tuples where the value is the same will be further sorted by the alphabetical order of the key.

At the end we write a nice `for` loop which does a multiple assignment iteration and prints out the ten most common words by iterating through a slice of the list (`lst[:10]`).

So now the output finally looks like what we want for our word frequency analysis.

```
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

The fact that this complex data parsing and analysis can be done with an easy-to-understand 19-line Python program is one reason why Python is a good choice as a language for exploring information.

This page titled [10.6: The most common words](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.6: The most common words](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.8: Sequences- strings, lists, and tuples - Oh My!

I have focused on lists of tuples, but almost all of the examples in this chapter also work with lists of lists, tuples of tuples, and tuples of lists. To avoid enumerating the possible combinations, it is sometimes easier to talk about sequences of sequences.

In many contexts, the different kinds of sequences (strings, lists, and tuples) can be used interchangeably. So how and why do you choose one over the others?

To start with the obvious, strings are more limited than other sequences because the elements have to be characters. They are also immutable. If you need the ability to change the characters in a string (as opposed to creating a new string), you might want to use a list of characters instead.

Lists are more common than tuples, mostly because they are mutable. But there are a few cases where you might prefer tuples:

1. In some contexts, like a `return` statement, it is syntactically simpler to create a tuple than a list. In other contexts, you might prefer a list.
2. If you want to use a sequence as a dictionary key, you have to use an immutable type like a tuple or string.
3. If you are passing a sequence as an argument to a function, using tuples reduces the potential for unexpected behavior due to aliasing.

Because tuples are immutable, they don't provide methods like `sort` and `reverse`, which modify existing lists. However Python provides the built-in functions `sorted` and `reversed`, which take any sequence as a parameter and return a new sequence with the same elements in a different order.

This page titled [10.8: Sequences- strings, lists, and tuples - Oh My!](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.8: Sequences- strings, lists, and tuples - Oh My!](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.9: Debugging

Lists, dictionaries and tuples are known generically as *data structures*; in this chapter we are starting to see compound data structures, like lists of tuples, and dictionaries that contain tuples as keys and lists as values. Compound data structures are useful, but they are prone to what I call *shape errors*; that is, errors caused when a data structure has the wrong type, size, or composition, or perhaps you write some code and forget the shape of your data and introduce an error.

For example, if you are expecting a list with one integer and I give you a plain old integer (not in a list), it won't work.

When you are debugging a program, and especially if you are working on a hard bug, there are four things to try:

reading

Examine your code, read it back to yourself, and check that it says what you meant to say.

running

Experiment by making changes and running different versions. Often if you display the right thing at the right place in the program, the problem becomes obvious, but sometimes you have to spend some time to build scaffolding.

ruminating

Take some time to think! What kind of error is it: syntax, runtime, semantic? What information can you get from the error messages, or from the output of the program? What kind of error could cause the problem you're seeing? What did you change last, before the problem appeared?

retreating

At some point, the best thing to do is back off, undoing recent changes, until you get back to a program that works and that you understand. Then you can start rebuilding.

Beginning programmers sometimes get stuck on one of these activities and forget the others. Each activity comes with its own failure mode.

For example, reading your code might help if the problem is a typographical error, but not if the problem is a conceptual misunderstanding. If you don't understand what your program does, you can read it 100 times and never see the error, because the error is in your head.

Running experiments can help, especially if you run small, simple tests. But if you run experiments without thinking or reading your code, you might fall into a pattern I call "random walk programming", which is the process of making random changes until the program does the right thing. Needless to say, random walk programming can take a long time.

You have to take time to think. Debugging is like an experimental science. You should have at least one hypothesis about what the problem is. If there are two or more possibilities, try to think of a test that would eliminate one of them.

Taking a break helps with the thinking. So does talking. If you explain the problem to someone else (or even to yourself), you will sometimes find the answer before you finish asking the question.

But even the best debugging techniques will fail if there are too many errors, or if the code you are trying to fix is too big and complicated. Sometimes the best option is to retreat, simplifying the program until you get to something that works and that you understand.

Beginning programmers are often reluctant to retreat because they can't stand to delete a line of code (even if it's wrong). If it makes you feel better, copy your program into another file before you start stripping it down. Then you can paste the pieces back in a little bit at a time.

Finding a hard bug requires reading, running, ruminating, and sometimes retreating. If you get stuck on one of these activities, try the others.

This page titled [10.9: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.9: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.E: Tuples (Exercises)

Exercise 1: Revise a previous program as follows: Read and parse the "From" lines and pull out the addresses from the line. Count the number of messages from each person using a dictionary.

After all the data has been read, print the person with the most commits by creating a list of (count, email) tuples from the dictionary. Then sort the list in reverse order and print out the person who has the most commits.

```
Sample Line:
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

Enter a file name: mbox-short.txt
cwen@iupui.edu 5

Enter a file name: mbox.txt
zqian@umich.edu 195
```

Exercise 2: This program counts the distribution of the hour of the day for each of the messages. You can pull the hour from the "From" line by finding the time string and then splitting that string into parts using the colon character. Once you have accumulated the counts for each hour, print out the counts, one per line, sorted by hour as shown below.

Sample Execution:

```
python timeofday.py
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Exercise 3: Write a program that reads a file and prints the *letters* in decreasing order of frequency. Your program should convert all the input to lower case and only count the letters a-z. Your program should not count spaces, digits, punctuation, or anything other than the letters a-z. Find text samples from several different languages and see how letter frequency varies between languages. Compare your results with the tables at [Wikipedia.org/wiki/Letter_frequencies](https://en.wikipedia.org/wiki/Letter_frequencies).

1. Fun fact: The word "tuple" comes from the names given to sequences of numbers of varying lengths: single, double, triple, quadruple, quintuple, sextuple, septuple, etc.↵
2. Python does not translate the syntax literally. For example, if you try this with a dictionary, it will not work as might expect.↵

This page titled [10.E: Tuples \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [10.E: Tuples \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

10.G: Tuples (Glossary)

comparable

A type where one value can be checked to see if it is greater than, less than, or equal to another value of the same type. Types which are comparable can be put in a list and sorted.

data structure

A collection of related values, often organized in lists, dictionaries, tuples, etc.

DSU

Abbreviation of "decorate-sort-undecorate", a pattern that involves building a list of tuples, sorting, and extracting part of the result.

gather

The operation of assembling a variable-length argument tuple.

hashable

A type that has a hash function. Immutable types like integers, floats, and strings are hashable; mutable types like lists and dictionaries are not.

scatter

The operation of treating a sequence as a list of arguments.

shape (of a data structure)

A summary of the type, size, and composition of a data structure.

singleton

A list (or other sequence) with a single element.

tuple

An immutable sequence of elements.

tuple assignment

An assignment with a sequence on the right side and a tuple of variables on the left. The right side is evaluated and then its elements are assigned to the variables on the left.

This page titled [10.G: Tuples \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

11: Libraries

- [11.1: NumPy - Numerical Python](#)
- [11.2: SciPy - Scientific Computing in Python](#)
- [11.3: Pandas - Data Manipulation and Analysis](#)
- [11.4: Matplotlib — Data Visualization in Python](#)
- [11.5: Scikit-learn \(sklearn\) - Supervised and Unsupervised Learning](#)

This page titled [11: Libraries](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

11.1: NumPy - Numerical Python

NumPy

NumPy, short for Numerical Python, is a fundamental library in Python for scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more. The [NumPy official user guide](#) provides a comprehensive description of the library. A good place to start is the chapter: [NumPy: the absolute basics for beginners](#).

Jupyter Notebook with NumPy Examples:



Key features and functionalities of NumPy:

- **Broadcasting:** This powerful feature allows NumPy to perform operations on arrays of different shapes and sizes, automatically aligning them for computation where possible.
- **Integration with other libraries:** NumPy forms the foundation for many other popular scientific computing and machine learning libraries in Python, such as SciPy, Matplotlib, Pandas, and Scikit-learn.
- **Performance:** NumPy's core is written in C and C++, which enables it to perform numerical computations significantly faster than native Python lists, especially when dealing with large datasets.
- **ndarray object:** The core of NumPy is the ndarray (N-dimensional array) object, which is a powerful and efficient data structure for storing and manipulating numerical data. Unlike Python lists, NumPy arrays are fixed in size at creation and require all elements to be of the same data type, which contributes to their speed and memory efficiency.
- **Mathematical operations:** NumPy offers a wide range of mathematical functions that operate element-wise on arrays, including statistical operations (e.g., mean, median, standard deviation), linear algebra operations (e.g., dot product, matrix multiplication), Fourier transforms, and more.

NumPy is essential for tasks involving numerical data manipulation and analysis due to its speed, efficiency, and comprehensive set of tools. It is widely used in fields like data science, machine learning, scientific research, and engineering for tasks such as data cleaning, transformation, statistical analysis, and algorithm implementation.

Why Use NumPy?

The core of NumPy is its ndarray object, which efficiently stores and manipulates large arrays of homogeneous data. Here's why it's so popular:

- **Performance:** NumPy operations are implemented in C, making them significantly faster than standard Python list operations, especially for large datasets.
- **Memory Efficiency:** NumPy arrays consume less memory compared to Python lists because they store elements of the same data type contiguously in memory.
- **Vectorization:** NumPy allows you to perform operations on entire arrays without explicit loops, a concept known as vectorization. This not only improves performance but also leads to more concise and readable code.
- **Mathematical Functions:** It provides a vast collection of mathematical functions for array operations, linear algebra, Fourier transforms, and more.

Learning Objectives

- Creating NumPy Arrays
- Array Attributions and Operations
- Universal and Aggregate Function
- Indexing and Slicing
- Reshaping Arrays

PYTHON SESSION: READY

Installation

If NumPy is not already installed, you can use pip to install it:

Note

Bash or Command Terminal.
`pip install numpy`

NumPy Basics: The ndarray Object

The ndarray is the primary data structure in NumPy. Let's explore its creation and basic attributes.

Creating Arrays

You can create NumPy arrays from Python lists or tuples using the `np.array()` function.

✓ Example \(\PageIndex{1}\)

Python code for NumPy vector and matrix arrays

Solution

```
import numpy as np

# Create a 1D array (vector)
arr_1d = np.array([1, 2, 3, 4, 5])
print("1D Array:", arr_1d)

# Create a 2D array (matrix)
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:\n", arr_2d)

# Create a 3D array
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print("3D Array:\n", arr_3d)
```

NumPy also provides functions to create arrays with initial placeholder content:

✓ Example \(\PageIndex{2}\)

Python code for placeholder arrays

Solution

```
# Array of zeros
zeros_arr = np.zeros((3, 4)) # 3 rows, 4 columns
print("Array of Zeros:\n", zeros_arr)

# Array of ones
ones_arr = np.ones((2, 2))
print("Array of Ones:\n", ones_arr)

# Empty array (values are uninitialized, whatever is in memory)
empty_arr = np.empty((2, 3))
print("Empty Array:\n", empty_arr)

# Array with a range of values
range_arr = np.arange(0, 10, 2) # Start, Stop (exclusive), Step
print("Range Array:\n", range_arr)
```

PYTHON SESSION: READY

```
# Linearly spaced array
linspace_arr = np.linspace(0, 10, 5) # Start, Stop (inclusive), Number of elements
print("Linspace Array:", linspace_arr)
```

Array Attributes

Each ndarray has several important attributes:

- `.ndim`: The number of dimensions (axes) of the array.
- `.shape`: A tuple indicating the size of the array in each dimension.
- `.size`: The total number of elements in the array.
- `.dtype`: The data type of the elements in the array.

✓ Example \(\PageIndex{3}\)

Python code to display array attributes

Solution

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)
```

Array Operations

NumPy allows you to perform various operations on arrays, either element-wise or using built-in functions.

Element-wise Operations

Arithmetic operations (+, -, *, /) are performed element-wise.

✓ Example \(\PageIndex{4}\)

Python code for the element-wise Arithmetic operations (*,-,*,/)

Solution

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("Addition:", arr1 + arr2)
print("Subtraction:", arr1 - arr2)
print("Multiplication:", arr1 * arr2)
print("Division:", arr1 / arr2)
```

Broadcasting

Broadcasting is a powerful mechanism that allows NumPy to perform operations on arrays of different shapes. It implicitly "stretches" the smaller array to match the larger one, provided they are compatible.

✓ Example \(\PageIndex{5}\)

Python code for broadcasting

Solution

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
scalar = 10
print("Array + Scalar:\n", arr + scalar) # Scalar is broadcast to all elements
print("Array - Scalar:\n", arr - scalar)
print("Array * Scalar:\n", arr * scalar)
print("Array / Scalar:\n", arr / scalar)
```

Universal Functions (ufuncs)

NumPy provides many universal functions (ufuncs) that operate element-wise on arrays, such as `np.sqrt()` , `np.sin()` , `np.cos()` , `np.exp()` , etc.

✓ Example \(\PageIndex{6}\)

Python code for a couple of universal function

Solution

```
import numpy as np
arr = np.array([1, 4, 9])
print("Square Root:", np.sqrt(arr))
print("Sine:", np.sin(arr))
```

Aggregate Functions

NumPy offers functions to perform aggregations (e.g., sum, mean, min, max) along specified axes.

✓ Example \(\PageIndex{7}\)

Python code for example of aggregations

Solution

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Sum of all elements:", np.sum(arr))
print("Sum along columns (axis=0):", np.sum(arr, axis=0)) # Sum of [1,4], [2,5], [3,6]
print("Sum along rows (axis=1):", np.sum(arr, axis=1)) # Sum of [1,2,3], [4,5,6]

print("Mean of all elements:", np.mean(arr))
print("Maximum element:", np.max(arr))
```

Indexing and Slicing

Accessing and manipulating parts of NumPy arrays is similar to Python lists but with extended capabilities for multiple dimensions.

Basic Indexing

✓ Example \(\PageIndex{8}\)

Python code example for basic indexing

Solution

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("First element:", arr[0])
print("Last element:", arr[-1])
```

PYTHON SESSION: READY

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Element at (0, 1):", matrix[0, 1]) # Row 0, Column 1 (value 2)
print("Element at (2, 0):", matrix[2, 0]) # Row 2, Column 0 (value 7)
```

Slicing

Slicing allows you to extract subarrays. The syntax is start:stop:step.

✓ Example \(\PageIndex{9}\)

Python code example for array slicing

Solution

```
import numpy as np
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

print("Elements from index 2 to 5:", arr[2:6]) # [2 3 4 5]
print("Elements from beginning to index 4:", arr[:5]) # [0 1 2 3 4]
print("Elements from index 5 to end:", arr[5:]) # [5 6 7 8 9]
print("Every other element:", arr[::2]) # [0 2 4 6 8]
print("Reversed array:", arr[::-1]) # [9 8 7 6 5 4 3 2 1 0]

matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("First two rows:\n", matrix[:2, :])
print("All rows, last column:\n", matrix[:, 2])
print("Sub-matrix (rows 0-1, cols 1-2):\n", matrix[0:2, 1:3])
```

Boolean Indexing

You can use boolean arrays to select elements that satisfy a certain condition.

✓ Example \(\PageIndex{10}\)

Python code example for Boolean indexing

Solution

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
# Select elements greater than 30
filtered_arr = arr[arr > 30]
print("Filtered Array:", filtered_arr)

# Elements that are even
even_elements = arr[arr % 2 == 0]
print("Even Elements:", even_elements)
```

Reshaping Arrays

Changing the shape of an array without changing its data is a common operation.

reshape()

The `reshape()` method returns a new array with the specified shape.

✓ Example \(\PageIndex{11}\)

Python code examples for array reshaping

PYTHON SESSION: READY

Solution

```
import numpy as np
arr = np.arange(12) # [ 0 1 2 3 4 5 6 7 8 9 10 11]
print("Original Array:", arr)

# Reshape to a 3x4 matrix
reshaped_arr = arr.reshape(3, 4)
print("Reshaped to 3x4:\n", reshaped_arr)

# Reshape to a 2x2x3 3D array
reshaped_3d = arr.reshape(2, 2, 3)
print("Reshaped to 2x2x3:\n", reshaped_3d)

# Use -1 for an unknown dimension (NumPy calculates it)
reshaped_auto = arr.reshape(4, -1) # 4 rows, NumPy calculates columns
print("Reshaped with auto-dimension:\n", reshaped_auto)

# Transpose a 2D array
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Original Matrix:\n", matrix)
print("Matrix Transpose:\n", matrix.T)
```

flatten() and ravel()

Both `flatten()` and `ravel()` convert a multi-dimensional array into a 1D array.

- `flatten()` returns a copy of a multi-dimensional array collapsed into a single, one-dimensional array.
- `ravel()` returns a **view** of the original array (if possible), meaning changes to the raveled array might affect the original.

✓ Example \(\PageIndex{12}\)

Python code example for array flatten and ravel

Solution

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6]])

flat_copy = matrix.flatten()
print("Flattened (copy):", flat_copy)

flat_view = matrix.ravel()
print("Raveled (view):", flat_view)

# Demonstrate view vs. copy
flat_view[0] = 99
print("Original matrix after changing raveled view:\n", matrix)

flat_copy[0] = 100
print("Original matrix after changing flattened copy:\n", matrix) # No change
```

Linear Algebra with NumPy

[NumPy linear algebra](#) rely on BLAS and LAPACK to provide capabilities for several linear algebra operations; however, it is recommended to use `scipy.linalg` for a more robust linear algebra operations.

Dot Product

The dot product of two arrays (vectors or matrices) can be calculated using `np.dot()` or the `@` operator (for matrix multiplication). Note that matrix multiplication differs from Arithmetic multiplication.

PYTHON SESSION: READY

✓ Example \(\PageIndex{13}\)

Python code example for matrix multiplication which differs from the Arithmetic product

Solution

```
import numpy as np
# Vector dot product
vec1 = np.array([1, 2])
vec2 = np.array([3, 4])
dot_product = np.dot(vec1, vec2)
print("Vector Dot Product:", dot_product) # (1*3) + (2*4) = 11

# Matrix multiplication
mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[5, 6], [7, 8]])
matrix_product = np.dot(mat1, mat2)
# or using the @ operator to achieve the same result
matrix_product_at = mat1 @ mat2
print("Matrix Product (np.dot):\n", matrix_product)
print("Matrix Product (@ operator):\n", matrix_product_at)

# Matrix vector multiplication
mat1_vec1_product = mat1 @ vec1
print("Matrix Vector Product:\n", mat1_vec1_product)
```

Note: A vector in NumPy has shape (n,) which can be treated as either row or column vector in matrix/vector operations. In this example mat1 is (2,2) and vec1 is (2,). The product, mat1_vec1_product has shape (2,).

Determinant, Inverse, Eigenvalues

NumPy's linalg module provides functions for advanced linear algebra.

✓ Example \(\PageIndex{14}\)

Python code examples for matrix determinant, inverse, and eigenvalues

Solution

```
import numpy as np
from numpy import linalg

matrix = np.array([[1, 2], [3, 4]])

# Determinant
det = linalg.det(matrix)
print("Determinant:", det)

# Inverse
inv_matrix = linalg.inv(matrix)
print("Inverse Matrix:\n", inv_matrix)

# Eigenvalues and Eigenvectors
eigenvalues, eigenvectors = linalg.eig(matrix)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Linear Matrix Equations

Solve $Ax = b$ where A is $(M \times M)$ matrix of scalars and b is $(M \times K)$ matrix of scalars. **PYTHON SESSION: READY** square and nonsingular otherwise an error will occur.


```
x = numpy.linalg.solve(A,b)
```

? Exercise \(\PageIndex{1}\)

```
# Use np.linalg.solve to solve the system of equations:
#   x0 + 2*x1   = 1
# 3*x0 + 5 * x1 = 2
# print the answer
import numpy as np
A = np.array([[1, 2], [3, 5]])
b = np.array([1, 2])
x =
print('Solution to Ax = b:\n',x)
```

run restart restart & run all

Hello world!

Answer

```
x = np.linalg.solve(A, b)
print("Solution to Ax = b:\n", x)
Solution to Ax = b
[-1., 1.]
```

The b vector in this exercise is treated as a column vector for the solution $x = A^{-1} b$ with x presented in the same form as b.

Conclusion

This guide has covered the fundamental aspects of NumPy, from creating arrays and performing basic operations to advanced indexing, reshaping, and linear algebra. NumPy's efficiency and extensive functionality make it an indispensable tool for anyone working with numerical data in Python. 🚀 Understanding NumPy is a crucial step towards mastering scientific computing and data science with Python.

This page titled [11.1: NumPy - Numerical Python](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

11.2: SciPy - Scientific Computing in Python

SciPy, pronounced "Sigh-Pie," is an open-source Python library that builds on NumPy and provides a collection of algorithms and tools for scientific and technical computing. It's a fundamental library for scientists, engineers, and analysts working with numerical data. The [SciPy Documentation](#) provides a comprehensive review of this library.

Jupyter Notebook with SciPy Examples



Key Features

SciPy extends NumPy's capabilities by offering a wide range of specialized modules for common tasks in scientific computing, including:

- **Optimization:** Algorithms for finding minima and maxima of functions, fitting curves to data, and solving linear programming problems.
- **Integration:** Routines for numerical integration of functions and solving ordinary differential equations.
- **Interpolation:** Tools for estimating unknown values between known data points.
- **Signal Processing:** Functions for analyzing and manipulating signals, such as filtering and convolution.
- **Image Processing:** Basic image manipulation and analysis tools.
- **Statistics:** A comprehensive set of statistical functions, including probability distributions, hypothesis testing, and statistical tests.
- **Linear Algebra:** Advanced linear algebra routines beyond what NumPy provides, such as eigenvalue problems and matrix decompositions.
- **Sparse Matrices:** Tools for working with sparse matrices, which are matrices where most elements are zero, leading to memory and computational efficiency gains.

Why is it Important?

SciPy's importance lies in its ability to provide highly optimized and well-tested implementations of widely used scientific algorithms. This allows users to focus on problem-solving rather than reimplementing complex mathematical methods from scratch. It integrates seamlessly with NumPy, making it a powerful combination for data analysis, modeling, and simulation in various scientific and engineering domains.

Introduction to SciPy

SciPy (pronounced "Sigh-Pie") is an open-source Python library used for scientific and technical computing. It builds on NumPy, providing a large collection of algorithms and high-level commands for data manipulation and visualization. While NumPy provides the fundamental array object and basic array operations, SciPy provides a wide range of specialized tools for tasks such as optimization, integration, interpolation, signal processing, linear algebra, statistics, and more.

SciPy is organized into sub-packages, each focusing on a specific domain. This modular structure makes it easy to import only the necessary functions, keeping your code clean and efficient.

To use SciPy, you typically need to have both NumPy and SciPy installed. You can install them using pip:

```
pip install numpy scipy
```

Let's explore some of the most commonly used modules within SciPy with practical examples.

[scipy.optimize: Optimization and Root Finding](#)

The `scipy.optimize` module provides algorithms for minimizing (or maximizing) objective functions, including least squares, curve fitting, and root finding.

PYTHON SESSION: READY

Example: Minimizing a Function

Let's find the minimum of the function $f(x) = x^2 + 10 \sin(x)$.

```
import numpy as np
from scipy.optimize import minimize

# Define the function to minimize
def objective_function(x):
    return x**2 + 10 * np.sin(x)

# Initial guess for x
x0 = 0.0

# Perform the minimization
result = minimize(objective_function, x0)

print("--- scipy.optimize Example (Minimization) ---")
print(f"Function minimized: f(x) = x^2 + 10 * sin(x)")
print(f"Initial guess: x0 = {x0}")
print(f"Minimum found at x: {result.x[0]:.4f}")
print(f"Minimum function value: {result.fun:.4f}")
print("-" * 40)
```

runrestartrestart & run all

```
--- scipy.optimize Example (Minimization) ---
Function minimized: f(x) = x^2 + 10 * sin(x)
Initial guess: x0 = 0.0
Minimum found at x: -1.3064
Minimum function value: -7.9458
-----
```

Example: Root Finding

Let's find the root of the function $f(x) = x^3 - 2x - 5$.

```
from scipy.optimize import root

# Define the function for which to find the root
def equation_to_solve(x):
    return x**3 - 2*x - 5

# Initial guess for the root
x0_root = 2.0

# Find the root
sol = root(equation_to_solve, x0_root)

print("\n--- scipy.optimize Example (Root Finding) ---")
print(f"Function: f(x) = x^3 - 2x - 5")
print(f"Initial guess: x0 = {x0_root}")
print(f"Root found at x: {sol.x[0]:.4f}")
```

PYTHON SESSION: READY

```
print(f"Function value at root: {equation_to_solve(sol.x[0]):.4e}") # Should be close
print("-" * 40)
```

run restart restart & run all

```
--- scipy.optimize Example (Root Finding) ---
Function: f(x) = x^3 - 2x - 5
Initial guess: x0 = 2.0
Root found at x: 2.0946
Function value at root: -8.8818e-16
-----
```

[scipy.integrate: Numerical Integration](#)

This module provides functions for integrating functions, differential equations, and samples.

[Example: Numerical Integration \(Quadrature\)](#)

Let's integrate $f(x)=x^2$ from 0 to 1. The analytical answer is $1/3$.

```
from scipy.integrate import quad

# Define the function to integrate
def integrand(x):
    return x**2

# Perform the integration from 0 to 1
result_integrate, error = quad(integrand, 0, 1)

print("\n--- scipy.integrate Example (Quadrature) ---")
print(f"Function: f(x) = x^2")
print(f"Integral from 0 to 1: {result_integrate:.4f}")
print(f"Estimated absolute error: {error:.4e}")
print("-" * 40)
```

run restart restart & run all

```
--- scipy.integrate Example (Quadrature) ---
Function: f(x) = x^2
Integral from 0 to 1: 0.3333
Estimated absolute error: 3.7007e-15
-----
```

[scipy.linalg: Linear Algebra](#)

The `scipy.linalg` module provides highly optimized routines for linear algebra operations, building on top of BLAS and LAPACK. It offers more advanced functionality than `numpy.linalg`.

[Example: Solving a System of Linear Equations](#)

Solve the system: $\begin{bmatrix} 2x + y \\ x - 3y \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 2x + y \\ x - 3y \end{bmatrix}$

$\begin{bmatrix} 2x + y \\ x - 3y \end{bmatrix}$

$\begin{bmatrix} 2x + y \\ x - 3y \end{bmatrix}$

PYTHON SESSION: READY

```
\end{split}
\end{align}\]
```

This can be written in matrix form as $\begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \end{bmatrix}$

```
import numpy as np
from scipy.linalg import solve

# Define the coefficient matrix A
A = np.array([[2, 1], [1, -3]])

# Define the constant vector b
b = np.array([5, -1])

# Solve the system Ax = b
x_solution = solve(A, b)

print("\n--- scipy.linalg Example (Solving Linear Equations) ---")
print("System: 2x + y = 5, x - 3y = -1")
print(f"Coefficient matrix A:\n{A}")
print(f"Constant vector b:\n{b}")
print(f"Solution x (x, y): {x_solution}")
print("-" * 40)
```

[run](#)[restart](#)[restart & run all](#)

```
--- scipy.linalg Example (Solving Linear Equations) ---
System: 2x + y = 5, x - 3y = -1
Coefficient matrix A:
[[ 2  1]
 [ 1 -3]]
Constant vector b:
[ 5 -1]
Solution x (x, y): [2.  1.]
-----
```

[scipy.stats: Statistical Functions](#)

This module contains a large number of probability distributions (continuous and discrete), statistical tests, and descriptive statistics.

[Example: Normal Distribution and T-test](#)

Let's generate some random data from a normal distribution and perform a one-sample t-test.

```
import numpy as np
from scipy.stats import norm, ttest_1samp

# Generate random data from a normal distribution
# (mean=0, std=1, 100 samples)
data = norm.rvs(loc=0, scale=1, size=100, random_state=42)

# Calculate PDF and CDF for a specific value
```

PYTHON SESSION: READY

```
x_val = 1.0
pdf_at_x = norm.pdf(x_val)
cdf_at_x = norm.cdf(x_val)

# Perform a one-sample t-test
# Null hypothesis: the true mean of the sample is 0.1
test_mean = 0.1
statistic, p_value = ttest_1samp(data, test_mean)

print("\n--- scipy.stats Example (Normal Distribution & T-test) ---")
print(f"PDF of standard normal at x={x_val}: {pdf_at_x:.4f}")
print(f"CDF of standard normal at x={x_val}: {cdf_at_x:.4f}")
print(f"Mean of generated data: {np.mean(data):.4f}")
print(f"One-sample t-test against mean {test_mean}:")
print(f"  T-statistic: {statistic:.4f}")
print(f"  P-value: {p_value:.4f}")
if p_value < 0.05:
    print("    (Reject the null hypothesis at 5% significance level)")
else:
    print("    (Fail to reject the null hypothesis at 5% significance level)")
print("-" * 40)
```

run

restart

restart & run all

```
--- scipy.stats Example (Normal Distribution & T-test) ---
PDF of standard normal at x=1.0: 0.2420
CDF of standard normal at x=1.0: 0.8413
Mean of generated data: -0.1038
One-sample t-test against mean 0.1:
  T-statistic: -2.2446
  P-value: 0.0270
  (Reject the null hypothesis at 5% significance level)
-----
```

scipy.signal: Signal Processing

This module provides functions for signal processing, including filtering, convolution, and spectral analysis.

Example: Filtering a Signal

Let's create a noisy signal and apply a Butterworth filter to smooth it.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter

# Generate a noisy signal
fs = 1000          # Sampling frequency
f1 = 10            # Frequency of the first sine wave
f2 = 50            # Frequency of the second sine wave
t = np.linspace(0, 1, fs, endpoint=False) # Time vector
clean_signal = np.sin(2 * np.pi * f1 * t) + 0.2 * np.sin(2 * np.pi * f2 * t)
noisy_signal = clean_signal + np.random.randn(len(t))
```

PYTHON SESSION: READY

```
noisy_signal = clean_signal + noise

# Design a Butterworth low-pass filter
# Cutoff frequency is 20 Hz (normalized to Nyquist frequency)
cutoff_freq = 20
nyquist_freq = 0.5 * fs
normal_cutoff = cutoff_freq / nyquist_freq
order = 4 # Filter order

b, a = butter(order, normal_cutoff, btype='low', analog=False)

# Apply the filter to the noisy signal
filtered_signal = lfilter(b, a, noisy_signal)

# Plot the original and filtered signals
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(t, noisy_signal, label='Noisy Signal', color='red', alpha=0.5)
plt.plot(t, clean_signal, label='Clean Signal', color='blue')
plt.title('Noisy Signal vs Clean Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid()
plt.legend()
plt.subplot(2, 1, 2)
plt.plot(t, filtered_signal, label='Filtered Signal', color='green')
plt.title(f'Filtered Signal (Butterworth Low-pass, $f_c$ = {cutoff_freq} Hz)')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

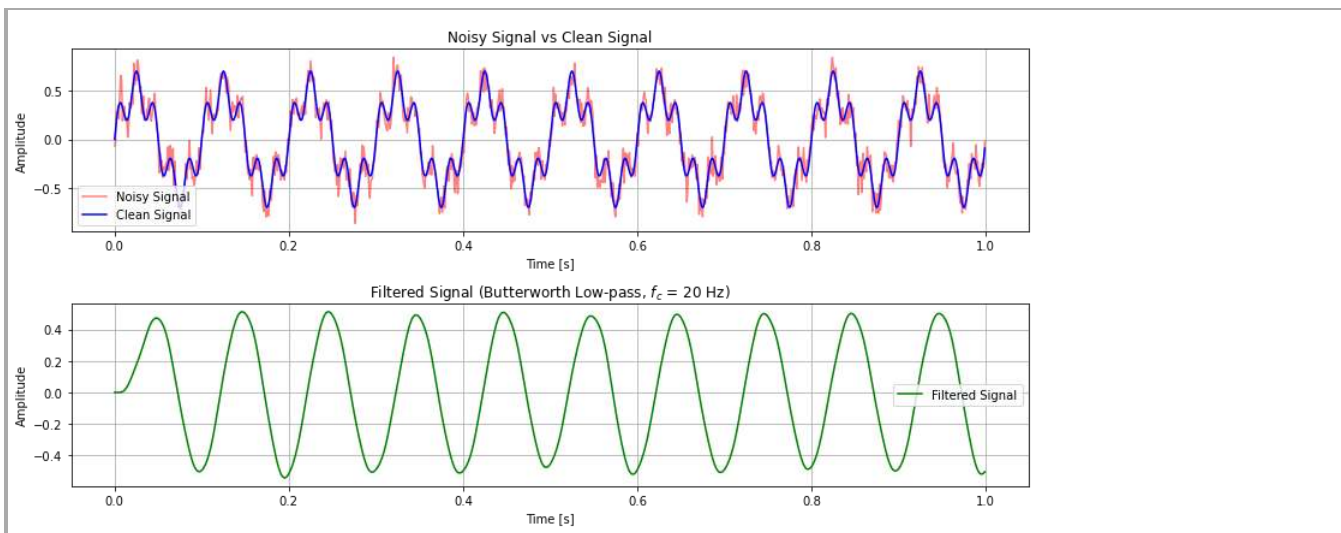
print("\n--- scipy.signal Example (Filtering) ---")
print(f"Original signal length: {len(noisy_signal)}")
print(f"Filtered signal length: {len(filtered_signal)}")
print(f"Filtering applied: 4th order Butterworth low-pass filter with {cutoff_freq} Hz")
print("-" * 40)
```

run

restart

restart & run all

PYTHON SESSION: READY



[scipy.interpolate: Interpolation](#)

The `scipy.interpolate` module provides tools for interpolating data. Interpolation is the process of estimating values between known data points.

[Example: 1-D Interpolation](#)

Let's interpolate a set of known data points using a cubic spline.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# Original data points
x_data = np.array([0, 1, 2, 3, 4, 5])
y_data = np.array([0, 0.8, 0.9, 0.1, -0.8, -1.0])

# Create an interpolation function (cubic spline)
f_interp = interp1d(x_data, y_data, kind='cubic')

# New points where we want to interpolate
x_new = np.linspace(0, 5, num=50, endpoint=True)

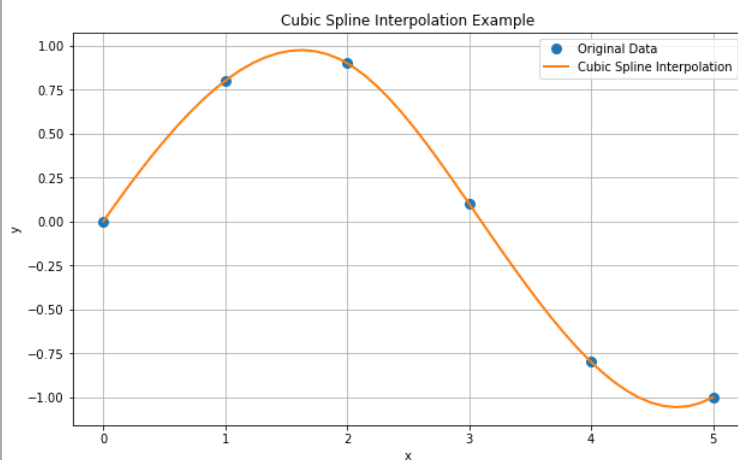
# Get the interpolated y values
y_interpolated = f_interp(x_new)

# Plotting the original data and the interpolated curve
plt.figure(figsize=(10, 6))
plt.plot(x_data, y_data, 'o', label='Original Data', markersize=8)
plt.plot(x_new, y_interpolated, '-', label='Cubic Spline Interpolation', linewidth=2)
plt.title('Cubic Spline Interpolation Example')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()

print( "\n--- scipy.interpolate Example (1-D Interpolation) ---")
```

PYTHON SESSION: READY


```
print(f"Original x data: {x_data}")
print(f"Original y data: {y_data}")
print(f"Interpolated y at x=0.5: {f_interp(0.5):.4f}")
print(f"Interpolated y at x=2.5: {f_interp(2.5):.4f}")
# In a real application, you would plot original and interpolated points
# to visualize the spline.
print("-" * 40)
```

Conclusion

SciPy is an indispensable library for anyone doing scientific computing in Python. Its rich collection of algorithms covers a vast array of mathematical, scientific, and engineering domains. By leveraging its highly optimized functions, you can perform complex analyses and simulations with relative ease and efficiency. This chapter has only scratched the surface of what SciPy can do, but it provides a solid foundation for exploring its capabilities further.

This page titled [11.2: SciPy - Scientific Computing in Python](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

11.3: Pandas - Data Manipulation and Analysis

Introduction to Pandas in Python

Pandas is a powerful, open-source Python library widely used for data manipulation and analysis. Built on top of the NumPy library, Pandas provides flexible and efficient data structures that make working with structured data intuitive and fast. It's a cornerstone for data scientists and analysts, enabling tasks from data cleaning and transformation to aggregation and visualization. The [pandas documentation](#) provides a comprehensive description of this library.

Jupyter Notebook with pandas examples



Why Pandas?

Before Pandas, data analysis in Python often involved more verbose and less intuitive methods. Pandas simplifies these tasks by offering:

- Fast and efficient data structures: Primarily Series and DataFrame, designed for handling large datasets.
- Integrated handling of missing data: Simplifies dealing with NaN values.
- Flexible reshaping and pivoting of data sets.
- Powerful, group-by functionality for performing split-apply-combine operations on data sets.
- Intelligent label-based slicing, fancy indexing, and subsetting of large data sets.
- Intuitive merging and joining of data sets.
- Robust I/O tools for loading data from flat files (CSV, delimited), Excel files, databases, and more.

Let's dive into the core data structures.

Pandas Data Structures: Series and DataFrame

Pandas introduces two primary data structures: Series and DataFrame.

Series

A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). It's similar to a NumPy array, but with an explicit index associated with each element.

Creating a Series

You can create a Series from a list, a NumPy array, or a dictionary.

```
import pandas as pd
import numpy as np

# 1. From a list
data_list = [10, 20, 30, 40, 50]
s1 = pd.Series(data_list)
print("Series from list:")
print(s1)
print("-" * 30)

# 2. From a NumPy array
data_array = np.array([100, 200, 300, 400])
s2 = pd.Series(data_array)
print("Series from NumPy array:")
print(s2)
print("-" * 30)

# 3. From a dictionary (keys become the index)
```

PYTHON SESSION: READY

```
data_dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
s3 = pd.Series(data_dict)
print("Series from dictionary:")
print(s3)
print("-" * 30)

# 4. Series with a custom index
s4 = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
print("Series with custom index:")
print(s4)
```

run restart restart & run all

Series from list:

```
0    10
1    20
2    30
3    40
4    50
```

dtype: int64

Series from NumPy array:

```
0    100
1    200
2    300
3    400
```

dtype: int64

Series from dictionary:

```
a     1
b     2
c     3
d     4
```

dtype: int64

Series with custom index:

```
x     10
y     20
z     30
```

dtype: int64

DataFrame

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet or a SQL table, or a dictionary of Series objects. It is the most commonly used Pandas object.

Creating a DataFrame

DataFrames can be created in various ways, most commonly from dictionaries of lists/Series, or from NumPy arrays.

```
import numpy as np
```

PYTHON SESSION: READY

```
# 1. From a dictionary of lists (most common)
# Each key becomes a column name, and its value (a list) becomes the column's data.
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}
df1 = pd.DataFrame(data)
print("DataFrame from dictionary of lists:")
print(df1)
print("-" * 30)

# 2. From a dictionary of Series
# Similar to a dictionary of lists, but values are Pandas Series.
data_series = {
    'ColA': pd.Series([1, 2, 3]),
    'ColB': pd.Series([4, 5, 6])
}
df2 = pd.DataFrame(data_series)
print("DataFrame from dictionary of Series:")
print(df2)
print("-" * 30)

# 3. From a list of dictionaries (each dictionary represents a row)
# The keys of the dictionaries become column names. If a key is missing in a dict, NaN
data_rows = [
    {'Name': 'Eve', 'Age': 28},
    {'Name': 'Frank', 'Age': 32, 'City': 'Miami'}
]
df3 = pd.DataFrame(data_rows)
print("DataFrame from list of dictionaries:")
print(df3)
print("-" * 30)

# 4. From a NumPy array (requires specifying column names and optionally an index)
# The array provides the values, and you provide the labels for rows (index) and columns
df4 = pd.DataFrame(np.random.rand(3, 2), # 3 rows, 2 columns of random numbers
                   index=['row1', 'row2', 'row3'],
                   columns=['colA', 'colB'])
print("DataFrame from NumPy array with custom index and columns:")
print(df4)
```

run

restart

restart & run all

DataFrame from dictionary of lists:

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago
			Houston

PYTHON SESSION: READY

DataFrame from dictionary of Series:

	ColA	ColB
0	1	4
1	2	5
2	3	6

DataFrame from list of dictionaries:

	Name	Age	City
0	Eve	28	NaN
1	Frank	32	Miami

DataFrame from NumPy array with custom index and columns:

	colA	colB
row1	0.412317	0.899009
row2	0.391096	0.718538
row3	0.040421	0.312776

Data Handling:

- **Input/Output:**
 - Robust tools for loading data from various formats, including CSV, Excel, databases, and HDF5, and for saving data to these formats.
- **Missing Data:**
 - Easy handling of missing data, typically represented as NaN (Not a Number) for numerical data or NaT (Not a Timestamp) for time-series data.

Data Manipulation:

- **Indexing and Selection:**
 - Powerful and flexible methods for selecting subsets of data using label-based (.loc), integer-based (.iloc), or mixed (.ix - deprecated) indexing.
- **Data Cleaning:**
 - Functions for handling missing values, duplicates, and data type conversions.
- **Data Transformation:**
 - Capabilities for reshaping, pivoting, merging, and joining datasets.
- **Grouping and Aggregation:**
 - Flexible groupby functionality to perform split-apply-combine operations, enabling aggregations and transformations on grouped data.
- **Time Series Functionality:**
 - Specific tools for working with time series data, including date range generation, frequency conversion, and moving window statistics.

Why Pandas is widely used:

- **Simplifies Data Analysis:**
 - Provides an intuitive and high-level API for complex data operations, reducing the amount of code required compared to raw Python or NumPy.
- **Performance:**
 - **PYTHON SESSION: READY** implemented in Cython for performance-critical sections, offering efficient data processing.
- **Flexibility.**

- Adaptable to various data analysis tasks, from simple data exploration to complex statistical modeling.
- **Integration:**
 - Seamlessly integrates with other Python libraries for scientific computing and data visualization, such as NumPy, Matplotlib, and scikit-learn.

Summary

Pandas is an essential Python library for data manipulation and analysis, providing powerful and flexible data structures like Series and DataFrame that simplify tasks such as data cleaning, transformation, aggregation, and visualization. It's widely used by data scientists and analysts for its efficiency and intuitive approach to working with structured data.

This page titled [11.3: Pandas - Data Manipulation and Analysis](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

11.4: Matplotlib — Data Visualization in Python

Introduction to Matplotlib

Matplotlib is a powerful and widely used open-source Python library for creating static, interactive, and animated visualizations in Python. It's a foundational tool for data scientists, researchers, and anyone who needs to present data graphically. It provides a comprehensive library for creating static, animated, and interactive visualizations in Python. It is widely used for generating plots, histograms, power spectra, bar charts, error charts, scatterplots, and much more. Matplotlib is designed to be as usable as MATLAB, with the advantage of being free and open-source.

While other libraries like Seaborn and Plotly build on top of Matplotlib or offer different approaches, Matplotlib remains the foundational plotting library in Python, offering fine-grained control over every aspect of a plot.

The [Matplotlib documentation](#) provides a comprehensive description of this library.

Jupyter Notebook with Matplotlib Examples:



Key Features

Matplotlib provides a comprehensive set of tools for generating a wide variety of plots and charts, including:

- **Line Plots:** For visualizing trends and relationships over time or continuous variables.
- **Scatter Plots:** To display the relationship between two numerical variables.
- **Bar Charts:** For comparing categorical data.
- **Histograms:** To show the distribution of a single numerical variable.
- **Pie Charts:** For representing proportions of a whole.
- **3D Plots:** For visualizing data in three dimensions.
- **Subplots:** The ability to create multiple plots within a single figure, allowing for side-by-side comparisons.
- **Customization:** Extensive options for customizing every aspect of a plot, including colors, labels, titles, fonts, and line styles.
- **Output Formats:** Support for saving plots in various file formats like PNG, JPG, PDF, SVG, and more.

Why is it Important?

Matplotlib's importance stems from its flexibility and its role as the **de facto standard for plotting in Python**. It integrates seamlessly with NumPy arrays and Pandas DataFrames, making it an essential component of the scientific Python ecosystem. While other plotting libraries exist (often building on Matplotlib or offering different aesthetics), Matplotlib provides the fundamental building blocks and control necessary for creating high-quality, publication-ready figures for a wide range of applications, from exploratory data analysis to presenting research findings.

To use Matplotlib, you typically need to have it installed. You can install it using pip:

```
pip install matplotlib
```

Let's explore some of the most common types of plots and functionalities within Matplotlib with practical examples.

Basic Plotting: Line Plots and Scatter Plots

The most fundamental use of Matplotlib is to create 2D plots. The `pyplot` module is the most commonly used interface.

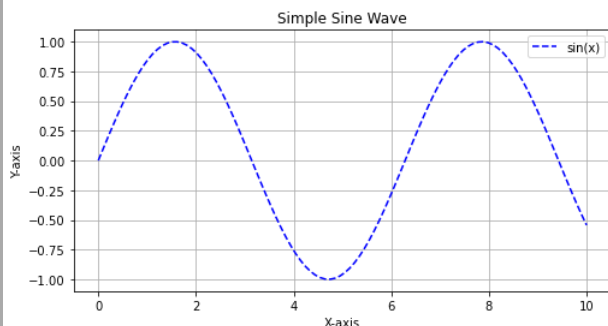
Example: Simple Line Plot

```
import matplotlib.pyplot as plt
import numpy as np

# Generate some data
x = np.linspace(0, 100)
y = np.sin(x)
```

```
# Create a simple line plot
plt.figure(figsize=(8, 4)) # Set figure size
plt.plot(x, y, label='sin(x)', color='blue', linestyle='--')
plt.title('Simple Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.legend()
plt.show() # Display the plot
```

run restart restart & run all



Example: Scatter Plot

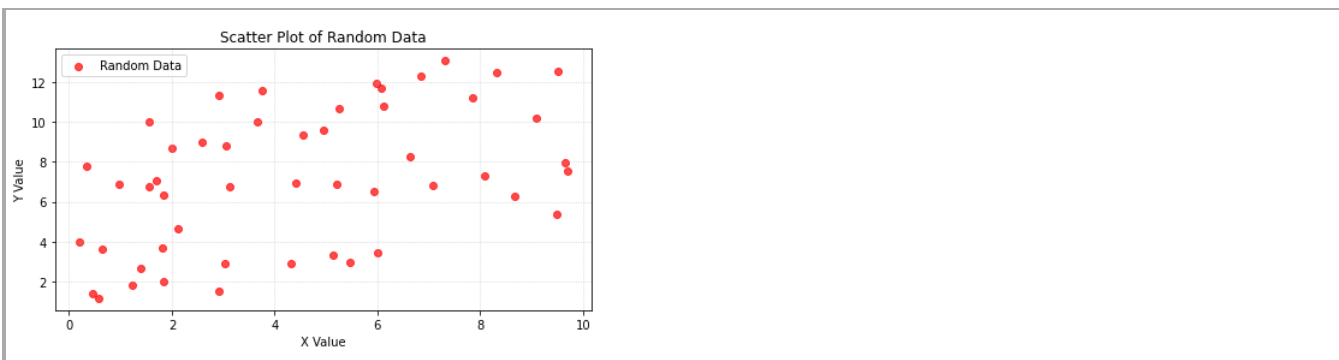
```
import matplotlib.pyplot as plt
import numpy as np

# Generate random data for scatter plot
np.random.seed(42) # for reproducibility
x_scatter = np.random.rand(50) * 10
y_scatter = np.random.rand(50) * 10 + x_scatter * 0.5 # Add some correlation

# Create a scatter plot
plt.figure(figsize=(8, 4))
plt.scatter(x_scatter, y_scatter, color='red', marker='o', alpha=0.7, label='Random Data')
plt.title('Scatter Plot of Random Data')
plt.xlabel('X Value')
plt.ylabel('Y Value')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.show()
```

run restart restart & run all

PYTHON SESSION: READY



Bar Charts and Histograms

Matplotlib is excellent for visualizing categorical data and distributions.

Example: Bar Chart

```
import matplotlib.pyplot as plt
import numpy as np

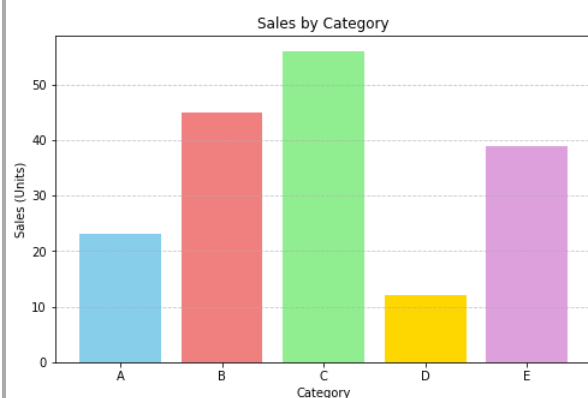
# Data for bar chart
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 12, 39]

# Create a bar chart
plt.figure(figsize=(8, 5))
plt.bar(categories, values, color=['skyblue', 'lightcoral', 'lightgreen', 'gold', 'plum'])
plt.title('Sales by Category')
plt.xlabel('Category')
plt.ylabel('Sales (Units)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

run

restart

restart & run all



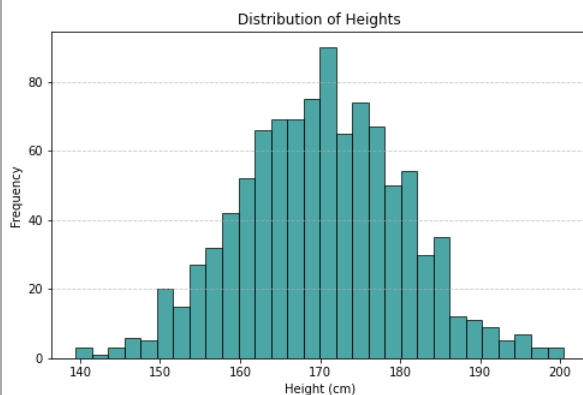
Example: Histogram

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for histogram (e.g., heights)
data_hist = np.random.normal(loc=170, scale=10, size=1000) # Mean 170, Std Dev 10
```

```
# Create a histogram
plt.figure(figsize=(8, 5))
plt.hist(data_hist, bins=30, color='teal', edgecolor='black', alpha=0.7)
plt.title('Distribution of Heights')
plt.xlabel('Height (cm)')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

run restart restart & run all



Subplots and Layouts

Matplotlib allows you to create multiple plots within a single figure using subplots, which is crucial for comparing different visualizations.

Example: Multiple Subplots

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data
x_multi = np.linspace(0, 2 * np.pi, 100)
y1_multi = np.sin(x_multi)
y2_multi = np.cos(x_multi)
y3_multi = np.tan(x_multi) # Will have asymptotes
y4_multi = np.exp(-x_multi/2) * np.sin(2*x_multi)

# Create a figure with 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('Multiple Subplots Example', fontsize=16)

# Plot 1: Sine wave
axes[0, 0].plot(x_multi, y1_multi, color='blue')
axes[0, 0].set_title('Sine Wave')
axes[0, 0].grid(True)

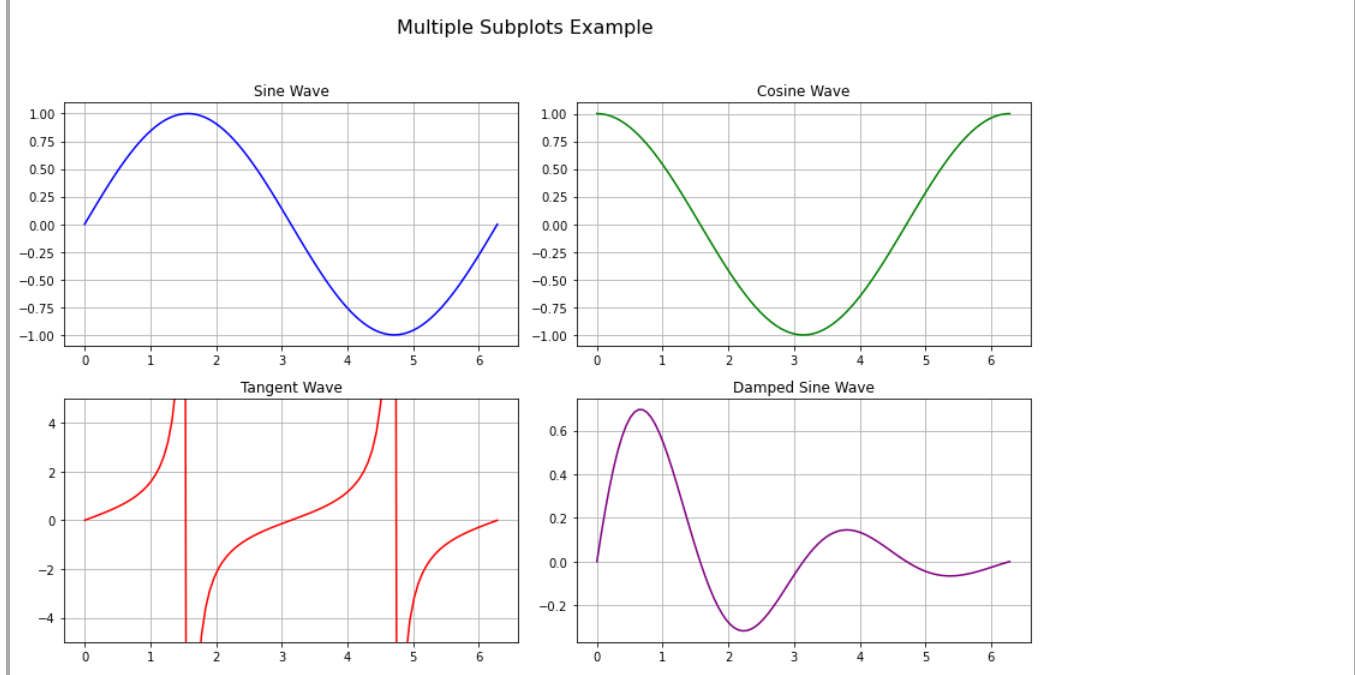
# Plot 2: Cosine wave
axes[0, 1].plot(x_multi, y2_multi, color='green')
axes[0, 1].set_title('Cosine Wave')
axes[0, 1].grid(True)
```

```
# Plot 3: Tangent wave (with limits to avoid asymptotes)
axes[1, 0].plot(x_multi, y3_multi, color='red')
axes[1, 0].set_title('Tangent Wave')
axes[1, 0].set_ylim(-5, 5) # Limit y-axis for better visualization
axes[1, 0].grid(True)

# Plot 4: Damped Sine wave
axes[1, 1].plot(x_multi, y4_multi, color='purple')
axes[1, 1].set_title('Damped Sine Wave')
axes[1, 1].grid(True)

plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title overlap
plt.show()
```

run restart restart & run all



Customizing Plots

Matplotlib offers extensive customization options for colors, markers, line styles, fonts, annotations, and more.

Example: Customizing a Plot

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data
x_custom = np.linspace(0, 5, 50)
y_custom = x_custom**2

# Create a plot with custom styles
plt.figure(figsize=(9, 6))
plt.plot(x_custom, y_custom, color='red', # Hex color
```

```

linestyle='-',          # Solid line
linewidth=3,           # Line thickness
marker='o',            # Circle markers
markersize=8,          # Marker size
markeredgecolor='black', # Marker border color
markerfacecolor='gold', # Marker fill color
label='$y = x^2$')      # LaTeX-style label

```

```

plt.title('Customized Parabola Plot', fontsize=18, fontweight='bold', color='darkblue')
plt.xlabel('X-axis', fontsize=14, color='gray')
plt.ylabel('Y-axis', fontsize=14, color='gray')

```

```

plt.grid(True, linestyle=':', alpha=0.8)
plt.legend(fontsize=12, loc='upper left')

```

```
# Add an annotation
```

```

plt.annotate('Local Minimum', xy=(0, 0), xytext=(1, 10),
            arrowprops=dict(facecolor='black', shrink=0.05),
            fontsize=10, color='green')

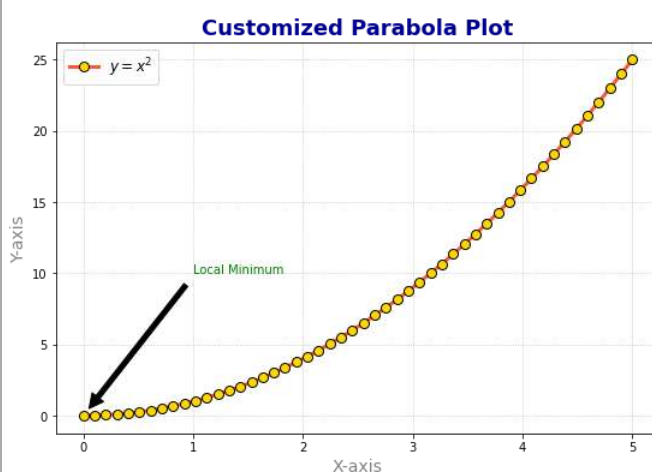
```

```
plt.show()
```

run

restart

restart & run all



3D Plotting

Matplotlib can also create 3D plots, although for very complex 3D visualizations, other libraries might be more specialized.

Example: 3D Surface Plot

```

import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D # Required for 3D plots

# Generate data for a 3D surface
x_3d = np.linspace(-5, 5, 100)
y_3d = np.linspace(-5, 5, 100)
X_3d, Y_3d = np.meshgrid(x_3d, y_3d)
Z_3d = X_3d**2 + Y_3d**2

```

PYTHON SESSION: READY

```
# Create a 3D figure
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

# Plot the surface
ax.plot_surface(X_3d, Y_3d, Z_3d, cmap='viridis', edgecolor='none')

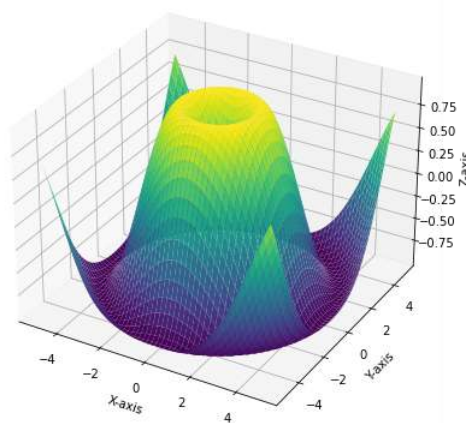
ax.set_title('3D Surface Plot of  $\sin(\sqrt{x^2 + y^2})$ ')
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')

plt.show()
```

run

restart

restart & run all

3D Surface Plot of $\sin(\sqrt{x^2 + y^2})$


Conclusion

Matplotlib is a powerful and flexible library for creating a wide variety of static, interactive, and animated visualizations in Python. Its extensive customization options allow users to produce publication-quality figures. While it can have a steeper learning curve for advanced customizations, its foundational role in the Python data science ecosystem makes it an essential tool for data analysis and presentation. This chapter has provided a glimpse into its capabilities, encouraging further exploration of its vast features.

This page titled [11.4: Matplotlib — Data Visualization in Python](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY



11.5: Scikit-learn (sklearn) - Supervised and Unsupervised Learning

Introduction to Scikit-learn

Scikit-learn (often abbreviated as sklearn) is a free software machine learning library for the Python programming language. It's built on NumPy, SciPy, and Matplotlib, and provides a wide range of supervised and unsupervised learning algorithms. Scikit-learn is known for its simplicity, efficiency, and ease of use, making it a popular choice for both beginners and experienced machine learning practitioners. The [scikit-learn documentation](#) provides a comprehensive description of this library.

Jupyter Notebook with sklearn Examples:



Key Features

Scikit-learn offers a comprehensive set of tools for various machine learning tasks, including:

- Classification: Identifying which category an item belongs to (e.g., spam detection, image recognition).
- Regression: Predicting a continuous value (e.g., predicting house prices, stock trends).
- Clustering: Grouping similar items together (e.g., customer segmentation, document analysis).
- Dimensionality Reduction: Reducing the number of features in a dataset while preserving important information (e.g., principal component analysis).
- Model Selection: Comparing, validating, and choosing parameters and models (e.g., cross-validation, grid search).
- Preprocessing: Preparing data for machine learning algorithms (e.g., scaling, imputation).
- Scikit-learn offers a comprehensive set of tools for various machine learning tasks, including:

Example

Scikit-learn Engineering Analysis Examples

Scikit-learn (sklearn) is a powerful Python library for machine learning, widely used in various fields, including engineering analysis. Here is an example of how to perform a basic engineering analysis task, such as predicting material properties, using sklearn.

Example: Predicting Material Hardness

This example demonstrates predicting a material's hardness based on its composition using a linear regression model.

Explanation:

- Data Preparation: A sample dataset representing material composition and hardness is created. In a real-world scenario, this would be loaded from a file (e.g., CSV, Excel).
- Feature and Target Definition: The independent variables (material composition) are defined as X, and the dependent variable (hardness) as y.
- Data Splitting: The data is split into training and testing sets to evaluate the model's generalization performance on unseen data.
- Model Training: A LinearRegression model from sklearn.linear_model is initialized and trained using the fit() method on the training data.
- Prediction: The trained model predicts hardness values for the test set using the predict() method.
- Model Evaluation: The mean_squared_error and root_mean_squared_error metrics are used to quantify the difference between predicted and actual hardness values.
- New Predictions: The trained model can then be used to predict the hardness of new, unobserved material compositions.

This example demonstrates a fundamental application of sklearn in engineering analysis. More complex scenarios might involve feature engineering, different machine learning algorithms (e.g., Support Vector Machines, Random Forests), and more elaborate evaluation metrics.

```
%pip install scikit-learn
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

PYTHON SESSION: READY

```
from sklearn.metrics import mean_squared_error
import numpy as np

# 1. Create a sample dataset (replace with your actual engineering data)
data = {
    'Carbon_Content': [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0],
    'Manganese_Content': [0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4],
    'Hardness_HV': [150, 180, 210, 240, 270, 300, 330, 360, 390, 420]
}
df = pd.DataFrame(data)

# 2. Define features (X) and target (y)
X = df[['Carbon_Content', 'Manganese_Content']]
y = df['Hardness_HV']

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=

# 4. Initialize and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 5. Make predictions on the test set
y_pred = model.predict(X_test)

# 6. Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("-" * 30)
print(f"Mean Squared Error: {mse:0.4e}")
print(f"Root Mean Squared Error: {rmse:0.4e}")

# 7. Use the trained model for new predictions
new_material_composition = pd.DataFrame([[0.55, 0.95]], columns=['Carbon_Content', 'M
predicted_hardness = model.predict(new_material_composition)
print(f"Predicted Hardness for new material: {predicted_hardness[0]:.2f} HV")
```

run

restart

restart & run all

```
Requirement already satisfied: scikit-learn in
/srv/conda/envs/notebook/lib/python3.8/site-packages (1.3.2)
Requirement already satisfied: numpy<2.0,>=1.17.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.23.5)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (3.5.0)
Requirement already satisfied: joblib>=1.1.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
```

PYTHON SESSION: READY

```
Requirement already satisfied: scipy>=1.5.0 in  
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-  
learn) (1.9.3)
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
-----
```

```
Mean Squared Error: 1.6156e-27
```

```
Root Mean Squared Error: 4.0194e-14
```

```
Predicted Hardness for new material: 285.00 HV
```

11.5: Scikit-learn (sklearn) - Supervised and Unsupervised Learning is shared under a [CC BY-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

PYTHON SESSION: READY

CHAPTER OVERVIEW

12: Model Building and Regression

[12.1: Functions for Model Building and Regression](#)

[12.2: Linear Regression](#)

[12.3: Optimization - Parameter Estimation for Nonlinear Functions](#)

[12.4: Summary](#)

This page titled [12: Model Building and Regression](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

12.1: Functions for Model Building and Regression

Regression and Model Building

Python offers several powerful libraries for regression and model building, each with its own set of functions and methodologies. Here's a breakdown of the most commonly used ones:

Regression

NumPy

While not a dedicated regression library, NumPy is fundamental for numerical operations in Python and is often used for foundational regression calculations or when implementing algorithms from scratch.

Relevant Functions:

- `numpy.polynomial.Polynomial.fit(x, y, deg)`: (Replaces `numpy.polyfit`) Fits a polynomial of degree `deg` to points `(x, y)`. Returns the coefficients of the polynomial.
- `numpy.polyval(p, x)`: Evaluates a polynomial `p` at specific `x` values.
- `numpy.linalg.lstsq(a, b)`: Solves the equation $ax = b$ by computing a vector `x` that minimizes the Euclidean 2-norm $\|b - ax\|^2$. This is the core of ordinary least squares.
- `numpy.corrcoef(x, y)`: Returns the Pearson product-moment correlation coefficients.
- `numpy.mean()`, `numpy.std()`: For calculating means and standard deviations, useful in manual calculations of regression coefficients.

SciPy

SciPy builds on NumPy and provides more advanced scientific computing capabilities, including optimization, signal processing, and statistics.

Relevant Functions:

- `scipy.stats.linregress(x, y)`: Calculates a linear least-squares regression for two sets of measurements. Returns slope, intercept, R-value, p-value, and standard error. It's great for simple linear regression.
- `scipy.optimize.curve_fit(f, xdata, ydata)`: Uses non-linear least squares to fit a function `f` to data. Useful for more complex, non-linear regression models where you define the function form.

Model Building

Scikit-learn (sklearn)

Scikit-learn is the go-to library for model building and machine learning in Python, providing a vast array of algorithms for supervised and unsupervised learning, including many regression models.

Commonly Used Regression Models/Functions:

- `sklearn.linear_model.LinearRegression`:
 - `LinearRegression()`: Initializes a linear regression model.
 - `.fit(X, y)`: Fits the linear model to the training data.
 - `.predict(X_new)`: Predicts target values for new data.
 - `.coef_`: Returns the estimated coefficients for the features.
 - `.intercept_`: Returns the independent term in the linear model.
 - `.score(X, y)`: Returns the coefficient of determination (R^2) of the prediction.
- `sklearn.linear_model.Ridge`:
 - `Ridge(alpha=1.0)`: Initializes a Ridge regression model (L2 regularization). `alpha` controls the strength of regularization.
 - `.fit(X, y)`: Fits the model.
 - `.predict(X_new)`: Predicts.
 - `.coef_`, `.intercept_`, `.score(X, y)`: Similar to `LinearRegression`.
- `sklearn.linear_model.Lasso`:
 - `Lasso(alpha=1.0)`: Initializes a Lasso regression model (L1 regularization). `alpha` controls the strength of regularization.

- `.fit(X, y)`: Fits the model.
- `.predict(X_new)`: Predicts.
- `.coef_`, `.intercept_`, `.score(X, y)`: Similar to `LinearRegression`.
- `sklearn.linear_model.ElasticNet`:
 - `ElasticNet(alpha=1.0, l1_ratio=0.5)`: Initializes an Elastic Net regression model (combination of L1 and L2 regularization). `l1_ratio` controls the mix between L1 and L2.
 - `.fit(X, y)`: Fits the model.
 - `.predict(X_new)`: Predicts.
- `sklearn.linear_model.BayesianRidge`:
 - `BayesianRidge()`: Initializes a Bayesian Ridge regression model.
- `sklearn.linear_model.SGDRegressor`:
 - `SGDRegressor()`: Implements linear regression with stochastic gradient descent.
- `sklearn.ensemble.RandomForestRegressor`:
 - `RandomForestRegressor(n_estimators=100)`: Initializes a Random Forest Regressor.
- `sklearn.ensemble.GradientBoostingRegressor`:
 - `GradientBoostingRegressor()`: Initializes a Gradient Boosting Regressor.
- `sklearn.tree.DecisionTreeRegressor`:
 - `DecisionTreeRegressor()`: Initializes a Decision Tree Regressor.
- `sklearn.svm.SVR`:
 - `SVR()`: Initializes a Support Vector Regressor.

Helper Functions for Model Building and Evaluation:

- `sklearn.model_selection.train_test_split(X, y, test_size=0.2, random_state=42)`: Splits data into training and testing sets.
- `sklearn.preprocessing.PolynomialFeatures(degree=2)`: Generates polynomial features for polynomial regression.
- `sklearn.pipeline.make_pipeline(*steps)`: Creates a pipeline for chaining multiple processing steps and models.
- `sklearn.metrics.mean_squared_error(y_true, y_pred)`: Calculates Mean Squared Error.
- `sklearn.metrics.r2_score(y_true, y_pred)`: Calculates the R2 score.
- `sklearn.metrics.mean_absolute_error(y_true, y_pred)`: Calculates Mean Absolute Error.
- `sklearn.model_selection.GridSearchCV`: For hyperparameter tuning.
- `sklearn.model_selection.KFold`, `sklearn.model_selection.StratifiedKFold`: For cross-validation.

This page titled [12.1: Functions for Model Building and Regression](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

12.2: Linear Regression

Introduction to Linear Regression

Building on our discussion of optimization and parameter estimation, let's delve into a widely used statistical method for modeling the relationship between variables: **Linear Regression**. While the "Introduction to Optimization" Canvas focused on nonlinear functions, linear regression is a foundational concept that often serves as a starting point for understanding how models are fitted to data.

What is Linear Regression?

Linear regression is a statistical method used to model the relationship between a dependent variable (often denoted as Y) and one or more independent variables (often denoted as X). The core idea is to find the "best-fitting" straight line (or a hyperplane in higher dimensions) that describes how the dependent variable changes as the independent variable(s) change.

The term "linear" refers to the fact that the model assumes a linear relationship between the parameters and the dependent variable. Even if the independent variables are transformed (e.g., X^2 , $\log(X)$), as long as the parameters are multiplied by these terms and added together, the model remains linear in its parameters.

Simple Linear Regression

The simplest form is simple linear regression, which involves one independent variable and one dependent variable. The mathematical model for simple linear regression is:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y is the dependent variable (the outcome you are trying to predict).
- X is the independent variable (the predictor).
- β_0 (beta-naught) is the **Y-intercept**, representing the expected value of Y when X is 0.
- β_1 (beta-one) is the **slope** of the line, representing the change in Y for a one-unit change in X .
- ϵ (epsilon) is the **error term** (or residual), representing the difference between the observed Y value and the predicted Y value. This accounts for variability not explained by the model.

Multiple Linear Regression

When there are two or more independent variables, it's called multiple linear regression:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Here, β_i represents the change in Y for a one-unit change in X_i , holding all other independent variables constant.

How are the Parameters (β values) Estimated?

The goal of linear regression is to find the values for the parameters ($\beta_0, \beta_1, \dots, \beta_n$) that minimize the sum of the squared differences between the observed values of Y and the values predicted by the linear model. This method is known as the **Ordinary Least Squares (OLS)** method.

The "error" or residual for each data point is $e_i = Y_i - \hat{Y}_i$, where $\hat{Y}_i$ is the predicted value. The objective function to minimize is the sum of squared residuals:

$$\text{Least Squares Minimization of } \sum_{i=1}^m (Y_i - (\beta_0 + \beta_1 X_{i1} + \dots + \beta_n X_{in}))^2$$

Unlike nonlinear regression, the parameters in linear regression can often be estimated directly using a closed-form mathematical solution (matrix algebra), although iterative optimization algorithms can also be used, especially for very large datasets.

Applications of Linear Regression

Linear regression is widely used in various fields for:

- **Prediction:** Forecasting future values (e.g., predicting house prices based on size and location).
- **Understanding Relationships:** Determining the strength and direction of the relationship between variables (e.g., how advertising spend affects sales).

PYTHON SESSION: READY

- **Trend Analysis:** Identifying trends over time.

While linear regression is powerful, it's important to remember its assumptions (e.g., linearity of relationship, independence of errors, homoscedasticity) and to use it appropriately.

Linear Regression

Linear Regression with SciPy

The basis for linear least squares regression is finding the line of best fit for a set of data points by minimizing the sum of the squares of the vertical distances (residuals or errors) between the observed data points and the predicted values from the line. Linear least squares regression models the relationship between a dependent variable (what you're trying to predict) and one or more independent variables (the predictors) by fitting a linear equation to the observed data. The "best fit" is determined by minimizing a specific error criterion. A residual (or error) is the difference between an observed value y_i of the dependent variable and the predicted value $\hat{y}_i$ from the linear regression line for a given independent variable value x_i .

$$e_i = y_i - \hat{y}_i$$

Example

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Generate sample data
#np.random.seed(42)
slope_act, intercept_act = 2, 4
sigma = 3
x_max, ndata = 5, 50
x = np.linspace(0, x_max, ndata)
# Generate y with some noise
y = slope_act * x + intercept_act + np.random.normal(0, sigma, ndata)

# 2. Perform linear regression using scipy.stats.linregress
slope, intercept, r_value, p_value, stderr_slope = stats.linregress(x, y)

# 3. Calculate the predicted y values
y_pred = slope * x + intercept

# Calculate the standard error of the regression estimate (s_y_x)
# This is the standard deviation of the residuals
residuals = y - y_pred
s_y_x = np.sqrt(np.sum(residuals**2) / (len(x) - 2))

# Calculate the standard error of the mean predicted value for each x
# This is an approximation for simplicity, a more rigorous approach
# involves the full covariance matrix and Student's t-distribution for confidence intervals
# For a given x_i, the standard error of the predicted mean is:
# SE_pred = s_y_x * sqrt(1/n + (x_i - mean(x))^2 / sum((x_j - mean(x))^2))
mean_x = np.mean(x)
sum_sq_diff_x = np.sum((x - mean_x)**2)
se_pred_mean = s_y_x * np.sqrt(1/len(x) + (x - mean_x)**2 / sum_sq_diff_x)

# Determine the critical t-value for the desired confidence level
# F interval and n-2 degrees of freedom
alpha = 0.05
```

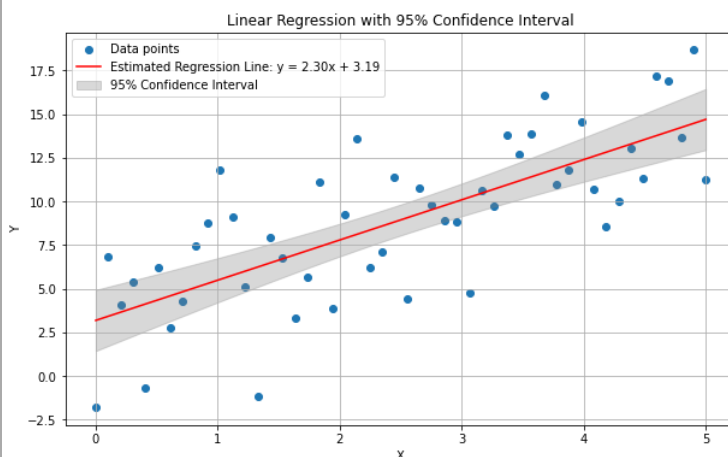
PYTHON SESSION: READY

```
degrees_freedom = len(x) - 2
t_critical = stats.t.ppf(1 - alpha/2, degrees_freedom)

# Calculate the upper and lower confidence bounds
upper_bound = y_pred + t_critical * se_pred_mean
lower_bound = y_pred - t_critical * se_pred_mean

# Plot the data, regression line, and confidence intervals
plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Data points')
plt.plot(x, y_pred, color='red', label=f'Estimated Regression Line: y = {slope:.2f}x - {intercept:.2f}')
plt.fill_between(x, lower_bound, upper_bound, color='gray', alpha=0.3, label='95% Confidence Interval')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Regression with 95% Confidence Interval')
plt.legend()
plt.grid(True)
plt.show()
```

run restart restart & run all



Minimizing the Sum of Squared Residuals (SSR or SSE)

To ensure that larger errors have a proportionally greater impact on the minimization, linear least squares regression squares each residual and then sums them up. The method then finds the line that makes this sum of squared residuals (SSR) or sum of squared errors (SSE) as small as possible.

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The model for a simple linear regression with one independent variable) is typically represented by the equation of a straight line:

$$\hat{y} = \beta_0 + \beta_1 x$$

The process of linear least squares regression involves calculating the specific values for β_0 and β_1 that minimize the sum of squared residuals for the given dataset. The following example uses the `linregress` function for the `scipy.stats` library to obtain the estimated slope, intercept, r value, p value of the slope, standard error for the slope and the standard error for the intercept.

Example Linear Ordinary Least Squares Regression with Scikit-learn (sklearn)

`LinearRegression` fits a linear model with p coefficients, $w = (w_1, w_2, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation. Mathematically it solves a problem

PYTHON SESSION: READY

$\sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \bar{X})^2}$

with (X) independent variables and (y) dependent variables. The least squares solution is computed using the singular value decomposition of (X) . If (X) is a matrix of shape $(N_{\text{samples}}, N_{\text{features}})$ this method has a cost of $O(N_{\text{samples}} N_{\text{features}}^2)$, assuming that $N_{\text{samples}} \geq N_{\text{features}}$

The Scikit-learn library provides a machine learning approach for linear regression which can be applied to either single or multi dimensional set of data from independent variables. It provides a robust method to perform linear regression in Python.

This example will cover:

1. **Generating Sample Data:** Creating synthetic data that has a linear relationship.
2. **Splitting Data:** Dividing the data into training and testing sets.
3. **Model Training:** Fitting a LinearRegression model to the training data.
4. **Prediction:** Making predictions on the test data.
5. **Evaluation:** Assessing the model's performance.
6. **Visualization:** Plotting the results to understand the linear fit.

Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
np.random.seed(42) # for reproducibility
slope = 3
intercept = 4
X = 2 * np.random.rand(100, 1) # 100 samples, 1 feature (x values)
y = intercept + slope * X + np.random.randn(100, 1) # y values with some noise
# Split data into training and testing dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = linear_model.LinearRegression()
# Train the model using the training data
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"\nModel Coefficient (Slope): {model.coef_[0][0]:.2f}")
print(f"Model Intercept: {model.intercept_[0]:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared (R2 Score): {r2:.2f}")
plt.figure(figsize=(10, 7))
# Plot the original training data
plt.scatter(X_train, y_train, color='blue', label='Training Data', alpha=0.7)
# Plot the original test data
plt.scatter(X_test, y_test, color='green', label='Test Data', alpha=0.7)
# Plot the regression line using the test data's X values and the model's predictions
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line (Prediction)')
plt.plot([], [], ' ', label=f'Slope: {model.coef_[0][0]:.2f}, Intercept: {model.intercept_[0]:.2f}')
plt.title('Linear Ordinary Least Squares Regression Fit with sklearn on Synthetic Data')
plt.xlabel('X (Feature)')
plt.ylabel('y (Target)')
plt
```

PYTHON SESSION: READY

```
plt.grid(True)
plt.show()
```

run

restart

restart & run all

```
Requirement already satisfied: scikit-learn in
/srv/conda/envs/notebook/lib/python3.8/site-packages (1.3.2)
Requirement already satisfied: scipy>=1.5.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.9.3)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (3.5.0)
Requirement already satisfied: joblib>=1.1.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.4.2)
Requirement already satisfied: numpy<2.0,>=1.17.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.23.5)
Note: you may need to restart the kernel to use updated packages.

Model Coefficient (Slope): 2.80
Model Intercept: 4.14
Mean Squared Error (MSE): 0.65
R-squared (R2 Score): 0.81
```

Multiple Linear Regression

Multiple linear regression is a statistical technique used to model the relationship between a dependent variable and two or more independent variables/features, $\backslash(N_{\text{features}} \backslash \geq 2\backslash)$, by fitting a linear equation to observed data. In Python, this can be efficiently implemented using libraries like **scikit-learn** or **statsmodels**.

For reliable results, multiple linear regression relies on several assumptions:

- No Significant Multicollinearity: Independent variables are not highly correlated with each other. High multicollinearity can make it difficult to determine the individual impact of each predictor.
- Linearity: There's a linear relationship between the dependent variable and each independent variable.
- Independence of Errors: The errors (residuals) are independent of each other.
- Homoscedasticity: The variance of the errors is constant across all levels of the independent variables.
- Normality of Errors: The errors are normally distributed.

The least squares regression function **sklearn.linear_model.LinearRegression** is demonstrated in the following example with home prices as a function of square foot space and number of bedrooms.

Multiple Linear Regression Example 1

```
import numpy as np
from sklearn import linear_model
import pandas as pd # Often used for handling multiple independent variables

# Sample Data Generation
# Let's consider a scenario where 'y' (house price) depends on 'X1' (size in sq ft)
# and 'X2' (number of bedrooms).
```

PYTHON SESSION: READY


```
data = {
    'Size_sqft': [1000, 1200, 1500, 1300, 1100, 1600, 1400, 1700, 1800, 1900],
    'Num_Bedrooms': [2, 3, 3, 2, 2, 4, 3, 4, 4, 5],
    'House_Price_k': [250, 300, 380, 320, 280, 420, 350, 450, 480, 520]
}
df = pd.DataFrame(data)

# Define independent variables (features) and dependent variable (target)
X_multi = df[['Size_sqft', 'Num_Bedrooms']]
y_multi = df['House_Price_k']

# Model Initialization and Training
model_multi = linear_model.LinearRegression()
model_multi.fit(X_multi, y_multi)

# Accessing Model Parameters
# Each coefficient corresponds to its respective independent variable.
print(f"Coefficients (slopes) for Size_sqft and Num_Bedrooms: {model_multi.coef_}")
print(f"Intercept: {model_multi.intercept_:.2f}")

# Making Predictions
# Predict the price of a house with 1450 sq ft and 3 bedrooms.
new_house = pd.DataFrame([[1450, 3]], columns=['Size_sqft', 'Num_Bedrooms'])
predicted_price = model_multi.predict(new_house)
print(f"Predicted price for a 1450 sqft, 3-bedroom house: ${predicted_price[0]:.2f}k")
```

run

restart

restart & run all

Collecting scikit-learn

Downloading scikit_learn-1.3.2-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.1 MB)
11.1/11.1 MB 84.9 MB/s

eta 0:00:0000:0100:01

Collecting threadpoolctl>=2.0.0

Downloading threadpoolctl-3.5.0-py3-none-any.whl (18 kB)

Collecting joblib>=1.1.1

Downloading joblib-1.4.2-py3-none-any.whl (301 kB)
301.8/301.8 kB 59.3 MB/s

eta 0:00:00

Requirement already satisfied: numpy<2.0,>=1.17.3 in /srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-learn) (1.23.5)

Requirement already satisfied: scipy>=1.5.0 in /srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-learn) (1.9.3)

Installing collected packages: threadpoolctl, joblib, scikit-learn
Successfully installed joblib-1.4.2 scikit-learn-1.3.2 threadpoolctl-3.5.0

Note: you may need to restart the kernel to use updated packages.

PYTHON SESSION: READY

es) for Size_sqft and Num_Bedrooms: [0.26103448

```
11.59482759]
Intercept: -40.60
Predicted price for a 1450 sqft, 3-bedroom house: $372.68k
```

Multiple Linear Regression Example 2

Compare scikit-learn with statsmodels.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.api as sm

data = {
    'TV': [230.1, 44.5, 17.2, 151.5, 180.8, 8.7, 57.5, 120.2, 8.6, 199.8],
    'Radio': [37.8, 39.3, 45.9, 41.3, 10.8, 48.9, 32.8, 19.6, 2.1, 2.6],
    'Newspaper': [69.2, 45.1, 69.3, 58.5, 58.4, 75.0, 23.5, 11.6, 1.0, 21.2],
    'Sales': [22.1, 10.4, 9.3, 18.5, 12.9, 7.2, 11.8, 13.2, 4.8, 10.6]
}
df = pd.DataFrame(data)
print("Sample Data:")
print(df)
# Define independent variables (X) and dependent variable (y)
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

# Split data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Create a Linear Regression model object
model_skl = LinearRegression()

# Train the model using the training data
model_skl.fit(X_train, y_train)

# Make predictions on the test set
y_pred_skl = model_skl.predict(X_test)

# Evaluate the model's performance
mse_skl = mean_squared_error(y_test, y_pred_skl)
r2_skl = r2_score(y_test, y_pred_skl)

print("\n--- Scikit-learn Results ---")
print(f"Coefficients ( $\beta_1$ ,  $\beta_2$ ,  $\beta_3$  for TV, Radio, Newspaper respectively): {model_skl.coef_}")
print(f"Intercept ( $\beta_0$ ): {model_skl.intercept_:.2f}")
print(f"Mean Squared Error (MSE): {mse_skl:.2f}")
print(f"R-squared ( $R^2$ ): {r2_skl:.2f}")

# Define independent variables (X) and dependent variable (y)
# For statsmodels, it's good practice to add a constant term for the intercept
X_sm = sm.add_constant(df[['TV', 'Radio', 'Newspaper']])
y_sm = y
```

PYTHON SESSION: READY

```
# Create and fit the OLS (Ordinary Least Squares) model
model_sm = sm.OLS(y_sm, X_sm).fit()

# Print the comprehensive summary of the regression results
print("\n--- Statsmodels Results ---")
print(model_sm.summary())
```

run

restart

restart & run all

```
Requirement already satisfied: scikit-learn in
/srv/conda/envs/notebook/lib/python3.8/site-packages (1.3.2)
Collecting statsmodels
  Downloading statsmodels-0.14.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (10.9 MB)
      10.9/10.9 MB 92.7 MB/s
eta 0:00:0000:0100:01
Requirement already satisfied: numpy<2.0,>=1.17.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.23.5)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (3.5.0)
Requirement already satisfied: scipy>=1.5.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.9.3)
Requirement already satisfied: joblib>=1.1.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.4.2)
Requirement already satisfied: pandas!=2.1.0,>=1.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
statsmodels) (1.5.2)
Collecting patsy>=0.5.4
  Downloading patsy-1.0.1-py2.py3-none-any.whl (232 kB)
      232.9/232.9 kB 41.9 MB/s
eta 0:00:00
Requirement already satisfied: packaging>=21.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
statsmodels) (24.0)
Requirement already satisfied: python-dateutil>=2.8.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
pandas!=2.1.0,>=1.0->statsmodels) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
pandas!=2.1.0,>=1.0->statsmodels) (2024.1)
Requirement already satisfied: six>=1.5 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from python-
dateutil>=2.8.1->pandas!=2.1.0,>=1.0->statsmodels) (1.16.0)
Installing collected packages: patsy, statsmodels
Successfully installed patsy-1.0.1 statsmodels-0.14.1
```

PYTHON SESSION: READY

Note: you may need to restart the kernel to use updated packages.

Sample Data:

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9
5	8.7	48.9	75.0	7.2
6	57.5	32.8	23.5	11.8
7	120.2	19.6	11.6	13.2
8	8.6	2.1	1.0	4.8
9	199.8	2.6	21.2	10.6

--- Scikit-learn Results ---

Coefficients (β_1 , β_2 , β_3 for TV, Radio, Newspaper respectively): [0.07468737 0.3029154 -0.06925603]

Intercept (β_0): -1.55

Mean Squared Error (MSE): 13.20

R-squared (R^2): -0.68

--- Statsmodels Results ---

OLS Regression Results

```

=====
Dep. Variable:          Sales    R-squared:
0.909
Model:                  OLS      Adj. R-squared:
0.864
Method:                 Least Squares    F-statistic:
20.07
Date:                   Wed, 16 Jul 2025    Prob (F-statistic):
0.00157
Time:                   15:59:36    Log-Likelihood:
-17.930
No. Observations:       10    AIC:
43.86
Df Residuals:           6    BIC:
45.07
Df Model:                3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025
const	-1.8555	1.589	1.168	0.287	-2.032

PYTHON SESSION: READY

```
TV          0.0633    0.009    7.165    0.000    0.042
0.085
Radio       0.2374    0.062    3.839    0.009    0.086
0.389
Newspaper   -0.0671    0.039    -1.699    0.140    -0.164
0.030
=====
```

```
Omnibus:                2.174    Durbin-Watson:
2.204
Prob(Omnibus):          0.337    Jarque-Bera (JB):
0.988
Skew:                   -0.762    Prob(JB):
0.610
Kurtosis:               2.783    Cond. No.
366.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretation of scikit-learn Results:

- Coefficients: These values indicate the change in 'Sales' for a one-unit increase in the respective advertising medium, holding other variables constant. For example, `model_skl.coef_[0]` is the coefficient for 'TV'.
- Intercept: This is the predicted 'Sales' when all independent variables ('TV', 'Radio', 'Newspaper') are zero.
- Mean Squared Error (MSE): This measures the average of the squares of the errors or deviations. A lower MSE indicates a better fit.
- R-squared (R^2): This represents the proportion of the variance in the dependent variable ('Sales') that is predictable from the independent variables. An R^2 of 1 indicates a perfect fit, while 0 indicates no linear relationship.

Interpretation of statsmodels Results:

The statsmodels summary is rich with information:

- Coefficients (coef): Same as in scikit-learn, these are the estimated beta values.
- std err (Standard Error): Measures the precision of the coefficient estimates. Smaller standard errors indicate more precise estimates.
- t and P>|t| (t-value and P-value):
 - The t-value is the coefficient divided by its standard error.
 - The P-value ($P>|t|$) indicates the probability of observing such a t-value if the true coefficient were zero (i.e., no relationship). A common threshold for statistical significance is $P\text{-value} < 0.05$. If the P-value is less than 0.05, we typically consider the independent variable to be a statistically significant predictor of the dependent variable.
- [0.025, 0.975] (Confidence Interval): Provides a range within which the true coefficient is likely to fall with 95% confidence.
- R-squared and Adj. R-squared: Similar to scikit-learn, they tell us how much variance in 'Sales' is explained by the independent variables. Adjusted R-squared is generally preferred when comparing models with different numbers of independent variables, as it accounts for the number of predictors.
- F-statistic and Prob (F-statistic): These assess the overall statistical significance of the entire regression model. A low Prob (F-statistic) (e.g., < 0.05) suggests that at least one of the independent variables significantly predicts the dependent variable.

PYTHON SESSION: READY

Both scikit-learn and statsmodels are valuable tools for multiple linear regression in Python, serving different primary purposes: scikit-learn for predictive modeling and statsmodels for statistical inference and hypothesis testing.

Example 3: Predicting Concrete Compressive Strength

Concrete strength is a crucial factor in civil engineering. It's influenced by the mix proportions of its ingredients. The following are variables for this model:

- Dependent Variable (Y): Concrete Compressive Strength (MPa)
- Independent Variables (X):
 - Cement (kg/m³)
 - Blast Furnace Slag (kg/m³)
 - Fly Ash (kg/m³)
 - Water (kg/m³)
 - Superplasticizer (kg/m³)
 - Coarse Aggregate (kg/m³)
 - Fine Aggregate (kg/m³)
 - Age (days)

Since actual concrete strength data can be extensive, a small, representative synthetic dataset is created for this simulation. In a real engineering scenario, this data would come from laboratory experiments or field measurements.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.api as sm

# Synthetic Data for Concrete Compressive Strength
# (Actual datasets are much larger and more complex)
np.random.seed(42) # for reproducibility
data = {
    'Cement': np.random.uniform(100, 500, 20),
    'BlastFurnaceSlag': np.random.uniform(0, 300, 20),
    'FlyAsh': np.random.uniform(0, 200, 20),
    'Water': np.random.uniform(150, 250, 20),
    'Superplasticizer': np.random.uniform(0, 20, 20),
    'CoarseAggregate': np.random.uniform(800, 1200, 20),
    'FineAggregate': np.random.uniform(600, 900, 20),
    'Age': np.random.randint(7, 90, 20),
    # Compressive Strength - simplified linear relationship + noise
    'CompressiveStrength': (
        5 + 0.05 * np.random.uniform(100, 500, 20) + # Cement
        0.03 * np.random.uniform(0, 300, 20) +      # Slag
        0.02 * np.random.uniform(0, 200, 20) -      # Fly Ash (slight negative impact
        0.1 * np.random.uniform(150, 250, 20) +      # Water (negative impact)
        0.2 * np.random.uniform(0, 20, 20) +         # Superplasticizer
        0.01 * np.random.uniform(800, 1200, 20) +    # Coarse Aggregate
        0.01 * np.random.uniform(600, 900, 20) +    # Fine Aggregate
        0.3 * np.random.randint(7, 90, 20) +         # Age
        np.random.normal(0, 5, 20) # Noise
    )
}
df = pd.DataFrame(data)
```

PYTHON SESSION: READY

```
# Ensure strength is positive and somewhat realistic
df['CompressiveStrength'] = df['CompressiveStrength'].apply(lambda x: max(10, x)).round(1)

#print("Sample Concrete Compressive Strength Data:")
#print(df.head())
# Define independent variables (X) and dependent variable (y)
X = df[['Cement', 'BlastFurnaceSlag', 'FlyAsh', 'Water',
        'Superplasticizer', 'CoarseAggregate', 'FineAggregate', 'Age']]
y = df['CompressiveStrength']

# Split data into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a Linear Regression model object
model_skl = LinearRegression()

# Train the model using the training data
model_skl.fit(X_train, y_train)

# Make predictions on the test set
y_pred_skl = model_skl.predict(X_test)

# Evaluate the model's performance
mse_skl = mean_squared_error(y_test, y_pred_skl)
rmse_skl = np.sqrt(mse_skl) # Root Mean Squared Error
r2_skl = r2_score(y_test, y_pred_skl)

print("\n--- Scikit-learn Results (for Prediction) ---")
print("Coefficients for each constituent (Cement to Age):")
for i, col in enumerate(X.columns):
    print(f"    {col}: {model_skl.coef_[i]:.4f}")
print(f"Intercept: {model_skl.intercept_:.2f}")
print(f"Mean Squared Error (MSE): {mse_skl:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_skl:.2f}") # Easier to interpret than MSE
print(f"R-squared (R²): {r2_skl:.2f}")

# Example prediction: Predict strength for a new mix
new_mix = pd.DataFrame([[350, 50, 20, 180, 5, 1000, 700, 28]],
                        columns=X.columns)
predicted_strength = model_skl.predict(new_mix)
print(f"\nPredicted Compressive Strength for a new mix: {predicted_strength[0]:.2f} MPa")

# statsmodels analysis
# Add a constant (intercept) term to the independent variables
X_sm = sm.add_constant(df[['Cement', 'BlastFurnaceSlag', 'FlyAsh', 'Water',
                           'Superplasticizer', 'CoarseAggregate', 'FineAggregate', 'Age']])
y_sm = df['CompressiveStrength']

# Create and fit the OLS (Ordinary Least Squares) model
model_sm = sm.OLS(y_sm, X_sm).fit()
```

PYTHON SESSION: READY

```
# Print the comprehensive summary of the regression results
print("\n--- Statsmodels Results (for Statistical Inference) ---")
print(model_sm.summary())
```

run

restart

restart & run all

```
Collecting scikit-learn
  Downloading scikit_learn-1.3.2-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.1 MB)
      11.1/11.1 MB 74.5 MB/s
eta 0:00:0000:010:01
Collecting statsmodels
  Downloading statsmodels-0.14.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (10.9 MB)
      10.9/10.9 MB 103.0 MB/s
eta 0:00:0000:0100:01
Requirement already satisfied: scipy>=1.5.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.9.3)
Requirement already satisfied: numpy<2.0,>=1.17.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from scikit-
learn) (1.23.5)
Collecting joblib>=1.1.1
  Downloading joblib-1.4.2-py3-none-any.whl (301 kB)
      301.8/301.8 kB 63.9 MB/s
eta 0:00:00
Collecting threadpoolctl>=2.0.0
  Downloading threadpoolctl-3.5.0-py3-none-any.whl (18 kB)
Requirement already satisfied: pandas!=2.1.0,>=1.0 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
statsmodels) (1.5.2)
Requirement already satisfied: packaging>=21.3 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
statsmodels) (24.0)
Collecting patsy>=0.5.4
  Downloading patsy-1.0.1-py2.py3-none-any.whl (232 kB)
      232.9/232.9 kB 52.2 MB/s
eta 0:00:00
Requirement already satisfied: python-dateutil>=2.8.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
pandas!=2.1.0,>=1.0->statsmodels) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from
pandas!=2.1.0,>=1.0->statsmodels) (2024.1)
Requirement already satisfied: six>=1.5 in
/srv/conda/envs/notebook/lib/python3.8/site-packages (from python-
dateutil>=2.8.1->pandas!=2.1.0,>=1.0->statsmodels) (1.16.0)
Installing collected packages: threadpoolctl, patsy, joblib, scikit-
```

PYTHON SESSION: READY

Successfully installed joblib-1.4.2 patsy-1.0.1 scikit-learn-1.3.2
statsmodels-0.14.1 threadpoolctl-3.5.0
Note: you may need to restart the kernel to use updated packages.

```
--- Scikit-learn Results (for Prediction) ---
Coefficients for each constituent (Cement to Age):
Cement: 0.0112
BlastFurnaceSlag: -0.0080
FlyAsh: 0.0179
Water: 0.0435
Superplasticizer: 0.3829
CoarseAggregate: 0.0161
FineAggregate: 0.0186
Age: 0.0564
Intercept: -7.78
Mean Squared Error (MSE): 117.65
Root Mean Squared Error (RMSE): 10.85
R-squared (R2): -0.45
```

Predicted Compressive Strength for a new mix: 36.53 MPa

```
--- Statsmodels Results (for Statistical Inference) ---
OLS Regression Results
```

```
=====
Dep. Variable:          CompressiveStrength    R-squared:
0.227
Model:                  OLS                  Adj. R-squared:
-0.335
Method:                 Least Squares        F-statistic:
0.4044
Date:                  Wed, 16 Jul 2025      Prob (F-statistic):
0.896
Time:                  18:37:33              Log-Likelihood:
-70.722
No. Observations:      20                   AIC:
159.4
Df Residuals:          11                   BIC:
168.4
Df Model:              8
Covariance Type:       nonrobust
=====
```

```
=====
               coef      std err          t      P>|t|
-----
const         -4.1808      38.362      -0.109      0.915
=====
```

PYTHON SESSION: READY

4

Cement	0.0047	0.031	0.152	0.882
-0.063	0.072			
BlastFurnaceSlag	0.0020	0.042	0.048	0.963
-0.091	0.095			
FlyAsh	0.0171	0.045	0.382	0.710
-0.081	0.115			
Water	0.0928	0.091	1.015	0.332
-0.108	0.294			
Superplasticizer	0.5224	0.527	0.992	0.343
-0.637	1.682			
CoarseAggregate	0.0210	0.023	0.928	0.373
-0.029	0.071			
FineAggregate	-0.0104	0.042	-0.247	0.810
-0.103	0.083			
Age	0.0921	0.151	0.610	0.555
-0.240	0.425			

Omnibus:	2.366	Durbin-Watson:
1.765		
Prob(Omnibus):	0.306	Jarque-Bera (JB):
1.118		
Skew:	-0.075	Prob(JB):
0.572		
Kurtosis:	1.851	Cond. No.
2.01e+04		

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.01e+04. This might indicate that there are strong multicollinearity or other numerical problems.

Engineering Relevance of scikit-learn Output:

- Coefficients: An engineer can interpret these to understand the relative impact of each ingredient on strength. For example, a positive coefficient for 'Cement' means more cement generally leads to higher strength, while a negative coefficient for 'Water' means more water (assuming all else constant) tends to reduce strength.
- RMSE: Provides an intuitive measure of the typical error in predictions, in the same units as the dependent variable (MPa). An engineer wants to minimize this.
- R²: Indicates how well the model explains the variability in concrete strength. A high R² suggests the model's inputs are good predictors.

Engineering Relevance of statsmodels Output:

- P-values ($P > |t|$): This is perhaps the most critical output for an engineer designing concrete mixes. A low p-value (e.g., < 0.05) for a specific ingredient indicates that its quantity has a statistically significant impact on concrete strength. For instance, if 'Water' has a low p-value, it confirms that controlling the water content is statistically important for strength.

PYTHON SESSION: READY

- Confidence Intervals ([0.025, 0.975]): Provide a range for the true effect of each ingredient. An engineer can use this to understand the variability and robustness of the relationships.
- F-statistic and Prob (F-statistic): These indicate if the overall model is statistically significant. A low p-value here means that the chosen mix of ingredients (as a whole) significantly predicts concrete strength.
- R-squared and Adjusted R-squared: Similar to scikit-learn, but often accompanied by more detailed statistical tests and diagnostics, which are valuable for a deeper understanding of model fit and assumptions.

This engineering example demonstrates how multiple linear regression can be used not just for prediction but also for gaining actionable insights into complex material behaviors, allowing engineers to optimize designs, control quality, and understand the fundamental relationships between input parameters and performance.

Summary

Choosing the right method for linear regression in Python depends on your goals and the level of detail you need. The two most popular libraries for this task are **scikit-learn** and **statsmodels**. There is also a simpler, more direct approach using **NumPy** for basic cases.

Scikit-learn

Scikit-learn is a powerful and widely used machine learning library. It is the go-to choice for building predictive models.

- **When to use it:** Use scikit-learn when your primary goal is to **build a predictive model** and integrate it into a larger machine learning pipeline. It's excellent for tasks like:
 - Making predictions on new data.
 - Evaluating model performance with metrics like R², Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).
 - Performing other machine learning tasks like classification, clustering, and cross-validation, as it has a consistent API.
 - Working with multiple linear regression (multiple independent variables).
- How it works: You use the LinearRegression class from sklearn.linear_model. The process is typically:
 1. Import LinearRegression.
 2. Create an instance of the model: `model = LinearRegression()`.
 3. Fit the model to your data: `model.fit(X, y)`, where X is your feature matrix and y is your target variable.
 4. Access the coefficients (`model.coef_`) and intercept (`model.intercept_`).
 5. Make predictions with `model.predict(X_new)`.

Statsmodels

Statsmodels is a library that focuses on providing statistical models and tests. It is more similar to the output you would get from statistical software like R.

- **When to use it:** Use statsmodels when your focus is on **statistical inference** and understanding the relationships between variables. It provides a detailed summary of the model, which is crucial for:
 - Examining p-values to determine the statistical significance of each independent variable.
 - Checking for multicollinearity and other regression diagnostics.
 - Performing hypothesis testing.
 - Getting a comprehensive summary table with many statistical details.
- How it works: You can use the OLS (Ordinary Least Squares) method from statsmodels.api or statsmodels.formula.api. The formula API is particularly useful as it allows you to specify the model using R-style formulas.
 1. Import statsmodels.api as sm or statsmodels.formula.api as smf.
 2. Add a constant to your independent variables if you are using the sm API: `X = sm.add_constant(X)`.
 3. Define and fit the model: `model = sm.OLS(y, X).fit()`.
 4. Get the detailed summary table with `print(model.summary())`.

NumPy

NumPy is the fundamental package for scientific computing in Python that can also be used for linear regression, especially for simple

PYTHON SESSION: READY

- Use `numpy.linalg.lstsq` for a very simple linear regression problems (e.g., a single predictor variable).

- The `Polynomial.fit` can be used to fit a polynomial (including a degree 1 polynomial) to your data.
- You can also manually calculate the coefficients using linear algebra functions within NumPy.

Summary Table

Method	Best for...	Key Features
scikit-learn	Predictive modeling, machine learning pipelines	Simple API, performance metrics, easy integration with other ML tools.
statsmodels	Statistical inference, hypothesis testing	Detailed summary output, p-values, regression diagnostics, R-style formulas.
NumPy	Simple linear regression, manual calculations	Direct, low-level control, good for understanding the fundamentals.

For most users doing data science and machine learning, **scikit-learn** is the most common and practical choice. If your work is more rooted in classical statistics and you need to formally test hypotheses about your variables, **statsmodels** is the superior option.

Conclusion

Multiple linear regression is a powerful and widely used tool for understanding and predicting relationships between variables. Python's **scikit-learn** provides a straightforward way to build predictive models, while **statsmodels** offers in-depth statistical insights crucial for hypothesis testing and understanding the underlying relationships.

This page titled [12.2: Linear Regression](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

12.3: Optimization - Parameter Estimation for Nonlinear Functions

Introduction to Optimization: Parameter Estimation for Nonlinear Functions

Welcome! This document will introduce you to the fascinating and incredibly useful field of **optimization**, specifically focusing on its application in **parameter estimation for nonlinear functions**. Whether you're analyzing scientific data, designing engineering systems, or building predictive models, understanding these concepts is fundamental to extracting meaningful insights and creating efficient solutions.

What is Optimization?

At its core, **optimization** is the process of finding the "best" solution from a set of available alternatives. In a mathematical context, it typically involves either:

- **Minimizing** an objective function (e.g., minimizing cost, error, or weight).
- **Maximizing** an objective function (e.g., maximizing profit, efficiency, or performance).

This "best" solution is found by systematically adjusting a set of "decision variables" while adhering to various "constraints" that define the boundaries of what is feasible or acceptable.

What is Parameter Estimation?

Parameter estimation is the process of determining the values of parameters within a mathematical model that best describe a given set of observed data. Imagine you have a set of measurements, and you believe there's an underlying mathematical relationship connecting your independent and dependent variables. Parameter estimation helps you find the specific numbers (the parameters) that make your model's predictions align as closely as possible with your actual observations.

Nonlinear optimization is a fundamental tool across many engineering disciplines due to the inherent nonlinearities in real-world systems. Here are several prominent examples:

1. Aerospace Engineering:

- **Aircraft and Rocket Trajectory Optimization:** Calculating the most fuel-efficient flight paths for aircraft or optimal trajectories for rockets and satellites to minimize fuel consumption or maximize payload, often considering complex aerodynamic forces, gravity, and thrust, which are highly nonlinear.
- **Aircraft Wing Design:** Optimizing the shape of an aircraft wing (airfoil) to maximize lift-to-drag ratio at various flight conditions, which involves nonlinear fluid dynamics equations.
- **Satellite Constellation Design:** Determining the optimal number, orbits, and phasing of satellites to provide continuous coverage over a specific area with minimal cost.

2. Mechanical Engineering:

- **Robotics:** Optimizing robot arm movements for speed, energy efficiency, or precision, considering the complex kinematics and dynamics of multi-jointed systems.
- **Engine Design:** Optimizing engine parameters (e.g., fuel injection timing, valve lift) for maximum power output, fuel efficiency, or minimal emissions, where combustion processes and fluid flows are highly nonlinear.
- **Vibration Control:** Designing passive or active vibration damping systems to minimize unwanted oscillations in structures or machinery.
- **Mechanism Design:** Optimizing the dimensions of linkages and gears in mechanical systems to achieve desired motion profiles or force transmission characteristics.

3. Chemical Engineering:

- **Process Optimization:** Maximizing the yield of a chemical reaction, minimizing energy consumption in a separation process (like distillation), or optimizing reactor design parameters (temperature, pressure, catalyst concentration) to improve efficiency and product quality. Reaction kinetics are often nonlinear.
- **Supply Chain Optimization:** Determining optimal production schedules, inventory levels, and transportation routes for chemical products to minimize costs and meet demand, often with nonlinear cost functions or capacity constraints.

4. Electrical Engineering:

- **PYTHON SESSION: READY** Optimizing the geometry of antennas to achieve desired radiation patterns, gain, and impedance matching.

- **Circuit Design:** Minimizing power consumption or maximizing signal integrity in complex electronic circuits, where component behaviors can be nonlinear.
- **Power Systems:** Optimizing power flow in electrical grids to minimize transmission losses, ensure stability, and manage load distribution efficiently.

5. Civil and Structural Engineering:

- **Bridge and Building Design (as discussed previously):** Minimizing material weight while satisfying complex stress, deflection, and stability constraints under various loading conditions.
- **Foundation Design:** Optimizing the dimensions and placement of foundations to minimize settlement and ensure structural stability given nonlinear soil-structure interaction.
- **Water Resources Management:** Optimizing the operation of reservoir systems to meet water demand for irrigation, power generation, and flood control, considering nonlinear flow dynamics and demand patterns.

6. Materials Science and Engineering:

- **Material Design:** Optimizing the composition and microstructure of new materials to achieve desired properties (e.g., strength, conductivity, thermal resistance), often involving complex nonlinear relationships between composition and properties.
- **Manufacturing Process Optimization:** Fine-tuning parameters in manufacturing processes (e.g., temperature profiles in heat treatment, pressure in molding) to minimize defects and maximize product quality.

In all these examples, the objective function (what you want to minimize or maximize) and/or the constraints (the rules that must be followed) involve nonlinear relationships, making nonlinear optimization techniques indispensable.

Why Nonlinear Functions?

Many real-world phenomena exhibit relationships that cannot be accurately described by simple straight lines or linear combinations. In these cases, **nonlinear functions** are essential for capturing the true complexity of the system. A nonlinear function is simply one where the relationship between the independent variables and the dependent variable is not a straight line, and the parameters themselves might be part of exponents, products, or other complex terms.

The Goal: Best Fit Parameters

When we combine optimization with parameter estimation for nonlinear functions, our primary goal is to find the set of parameters for our chosen nonlinear model that provides the "**best fit**" to our observed data. This "best fit" is typically defined by minimizing the difference (or "error") between the values predicted by our model and the actual observed values.

This process often involves an iterative approach, where an optimization algorithm systematically adjusts the parameters, evaluates how well the model fits the data (e.g., by calculating the sum of squared errors), and then continues to refine the parameters until the error is minimized to an acceptable degree. This allows us to derive a robust mathematical representation of the underlying process, enabling predictions, analysis, and informed decision-making.

SciPy Optimize Curve_fit

To find the coefficients for a nonlinear function given independent and dependent data in Python, you should use the `scipy.optimize.curve_fit` function.

The `scipy.optimize.curve_fit` is a powerful function that performs **nonlinear least squares** to fit a user-defined function (your nonlinear model) to data. It works by minimizing the sum of the squared residuals, where a residual is the difference between the observed (dependent) data and the values predicted by your model.

How it Works

1. Define your nonlinear function: You need to express your nonlinear model as a Python function. This function will take the independent variable(s) as the first argument, followed by the parameters (coefficients) you want to fit.
2. Provide your data: You'll supply your independent data (x-values) and dependent data (y-values) to `curve_fit`.
3. Initial guess: `curve_fit` often benefits from an initial guess for the parameters. While optional, a good initial guess can help the optimization algorithm converge more quickly and avoid local minima, especially for complex nonlinear functions.
4. Output: `curve_fit` returns two primary things:
5. At **PYTHON SESSION: READY** values for the parameters (coefficients) of your function.
 - `popt`: An array containing the optimal values for the parameters (coefficients) of your function.

- pcov: The estimated covariance of popt. This matrix can be used to calculate the standard errors of your fitted parameters.

Example Usage

Let's say you want to fit an exponential function of the form $y = e^{bx}$ to some data.

```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# 1. Generate some sample data (for demonstration purposes)
# In a real scenario, you'd load your actual independent and dependent data
x_data = np.linspace(0, 4, 50)
true_a = 2.5
true_b = 1.3
y_data = true_a * np.exp(true_b * x_data) + 10*np.random.normal(0, 0.5, x_data.size)

# 2. Define your nonlinear function
def exponential_function(x, a, b):
    return a * np.exp(b * x)

# 3. Use curve_fit to find the coefficients
# p0 is an optional initial guess for the parameters [a, b]
initial_guess = [1.0, 1.0]
popt, pcov = curve_fit(exponential_function, x_data, y_data, p0=initial_guess)

# Extract the optimized coefficients
a_fit, b_fit = popt

print(f"Fitted 'a' coefficient: {a_fit:.4f}")
print(f"Fitted 'b' coefficient: {b_fit:.4f}")
print(f"Covariance matrix diagonal values: {np.diag(pcov)}")

# 4. Plot the original data and the fitted curve
plt.figure(figsize=(8, 6))
plt.scatter(x_data, y_data, label='Original Data')
plt.plot(x_data, exponential_function(x_data, a_fit, b_fit), color='red', label='Fitted Curve')
plt.title('Nonlinear Curve Fitting with scipy.optimize.curve_fit')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()
```

runrestartrestart & run all

```
Fitted 'a' coefficient: 2.7119
Fitted 'b' coefficient: 1.2775
Covariance matrix diagonal values: [0.01276081 0.0001289 ]
```

PYTHON SESSION: READY

Considerations

- **Model Selection:** The most critical step is choosing the correct nonlinear function (model) that best describes the relationship between your independent and dependent variables. Incorrect model selection will lead to poor fits.
- **Initial Guesses (p_0):** For some complex models, providing a reasonable p_0 (initial guess for the parameters) is crucial for `curve_fit` to converge to the correct solution. If `curve_fit` struggles, try different initial guesses or visualize your data to estimate starting values.
- **Bounds:** You can also specify bounds for your parameters in `curve_fit` if you know that certain coefficients must fall within a specific range. This can help guide the optimization.
- **Error Analysis:** The `pcov` output allows you to estimate the uncertainties of your fitted parameters, which is important for understanding the reliability of your model. A covariance matrix with large diagonal terms is an indication of an incorrect model for the data or too many terms in the model.

This page titled [12.3: Optimization - Parameter Estimation for Nonlinear Functions](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

12.4: Summary

Linear Regression and Model Building

Linear regression is a fundamental supervised learning algorithm used for modeling the relationship between a dependent variable and one or more independent variables by fitting a linear equation to the observed data.

Linear Regression

At its core, linear regression assumes a linear relationship between the input variables (features) and the single output variable (target). The goal is to find the best-fitting straight line (or hyperplane in higher dimensions) that minimizes the sum of the squared differences between the observed and predicted values.

The general form of a linear regression equation is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon \quad (12.4.1)$$

Where:

- Y is the dependent variable (the target we want to predict).
- $X_1, X_2, \dots, X_n$ are the independent variables (features).
- β_0 is the y-intercept (the value of Y when all X 's are zero).
- $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients (slopes) for each independent variable, representing the change in Y for a one-unit change in the corresponding X , holding other X 's constant.
- ϵ is the error term, representing the irreducible error in the model.

The objective of training a linear regression model is to estimate the coefficients (β values) that best fit the data. This is typically done using the Ordinary Least Squares (OLS) method, which minimizes the sum of the squared residuals (the differences between actual and predicted Y values).

Model Building Process

Building a robust predictive model, including linear regression, generally follows these steps:

1. Data Collection and Preparation:

- **Gather Data:** Obtain relevant datasets.
- **Feature Engineering:** Create new features or transform existing ones to improve model performance.
- **Handle Missing Values:** Impute or remove missing data points.
- **Outlier Detection and Treatment:** Identify and manage extreme values that can disproportionately affect the model.
- **Feature Scaling:** Standardize or normalize features, especially important for algorithms sensitive to feature scales (though less critical for basic linear regression, it's good practice for many ML models).

2. Data Splitting:

- Divide your dataset into training and testing sets (e.g., 70-80% for training, 20-30% for testing). The training set is used to train the model, and the test set is used to evaluate its performance on unseen data.

3. Model Training:

- Select a linear regression algorithm (e.g., from Scikit-learn, Statsmodels).
- Fit the model to the training data, allowing it to learn the coefficients.

4. Model Evaluation:

- Assess the model's performance on the test set using various metrics. Common metrics for regression include:
 - **Mean Absolute Error (MAE):** Average of the absolute differences between predictions and actual values.
 - **Mean Squared Error (MSE):** Average of the squared differences. Penalizes larger errors more.
 - **Root Mean Squared Error (RMSE):** Square root of MSE, in the same units as the target variable.
 - **R-squared (R2):** Represents the proportion of the variance in the dependent variable that is predictable from the independent variables. A higher R2 indicates a better fit.

5. Prediction:

- Once satisfied with the model's performance, use it to make predictions on new, unseen data.

Python Implementations

Python offers several powerful libraries for performing linear regression, each with its own strengths:

- **NumPy**: Provides basic array operations and can be used for manual implementation of linear regression, especially for simple cases or understanding the underlying math.
- **SciPy**: Offers scientific computing tools, including a `linregress` function for simple linear regression (one independent variable).
- **Scikit-learn**: The go-to library for machine learning. Provides a robust `LinearRegression` class for simple and multiple linear regression, focusing on prediction performance.
- **Statsmodels**: A library for statistical modeling and econometric analysis. It provides more detailed statistical output, including p-values, confidence intervals, and various statistical tests, making it suitable for statistical inference and understanding the significance of variables.

This page titled [12.4: Summary](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

CHAPTER OVERVIEW

13: Statistics, Probability, and Interpolation

- [13.1: Statistics and Histograms](#)
- [13.2: Normal Distribution](#)
- [13.3: Random Number Generation](#)
- [13.4: Interpolation](#)
- [13.5: Summary](#)

This page titled [13: Statistics, Probability, and Interpolation](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

13.1: Statistics and Histograms

Introduction to Statistics and Histograms

This document provides an introduction to the fundamental concepts of statistics and a widely used tool for data visualization: histograms. Understanding these topics is crucial for anyone working with data, as they form the bedrock of data analysis and interpretation.

Introduction to Statistics

Statistics is the science of collecting, organizing, analyzing, interpreting, and presenting data. It provides the tools and methods necessary to make sense of information, draw meaningful conclusions, and make informed decisions in the face of uncertainty.

The primary goals of statistics include:

- **Summarizing Data:** Condensing large datasets into understandable summaries.
- **Making Inferences:** Drawing conclusions about a larger population based on a smaller sample of data.
- **Identifying Relationships:** Discovering patterns, trends, and correlations between different variables.
- **Forecasting:** Predicting future outcomes or behaviors.

Statistics is broadly divided into two main branches:

- **Descriptive Statistics:** This branch focuses on methods for organizing, summarizing, and presenting data in a clear and informative way. It involves calculating measures like averages (mean, median, mode), measures of spread (range, variance, standard deviation), and creating visual summaries (like charts and graphs). The goal is simply to describe the characteristics of the data at hand, without making generalizations beyond that data.
- **Inferential Statistics:** This branch uses data from a sample to make predictions or inferences about a larger population. It involves techniques like hypothesis testing, confidence intervals, and regression analysis. The challenge here is to account for the uncertainty introduced by using a sample rather than the entire population.

In essence, statistics helps us transform raw data into valuable insights, enabling us to understand complex phenomena and solve problems across diverse fields, from science and engineering to business and social sciences.

Histograms

A **histogram** is a graphical representation of the distribution of numerical data. It is similar to a bar chart, but with a crucial difference: a histogram groups numbers into ranges (called "bins") and then plots the frequency (or count) of values that fall into each bin.

Purpose of a Histogram: The main purpose of a histogram is to visually represent the shape of the data's distribution. It helps us understand:

- **Central Tendency:** Where the data values tend to cluster (e.g., unimodal, bimodal).
- **Spread/Variability:** How spread out the data values are.
- **Skewness:** Whether the data is symmetrical or skewed to one side (left-skewed or right-skewed).
- **Outliers:** The presence of unusual data points that fall far from the main body of the data.

How a Histogram is Constructed:

1. **Collect Data:** Gather a set of numerical data.
2. **Define Bins:** Divide the entire range of data values into a series of intervals called "bins." Each bin represents a range of values, and all bins typically have the same width. The choice of bin width can significantly affect the appearance of the histogram.
3. **Count Frequencies:** For each bin, count how many data points fall within that specific range. This count is the "frequency" of that bin.
4. **Draw Bars:** On a graph, draw a bar for each bin. The width of the bar corresponds to the bin width, and the height of the bar corresponds to the frequency (or relative frequency) of data points in that bin. There are no gaps between the bars in a histogram, indicating that the bins cover a continuous range of values.

PYTHON SESSION: READY

Example:

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# Parameters for the Gaussian distribution
mean = 1
std_dev = 2
num_values = 1000

# 1. Generate a Gaussian distributed random sequence
# np.random.normal(loc=mean, scale=std_dev, size=num_values) generates
# random samples from a normal (Gaussian) distribution.
# loc is the mean, scale is the standard deviation, and size is the number of samples
random_sequence = np.random.normal(loc=mean, scale=std_dev, size=num_values)

# 2. Create a histogram to display the distribution
# plt.hist() creates a histogram.
# The 'bins' argument defines the number of bins or the bin edges. 'auto' lets matplotlib
# 'density=True' normalizes the histogram so that the sum of the bar areas equals 1,
# which is necessary for superimposing a probability density function.
plt.hist(random_sequence, bins='auto', density=True, alpha=0.7, color='skyblue', label='Histogram')

# 3. Define the Gaussian (Normal) Probability Density Function (PDF)
# The PDF for a normal distribution is given by:
#  $f(x) = (1 / (\sigma * \sqrt{2 * \pi})) * \exp(-0.5 * ((x - \mu) / \sigma)^2)$ 
# where mu is the mean and sigma is the standard deviation.

# Generate x-values for the theoretical Gaussian curve
# We want the curve to span the range of our generated data for better visualization.
# np.linspace creates evenly spaced numbers over a specified interval.
x = np.linspace(min(random_sequence), max(random_sequence), 1000)
# Calculate the PDF values for these x-values using scipy.stats.norm.pdf
# This is a convenient way to get the theoretical Gaussian curve.
pdf_values = stats.norm.pdf(x, loc=mean, scale=std_dev)

# 4. Superimpose the Gaussian function on the histogram
# plt.plot() draws a line plot.
# We plot the x-values against their corresponding PDF values.
plt.plot(x, pdf_values, color='red', linestyle='--', linewidth=2, label='Theoretical Gaussian PDF')

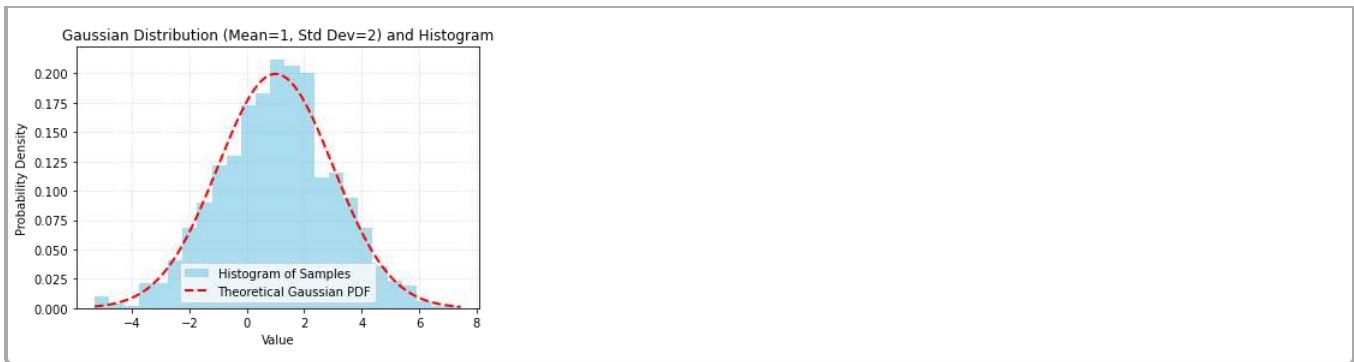
# Add labels and title for better readability
plt.xlabel('Value')
plt.ylabel('Probability Density')
plt.title(f'Gaussian Distribution (Mean={mean}, Std Dev={std_dev}) and Histogram')
plt.legend() # Display the legend to identify the histogram and the PDF curve
plt.grid(True, linestyle=':', alpha=0.6) # Add a grid for easier reading
plt.show() # Display the plot
```

run

restart

restart & run all

PYTHON SESSION: READY



Example:

Imagine you have the test scores of 50 students. To create a histogram, you might define bins like 50-59, 60-69, 70-79, 80-89, and 90-100. Then, you count how many students scored in each range and draw bars accordingly. For example, assume a gaussian distribution with mean 75 and standard deviation 10. A quick glance at the histogram would tell you if most students scored high, low, or if scores were evenly distributed.

Histograms are powerful tools for exploratory data analysis, providing immediate visual insights into the underlying patterns and characteristics of a dataset before more complex statistical analyses are performed.

This page titled [13.1: Statistics and Histograms](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

13.2: Normal Distribution

Normal Distribution

The **normal distribution**, also known as the Gaussian distribution or bell curve, is one of the most important and frequently encountered probability distributions in statistics. It is characterized by its symmetrical, bell-shaped curve, where the majority of data points cluster around the mean, and the frequency decreases as you move further away from the mean.

Key Characteristics of a Normal Distribution:

- **Symmetry:** The distribution is perfectly symmetrical around its mean. This means the left side of the curve is a mirror image of the right side.
- **Mean, Median, and Mode are Equal:** In a perfectly normal distribution, the mean, median, and mode all coincide at the peak of the curve.
- **Bell-Shaped:** The graph of the normal distribution has a distinctive bell shape.
- **Asymptotic to the X-axis:** The tails of the curve extend indefinitely in both directions, approaching but never quite touching the x-axis.
- **Defined by Two Parameters:** A normal distribution is completely defined by its mean (μ) and its standard deviation (σ).
 - The mean (μ) determines the center (location) of the distribution.
 - The standard deviation (σ) determines the spread (width) of the distribution. A smaller standard deviation means the data is more tightly clustered around the mean, resulting in a taller, narrower bell curve. A larger standard deviation means the data is more spread out, resulting in a shorter, wider bell curve.

Importance of Normal Distribution: The normal distribution is widely used in statistics for several reasons:

- Many natural phenomena (e.g., heights, blood pressure, measurement errors) tend to follow a normal distribution.
- It is a foundational concept for many inferential statistical tests.
- The Central Limit Theorem states that the distribution of sample means will tend to be normal, regardless of the shape of the population distribution, as the sample size increases.

Python Example for Normal Distribution and Histogram:

The example in section 13.1 includes Python code that demonstrates how to generate data that follows a normal distribution using NumPy and then visualize its distribution using a histogram with Matplotlib.

This page titled [13.2: Normal Distribution](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

13.3: Random Number Generation

Random Number Generators in Python

Random number generators (RNGs) are algorithms that produce sequences of numbers that appear to be random. In computing, these are typically **pseudo-random number generators (PRNGs)**, meaning they are not truly random but rather deterministic algorithms that produce sequences of numbers that pass statistical tests for randomness. They start from an initial value called a "seed," and if the same seed is used, the same sequence of "random" numbers will be generated. This determinism is often desirable for reproducibility in scientific simulations and testing.

Why are Random Numbers Important? Random numbers are fundamental in many areas, including:

- **Simulations:** Modeling real-world phenomena (e.g., weather patterns, stock market behavior, particle physics).
- **Statistical Sampling:** Selecting representative samples from a population for surveys or experiments.
- **Cryptography:** Generating keys and secure communication protocols.
- **Games:** Creating unpredictable outcomes (e.g., dice rolls, card shuffles).
- **Machine Learning:** Initializing weights in neural networks, splitting datasets, and regularization techniques.

Random Number Generation in Python:

Python provides two primary modules for generating random numbers:

1. **random module:** This is the standard library module for generating pseudo-random numbers. It's suitable for most common tasks like generating random integers, floats, or selecting random items from a sequence.
2. **numpy.random module:** Part of the NumPy library, this module is highly optimized for generating large arrays of random numbers, especially for numerical and scientific computing. It offers a wider variety of distributions (e.g., normal, uniform, binomial) and is generally preferred for performance when dealing with large datasets.

Key Concepts:

- **Seed:** The starting point for the PRNG. If you set the same seed, you'll get the same sequence of random numbers. This is useful for debugging and ensuring reproducibility.
 - `random.seed(value)`
 - `np.random.seed(value)`
- **Uniform Distribution:** By default, many random number functions generate numbers from a uniform distribution, meaning every value within a given range has an equal chance of being generated.
- **Other Distributions:** Both modules can generate numbers from various probability distributions (e.g., normal, exponential, Poisson).

Python Example for Random Number Generation:

The following code demonstrates basic random number generation using both the `random` and `numpy.random` modules.

Example:

```
import numpy as np

# Set a seed for reproducibility (optional)
# If you want the same "random" sequence every time you run the script,
# uncomment the line below and set an integer seed.
# np.random.seed(42)

# Generate a random sequence of integers
# np.random.randint(low, high, size) generates random integers from low (inclusive)
# to high (exclusive).
# Here, we generate 10 random integers between 0 and 99 (inclusive).
rand_nums = np.random.randint(0, 100, 10)
```

PYTHON SESSION: READY


```
print(f"Random sequence of integers (0-99): {random_integers}")

# Generate a random sequence of floating-point numbers
# np.random.rand(d0, d1, ..., dn) creates an array of the given shape
# and populates it with random samples from a uniform distribution over [0, 1).
# Here, it generates 5 random floats.
random_floats_uniform = np.random.rand(5)
print(f"Random sequence of floats (uniform [0, 1)): {random_floats_uniform}")

# Generate a random sequence from a standard normal distribution (mean=0, std=1)
# np.random.randn(d0, d1, ..., dn) creates an array of the given shape
# and populates it with random samples from a standard normal distribution.
# Here, it generates 7 random floats from a normal distribution.
random_floats_normal = np.random.randn(7)
print(f"Random sequence of floats (standard normal): {random_floats_normal}")

# Generate a random choice from a given list or array
# np.random.choice(a, size, replace, p) generates random samples from a given 1-D array
# 'a' is the array to choose from.
# 'size' is the number of samples to draw.
# 'replace=True' means a sample can be chosen multiple times.
# 'replace=False' means a sample can be chosen only once.
# 'p' is the probabilities associated with each entry in 'a'.
my_list = ['apple', 'banana', 'cherry', 'date', 'elderberry']
random_choices = np.random.choice(my_list, 3, replace=False) # Choose 3 unique fruits
print(f"Random choices from a list (unique): {random_choices}")

# Generate a random permutation of a sequence
# np.random.permutation(x) randomly permutes a sequence or returns a permuted range.
original_array = np.arange(10) # Array from 0 to 9
permuted_array = np.random.permutation(original_array)
print(f"Original array: {original_array}")
print(f"Permuted array: {permuted_array}")
```

run restart restart & run all

```
Random sequence of integers (0-99): [ 6 13  8 64 15 19 93 65 89 89]
Random sequence of floats (uniform [0, 1)): [0.02079568 0.88925064
0.50288663 0.10124879 0.8108217 ]
Random sequence of floats (standard normal): [-1.2832463  1.63585608
0.29387082 -0.23494306  0.27469705  0.06441582
0.2056479 ]
Random choices from a list (unique): ['cherry' 'apple' 'banana']
Original array: [0 1 2 3 4 5 6 7 8 9]
Permuted array: [6 0 3 8 9 7 1 4 2 5]
```

This page titled [13.3: Random Number Generation](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

13.4: Interpolation

Interpolation in Python

Interpolation is a method of constructing new data points within the range of a discrete set of known data points. In simpler terms, when you have a few data points and you want to estimate values between those points, interpolation comes into play. It's essentially "filling in the gaps" in your data based on the existing pattern.

Why is Interpolation Important? Interpolation is a crucial technique in various fields:

- **Data Analysis and Visualization:** Creating smooth curves from discrete data points for better visual representation.
- **Signal Processing:** Reconstructing continuous signals from sampled data.
- **Image Processing:** Resizing images, rotating, or performing other transformations where pixel values need to be estimated.
- **Numerical Methods:** Approximating functions, solving differential equations, or integrating functions numerically.
- **Engineering and Science:** Estimating values in experiments where continuous measurements are not feasible or available.

Common Interpolation Methods: There are several methods for interpolation, each with its own characteristics and suitability for different types of data:

- **Linear Interpolation:** The simplest method, where two adjacent data points are connected by a straight line, and values between them are estimated along this line.
- **Polynomial Interpolation:** Fits a polynomial curve through a set of data points. While it can be very accurate, high-degree polynomials can sometimes lead to oscillations (Runge's phenomenon).
- **Spline Interpolation:** Uses a series of low-degree polynomials (splines) to fit segments of the data. This method is generally more stable and produces smoother curves than a single high-degree polynomial. Common types include cubic splines.

Interpolation in Python:

Python's SciPy library, specifically the `scipy.interpolate` module, provides a comprehensive set of tools for various interpolation tasks. It offers functions for 1D, 2D, and higher-dimensional interpolation, as well as different interpolation methods.

Python Example for Interpolation:

The following code demonstrates linear and cubic spline interpolation using `scipy.interpolate` and visualizes the results with Matplotlib.

Example:

```
import numpy as np
from scipy.interpolate import interp1d, CubicSpline
import matplotlib.pyplot as plt

# --- 1. Linear Interpolation (interp1d) ---

# Original data points
x_original = np.array([0, 1, 2, 3, 4, 5])
y_original = np.array([0, 0.8, 0.9, 0.1, -0.8, -1])

# Create a linear interpolation function
# 'kind' can be 'linear', 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', etc.
# 'linear' is the default.
f_linear = interp1d(x_original, y_original, kind='linear')

# Points where we want to interpolate
x_new_linear = np.linspace(0, 5, num=50, endpoint=True)
y_new_linear = f_linear(x_new_linear)
```

PYTHON SESSION: READY

```
# --- 2. Cubic Spline Interpolation (CubicSpline) ---

# Original data points (can be the same or different)
x_cs = np.array([0, 1, 2, 3, 4, 5])
y_cs = np.array([0, 0.8, 0.9, 0.1, -0.8, -1])

# Create a cubic spline interpolation function
# 'bc_type' (boundary conditions) can be 'not-a-knot', 'natural', 'clamped', etc.
cs = CubicSpline(x_cs, y_cs)

# Points where we want to interpolate
x_new_cs = np.linspace(0, 5, num=50, endpoint=True)
y_interpolated_cs = cs(x_new_cs)

# --- 3. Plotting the results (optional, requires matplotlib) ---
plt.figure(figsize=(10, 6))

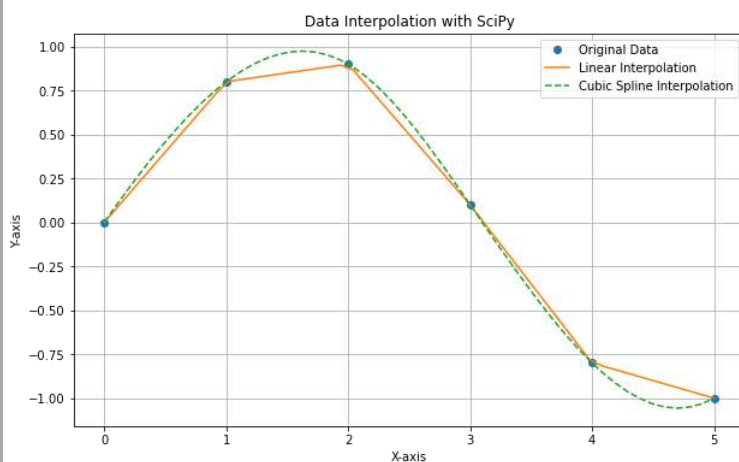
# Plot original data
plt.plot(x_original, y_original, 'o', label='Original Data')

# Plot linear interpolation
plt.plot(x_new_linear, y_interpolated_linear, '-', label='Linear Interpolation')

# Plot cubic spline interpolation
plt.plot(x_new_cs, y_interpolated_cs, '--', label='Cubic Spline Interpolation')

plt.title('Data Interpolation with SciPy')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()
```

run restart restart & run all



This page is shared under a [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

13.5: Summary

Summary

Statistical processing in Python involves several key areas in statistics, probability, and interpolation:

1. **Descriptive Statistics and Visualization:** Python, particularly with libraries like NumPy and Matplotlib, is used to organize, summarize, and present data. This includes calculating measures of central tendency (like mean) and spread (like standard deviation), and creating visual summaries such as **histograms**. Histograms are crucial for understanding the distribution of numerical data, revealing its shape, central tendency, spread, and skewness.
2. **Working with Distributions:** Python allows for the generation and analysis of data conforming to various probability distributions. The **Normal Distribution** (or Gaussian distribution) is a prime example, being a symmetrical, bell-shaped curve defined by its mean and standard deviation. Python libraries enable generating data that follows such distributions and visualizing them to understand their characteristics.
3. **Random Number Generation:** For simulations, statistical sampling, and other applications requiring randomness, Python's random and numpy.random modules are essential. These **pseudo-random number generators** produce sequences of numbers that appear random, allowing for reproducible experiments by setting a "seed." They can generate numbers from uniform, normal, and other specified distributions.
4. **Data Interpolation:** While not strictly a statistical analysis method, **interpolation** in Python (using scipy.interpolate) is a critical data processing technique. It allows for the estimation of new data points within the range of known data, effectively "filling in the gaps" to create smoother representations or approximate values. This is vital for data analysis, visualization, and numerical methods.

In summary, Python provides robust tools, primarily through NumPy, SciPy, and Matplotlib, for performing a wide range of statistical processing tasks, from basic data description and visualization to generating random data and advanced numerical techniques like interpolation.

This page titled [13.5: Summary](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

CHAPTER OVERVIEW

14: Linear Algebra Equations

- [14.1: Matrix Methods for Linear Equations](#)
- [14.2: Well-Determined Fully Specified System](#)
- [14.3: Underdetermined Systems](#)
- [14.4: Overdetermined Systems](#)
- [14.5: A General Solution](#)
- [14.6: Summary](#)

This page titled [14: Linear Algebra Equations](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

14.1: Matrix Methods for Linear Equations

Systems of linear equations are fundamental in various fields of engineering, science, and mathematics. While simple systems can be solved using substitution or elimination, more complex systems with many variables become cumbersome. This is where **matrix methods** provide a powerful and systematic approach to represent and solve such systems efficiently.

System Representation in Matrix Form

A system of m linear equations with n variables can be compactly written in matrix form as:

$$Ax = B \quad (14.1.1)$$

Where:

- A is the **coefficient matrix** (an $m \times n$ matrix containing the coefficients of the variables).
- x is the **variable vector** (an $n \times 1$ column vector containing the unknown variables).
- B is the **constant vector** (an $m \times 1$ column vector containing the constants on the right-hand side of the equations).

For example, consider the system of m linear equations with n unknowns:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ \vdots &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned} \quad (14.1.2)$$

This system can be written in matrix form as:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \quad (14.1.3)$$

There are several possible solutions for a set of m linear equations with n unknowns, depending on the relationship between m and n , and the nature of the equations themselves. Here's a breakdown of the possibilities:

General Concepts

Before diving into specific cases, here are some fundamental ideas:

- **Consistent System:** A system of linear equations is consistent if it has at least one solution.
- **Inconsistent System:** A system of linear equations is inconsistent if it has no solutions.
- **Unique Solution:** A system has a unique solution if there is exactly one set of values for the unknowns that satisfies all equations.
- **Infinitely Many Solutions:** A system has infinitely many solutions if there are an infinite number of sets of values for the unknowns that satisfy all equations. This typically occurs when there are "redundant" equations or "free variables."

Case Analysis

Consider the possibilities based on the relationship between m (number of equations) and n (number of unknowns).

$m=n$ (Square System)

When the number of equations equals the number of unknowns, it is a well-determined "square" system.

- **Unique Solution:** This is the most common and often desired outcome. A unique solution exists if and only if the coefficient matrix is **invertible (non-singular)**. In terms of determinants, this means the determinant of the coefficient matrix is non-zero ($\det(A) \neq 0$). You can solve such systems using methods like Gaussian elimination, Cramer's Rule, or matrix inversion.
- **No Solution (Inconsistent):** If the determinant of the coefficient matrix is zero ($\det(A) = 0$), the system might be inconsistent. This happens when the equations are contradictory (e.g., $x + y = 5$ and $2x + 2y = 3$). In Gaussian elimination, you would end up with a row like $[0 \ 0 \ \dots \ 0 \ | \ c]$ where $c \neq 0$.

- **Infinitely Many Solutions:** If the determinant of the coefficient matrix is zero ($\det(A) = 0$), the system might have infinitely many solutions. This happens when the equations are dependent (e.g., $2x + 3y = 5$ and $4x + 6y = 10$). One equation is essentially a multiple of another. In Gaussian elimination, you would end up with one or more rows of all zeros ($[0 \ 0 \ \dots \ 0 \ | \ 0]$), indicating free variables

$m < n$ (Underdetermined System)

When the number of equations is less than the number of unknowns, the system is "underdetermined." This implies there are more variables than constraints, which usually leads to multiple solutions.

- **Infinitely Many Solutions:** This is the most common scenario for underdetermined systems. If the system is consistent, there will be infinitely many solutions. You will typically have "free variables" that can take on any value, and the other variables will be expressed in terms of these free variables. Gaussian elimination will often result in rows of zeros, indicating these free variables.
- **No Solution (Inconsistent):** It is still possible for an underdetermined system to be inconsistent. This occurs if some of the equations are contradictory, even with fewer constraints than variables. For example, $x + y + z = 5$ and $x + y + z = 3$.
- **Unique Solution:** A unique solution is **not possible** for a consistent underdetermined system. If a solution exists, there will always be infinitely many because you have more unknowns than independent equations to uniquely determine them.

$m > n$ (Overdetermined System)

When the number of equations is greater than the number of unknowns, the system is "overdetermined." This implies more constraints than variables, making it more likely for inconsistencies.

- **No Solution (Inconsistent):** This is the most frequent outcome for overdetermined systems. It's often difficult for a single set of unknown values to satisfy all the equations simultaneously if there are too many independent constraints. Gaussian elimination will often lead to a row like $[0 \ 0 \ \dots \ 0 \ | \ c]$ where $c \neq 0$.
- **Unique Solution:** A unique solution is possible, but it's a specific case. This happens when n of the equations are linearly independent and consistent, and the remaining $m - n$ equations are redundant (i.e., they are linear combinations of the first n equations and are consistent with them). You can think of it as solving a subset of n equations and then checking if this solution satisfies the remaining $m - n$ equations.
- **Infinitely Many Solutions:** This is also a possibility, though less common than no solution. This occurs when the n unknowns are not uniquely determined by any subset of the equations, and the remaining equations are consistent and redundant. For instance, if you have many identical or dependent equations.

Methods for Solving and Determining Solutions

To determine which solution type applies and to find the solutions, you typically use:

1. **Gaussian Elimination (Row Echelon Form):** This is the most robust and widely used method. By transforming the augmented matrix into row echelon form or reduced row echelon form, you can easily identify:
 - **Inconsistency:** A row of the form $[0 \ 0 \ \dots \ 0 \ | \ c]$ where $c \neq 0$.
 - **Unique Solution:** A pivot position in every column corresponding to an unknown.
 - **Infinitely Many Solutions:** Columns without pivot positions (indicating free variables) and no inconsistent rows.
1. **Determinants (for $m = n$):** As mentioned, the determinant of the coefficient matrix can quickly tell you if a unique solution exists (non-zero determinant). If it's zero, further analysis (like Gaussian elimination) is needed to distinguish between no solution and infinitely many solutions.
2. **Matrix Inversion (for $m = n$ and unique solution):** If the coefficient matrix A is invertible, then $Ax = b$ has a unique solution $x = A^{-1}b$.
3. **Rank of a Matrix:** The rank of the coefficient matrix (A) and the rank of the augmented matrix ($[A \ | \ b]$) are crucial for formal determination:
 - **Consistent System:** $\text{rank}(A) = \text{rank}([A \ | \ b])$
 - **Inconsistent System:** $\text{rank}(A) < \text{rank}([A \ | \ b])$
 - **Unique Solution:** $\text{rank}(A) = \text{rank}([A \ | \ b]) = n$ (number of unknowns)
 - **Infinitely Many Solutions:** $\text{rank}(A) = \text{rank}([A \ | \ b]) < n$

In summary, the number of equations and unknowns, along with the specific coefficients and constants, dictate whether a system of linear equations has a unique solution, no solution, or infinitely many solutions. Gaussian elimination and the concept of matrix rank are the most powerful tools for analyzing these possibilities.

A linear system may behave in any one of three possible ways:

1. The system has a unique solution if $n = m$ and A is nonsingular.
2. The system has infinitely many solutions when $n > m$.
3. The system has no solution if $n < m$.

A system of linear equations has a non-unique solution when it has either infinitely many solutions or no solution. In contrast, is a single, specific solution that satisfies all equations in the system.

Here's a more detailed breakdown:

1. Infinite Solutions:

- In a linear system, if there are more variables than independent equations, the system will typically have infinitely many solutions.
- Geometrically, this means the equations' lines or planes intersect along a line or a plane, rather than at a single point.
- Examples include systems where one equation is a multiple of another, or where equations represent the same line or plane.

2. No Solution:

- If a system of equations is inconsistent, meaning the equations contradict each other, then there is no solution.
- Geometrically, this means the lines or planes represented by the equations do not intersect at any point.
- For example, if two equations have the same variable terms but different constant terms, they represent parallel lines and have no solution.

3. Determining Non-uniqueness:

- Rank of Matrices: In linear algebra, the ranks of the coefficient matrix (A) and the augmented matrix ($A|b$) are used to determine the nature of solutions.
 - If the rank of A is less than the rank of ($A|b$), there are no solutions.
 - If the rank of A is equal to the rank of ($A|b$), but less than the number of variables, there are infinitely many solutions.
- Determinant of the Coefficient Matrix:

For a square matrix (same number of equations and variables), if the determinant of the coefficient matrix is zero, the system either has no solution or infinitely many solutions.

4. Example:

- Consider the system:

$$\begin{aligned} 2x + 3y &= 5 \\ 4x + 6y &= 10 \end{aligned} \tag{14.1.4}$$

This system has infinitely many solutions because the second equation is simply twice the first equation, meaning they represent the same line.

5. Singular Matrices:

- A matrix is considered singular if it is not invertible (has no inverse).
- A singular matrix is associated with linear systems that have either no solution or infinitely many solutions.

This page titled [14.1: Matrix Methods for Linear Equations](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by

A linear system of equations is a set of n linear equations with n unknown variables. A system is considered **well-determined** and **fully specified** if it has exactly one unique solution. This typically means:

1. **Number of Equations = Number of Variables:** There are as many independent equations as there are unknown variables.
2. **Non-Singular Coefficient Matrix:** The determinant of the coefficient matrix is non-zero. If the determinant is zero, the matrix is singular, and the system either has no solutions or infinitely many solutions.

An n-th order linear system can be represented in matrix form as:

Where:

- $\backslash(A\backslash)$ is the $\backslash(n \times n)$ **coefficient matrix** containing the coefficients of the variables.
- $\backslash(x\backslash)$ is the $\backslash(n \times 1)$ **variable vector** containing the unknown variables $\backslash(\left(x_1,x_2,...,x_n\right)\backslash)$.
- $\backslash(b\backslash)$ is the $\backslash(n \times 1)$ **constant vector** containing the constants on the right-hand side of the equations.

For example, a 3rd order system would look like:

In matrix form:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Python's NumPy library provides highly optimized functions for numerical operations, including solving linear systems. The most straightforward and recommended method is using `numpy.linalg.solve()`. This function efficiently computes the solution x for the equation $(Ax = b)$.

Why numpy.linalg.solve?

- **Efficiency:** It uses advanced numerical algorithms (like LU decomposition) that are much more efficient and numerically stable than explicitly calculating the inverse of A and multiplying it by b (i.e., $x = A^{-1}b$), especially for large systems.
- **Robustness:** It handles various cases and provides error messages if the matrix is singular or ill-conditioned.

Steps to Solve:

1. **Import NumPy:** Start by importing the numpy library.
2. **Define Coefficient Matrix (A):** Create a NumPy array representing the coefficient matrix A.
3. **Define Constant Vector (b):** Create a NumPy array representing the constant vector b.
4. **Solve the System:** Use `numpy.linalg.solve(A, b)` to get the solution vector x.
5. **Interpret Results:** The returned vector x contains the values of the unknown variables.

If the **PYTHON SESSION: READY** lar (i.e., its determinant is zero), `numpy.linalg.solve` will raise a `LinAlgError`, indicating that the system does not have a unique solution.

Example for a Well Determined Non-Singular System

```
import numpy as np

# --- Example 1: Well-Determined System (as before) ---
print("--- Example 1: Well-Determined System ---")
# Let's consider a 3x3 linear system:
# 2x + 3y - z = 1
# 4x - y + 2z = 8
# x + 2y + 5z = 10

A_well_determined = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [1, 2, 5]
])
b_well_determined = np.array([1, 8, 10])

print("Coefficient Matrix (A_well_determined):\n", A_well_determined)
print("\nConstant Vector (b_well_determined):\n", b_well_determined)

try:
    det_A_well_determined = np.linalg.det(A_well_determined)
    print(f"\nDeterminant of A_well_determined: {det_A_well_determined:.4f}")
    if np.isclose(det_A_well_determined, 0):
        print("Warning: The determinant is close to zero. The matrix might be singular")
    else:
        print("The matrix A_well_determined is non-singular, indicating a unique solution")

    x_well_determined = np.linalg.solve(A_well_determined, b_well_determined)
    print("\nSolution Vector (x_well_determined):\n", x_well_determined)
    verification_well_determined = A_well_determined @ x_well_determined
    print("\nVerification (A_well_determined * x_well_determined):\n", verification_well_determined)
    print("Is A_well_determined * x_well_determined approximately equal to b_well_determined?")

except np.linalg.LinAlgError as e:
    print(f"\nError: Could not solve the system for A_well_determined. {e}")
    print("This usually means the coefficient matrix is singular (determinant is zero)")
except Exception as e:
    print(f"\nAn unexpected error occurred for A_well_determined: {e}")
```

run

restart

restart & run all

--- Example 1: Well-Determined System ---

Coefficient Matrix (A_well_determined):

```
[[ 2  3 -1]
 [ 4 -1  2]
 [ 1  2  5]]
```

Constant Vector (b_well_determined):

PYTHON SESSION: READY

Determinant of A_well_determined: -81.0000
The matrix A_well_determined is non-singular, indicating a unique solution exists.

Solution Vector (x_well_determined):
[1.17283951 0.12345679 1.71604938]

Verification (A_well_determined * x_well_determined):
[1. 8. 10.]
Is A_well_determined * x_well_determined approximately equal to b_well_determined? True

Example for a Well Determined Singular Valued System

```
print("\n\n--- Example 2: Singular Matrix ---")
# Consider a system where one row is a linear combination of others.
# This makes the matrix singular and the system either has no solution or infinite so
# Example:
# x + y = 2
# 2x + 2y = 4 (This is just 2 times the first equation)
# x - y = 0

A_singular = np.array([
    [1, 1],
    [2, 2]
])
b_singular_consistent = np.array([2, 4]) # Consistent system (infinite solutions)
b_singular_inconsistent = np.array([2, 5]) # Inconsistent system (no solutions)

print("Coefficient Matrix (A_singular):\n", A_singular)
print("\nConstant Vector (b_singular_consistent):\n", b_singular_consistent)
print("Constant Vector (b_singular_inconsistent):\n", b_singular_inconsistent)

try:
    det_A_singular = np.linalg.det(A_singular)
    print(f"\nDeterminant of A_singular: {det_A_singular:.4f}")

    if np.isclose(det_A_singular, 0):
        print("The determinant is zero. The matrix A_singular is singular.")
        print("This system does not have a unique solution.")
    else:
        print("The matrix A_singular is non-singular, indicating a unique solution exists.")

    # Attempt to solve with the consistent singular system
    print("\nAttempting to solve A_singular * x = b_singular_consistent:")
    x_singular_consistent = np.linalg.solve(A_singular, b_singular_consistent)
    print("Solution Vector (x_singular_consistent):\n", x_singular_consistent) # This
except np.linalg.LinAlgError as e:
    print(f"\nCaught LinAlgError for A_singular * x = b_singular_consistent: {e}")
    # PYTHON SESSION: READY, numpy.linalg.solve raises an error for a singular matrix.
except Exception as e:
```

```

print(f"\nAn unexpected error occurred for A_singular * x = b_singular_consistent")

print("\nAttempting to solve A_singular * x = b_singular_inconsistent:")
try:
    # Attempt to solve with the inconsistent singular system
    x_singular_inconsistent = np.linalg.solve(A_singular, b_singular_inconsistent)
    print("Solution Vector (x_singular_inconsistent):\n", x_singular_inconsistent) #

except np.linalg.LinAlgError as e:
    print(f"\nCaught LinAlgError for A_singular * x = b_singular_inconsistent: {e}")
    print("As expected, numpy.linalg.solve raises an error for a singular matrix, regardless of consistency.")
except Exception as e:
    print(f"\nAn unexpected error occurred for A_singular * x = b_singular_inconsistent")

```

```

--- Example 2: Singular Matrix ---
Coefficient Matrix (A_singular):
[[1 1]
 [2 2]]

Constant Vector (b_singular_consistent):
[2 4]
Constant Vector (b_singular_inconsistent):
[2 5]

Determinant of A_singular: 0.0000
The determinant is zero. The matrix A_singular is singular.
This system does not have a unique solution.

Attempting to solve A_singular * x = b_singular_consistent:

Caught LinAlgError for A_singular * x = b_singular_consistent:
Singular matrix
As expected, numpy.linalg.solve raises an error for a singular matrix.

Attempting to solve A_singular * x = b_singular_inconsistent:

Caught LinAlgError for A_singular * x = b_singular_inconsistent:
Singular matrix
As expected, numpy.linalg.solve raises an error for a singular matrix,
regardless of consistency.

```

Summary

A well-determined, fully specified n th-order linear system has a unique solution. In Python, the most efficient and robust way to solve such a system (represented as $(Ax=b)$) is by using the `numpy.linalg.solve()` function from the NumPy library.

Here's a summary of the steps involved:

1. **PYTHON SESSION: READY** the coefficient matrix A and constant vector b as NumPy arrays.

2. **Solve with** `numpy.linalg.solve()`: Pass the coefficient matrix `A` and the constant vector `b` to `np.linalg.solve(A, b)`. This function will return the solution vector `x`.
3. **Handle Errors:** If the matrix `A` is singular (i.e., its determinant is zero, meaning no unique solution exists), `numpy.linalg.solve()` will raise a `LinAlgError`. It's good practice to wrap the call in a `try-except` block to gracefully handle this issue.

The NumPy `linalg.solve` method is preferred over calculating the inverse of `A` explicitly $(x=A^{-1}b)$ because `numpy.linalg.solve()` is more numerically stable and computationally efficient, especially for larger systems.

This page titled [14.2: Well-Determined Fully Specified System](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

14.3: Underdetermined Systems

Solving Underdetermined Systems of Linear Equations

An underdetermined system of linear equations is a system where there are more variables than equations. Such a system typically has infinitely many solutions, or no solutions at all (though the latter is less common when discussing "solutions"). When solutions exist, they can often be expressed in terms of free variables.

Consider a linear system in the form $(Ax=b)$, where:

- A is an $(m \times n)$ matrix, with $(m < n)$, fewer equations than variables.
- x is an $(n \times 1)$ column vector of variables.
- b is an $(m \times 1)$ column vector of constants.

Since there are more variables than equations, the system has at least $n-m$ free variables. This means we can express some variables in terms of others.

General Approach

1. **Identify the rank of the matrix A :** The rank of matrix A determines the number of linearly independent rows (or columns). For a system to have solutions, the rank of the augmented matrix $(\left[A \mid b \right])$ must be equal to the rank of A .
2. **Find a particular solution:** One way to find a particular solution is to use the pseudoinverse of A . The pseudoinverse, denoted (A^+) , can be used to find the minimum-norm solution (the solution with the smallest Euclidean norm) when infinite solutions exist. This solution is given by $(x_p = A^+ b)$.
3. **Find the null space of A :** The null space of A consists of all vectors (x_n) such that $(A x_n = 0)$. Any vector in the null space can be added to a particular solution to form another valid solution.
4. **Construct the general solution:** The general solution to an underdetermined system is the sum of a particular solution and any vector from the null space: $(x = x_p + x_h)$, where $(x_h \in \text{null}(A))$.

In Python, libraries like the [NumPy](#) and [SciPy](#) provide powerful linear algebra tools to handle the pseudoinverse operation and for finding the null space.

NumPy also provides the [numpy.linalg.lstsq](#) function to compute a least-squares solution to a linear matrix equation. While primarily designed for overdetermined systems (where it finds the "best fit" solution), it can also be used for underdetermined systems. For underdetermined systems, `lstsq` returns the minimum-norm solution (similar to using the pseudoinverse).

When `lstsq` is used for an underdetermined system, it returns the solution that has the smallest Euclidean norm (i.e., the "shortest" vector (x) that satisfies the equations). This is often the particular solution (x_p) .

Example:

Let's consider a simple underdetermined system:

$$\begin{aligned} &\begin{aligned} &\begin{aligned} &x + 2y - z &= 5 \\ &3x - y + 2z &= 1 \end{aligned} \end{aligned} \end{aligned}$$

Here, we have 2 equations and 3 variables $(\left(x, y, z \right))$. The matrix A would be:

$$\begin{bmatrix} 1 & 2 & -1 \\ 3 & -1 & 2 \end{bmatrix}$$

The vector b would be:

$$\begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

PYTHON SESSION: READY

1
 $\end{bmatrix}$

We can use NumPy's `np.linalg.pinv` to find the pseudoinverse and then a particular solution where `scipy.linalg.null_space` would provide the null space.

[Example in Python:](#)

Compare the results from the pseudoinvers and the least-squares (`lstsq`)

```
# --- Example: Underdetermined System ---
import numpy as np
from scipy.linalg import null_space

# Define the coefficient matrix A and the constant vector b
A = np.array([[1, 2, -1],
              [3, -1, 2]])

b = np.array([5, 1])

print("Matrix A:\n", A)
print("\nVector b:\n", b)

# --- Step 1: Check for consistency (optional, but good practice) ---
# For an underdetermined system, if solutions exist, they are infinite.
# The rank of A must be equal to the rank of the augmented matrix [A|b]
# for a solution to exist.
rank_A = np.linalg.matrix_rank(A)
augmented_A = np.hstack((A, b.reshape(-1, 1))) # Create augmented matrix
rank_augmented_A = np.linalg.matrix_rank(augmented_A)

print(f"\nRank of A: {rank_A}")
print(f"Rank of augmented A: {rank_augmented_A}")

if rank_A != rank_augmented_A:
    print("\nNo solution exists for this system.")
else:
    print("\nSolutions exist for this system (potentially infinite).")

# --- Solution Method 1: Using np.linalg.pinv (Pseudoinverse) ---
# The pseudoinverse (Moore-Penrose inverse) gives the minimum-norm solution.
print("\n--- Solving using np.linalg.pinv (Pseudoinverse) ---")
A_pinv = np.linalg.pinv(A)
x_particular_pinv = A_pinv @ b

print("\nPseudoinverse of A:\n", A_pinv)
print("\nParticular solution (minimum-norm) from pinv:\n", x_particular_pinv)

# Verify the particular solution
print("\nVerification (A @ x_particular_pinv):\n", A @ x_particular_pinv)

# --- Finding the null space of A (applicable to both methods for general solution)
# The null space contains all vectors x_n such that A @ x_n = 0.
# A combination of these basis vectors can be added to the particular solution to find the general solution.
```

PYTHON SESSION: READY

```
# scipy.linalg.null_space returns an orthonormal basis for the null space.
null_space_basis = null_space(A)

print("\nBasis for the null space of A:\n", null_space_basis)

# --- Construct the general solution ---
# The general solution is  $x = x_{\text{particular\_pinv}} + c_1 v_1 + c_2 v_2 + \dots$ 
# where  $v_1, v_2, \dots$  are the basis vectors of the null space, and  $c_1, c_2, \dots$  are scalars.

print("\nGeneral Solution Form (using pinv's particular solution):")
print("x = x_particular_pinv + c * null_space_basis_vector")
# Assuming one basis vector for simplicity, adjust if null_space_basis has more columns
if null_space_basis.shape[1] > 0:
    print(f"x = {x_particular_pinv} + c * {null_space_basis[:, 0]}")
else:
    print("Null space is trivial (only the zero vector).")

# Example: Let's pick an arbitrary scalar 'c' (e.g., c = 10)
if null_space_basis.shape[1] > 0:
    c = 10
    another_solution = x_particular_pinv + c * null_space_basis[:, 0]
    print(f"\nExample solution with c = {c}:\n", another_solution)

    # Verify this new solution
    print("\nVerification (A @ another_solution):\n", A @ another_solution)

    # Example: Let's pick an arbitrary scalar 'c' (e.g., c = -5)
    c = -5
    yet_another_solution = x_particular_pinv + c * null_space_basis[:, 0]
    print(f"\nExample solution with c = {c}:\n", yet_another_solution)

    # Verify this new solution
    print("\nVerification (A @ yet_another_solution):\n", A @ yet_another_solution)

# --- Solution Method 2: Using np.linalg.lstsq ---
# For underdetermined systems, lstsq returns the minimum-norm solution.
print("\n--- Solving using np.linalg.lstsq ---")
x_lstsq, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)

print("\nSolution from np.linalg.lstsq (minimum-norm):\n", x_lstsq)
print("Residuals (should be close to zero for consistent systems):\n", residuals)
print("Rank of A from lstsq:\n", rank)
print("Singular values of A:\n", s)

# Verify the lstsq solution
print("\nVerification (A @ x_lstsq):\n", A @ x_lstsq)

# Note: x_particular_pinv and x_lstsq should be identical for consistent underdetermined systems.
print("\nComparison of solutions:")
print(f"x_particular_pinv: {x_particular_pinv}")
print(f"x_lstsq: {x_lstsq}")
```

PYTHON SESSION: READY


```
print(f"Solution from lstsq: {x_lstsq}")
print(f"Are they close? {np.allclose(x_particular_pinv, x_lstsq)}")
```

run

restart

restart & run all

Matrix A:

```
[[ 1  2 -1]
 [ 3 -1  2]]
```

Vector b:

```
[5 1]
```

Rank of A: 2

Rank of augmented A: 2

Solutions exist for this system (potentially infinite).

--- Solving using np.linalg.pinv (Pseudoinverse) ---

Pseudoinverse of A:

```
[[ 0.20481928  0.22891566]
 [ 0.3253012  -0.04819277]
 [-0.14457831  0.13253012]]
```

Particular solution (minimum-norm) from pinv:

```
[ 1.25301205  1.57831325 -0.59036145]
```

Verification (A @ x_particular_pinv):

```
[5. 1.]
```

Basis for the null space of A:

```
[[ -0.32929278]
 [ 0.5488213 ]
 [ 0.76834982]]
```

General Solution Form (using pinv's particular solution):

$x = x_{\text{particular_pinv}} + c * \text{null_space_basis_vector}$

```
x = [ 1.25301205  1.57831325 -0.59036145] + c * [-0.32929278
0.5488213      0.76834982]
```

Example solution with c = 10:

```
[-2.03991575  7.06652625  7.09313675]
```

Verification (A @ another_solution):

```
[5. 1.]
```

Example solution with c = -5:

```
[ 1.25301205  1.57831325 -0.59036145] + (-5) * [-0.32929278
0.5488213      0.76834982]
```

PYTHON SESSION: READY

```
Verification (A @ yet_another_solution):  
[5. 1.]  
  
--- Solving using np.linalg.lstsq ---  
  
Solution from np.linalg.lstsq (minimum-norm):  
[ 1.25301205  1.57831325 -0.59036145]  
Residuals (should be close to zero for consistent systems):  
[]  
Rank of A from lstsq:  
2  
Singular values of A:  
[3.75807206 2.42423068]  
  
Verification (A @ x_lstsq):  
[5. 1.]  
  
Comparison of solutions:  
Solution from pinv: [ 1.25301205  1.57831325 -0.59036145]  
Solution from lstsq: [ 1.25301205  1.57831325 -0.59036145]  
Are they close? True
```

This page titled [14.3: Underdetermined Systems](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

14.4: Overdetermined Systems

Solving Overdetermined Systems of Linear Equations

An overdetermined system of linear equations is a system where there are more equations than variables. This means $m > n$ in the equation $(Ax=b)$, where A is an $(m \times n)$ matrix.

Characteristics of Overdetermined Systems

- **More Equations than Variables:** The defining characteristic is that you have more constraints (equations) than unknowns (variables).
- **No Exact Solution (Generally):** In most real-world scenarios, an overdetermined system will not have an exact solution that satisfies all equations simultaneously. This is because the equations often represent measurements or observations that contain some level of error or inconsistency.
- **Inconsistency:** The lines (or planes, hyperplanes) represented by the equations typically do not intersect at a single common point.

The Least Squares Solution

Since an exact solution is usually not possible, the goal for overdetermined systems is to find the "best approximate" solution. This is where the **least squares method** comes in. The least squares solution is the vector x that minimizes the sum of the squares of the residuals. A residual is the difference between the observed value (from b) and the value predicted by the model (Ax) .

Mathematically, minimize the Euclidean norm of the residual vector: $(\min_x \|Ax - b\|^2)$

This minimization problem has a unique solution (if A has full column rank), given by the **normal equations**: $(A^T A x = A^T b)$

The solution x can then be found by: $(x = (A^T A)^{-1} A^T b)$

The term $((A^T A)^{-1} A^T)$ is also known as the pseudoinverse of A (specifically, the left pseudoinverse, (A^+)), which is particularly useful when A has full column rank. So, $(x = A^+ b)$.

Why Least Squares?

The least squares approach is widely used because:

- It provides a single, unique solution even when an exact solution doesn't exist.
- It is computationally efficient.
- It has strong statistical foundations (e.g., it corresponds to the maximum likelihood estimator under assumptions of normally distributed errors).

Solving in Python with NumPy

NumPy's `numpy.linalg.lstsq` function is the standard and most robust way to solve overdetermined systems in Python. It uses a numerically stable method (Singular Value Decomposition - SVD) to compute the least squares solution.

The `lstsq` function returns:

1. (x) : The least-squares solution.
2. residuals: The sum of squared residuals, or an empty array if the rank of A is less than the number of variables or if b is one dimensional.
3. rank: The effective rank of matrix A .
4. s : The singular values of A .

The (x) returned by `lstsq` is the vector that minimizes $(\|Ax - b\|^2)$. While (Ax) will not be exactly equal to b , it will be the closest possible approximation in a least-squares sense.

Example

```
import numpy as np

# DETERMINED SYSTEM:
# x + y = 2
```

```
# 2x - y = 1
# x + 2y = 4
A_over = np.array([[1, 1],
                   [2, -1],
                   [1, 2]])

b_over = np.array([2, 1, 4])

print("\nMatrix A (Overdetermined):\n", A_over)
print("\nVector b (Overdetermined):\n", b_over)

# --- Solving Overdetermined System using np.linalg.lstsq ---
# For overdetermined systems, lstsq finds the least-squares solution.
print("\n--- Solving Overdetermined System using np.linalg.lstsq ---")
x_lstsq_over, residuals_over, rank_over, s_over = np.linalg.lstsq(A_over, b_over, rcond=None)

print("\nLeast-squares solution from np.linalg.lstsq (Overdetermined):\n", x_lstsq_over)
print("Residuals (sum of squared residuals):", residuals_over)
print("Rank of A_over from lstsq:\n", rank_over)
print("Singular values of A_over:\n", s_over)

# Verify the solution (A @ x should be close to b, but not necessarily equal)
print("\nVerification (A_over @ x_lstsq_over):\n", A_over @ x_lstsq_over)
print("\nOriginal b_over:\n", b_over)
print("\nDifference (A_over @ x_lstsq_over - b_over):\n", A_over @ x_lstsq_over - b_over)

# The residuals array returned by lstsq is the sum of squared residuals,
# or an empty array if rank(A) < n or if b is 1-D.
# To get individual residuals:
individual_residuals_over = A_over @ x_lstsq_over - b_over
print("\nIndividual residuals (A_over @ x - b_over):\n", individual_residuals_over)
print("Sum of squared individual residuals:", np.sum(individual_residuals_over**2))
```

run

restart

restart & run all

Matrix A (Overdetermined):

```
[[ 1  1]
 [ 2 -1]
 [ 1  2]]
```

Vector b (Overdetermined):

```
[2 1 4]
```

--- Solving Overdetermined System using np.linalg.lstsq ---

Least-squares solution from np.linalg.lstsq (Overdetermined):

```
[1.11428571 1.31428571]
```

Residuals (sum of squared residuals): [0.25714286]

Rank of A over from lstsq:

PYTHON SESSION: READY

Singular values of A_over:

```
[2.64575131 2.23606798]
```

```
Verification (A_over @ x_lstsq_over):  
[2.42857143 0.91428571 3.74285714]
```

```
Original b_over:  
[2 1 4]
```

```
Difference (A_over @ x_lstsq_over - b_over):  
[ 0.42857143 -0.08571429 -0.25714286]
```

```
Individual residuals (A_over @ x - b_over):  
[ 0.42857143 -0.08571429 -0.25714286]
```

```
Sum of squared individual residuals: 0.25714285714285734
```

This page titled [14.4: Overdetermined Systems](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

14.5: A General Solution

Introduction to A General Solution

When dealing with systems of linear equations, we often encounter scenarios where the standard matrix inverse isn't applicable—either because the matrix is not square (meaning we have more equations than unknowns, or vice versa) or because it's singular. In such cases, finding a unique solution can be challenging, or even impossible in the traditional sense.

This document introduces the Moore-Penrose pseudo-inverse, a powerful generalization of the matrix inverse that allows us to find a "best-fit" or general solution to any system of linear equations, regardless of whether the matrix is square or invertible. It provides a robust method for tackling problems that are either overdetermined (more equations than unknowns) or underdetermined (fewer equations than unknowns), offering a unique solution that minimizes the error.

Let's explore how the Moore-Penrose pseudo-inverse provides a comprehensive approach to solving linear systems.

In NumPy, the Moore-Penrose pseudo-inverse of a matrix is computed using the `numpy.linalg.pinv()` function. This function provides a generalized inverse that exists for any matrix, including rectangular and singular matrices where a traditional inverse does not exist.

Key aspects of `numpy.linalg.pinv()`:

- **Generalized Inverse:**
- **Singular Value Decomposition (SVD):**
- **Handling Singular Values:**
- **Applications:** The pseudo-inverse is widely used in various fields, including:
 - **Least-squares regression:** Finding the "best approximate solution" to over-determined or under-determined systems of linear equations.
 - **Machine learning:** In algorithms like linear regression or support vector machines.
 - **Signal processing:** For inverse problems and system identification.

Moore-Penrose Pseudo-Inverse

The Moore-Penrose pseudo-inverse (often denoted as $A^{\dagger}$) is a generalization of the inverse matrix. It allows us to find a "best-fit" solution to a system of linear equations $(Ax = b)$, even when the matrix A is not square or is singular (non-invertible).

For a system $(Ax = b)$, the pseudo-inverse solution is given by $(x = A^{\dagger}b)$.

This solution has specific properties:

1. If a unique solution exists, the pseudo-inverse finds it.
2. If the system is overdetermined (more equations than unknowns, A is tall), the pseudo-inverse finds the least-squares solution that minimizes $\|Ax - b\|^2$.
3. If the system is underdetermined (fewer equations than unknowns, A is wide), the pseudo-inverse finds the solution with the minimum Euclidean norm ($\|x\|^2$) among all possible solutions.

Examples:

Example 1: Well-determined system with square and invertible A matrix

In this case, the pseudo-inverse will behave like the regular inverse.

```
import numpy as np
print("--- Example 1: Well-determined System (Unique Solution) ---")
A1 = np.array([[2, 1],
               [1, 3]])
b1 = np.array([4, 7])

print("Matrix A1:\n", A1)
print("b1:", b1)
```

PYTHON SESSION: READY

```
# Calculate the pseudo-inverse of A1
A1_plus = np.linalg.pinv(A1)
print("Pseudo-inverse of A1:\n", A1_plus)

# Solve for x1
x1 = A1_plus @ b1
print("Solution x1 (A1+ @ b1):\n", x1)
print("Verification (A1 @ x1):\n", A1 @ x1)
print("-" * 50 + "\n")
```

run restart restart & run all

```
--- Example 1: Well-determined System (Unique Solution) ---
Matrix A1:
[[2 1]
 [1 3]]
Vector b1:
[4 7]
Pseudo-inverse of A1:
[[ 0.6 -0.2]
 [-0.2  0.4]]
Solution x1 (A1+ @ b1):
[1. 2.]
Verification (A1 @ x1):
[4. 7.]
-----
```

Example 2: Overdetermined System (more equations than unknowns)

No exact solution exists, so the pseudo-inverse finds the least-squares solution.

```
import numpy as np
import matplotlib.pyplot as plt
print("--- Example 2: Overdetermined System (Least-Squares Solution) ---")
A2 = np.array([[1, 1],
               [2, 1],
               [3, 1]])
b2 = np.array([2, 3, 5]) # This system is inconsistent

print("Matrix A2:\n", A2)
print("Vector b2:\n", b2)

# Calculate the pseudo-inverse of A2
A2_plus = np.linalg.pinv(A2)
print("Pseudo-inverse of A2:\n", A2_plus)

# Solve for x2 (least-squares solution)
x2 = A2_plus @ b2
print("Solution x2 (A2+ @ b2 - Least-Squares):\n", x2)
print("Verification (A2 @ x2) - Note the error:\n", A2 @ x2)
print("Residual (||A2 @ x2 - b2||):\n", np.linalg.norm(A2 @ x2 - b2))
```

PYTHON SESSION: READY

```
print("-" * 50 + "\n")
x = np.linspace(1,2,200)
y1 = -A2[0,0]*x + b2[0]
y2 = -A2[1,0]*x + b2[1]
y3 = -A2[2,0]*x + b2[2]
plt.figure(figsize=(8, 6))
plt.plot(x, y1, label='Line 1: $x_1 + x_2 = 2$')
plt.plot(x, y2, label='Line 2: $2 x_1 + x_2 = 3$')
plt.plot(x, y3, label='Line 3: $3 x_1 + x_2 = 5$')
plt.scatter(x2[0:],x2[1:], color='red', label='Least Squares Solution')
plt.legend()
plt.title('Overdetermined System Visualization')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.grid()
plt.show()
```

run

restart

restart & run all

```
--- Example 2: Overdetermined System (Least-Squares Solution) ---
Matrix A2:
[[1 1]
 [2 1]
 [3 1]]
Vector b2:
[2 3 5]
Pseudo-inverse of A2:
[[-5.00000000e-01 -2.78472776e-16  5.00000000e-01]
 [ 1.33333333e+00  3.33333333e-01 -6.66666667e-01]]
Solution x2 (A2+ @ b2 - Least-Squares):
[1.5      0.33333333]
Verification (A2 @ x2) - Note the error:
[1.83333333  3.33333333  4.83333333]
Residual (||A2 @ x2 - b2||):
0.408248290463863
-----
```

Overdetermined system with three equations and two unknowns shown with least squares solution.

Example 3: Underdetermined System (fewer equations than unknowns)

Many solutions exist, the pseudo-inverse finds the one with minimum norm.

```
import numpy as np
print("--- Example 3: Underdetermined System (Minimum-Norm Solution) ---")
A3 = np.array([[1, 2, 3],
               [4, 5, 6]])
b3 = np.array([7, 8])

print(PYTHON SESSION: READY A3)
print(vector b3, b3)
```



```
# Calculate the pseudo-inverse of A3
A3_plus = np.linalg.pinv(A3)
print("Pseudo-inverse of A3:\n", A3_plus)

# Solve for x3 (minimum-norm solution)
x3 = A3_plus @ b3
print("Solution x3 (A3+ @ b3 - Minimum-Norm):\n", x3)
print("Verification (A3 @ x3):\n", A3 @ x3)
print("Norm of x3 (||x3||):\n", np.linalg.norm(x3))
print("-" * 50 + "\n")
```

run restart restart & run all

```
--- Example 3: Underdetermined System (Minimum-Norm Solution) ---
Matrix A3:
[[1 2 3]
 [4 5 6]]
Vector b3:
[7 8]
Pseudo-inverse of A3:
[[-0.94444444  0.44444444]
 [-0.11111111  0.11111111]
 [ 0.72222222 -0.22222222]]
Solution x3 (A3+ @ b3 - Minimum-Norm):
[-3.05555556  0.11111111  3.27777778]
Verification (A3 @ x3):
[7. 8.]
Norm of x3 (||x3||):
4.482476167543176
-----
```

Example 4: Singular (non-invertible square) Matrix

Even though a square and singular matrix system, does not have a regular inverse, the pseudo-inverse can still find a solution (least-squares if inconsistent, minimum-norm if consistent with multiple solutions).

```
import numpy as np
print("--- Example 4: Singular Square Matrix ---")
A4 = np.array([[1, 2],
               [2, 4]]) # This matrix is singular (rows are linearly dependent)
b4 = np.array([3, 6]) # This system is consistent (multiple solutions)

print("Matrix A4:\n", A4)
print("Vector b4:\n", b4)

# Calculate the pseudo-inverse of A4
A4_plus = np.linalg.pinv(A4)
print("Pseudo-inverse of A4:\n", A4_plus)
```

PYTHON SESSION: READY

```
# Solve for x4
x4 = A4_plus @ b4
print("Solution x4 (A4+ @ b4 - Minimum-Norm):\n", x4)
print("Verification (A4 @ x4):\n", A4 @ x4)
print("Norm of x4 (||x4||):\n", np.linalg.norm(x4))
print("-" * 50 + "\n")
```

run

restart

restart & run all

--- Example 4: Singular Square Matrix ---

Matrix A4:

```
[[1 2]
 [2 4]]
```

Vector b4:

```
[3 6]
```

Pseudo-inverse of A4:

```
[[0.04 0.08]
 [0.08 0.16]]
```

Solution x4 (A4+ @ b4 - Minimum-Norm):

```
[0.6 1.2]
```

Verification (A4 @ x4):

```
[3. 6.]
```

Norm of x4 (||x4||):

```
1.3416407864998734
```

Interpretation of Results:

1. For well-determined systems, x will be the exact unique solution.
2. For overdetermined systems, x will be the least-squares approximation. The residual ($\|Ax - b\|$) will indicate how well the solution fits.
3. For underdetermined systems, x will be one of infinitely many solutions, specifically the one with the smallest Euclidean norm.

This page titled [14.5: A General Solution](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

14.6: Summary

Summary: Solving Systems of Linear Equations

This chapter delves into the fundamental concepts and practical methods for solving systems of linear equations, a cornerstone of mathematics, science, and engineering. A linear system can be generally represented as $Ax=b$, where A is the coefficient matrix, x is the vector of variables, and b is the constant vector. The nature of the solution depends critically on the relationship between the number of equations (m) and the number of variables (n).

1. Well-Determined Systems ($m=n$)

When the number of equations equals the number of variables, the system is considered **well-determined**. If the coefficient matrix A is non-singular (i.e., its determinant is non-zero, and it has full rank), there exists a **unique solution**. This solution can be found directly by matrix inversion ($x = A^{-1}b$) or, more robustly and efficiently in practice, using numerical solvers like `numpy.linalg.solve` in Python, which employ techniques like LU decomposition.

2. Underdetermined Systems ($m<n$)

In **underdetermined systems**, there are fewer equations than variables. Such systems typically have **infinitely many solutions**, provided they are consistent (i.e., a solution exists). The solution set forms an affine subspace. To describe all possible solutions, we combine two components:

- **A Particular Solution (x_p):** This is often the minimum-norm solution, meaning the solution vector with the smallest Euclidean length. It can be found using the pseudoinverse of A (A^+), where $x_p = A^+b$, or by using `numpy.linalg.lstsq`, which yields this minimum-norm solution for underdetermined cases.
- **The Null Space (x_h):** This is the set of all vectors that, when multiplied by A , result in the zero vector ($Ax_h = 0$). The null space provides the "homogeneous" part of the solution. A basis for the null space can be found using functions like `scipy.linalg.null_space`.

The **general solution** for an underdetermined system is the sum of a particular solution and any vector from the null space: $x = x_p + x_h$.

3. Overdetermined Systems ($m>n$)

Overdetermined systems have more equations than variables. Due to potential inconsistencies or errors in real-world data, these systems generally **do not have an exact solution** that satisfies all equations simultaneously. Instead, the objective is to find the "best approximate" solution. This is achieved through the **least squares method**, which minimizes the sum of the squares of the differences between the observed values and the values predicted by the model ($\min_x \|Ax - b\|^2$). The `numpy.linalg.lstsq` function is the standard tool in Python for computing this least-squares solution, which is robustly found via Singular Value Decomposition (SVD).

4. General Solution Concept

The concept of a "general solution" is most relevant for underdetermined systems, where it encompasses all possible solutions by combining a specific solution with the vectors that lie in the null space of the coefficient matrix. For well-determined systems, the general solution is simply the unique solution itself. For overdetermined systems, since an exact solution is rare, the "solution" refers to the best approximate (least squares) fit.

In summary, understanding the relationship between the number of equations and variables is crucial for determining the nature of solutions to linear systems. Python's NumPy and SciPy libraries provides powerful and efficient tools like `np.linalg.solve`, `np.linalg.pinv`, and `np.linalg.lstsq` to handle all types of linear systems, whether they are well-determined, underdetermined, or overdetermined.

This page titled 14.6: Summary is shared under a CC BY-SA 4.0 license and was authored, remixed, and/or curated by .

CHAPTER OVERVIEW

15: Numerical Methods for Calculus and Differential Equations

[15.1: Numerical Integration](#)

[15.2: Numerical Differentiation](#)

[15.3: First-Order Differential Equations](#)

[15.4: Higher-Order Differential Equations](#)

[15.5: Summary](#)

This page titled [15: Numerical Methods for Calculus and Differential Equations](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

15.1: Numerical Integration

Numerical Integration in Python

Numerical integration, also known as numerical quadrature, is a technique used to approximate the definite integral of a function. It's particularly useful when:

1. **Analytical solutions are difficult or impossible:** Many functions do not have a simple antiderivative that can be expressed in terms of elementary functions.
2. **The function is only known at discrete points:** In experimental data or simulations, we might only have a set of data points (x_i, y_i) rather than a continuous function.

The core idea behind numerical integration is to approximate the area under the curve of a function $f(x)$ over an interval $[a, b]$ by dividing the interval into smaller subintervals and summing the areas of simple geometric shapes (like rectangles or trapezoids) that approximate the function within each subinterval.

Common Numerical Integration Methods

Two of the most common and fundamental methods are the Trapezoidal Rule and Simpson's Rule.

1. The Trapezoidal Rule

The Trapezoidal Rule approximates the area under the curve by dividing the interval $[a, b]$ into n subintervals of equal width, $h = (b - a)/n$. Within each subinterval, it approximates the area using a trapezoid.

The formula for the Trapezoidal Rule is:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-1}) + f(x_n) \right]$$

Where:

- h is the width of each subinterval.
- $(x_0, x_1, \dots, x_n)$ are the equally spaced points dividing the interval.

This method is relatively simple to implement and provides a good approximation for many functions.

2. Simpson's Rule

Simpson's Rule is a more accurate method than the Trapezoidal Rule, as it approximates the area under the curve using parabolas instead of straight lines. To use Simpson's Rule, the number of subintervals n must be an even number.

The formula for Simpson's Rule using Lagrange's formula is:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n) \right]$$

Where:

- $h = (b - a)/n$ is the width of each subinterval.
- The coefficients for the function values alternate between 4 and 2, starting and ending with 1.

Simpson's Rule generally provides a more accurate approximation for a given number of subintervals compared to the Trapezoidal Rule, especially for functions that are smooth.

3. Python Examples

The `scipy.integrate` sub-package provides several integration techniques including an ordinary differential equation integrator. An overview of the module is provided by the help command: `help(integrate)`

 Note

NAME

scipy.integrate

DE

PYTHON SESSION: READY

Integration and ODEs (:mod:`scipy.integrate`)

Integrating functions, given function object

quad -- General purpose integration
quad_vec -- General purpose integration of vector-valued functions
dblquad -- General purpose double integration
tplquad -- General purpose triple integration
nquad -- General purpose N-D integration
fixed_quad -- Integrate func(x) using Gaussian quadrature of order n
quadrature -- Integrate with given tolerance using Gaussian quadrature
romberg -- Integrate func using Romberg integration
newton_cotes -- Weights and error coefficient for Newton-Cotes integration
qmc_quad -- N-D integration using Quasi-Monte Carlo quadrature
IntegrationWarning -- Warning on issues during integration
AccuracyWarning -- Warning on issues during quadrature integration

Integrating functions, given fixed samples

trapezoid -- Use trapezoidal rule to compute integral.
cumulative_trapezoid -- Use trapezoidal rule to cumulatively compute integral.
simpson -- Use Simpson's rule to compute integral from samples.
romb -- Use Romberg Integration to compute integral from
-- (2*k + 1) evenly-spaced samples.

```
import scipy.integrate as integrate
import numpy as np

# Define the function to integrate
def f(x):
    return np.sin(x)

# Integrate f(x) from 0 to pi
result, error = integrate.quad(f, 0, np.pi)
print(f"Result of integration: {result}")
print(f"Estimated absolute error: {error}")
```

run restart restart & run all

Result of integration: 2.0
Estimated absolute error: 2.220446049250313e-14

```
import scipy.integrate as integrate
import numpy as np

# Sample data
x = np.array([0, 1, 2, 3, 4])
y = PYTHON SESSION: READY 9, 16]) # Example: y = x**2
```

```
# Integrate using the trapezoidal rule
result = integrate.trapezoid(y, x)
print(f"Result of integration using trapezoidal: {result}")
```

Result of integration using trapezoidal: 22.0

```
import scipy.integrate as integrate
import numpy as np

# Sample data (requires an odd number of points for standard Simpson's rule)
x = np.array([0, 0.5, 1, 1.5, 2])
y = np.array([0, 0.25, 1, 2.25, 4]) # Example: y = x**2

# Integrate using Simpson's rule
result = integrate.simpson(y, x)
print(f"Result of integration using simpson: {result}")
```

Result of integration using simpson: 2.6666666666666665

This page titled [15.1: Numerical Integration](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

PYTHON SESSION: READY

15.2: Numerical Differentiation

Numerical Differentiation in Python

Numerical differentiation is the process of approximating the derivative of a function using its values at discrete points. It's particularly useful in situations where:

1. **The function's analytical derivative is difficult or impossible to find:** Some complex functions may not have a simple, closed-form derivative.
2. **The function is only known through discrete data points:** In experimental data or simulations, we often have a series of (x,y) pairs without an explicit function definition.

The core idea behind numerical differentiation is to approximate the slope of the tangent line (the derivative) at a point by using the slope of a secant line connecting nearby points on the function. This is based on the definition of the derivative:

$$f' = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (15.2.1)$$

By choosing a small, finite value for h , we can approximate this limit.

Common Numerical Differentiation Methods (Finite Difference Methods)

There are several finite difference methods, each with its own advantages and accuracy characteristics. The most common ones are:

1. Forward Difference

The forward difference approximation uses the function value at the current point x and a point slightly ahead, $x + h$.

The formula is:

$$f' \approx \frac{f(x+h) - f(x)}{h} \quad (15.2.2)$$

This method is simple but introduces a truncation error that is proportional to h .

2. Backward Difference

The backward difference approximation uses the function value at the current point x and a point slightly behind, $x - h$.

The formula is:

$$f' \approx \frac{f(x) - f(x-h)}{h} \quad (15.2.3)$$

Similar to the forward difference, this method also has a truncation error proportional to h .

3. Central Difference

The central difference approximation uses function values from points both ahead ($x + h$) and behind ($x - h$) the current point x . This method is generally more accurate than the forward or backward difference methods for the same step size h .

The formula is:

$$f' \approx \frac{f(x+h) - f(x-h)}{2h} \quad (15.2.4)$$

The central difference method has a truncation error proportional to h^2 , meaning that as h gets smaller, the error decreases much faster than with the forward or backward methods. This makes it the preferred method when accuracy is critical and data points on both sides of x are available.

Considerations

1. Choosing the Step Size (h)

The choice of the step size h is crucial.

- **Too large h :** Leads to a poor approximation of the tangent, resulting in a large truncation error.

- **Too small h:** Can lead to significant round-off errors due to floating-point precision limitations when subtracting nearly equal numbers ($f(x+h) - f(x)$ or $f(x+h) - f(x-h)$) .

Finding an optimal h often involves a balance between these two types of errors.

2. Sensitive to Noise

Numerical differentiation provides an approximation, not an exact analytical derivative. It can be sensitive to noise in the data. Given the bandwidth of a signal to be differentiated, a digital filter can be designed to provide differentiation within the bandwidth of the signal and attenuation for higher frequency noise components.

This page titled [15.2: Numerical Differentiation](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

15.3: First-Order Differential Equations

First-Order Differential Equation Initial Value Problem

This document describes how to approach and solve a first-order ordinary differential equation (ODE) initial value problem (IVP) using Python.

Understanding the Problem

A first-order ordinary differential equation is an equation involving a function and its first derivative. An initial value problem adds a specific condition, known as the initial condition, which specifies the value of the function at a particular point. This condition is crucial because it allows us to find a unique solution among the infinite possible solutions to the differential equation.

The general form of a first-order ODE is often expressed as: $\frac{dy}{dx} = f(x, y)$

And an initial value problem adds: $y(x_0) = y_0$

Here, $f(x, y)$ is a given function, and (x_0, y_0) is the initial condition.

Python's Role in Solving ODEs

Python, with its rich ecosystem of scientific computing libraries, provides powerful tools for both analytical and numerical solutions to ODEs.

1. **Numerical Solutions (Approximate):** Many real-world differential equations do not have simple analytical solutions. In such cases, numerical methods are employed to approximate the solution. Python's SciPy library, specifically the `scipy.integrate.solve_ivp` function, is the standard tool for this. It implements various numerical integration techniques (e.g., Runge-Kutta methods) to step through the solution space and approximate the function's values at discrete points.

- **Process:**

- Define a Python function that represents $f(x, y)$ (i.e., the right-hand side of the ODE $\frac{dy}{dx} = f(x, y)$). This function takes (x) and (y) as arguments and returns the value of the derivative.
- Specify the interval over which you want to solve the ODE (e.g., $[x_{\text{start}}, x_{\text{end}}]$).
- Provide the initial condition (x_0, y_0) .
- Call `scipy.integrate.solve_ivp`, passing the derivative function, the time span, the initial value, and optionally, the method and points at which to evaluate the solution.
- The function returns an object containing the solution, typically (t) (the independent variable values) and (y) (the corresponding dependent variable values).
- The results can then be plotted using Matplotlib to visualize the solution curve.

2. **Analytical Solutions (Symbolic):** For simpler ODEs, especially those that can be solved explicitly, libraries like SymPy can be used to find symbolic (exact) solutions. SymPy allows you to define symbolic variables and functions, express the differential equation, and then use its `dsolve` function to find the general solution. If an initial condition is provided, SymPy can often apply it to find the particular solution.

- **Process:**

- Define symbolic variables (e.g., (x, y)) and functions (e.g., $(y(x))$).
- Formulate the differential equation using `Eq` and `Derivative` from SymPy.
- Use `dsolve` to solve the equation.
- Apply the initial condition to determine constants of integration.

Key Considerations

- **Type of Solution:** Choose between analytical and numerical methods based on the complexity of the ODE and whether an exact solution is required or an approximation is sufficient.
- **Accuracy and Stability (for Numerical Methods):** Different numerical methods have varying levels of accuracy and stability. `solve_ivp` provides options to select methods suitable for different problem types (e.g., RK45 for non-stiff problems, BDF for stiff problems).
- **In PYTHON SESSION: READY** numerical solutions, remember that the output is a set of discrete points approximating the continuous solution curve.

By leveraging these powerful Python libraries, you can effectively solve and analyze first-order differential equation initial value problems, whether seeking exact symbolic forms or accurate numerical approximations.

Example:

Find the solution for the first order differential equation initial value problem:

$$\frac{dy(t)}{dt} + \frac{y(t)}{4} = x(t)$$

where $x(t) = \cos(3t)$ and $y(0) = -1$

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def x(t):
    return np.cos(3 * t)

def f(t, y):
    dydt = x(t) - y / 4
    return dydt

tspan = np.linspace(0, 25, 1000)
yinit = [-1]

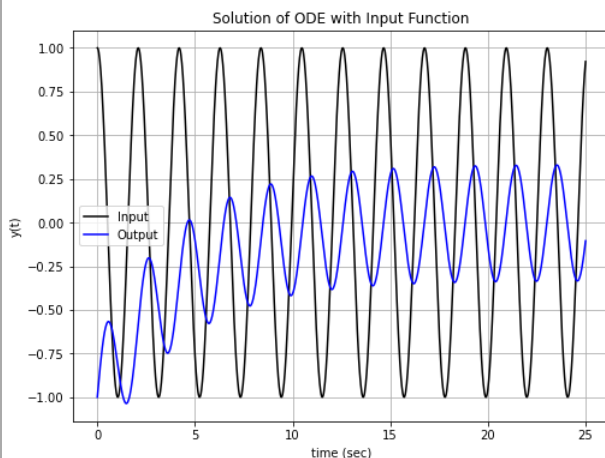
sol = solve_ivp(f, (0, 25), yinit, t_eval=tspan, rtol=1e-5)

plt.figure(figsize=(8, 6))
plt.plot(sol.t, x(sol.t), 'k-', label='Input')
plt.plot(sol.t, sol.y[0], 'b-', label='Output')
plt.legend(loc='best')
plt.xlabel('time (sec)')
plt.ylabel('y(t)')
plt.title('Solution of ODE with Input Function')
plt.grid(True)
plt.show()
```

run

restart

restart & run all



This page is licensed under a [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/) license and was authored, remixed, and/or curated by .

15.4: Higher-Order Differential Equations

Higher-Order Differential Equations, Initial Value Problem

This document outlines the process for solving initial value problems involving systems of higher-order ordinary differential equations (ODEs) using Python. This is a common task in various scientific and engineering fields, where phenomena are often described by coupled differential equations.

Understanding the Problem

A **higher-order differential equation** is one that involves derivatives of an order greater than one (e.g., second derivative, third derivative, etc.). A **system of differential equations** involves multiple coupled ODEs, where the rate of change of one variable might depend on other variables in the system. An **initial value problem (IVP)** for such a system means that for each dependent variable, its value and the values of its derivatives up to one less than its highest order are specified at a particular initial point.

The general form of a system of higher-order ODEs can be complex, but the key to solving them numerically in Python lies in a fundamental transformation.

Key Concept: Reduction to a System of First-Order ODEs

Most numerical solvers, including those in SciPy, are designed to solve systems of *first-order* ODEs. Therefore, the crucial first step in solving a higher-order ODE or a system of higher-order ODEs is to **reduce them to an equivalent system of first-order ODEs**.

How to Perform the Reduction:

For each higher-order differential equation, you introduce new dependent variables for each derivative up to one less than the highest order.

Example 1: Consider a single second-order ODE: $\frac{d^2y(t)}{dt^2} = f\left(t, y, \frac{dy}{dt}\right)$ with initial conditions $y(t_0) = y_0$ and $y'(t_0) = y'_0$.

We can transform this into a system of two first-order ODEs by defining new variables: Let $y_1 = y$ and let $y_2 = \frac{dy}{dt}$.

Then, the second-order ODE is transformed into the following system of first-order ODEs:

$$\begin{aligned} &\begin{aligned} &\begin{aligned} &y_1' &= y_2 \\ &y_2' &= f(t, y_1, y_2) \end{aligned} \end{aligned} \end{aligned}$$

The initial conditions also transform: $y_1(t_0) = y_0$ and $y_2(t_0) = y'_0$

Example 2: Consider the second-order differential equation: $y'' + 3y' + 2y = x(t)$

1. Then: $f(t, y, y') = -3y' - 2y + x(t)$
2. Let $y_1 = y$ and $y_2 = y'$
3. To obtain two 1st order ODEs:
$$\begin{aligned} y_1' &= y_2 \\ y_2' &= -2y_1 - 3y_2 + x(t) \end{aligned}$$
4. Initial conditions: $y(t_0) = y_0$ and $y'(t_0) = y'_0$
$$\begin{aligned} y_1(t_0) &= y_0 \\ y_2(t_0) &= y'_0 \end{aligned}$$

This principle extends to systems with multiple higher-order equations; each higher-order equation contributes new variables to the first-order system.

Python's Role in Solving Systems of ODEs

Python's SciPy library, particularly the `scipy.integrate.solve_ivp` function, is the go-to tool for numerically solving these systems. While SymPy can handle some analytical solutions for simpler systems, numerical methods are

PYTHON SESSION: READY

more commonly applied due to the complexity of most real-world systems.

Numerical Solution Process (using `scipy.integrate.solve_ivp`)

1. Define the System Function:

- Create a Python function that represents the right-hand side of your *transformed* system of first-order ODEs.
- This function typically takes two arguments: the independent variable (e.g., x or t) and a vector (NumPy array) of the dependent variables (e.g., $Y = [y_1, y_2, \dots, y_n]$).
- It must return a vector (NumPy array) containing the derivatives of each dependent variable in the same order as they appear in the input vector Y .
- **Example for the second-order ODE above:**

```
def system_of_odes(t, Y):  
    y1, y2 = Y # Unpack the dependent variables  
    dydt = [  
        y2, # dy1/dt = y2  
        f(t, y1, y2) # dy2/dt = f(x, y1, y2)  
    ]  
    return dydt
```

(Note: $f(t, y_1, y_2)$ would be your specific function for the second derivative.)

2. Define Initial Conditions:

- Create a list or NumPy array containing the initial values for *all* the dependent variables in your transformed first-order system. The order must match the order expected by your `system_of_odes` function.
- **Example:** `initial_conditions = [y_0, y'_0]`

3. Specify the Integration Interval:

- Define the range of the independent variable over which you want to solve the system (e.g., `t_span = [x_start, x_end]`).

4. Call `scipy.integrate.solve_ivp`:

- Pass your system function, the integration interval, and the initial conditions to `solve_ivp`.
- You can also specify `t_eval` to get solutions at specific points, and `method` to choose the numerical integration algorithm (e.g., 'RK45', 'BDF').

5. Interpret Results:

- The solution object returned by `solve_ivp` contains attributes like t (the independent variable values at which the solution was computed) and y (a 2D array where each row corresponds to one of your dependent variables, $y[0]$ for y_1 , $y[1]$ for y_2 , etc.).
- You can then extract the specific solution variables you are interested in and plot them using Matplotlib.

Conclusion

Solving systems of higher-order differential equations in Python provides a powerful capability. It is primarily enabled by utilizing the `scipy.integrate.solve_ivp` function. The critical step is always to convert the higher-order problem into an equivalent system of first-order ODEs. Once this transformation is done, Python's scientific libraries provide an efficient and robust framework for obtaining numerical solutions and visualizing their behavior.

Example

```
from scipy.integrate import solve_ivp  
import numpy as np  
import matplotlib.pyplot as plt  
  
# A function f(t, y) is defined elsewhere for your specific problem  
# For example, solve  $d^2y/dt^2 + dy/dt + y = 0$ ,  $y(0)=1$ ,  $dy/dt(0)=0$ 
```

```
# This means  $f(x, y_1, y_2) = -y_2 - y_1$ 

def f(t, y1, y2):
    return -y2 - y1

def system_of_odes(t, Y):
    y1, y2 = Y
    dydt = [
        y2,
        f(t, y1, y2)
    ]
    return dydt

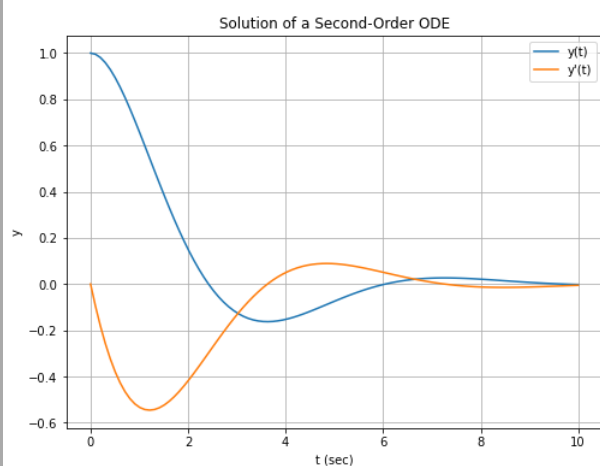
t_span = [0, 10] # Solve from x=0 to x=10
initial_conditions = [1, 0] #  $y(0)=1$ ,  $dy/dt(0)=0$  (which is  $y_2(0)=0$ )

# Solve the IVP
solution = solve_ivp(system_of_odes, t_span, initial_conditions, dense_output=True)

# Evaluate the solution at specific points for plotting
t_eval = np.linspace(t_span[0], t_span[1], 100)
y_solution = solution.sol(t_eval) # This returns [y1_values, y2_values]

# Plotting the solution for y (which is y1)
plt.figure(figsize=(8, 6))
plt.plot(t_eval, y_solution[0], label='y(t)')
plt.plot(t_eval, y_solution[1], label="y'(t)") # Plotting dy/dt as well
plt.xlabel('t (sec)')
plt.ylabel('y')
plt.title('Solution of a Second-Order ODE')
plt.legend()
plt.grid(True)
plt.show()
```

run restart restart & run all



This page titled **PYTHON SESSION: READY** Differential Equations is shared under a [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/) license and was authored, remixed, and/or curated by .

15.5: Summary

Numerical Methods: Integration, Differentiation, and Ordinary Differential Equations

Numerical methods are essential tools in scientific computing for approximating solutions to mathematical problems that are difficult or impossible to solve analytically. This summary covers numerical integration, numerical differentiation, and solving initial value ordinary differential equations using Python's NumPy and SciPy libraries.

Numerical Integration

Numerical integration, also known as quadrature, is the process of approximating the definite integral of a function. It's used when an antiderivative cannot be found analytically or when the function is only known at discrete points (e.g., experimental data).

Concepts: The definite integral of a function $f(x)$ from a to b is represented as $\int_a^b f(x)dx$. Numerical methods approximate this area by dividing it into smaller shapes (rectangles, trapezoids, parabolas) and summing their areas.

Python Implementations:

- **NumPy:** While NumPy itself doesn't offer dedicated integration routines, it provides the building blocks for simple methods like the Riemann sum or trapezoidal rule if you have discrete data points.
 - `np.trapz(y, x)`: Computes the integral using the composite trapezoidal rule.
 - `np.sum()`: Can be used for basic Riemann sums.
- **SciPy** (`scipy.integrate`): This module provides a comprehensive set of functions for numerical integration.
 - `scipy.integrate.quad(func, a, b)`: For integrating a function over a given interval. It's generally the most robust and accurate for single integrals of functions.
 - `scipy.integrate.trapz(y, x)`: Same as NumPy's `trapz`, useful for integrating data points.
 - `scipy.integrate.simpson(y, x)`: Computes the integral using Simpson's rule, often more accurate than the trapezoidal rule for smooth functions.

Numerical Differentiation

Numerical differentiation is the process of approximating the derivative of a function using its values at a set of discrete points. It's used when an analytical derivative is unknown, too complex, or when dealing with experimental data.

Concepts: The derivative of a function $f(x)$ at a point x is defined as $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. Numerical methods approximate this limit using finite differences.

- **Forward Difference:** $\frac{f(x+h) - f(x)}{h}$
- **Backward Difference:** $\frac{f(x) - f(x-h)}{h}$
- **Central Difference:** $\frac{f(x+h) - f(x-h)}{2h}$ (generally more accurate)

Python Implementations:

- **NumPy:** You can implement finite difference formulas directly using NumPy arrays.
 - `np.diff(arr)`: Computes the N-th discrete difference along the given axis. Useful for approximating derivatives of discrete data.
- **SciPy** (`scipy.misc` or `manual`):
 - `scipy.misc.derivative(func, x0, dx, n=1)`: Approximates the n-th derivative of a function `func` at point `x0` with step size `dx`. This function is convenient but might not be the most robust for all cases.
 - A more controlled method utilizes a derivative digital filter that differentiates low frequency components in the signal and attenuates the higher frequency noise components.

Solving Initial Value Ordinary Differential Equations (ODEs)

An Ordinary Differential Equation (ODE) is an equation that relates a function with its derivatives. An Initial Value Problem (IVP) for an ODE specifies the value of the unknown function (and possibly its derivatives) at a given point, called the initial condition.

Concepts:

- **First-Order ODE:** An ODE involving only the first derivative, e.g., $\frac{dy}{dt} = f(t, y)$.
- **Higher-Order ODE:** An ODE involving derivatives higher than the first can be converted into a system of first-order ODEs.

Converting Higher-Order ODEs to First-Order Systems: A common technique to solve higher-order ODEs numerically is to transform them into a system of first-order ODEs. For example, a second-order ODE $\frac{d^2y}{dt^2} = f(t, y, \frac{dy}{dt})$ can be converted by introducing new variables: Let $y_1 = y$ Let $y_2 = \frac{dy}{dt}$ Then, we have a system of first-order ODEs: $\frac{dy_1}{dt} = y_2$ and $\frac{dy_2}{dt} = f(t, y_1, y_2)$

Python Implementations:

- **NumPy:** NumPy is used to define the arrays for the state variables and the right-hand side of the ODE system.
- **SciPy** (`scipy.integrate.solve_ivp`): This is the primary function in SciPy for solving initial value problems for systems of ODEs.
 - `scipy.integrate.solve_ivp(fun, t_span, y0, method='RK45', ...)`
 - `fun`: A function describing the right-hand side of the ODE system. It takes (t, y) as arguments and returns dy/dt.
 - `t_span`: A tuple (`t_start`, `t_end`) defining the interval of integration.
 - `y0`: Initial conditions (a NumPy array).
 - `method`: The integration method (e.g., 'RK45' for Runge-Kutta 4(5), 'LSODA' for stiff problems).

These numerical methods are fundamental for solving a wide range of problems in physics, engineering, finance, and data science where analytical solutions are not feasible.

This page titled [15.5: Summary](#) is shared under a [CC BY-SA 4.0](#) license and was authored, remixed, and/or curated by .

CHAPTER OVERVIEW

16: Regular Expressions

- [16.1: Regular Expressions](#)
- [16.2: Character matching in regular expressions](#)
- [16.3: Extracting data using regular expressions](#)
- [16.4: Combining searching and extracting](#)
- [16.5: Escape Character](#)
- [16.6: Bonus section for Unix / Linux users](#)
- [16.7: Debugging](#)
- [16.E: Regular Expressions \(Exercises\)](#)
- [16.G: Regular Expressions \(Glossary\)](#)
- [16.S: Regular Expressions \(Summary\)](#)

This page titled [16: Regular Expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

16.1: Regular Expressions

So far we have been reading through files, looking for patterns and extracting various bits of lines that we find interesting. We have been using string methods like `split` and `find` and using lists and string slicing to extract portions of the lines.

This task of searching and extracting is so common that Python has a very powerful library called *regular expressions* that handles many of these tasks quite elegantly. The reason we have not introduced regular expressions earlier in the book is because while they are very powerful, they are a little complicated and their syntax takes some getting used to.

Regular expressions are almost their own little programming language for searching and parsing strings. As a matter of fact, entire books have been written on the topic of regular expressions. In this chapter, we will only cover the basics of regular expressions. For more detail on regular expressions, see:

http://en.Wikipedia.org/wiki/Regular_expression

<https://docs.python.org/3.5/library/re.html>

The regular expression library `re` must be imported into your program before you can use it. The simplest use of the regular expression library is the `search()` function. The following program demonstrates a trivial use of the search function.

```
# Search for lines that contain 'From'
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line):
        print(line)

# Code: http://www.py4e.com/code3/re01.py
```

We open the file, loop through each line, and use the regular expression `search()` to only print out lines that contain the string "From:". This program does not use the real power of regular expressions, since we could have just as easily used `line.find()` to accomplish the same result.

The power of the regular expressions comes when we add special characters to the search string that allow us to more precisely control which lines match the string. Adding these special characters to our regular expression allow us to do sophisticated matching and extraction while writing very little code.

For example, the caret character is used in regular expressions to match "the beginning" of a line. We could change our program to only match lines where "From:" was at the beginning of the line as follows:

```
# Search for lines that start with 'From'
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From:', line):
        print(line)

# Code: http://www.py4e.com/code3/re02.py
```

Now we will only match lines that *start with* the string "From:". This is still a very simple example that we could have done equivalently with the `startswith()` method from the string library. But it serves to introduce the notion that regular expressions contain special action characters that give us more control as to what will match the regular expression.

This page titled [16.1: Regular Expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.1: Regular Expressions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.2: Character matching in regular expressions

There are a number of other special characters that let us build even more powerful regular expressions. The most commonly used special character is the period or full stop, which matches any character.

In the following example, the regular expression "F.m:" would match any of the strings "From:", "Fxxm:", "F12m:", or "F!@m:" since the period characters in the regular expression match any character.

```
# Search for lines that start with 'F', followed by
# 2 characters, followed by 'm:'
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^F..m:', line):
        print(line)

# Code: http://www.py4e.com/code3/re03.py
```

This is particularly powerful when combined with the ability to indicate that a character can be repeated any number of times using the "*" or "+" characters in your regular expression. These special characters mean that instead of matching a single character in the search string, they match zero-or-more characters (in the case of the asterisk) or one-or-more of the characters (in the case of the plus sign).

We can further narrow down the lines that we match using a repeated *wild card* character in the following example:

```
# Search for lines that start with From and have an at sign
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From:.*@', line):
        print(line)

# Code: http://www.py4e.com/code3/re04.py
```

The search string " ^ From:.*@" will successfully match lines that start with "From:", followed by one or more characters (".+"), followed by an at-sign. So this will match the following line:

```
From: stephen.marquard@uct.ac.za
```

You can think of the ".*" wildcard as expanding to match all the characters between the colon character and the at-sign.

```
From:.*@
```

It is good to think of the plus and asterisk characters as "pushy". For example, the following string would match the last at-sign in the string as the ".*" pushes outwards, as shown below:

```
From: stephen.marquard@uct.ac.za, csev@umich.edu, and cwen @ iupui.edu
```

It is possible to tell an asterisk or plus sign not to be so "greedy" by adding another character. See the detailed documentation for information on turning off the greedy behavior.

This page titled [16.2: Character matching in regular expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.2: Character matching in regular expressions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.3: Extracting data using regular expressions

If we want to extract data from a string in Python we can use the `findall()` method to extract all of the substrings which match a regular expression. Let's use the example of wanting to extract anything that looks like an email address from any line regardless of format. For example, we want to pull the email addresses from each of the following lines:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
            for <source@collab.sakaiproject.org>;
Received: (from apache@localhost)
Author: stephen.marquard@uct.ac.za
```

We don't want to write code for each of the types of lines, splitting and slicing differently for each line. This following program uses `findall()` to find the lines with email addresses in them and extract one or more addresses from each of those lines.

The `findall()` method searches the string in the second argument and returns a list of all of the strings that look like email addresses. We are using a two-character sequence that matches a non-whitespace character (`\S`).

The output of the program would be:

```
['csev@umich.edu', 'cwen@iupui.edu']
```

Translating the regular expression, we are looking for substrings that have at least one non-whitespace character, followed by an at-sign, followed by at least one more non-whitespace character. The "`\S+`" matches as many non-whitespace characters as possible.

The regular expression would match twice (csev@umich.edu and cwen@iupui.edu), but it would not match the string "`@2PM`" because there are no non-blank characters *before* the at-sign. We can use this regular expression in a program to read all the lines in a file and print out anything that looks like an email address as follows:

```
# Search for lines that have an at sign between characters
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('\S+@\S+', line)
    if len(x) > 0:
        print(x)

# Code: http://www.py4e.com/code3/re06.py
```

We read each line and then extract all the substrings that match our regular expression. Since `findall()` returns a list, we simply check if the number of elements in our returned list is more than zero to print only lines where we found at least one substring that looks like an email address.

If we run the program on `mbox.txt` we get the following output:

PYTHON SESSION: READY

```
['wagnermr@iupui.edu']
['cwen@iupui.edu']
['<postmaster@collab.sakaiproject.org>']
['<200801032122.m03LMFo4005148@nakamura.uits.iupui.edu>']
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
['apache@localhost']
['source@collab.sakaiproject.org;']
```

Some of our email addresses have incorrect characters like " < " or ";" at the beginning or end. Let's declare that we are only interested in the portion of the string that starts and ends with a letter or a number.

To do this, we use another feature of regular expressions. Square brackets are used to indicate a set of multiple acceptable characters we are willing to consider matching. In a sense, the " \ S" is asking to match the set of "non-whitespace characters". Now we will be a little more explicit in terms of the characters we will match.

Here is our new regular expression:

```
[a-zA-Z0-9]\S*\S*[a-zA-Z]
```

This is getting a little complicated and you can begin to see why regular expressions are their own little language unto themselves. Translating this regular expression, we are looking for substrings that start with a *single* lowercase letter, uppercase letter, or number "[a-zA-Z0-9]", followed by zero or more non-blank characters (" \ S*"), followed by an at-sign, followed by zero or more non-blank characters (" \ S*"), followed by an uppercase or lowercase letter. Note that we switched from "+" to "*" to indicate zero or more non-blank characters since "[a-zA-Z0-9]" is already one non-blank character. Remember that the "*" or "+" applies to the single character immediately to the left of the plus or asterisk.

If we use this expression in our program, our data is much cleaner:

```
# Search for lines that have an at sign between characters
# The characters must be a letter or number
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('[a-zA-Z0-9]\S*\S*[a-zA-Z]', line)
    if len(x) > 0:
        print(x)

# Code: http://www.py4e.com/code3/re07.py
```

```
...
['wagnermr@iupui.edu']
['cwen@iupui.edu']
['postmaster@collab.sakaiproject.org']
['200801032122.m03LMFo4005148@nakamura.uits.iupui.edu']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['apache@localhost']
```

PYTHON SESSION: READY

Notice that on the "source@collab.sakaiproject.org" lines, our regular expression eliminated two letters at the end of the string (" > ;"). This is because when we append "[a-zA-Z]" to the end of our regular expression, we are demanding that whatever string the regular expression parser finds must end with a letter. So when it sees the " > " after "sakaiproject.org > ;" it simply stops at the last "matching" letter it found (i.e., the "g" was the last good match).

Also note that the output of the program is a Python list that has a string as the single element in the list.

This page titled [16.3: Extracting data using regular expressions](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.3: Extracting data using regular expressions](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

16.4: Combining searching and extracting

If we want to find numbers on lines that start with the string "X-" such as:

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
```

we don't just want any floating-point numbers from any lines. We only want to extract numbers from lines that have the above syntax.

We can construct the following regular expression to select the lines:

```
^X-.*: [0-9.]+
```

Translating this, we are saying, we want lines that start with "X-", followed by zero or more characters (".*"), followed by a colon (":") and then a space. After the space we are looking for one or more characters that are either a digit (0-9) or a period "[0-9.]+". Note that inside the square brackets, the period matches an actual period (i.e., it is not a wildcard between the square brackets).

This is a very tight expression that will pretty much match only the lines we are interested in as follows:

```
# Search for lines that start with 'X' followed by any non
# whitespace characters and ':'
# followed by a space and any number.
# The number can include a decimal.
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^X\S*: [0-9.]+', line):
        print(line)

# Code: http://www.py4e.com/code3/re10.py
```

When we run the program, we see the data nicely filtered to show only the lines we are looking for.

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.6178
X-DSPAM-Probability: 0.0000
```

But now we have to solve the problem of extracting the numbers. While it would be simple enough to use `split`, we can use another feature of regular expressions to both search and parse the line at the same time.

Parentheses are another special character in regular expressions. When you add parentheses to a regular expression, they are ignored when matching the string. But when you are using `findall()`, parentheses indicate that while you want the whole expression to match, you only are interested in extracting a portion of the substring that matches the regular expression.

So we make the following change to our program:


```
# Search for lines that start with 'X' followed by any
# non whitespace characters and ':' followed by a space
# and any number. The number can include a decimal.
# Then print the number if it is greater than zero.
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('^X\S*: ([0-9.]+)', line)
    if len(x) > 0:
        print(x)

# Code: http://www.py4e.com/code3/re11.py
```

Instead of calling `search()`, we add parentheses around the part of the regular expression that represents the floating-point number to indicate we only want `findall()` to give us back the floating-point number portion of the matching string.

The output from this program is as follows:

```
['0.8475']
['0.0000']
['0.6178']
['0.0000']
['0.6961']
['0.0000']
..
```

The numbers are still in a list and need to be converted from strings to floating point, but we have used the power of regular expressions to both search and extract the information we found interesting.

As another example of this technique, if you look at the file there are a number of lines of the form:

```
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
```

If we wanted to extract all of the revision numbers (the integer number at the end of these lines) using the same technique as above, we could write the following program:

```
# Search for lines that start with 'Details: rev='
# followed by numbers and '.'
# Then print the number if it is greater than zero
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('^Details:.*rev=([0-9.]+)', line)
    if len(x) > 0:
        print(x)

# Code: http://www.py4e.com/code3/re12.py
```

Translating our regular expression, we are looking for lines that start with "Details:", followed by any number of characters (".*"), followed by "rev=", and then by one or more digits. We want to find lines that match the entire expression but we only want to extract the integer number at the end of the line, so we surround "[0-9]+" with parentheses.

When we run the program, we get the following output:

```
['39772']  
['39771']  
['39770']  
['39769']  
...
```

Remember that the "[0-9]+" is "greedy" and it tries to make as large a string of digits as possible before extracting those digits. This "greedy" behavior is why we get all five digits for each number. The regular expression library expands in both directions until it encounters a non-digit, or the beginning or the end of a line.

Now we can use regular expressions to redo an exercise from earlier in the book where we were interested in the time of day of each mail message. We looked for lines of the form:

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

and wanted to extract the hour of the day for each line. Previously we did this with two calls to `split`. First the line was split into words and then we pulled out the fifth word and split it again on the colon character to pull out the two characters we were interested in.

While this worked, it actually results in pretty brittle code that is assuming the lines are nicely formatted. If you were to add enough error checking (or a big try/except block) to insure that your program never failed when presented with incorrectly formatted lines, the code would balloon to 10-15 lines of code that was pretty hard to read.

We can do this in a far simpler way with the following regular expression:

```
^From .* [0-9][0-9]:
```

The translation of this regular expression is that we are looking for lines that start with "From " (note the space), followed by any number of characters (".*"), followed by a space, followed by two digits "[0-9][0-9]", followed by a colon character. This is the definition of the kinds of lines we are looking for.

In order to pull out only the hour using `findall()`, we add parentheses around the two digits as follows:

```
^From .* ([0-9][0-9]):
```

This results in the following program:

```
# Search for lines that start with From and a character  
# followed by a two digit number between 00 and 99 followed by ':'  
# Then print the number if it is greater than zero  
import re  
hand = open('mbox-short.txt')  
for line in hand:  
    line = line.rstrip()  
    x = re.findall('^From .* ([0-9][0-9]):', line)  
    if len(x) > 0: print(x)  
  
# Code: http://www.py4e.com/code3/re13.py
```

When the program runs, it produces the following output:

```
[ '09' ]  
[ '18' ]  
[ '16' ]  
[ '15' ]  
...  

```

This page titled [16.4: Combining searching and extracting](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- **11.4: Combining searching and extracting** by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.5: Escape Character

Since we use special characters in regular expressions to match the beginning or end of a line or specify wild cards, we need a way to indicate that these characters are "normal" and we want to match the actual character such as a dollar sign or caret.

We can indicate that we want to simply match a character by prefixing that character with a backslash. For example, we can find money amounts with the following regular expression.

```
import re
x = 'We just received $10.00 for cookies.'
y = re.findall('\$[0-9.]+', x)
```

Since we prefix the dollar sign with a backslash, it actually matches the dollar sign in the input string instead of matching the "end of line", and the rest of the regular expression matches one or more digits or the period character. *Note:* Inside square brackets, characters are not "special". So when we say "[0-9.]", it really means digits or a period. Outside of square brackets, a period is the "wild-card" character and matches any character. Inside square brackets, the period is a period.

This page titled [16.5: Escape Character](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.5: Escape Character](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.6: Bonus section for Unix / Linux users

Support for searching files using regular expressions was built into the Unix operating system since the 1960s and it is available in nearly all programming languages in one form or another.

As a matter of fact, there is a command-line program built into Unix called *grep* (Generalized Regular Expression Parser) that does pretty much the same as the `search()` examples in this chapter. So if you have a Macintosh or Linux system, you can try the following commands in your command-line window.

```
$ grep '^From:' mbox-short.txt
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
```

This tells `grep` to show you lines that start with the string "From:" in the file `mbox-short.txt`. If you experiment with the `grep` command a bit and read the documentation for `grep`, you will find some subtle differences between the regular expression support in Python and the regular expression support in `grep`. As an example, `grep` does not support the non-blank character "`\S`" so you will need to use the slightly more complex set notation "`[^ ]`", which simply means match a character that is anything other than a space.

This page titled [16.6: Bonus section for Unix / Linux users](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.6: Bonus section for Unix / Linux users](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.7: Debugging

Python has some simple and rudimentary built-in documentation that can be quite helpful if you need a quick refresher to trigger your memory about the exact name of a particular method. This documentation can be viewed in the Python interpreter in interactive mode.

You can bring up an interactive help system using `help()` .

```
>>> help()

help> modules
```

If you know what module you want to use, you can use the `dir()` command to find the methods in the module as follows:

```
>>> import re
>>> dir(re)
[.. 'compile', 'copy_reg', 'error', 'escape', 'findall',
'finditer', 'match', 'purge', 'search', 'split', 'sre_compile',
'sre_parse', 'sub', 'subn', 'sys', 'template']
```

You can also get a small amount of documentation on a particular method using the `dir` command.

```
>>> help (re.search)
Help on function search in module re:

search(pattern, string, flags=0)
    Scan through string looking for a match to the pattern, returning
    a match object, or None if no match was found.

>>>
```

The built-in documentation is not very extensive, but it can be helpful when you are in a hurry or don't have access to a web browser or search engine.

This page titled [16.7: Debugging](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.7: Debugging](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.E: Regular Expressions (Exercises)

Exercise 1: Write a simple program to simulate the operation of the `grep` command on Unix. Ask the user to enter a regular expression and count the number of lines that matched the regular expression:

```
$ python grep.py
Enter a regular expression: ^Author
mbox.txt had 1798 lines that matched ^Author

$ python grep.py
Enter a regular expression: ^X-
mbox.txt had 14368 lines that matched ^X-

$ python grep.py
Enter a regular expression: java$
mbox.txt had 4218 lines that matched java$
```

Exercise 2: Write a program to look for lines of the form

```
`New Revision: 39772`
```

and extract the number from each of the lines using a regular expression and the `findall()` method. Compute the average of the numbers and print out the average.

```
Enter file:mbox.txt
38549.7949721

Enter file:mbox-short.txt
39756.9259259
```

This page titled [16.E: Regular Expressions \(Exercises\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.E: Regular Expressions \(Exercises\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

16.G: Regular Expressions (Glossary)

brittle code

Code that works when the input data is in a particular format but is prone to breakage if there is some deviation from the correct format. We call this "brittle code" because it is easily broken.

greedy matching

The notion that the "+" and "*" characters in a regular expression expand outward to match the largest possible string.

grep

A command available in most Unix systems that searches through text files looking for lines that match regular expressions. The command name stands for "Generalized Regular Expression Parser".

regular expression

A language for expressing more complex search strings. A regular expression may contain special characters that indicate that a search only matches at the beginning or end of a line or many other similar capabilities.

wild card

A special character that matches any character. In regular expressions the wild-card character is the period.

This page titled [16.G: Regular Expressions \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

16.S: Regular Expressions (Summary)

While this only scratched the surface of regular expressions, we have learned a bit about the language of regular expressions. They are search strings with special characters in them that communicate your wishes to the regular expression system as to what defines "matching" and what is extracted from the matched strings. Here are some of those special characters and character sequences:

- `^` Matches the beginning of the line.
- `$` Matches the end of the line.
- `.` Matches any character (a wildcard).
- `\s` Matches a whitespace character.
- `\S` Matches a non-whitespace character (opposite of `\s`).
- `*` Applies to the immediately preceding character and indicates to match zero or more of the preceding character(s).
- `*?` Applies to the immediately preceding character and indicates to match zero or more of the preceding character(s) in "non-greedy mode".
- `+` Applies to the immediately preceding character and indicates to match one or more of the preceding character(s).
- `+?` Applies to the immediately preceding character and indicates to match one or more of the preceding character(s) in "non-greedy mode".
- `[aeiou]` Matches a single character as long as that character is in the specified set. In this example, it would match "a", "e", "i", "o", or "u", but no other characters.
- `[a-z0-9]` You can specify ranges of characters using the minus sign. This example is a single character that must be a lowercase letter or a digit.
- `[^A-Za-z]` When the first character in the set notation is a caret, it inverts the logic. This example matches a single character that is anything *other than* an uppercase or lowercase letter.
- `( )` When parentheses are added to a regular expression, they are ignored for the purpose of matching, but allow you to extract a particular subset of the matched string rather than the whole string when using `findall()`.
- `\b` Matches the empty string, but only at the start or end of a word.
- `\B` Matches the empty string, but not at the start or end of a word.
- `\d` Matches any decimal digit; equivalent to the set `[0-9]`.
- `\D` Matches any non-digit character; equivalent to the set `[^0-9]`.

This page titled [16.S: Regular Expressions \(Summary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [11.S: Regular Expressions \(Summary\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

CHAPTER OVERVIEW

17: Object-Oriented Programming

As programs get to be millions of lines long, it becomes increasingly important to write code that is easy to understand. If you are working on a million line program, you can never keep the entire program in your mind at the same time. So we need ways to break the program into multiple smaller pieces so to solve a problem, fix a bug, or add a new feature we have less to look at. In a way, object oriented programming is a way to arrange your code so that you can zoom into 500 lines of the code, and understand it while ignoring the other 999,500 lines of code for the moment.

[17.1: Managing Larger Programs](#)

[17.2: Getting Started](#)

[17.3: Using Objects](#)

[17.4: Starting with Programs](#)

[17.5: Subdividing a Problem - Encapsulation](#)

[17.6: Our First Python Object](#)

[17.7: Classes as Types](#)

[17.8: Many Instances](#)

[17.9: Object Lifecycle](#)

[17.10: Inheritance](#)

[17.G: Object-Oriented Programming \(Glossary\)](#)

[17.S: Object-Oriented Programming \(Summary\)](#)

This page titled [17: Object-Oriented Programming](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

17.1: Managing Larger Programs

At the beginning of this book, we came up with four basic programming patterns which we use to construct programs:

- Sequential code
- Conditional code (if statements)
- Repetitive code (loops)
- Store and reuse (functions)

In later chapters, we explored simple variables as well as collection data structures like lists, tuples, and dictionaries.

As we build programs, we design data structures and write code to manipulate those data structures. There are many ways to write programs and by now, you probably have written some programs that are "not so elegant" and other programs that are "more elegant". Even though your programs may be small, you are starting to see how there is a bit of "art" and "aesthetic" to writing code.

As programs get to be millions of lines long, it becomes increasingly important to write code that is easy to understand. If you are working on a million line program, you can never keep the entire program in your mind at the same time. So we need ways to break the program into multiple smaller pieces so to solve a problem, fix a bug, or add a new feature we have less to look at.

In a way, object oriented programming is a way to arrange your code so that you can zoom into 500 lines of the code, and understand it while ignoring the other 999,500 lines of code for the moment.

This page titled [17.1: Managing Larger Programs](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.1: Managing Larger Programs](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.2: Getting Started

Like many aspects of programming it is necessary to learn the concepts of object oriented programming before you can use them effectively. So approach this chapter as a way to learn some terms and concepts and work through a few simple examples to lay a foundation for future learning. Throughout the rest of the book we will be using objects in many of the programs but we won't be building our own new objects in the programs.

The key outcome of this chapter is to have a basic understanding of how objects are constructed and how they function and most importantly how we make use of the capabilities of objects that are provided to us by Python and Python libraries.

This page titled [17.2: Getting Started](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.2: Getting Started](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.3: Using Objects

It turns out we have been using objects all along in this class. Python provides us with many built-in objects. Here is some simple code where the first few lines should feel very simple and natural to you.

But instead of focusing on what these lines accomplish, let's look at what is really happening from the point of view of object-oriented programming. Don't worry if the following paragraphs don't make any sense the first time you read them because we have not yet defined all these terms.

The first line is *constructing* an object of type *list*, the second and third lines are calling the `append()` *method*, the fourth line is calling the `sort()` *method*, and the fifth line is retrieving the item at position 0.

The sixth line is calling the `__getitem__()` *method* in the `stuff` list with a parameter of zero.

```
print (stuff.__getitem__(0))
```

The seventh line is an even more verbose way of retrieving the 0th item in the list.

```
print (list.__getitem__(stuff,0))
```

In this code, we are calling the `__getitem__` *method* in the `list` class and passing in the list (`stuff`) and the item we want retrieved from the list as parameters.

The last three lines of the program are completely equivalent, but it is more convenient to simply use the square bracket syntax to look up an item at a particular position in a list.

We can take a look into the capabilities of an object by looking at the output of the `dir()` *function*:

```
>>> stuff = list()
>>> dir(stuff)
['__add__', '__class__', '__contains__', '__delattr__',
 '__delitem__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__getitem__',
 '__gt__', '__hash__', '__iadd__', '__imul__', '__init__',
 '__iter__', '__le__', '__len__', '__lt__', '__mul__',
 '__ne__', '__new__', '__reduce__', '__reduce_ex__',
 '__repr__', '__reversed__', '__rmul__', '__setattr__',
 '__setitem__', '__sizeof__', '__str__', '__subclasshook__',
 'append', 'clear', 'copy', 'count', 'extend', 'index',
 'insert', 'pop', 'remove', 'reverse', 'sort']
>>>
```

The precise definition of `dir()` is that it lists the *methods* and *attributes* of a Python object.

The rest of this chapter will define all of the above terms so make sure to come back after you finish the chapter and re-read the above paragraphs to check your understanding.

This page titled [17.3: Using Objects](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.3: Using Objects](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

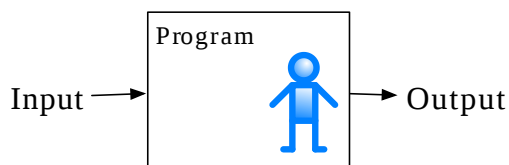
17.4: Starting with Programs

A program in its most basic form takes some input, does some processing, and produces some output. Our elevator conversion program demonstrates a very short but complete program showing all three of these steps.

```
usf = input('Enter the US Floor Number: ')
wf = int(usf) - 1
print('Non-US Floor Number is',wf)

# Code: http://www.py4e.com/code3/elev.py
```

If we think a bit more about this program, there is the "outside world" and the program. The input and output aspects are where the program interacts with the outside world. Within the program we have code and data to accomplish the task the program is designed to solve.



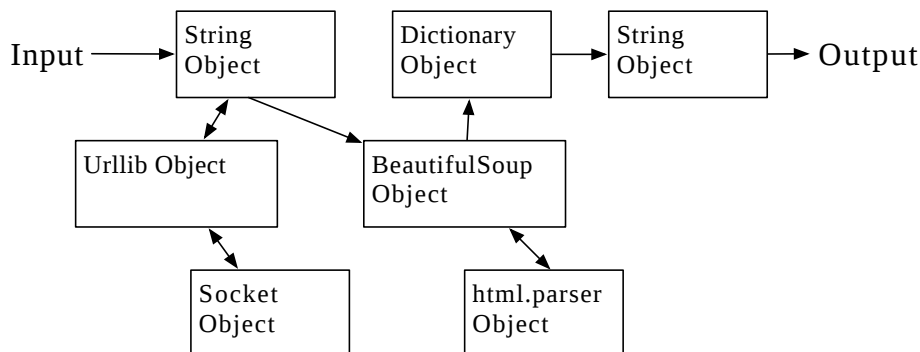
A Program

When we are "in" the program, we have some defined interactions with the "outside" world, but those interactions are well defined and generally not something we focus on. While we are coding we worry only about the details "inside the program".

One way to think about object oriented programming is that we are separating our program into multiple "zones". Each "zone" contains some code and data (like a program) and has well defined interactions with the outside world and the other zones within the program.

If we look back at the link extraction application where we used the BeautifulSoup library, we can see a program that is constructed by connecting different objects together to accomplish a task:

We read the URL into a string, and then pass that into `urllib` to retrieve the data from the web. The `urllib` library uses the `socket` library to make the actual network connection to retrieve the data. We take the string that we get back from `urllib` and hand it to BeautifulSoup for parsing. BeautifulSoup makes use of another object called `html.parser`¹ and returns an object. We call the `tags()` method in the returned object and then get a dictionary of tag objects, and loop through the tags and call the `get()` method for each tag to print out the 'href' attribute.



A Program as Network of Objects

We can draw a picture of this program and how the objects work together.

The key here is not to fully understand how this program works but to see how we build a network of interacting objects and orchestrate the interaction between the objects to create a program. It is also important to note that when you looked

PYTHON SESSION: READY

at that program several chapters back, you could fully understand what was going on in the program without even realizing that the program was "orchestrating the movement of data between objects". Back then it was just lines of code that got the job done.

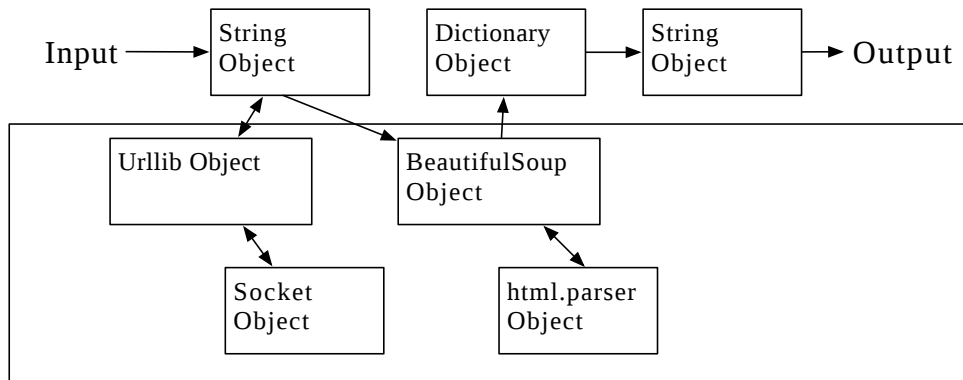
This page titled [17.4: Starting with Programs](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.4: Starting with Programs](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

PYTHON SESSION: READY

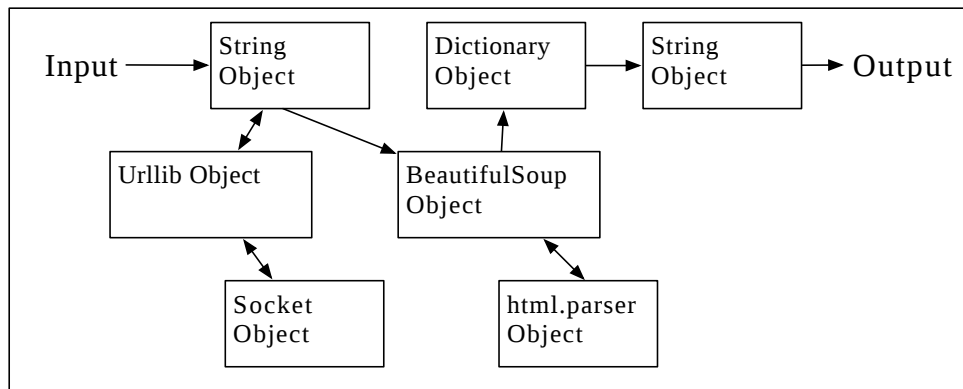
17.5: Subdividing a Problem - Encapsulation

One of the advantages of the object oriented approach is that it can hide complexity. For example, while we need to know how to use the `urllib` and BeautifulSoup code, we do not need to know how those libraries work internally. It allows us to focus on the part of the problem we need to solve and ignore the other parts of the program.



Ignoring Detail When Using an Object

This ability to focus on a part of a program that we care about and ignore the rest of the program is also helpful to the developers of the objects. For example the programmers developing BeautifulSoup do not need to know or care about how we retrieve our HTML page, what parts we want to read or what we plan to do with the data we extract from the web page.



Ignoring Detail When Building an Object

Another word we use to capture this idea that we ignore the internal detail of objects we use is "encapsulation". This means that we can know how to use an object without knowing how it internally accomplishes what we need done.

This page titled [17.5: Subdividing a Problem - Encapsulation](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.5: Subdividing a Problem - Encapsulation](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.6: Our First Python Object

At its simplest, an object is some code plus data structures that is smaller than a whole program. Defining a function allows us to store a bit of code and give it a name and then later invoke that code using the name of the function.

An object can contain a number of functions (which we call "methods") as well as data that is used by those functions. We call data items that are part of the object "attributes".

We use the `class` keyword to define the data and code that will make up each of the objects. The class keyword includes the name of the class and begins an indented block of code where we include the attributes (data) and methods (code).

Each method looks like a function, starting with the `def` keyword and consisting of an indented block of code. This example has one attribute (`x`) and one method (`party`). The methods have a special first parameter that we name by convention `self`.

Much like the `def` keyword does not cause function code to be executed, the `class` keyword does not create an object. Instead, the `class` keyword defines a template indicating what data and code will be contained in each object of type `PartyAnimal`. The class is like a cookie cutter and the objects created using the class are the cookies². You don't put frosting on the cookie cutter, you put frosting on the cookies - and you can put different frosting on each cookie.



A Class and Two Objects

If you continue through the example code, we see the first executable line of code:

```
an = PartyAnimal()
```

This is where we instruct Python to construct (e.g. create) an *object* or "instance of the class named `PartyAnimal`". It looks like a function call, but Python constructs the object with the right data and methods and returns the object which is then assigned to the variable `an`. This way this is quite similar to the following line which we have been using all along:

PYTHON SESSION: READY

```
counts = dict()
```

Here we are instructing Python to construct an object using the `dict` template (already present in Python), return the instance of dictionary and assign it to the variable `counts`.

When the `PartyAnimal` class is used to construct an object, the variable `an` is used to point to that object. We use `an` to access the code and data for that particular instance of a `PartyAnimal` object.

Each `PartyAnimal` object/instance contains within it a variable `x` and a method/function named `party`. We call that `party` method in this line:

```
an.party()
```

When the `party` method is called, the first parameter (which we call by convention `self`) points to the particular instance of the `PartyAnimal` object that `party` is called from within. Within the `party` method, we see the line:

```
self.x = self.x + 1
```

This syntax using the 'dot' operator is saying 'the `x` within `self`'. So each time `party()` is called, the internal `x` value is incremented by 1 and the value is printed out.

To help make sense of the difference between a global function and a method within a class/object, the following line is another way to call the `party` method within the `an` object:

```
PartyAnimal.party(an)
```

In this variation, we are accessing the code from within the `class` and explicitly passing the object pointer `an` in as the first parameter (i.e. `self` within the method). You can think of `an.party()` as shorthand for the above line.

When the program executes, it produces the following output:

```
So far 1
So far 2
So far 3
So far 4
```

The object is constructed, and the `party` method is called four times, both incrementing and printing the value for `x` within the `an` object.

This page titled [17.6: Our First Python Object](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.6: Our First Python Object](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.7: Classes as Types

As we have seen, in Python, all variables have a type. And we can use the built-in `dir` function to examine the capabilities of a variable. We can use `type` and `dir` with the classes that we create.

When this program executes, it produces the following output:

```
Type <class '__main__.PartyAnimal'>
Dir ['__class__', '__delattr__', ...
     '__sizeof__', '__str__', '__subclasshook__',
     '__weakref__', 'party', 'x']
Type <class 'int'>
Type <class 'method'>
```

You can see that using the `class` keyword, we have created a new type. From the `dir` output, you can see both the `x` integer attribute and the `party` method are available in the object.

This page titled [17.7: Classes as Types](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.7: Classes as Types](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.8: Many Instances

So far, we have been defining a class, making a single object, using that object, and then throwing the object away. But the real power in object oriented happens when we make many instances of our class.

When we are making multiple objects from our class, we might want to set up different initial values for each of the objects. We can pass data into the constructors to give each object a different initial value:

The constructor has both a `self` parameter that points to the object instance and then additional parameters that are passed into the constructor as the object is being constructed:

```
s = PartyAnimal('Sally')
```

Within the constructor, the line:

```
self.name = nam
```

Copies the parameter that is passed in (`nam`) into the `name` attribute within the object instance.

The output of the program shows that each of the objects (`s` and `j`) contain their own independent copies of `x` and `nam` :

```
Sally constructed
Sally party count 1
Jim constructed
Jim party count 1
Sally party count 2
```

This page titled [17.8: Many Instances](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.8: Many Instances](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.9: Object Lifecycle

In the previous examples, we are defining a class (template) and using that class to create an instance of that class (object) and then using the instance. When the program finishes, all the variables are discarded. Usually we don't think much about the creation and destruction of variables, but often as our objects become more complex, we need to take some action within the object to set things up as the object is being constructed and possibly clean things up as the object is being discarded.

If we want our object to be aware of these moments of construction and destruction, we add specially named methods to our object:

When this program executes, it produces the following output:

```
I am constructed
So far 1
So far 2
I am destructed 2
an contains 42
```

As Python is constructing our object, it calls our `__init__` method to give us a chance to set up some default or initial values for the object. When Python encounters the line:

```
an = 42
```

It actually 'throws our object away' so it can reuse the `an` variable to store the value `42`. Just at the moment when our `an` object is being 'destroyed' our destructor code (`__del__`) is called. We cannot stop our variable from being destroyed, but we can do any necessary cleanup right before our object no longer exists.

When developing objects, it is quite common to add a constructor to an object to set in initial values in the object, it is relatively rare to need a destructor for an object.

This page titled [17.9: Object Lifecycle](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.9: Object Lifecycle](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.10: Inheritance

Another powerful feature of object oriented programming is the ability to create a new class by extending an existing class. When extending a class, we call the original class the 'parent class' and the new class as the 'child class'.

For this example, we will move our `PartyAnimal` class into its own file:

```
class PartyAnimal:
    x = 0
    name = ''
    def __init__(self, nam):
        self.name = nam
        print(self.name, 'constructed')

    def party(self) :
        self.x = self.x + 1
        print(self.name, 'party count',self.x)

# Code: http://www.py4e.com/code3/party.py
```

Then, we can 'import' the `PartyAnimal` class in a new file and extend it as follows:

```
from party import PartyAnimal

class CricketFan(PartyAnimal):
    points = 0
    def six(self):
        self.points = self.points + 6
        self.party()
        print(self.name, "points",self.points)

s = PartyAnimal("Sally")
s.party()
j = CricketFan("Jim")
j.party()
j.six()
print(dir(j))

# Code: http://www.py4e.com/code3/party6.py
```

When we are defining the `CricketFan` object, we indicate that we are extending the `PartyAnimal` class. This means that all of the variables (`x`) and methods (`party`) from the `PartyAnimal` class are inherited by the `CricketFan` class.

You can see that within the `six` method in the `CricketFan` class, we can call the `party` method from the `PartyAnimal` class. The variables and methods from the parent class are *merged* into the child class.

As the program executes, we can see that the `s` and `j` are independent instances of `PartyAnimal` and `CricketFan` . The `j` object has additional capabilities beyond the `s` object.

```
Sally constructed
Sally party count 1
Jim constructed
Jim party count 1
Jim party count 2
Jim points 6
['__class__', '__delattr__', ... '__weakref__',
'name', 'party', 'points', 'six', 'x']
```

In the `dir` output for the `j` object (instance of the `CricketFan` class) you can see that it both has the attributes and methods of the parent class as well as the attributes and methods that were added when the class was extended to create the `CricketFan` class.

This page titled [17.10: Inheritance](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.10: Inheritance](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.G: Object-Oriented Programming (Glossary)

attribute

A variable that is part of a class.

class

A template that can be used to construct an object. Defines the attributes and methods that will make up the object.

child class

A new class created when a parent class is extended. The child class inherits all of the attributes and methods of the parent class.

constructor

An optional specially named method (`__init__`) that is called at the moment when a class is being used to construct an object. Usually this is used to set up initial values for the object.

destructor

An optional specially named method (`__del__`) that is called at the moment just before an object is destroyed. Destructors are rarely used.

inheritance

When we create a new class (child) by extending an existing class (parent). The child class has all the attributes and methods of the parent class plus additional attributes and methods defined by the child class.

method

A function that is contained within a class and the objects that are constructed from the class. Some object-oriented patterns use 'message' instead of 'method' to describe this concept.

object

A constructed instance of a class. An object contains all of the attributes and methods that were defined by the class. Some object-oriented documentation uses the term 'instance' interchangeably with 'object'.

parent class

The class which is being extended to create a new child class. The parent class contributes all of its methods and attributes to the new child class.

-
1. <https://docs.python.org/3/library/html.parser.html>↩
 2. Cookie image copyright CC-BY www.flickr.com/photos/dinner...es/23570475099↩
 3. If you are curious about where the list class is defined, take a look at (hopefully the URL won't change) github.com/python/cpython/blob/master/Objects/listobject.c - the list class is written in a language called "C". If you take a look at that source code and find it curious you might want to explore a few Computer Science courses.↩
-

This page titled [17.G: Object-Oriented Programming \(Glossary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.G: Object-Oriented Programming \(Glossary\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

17.S: Object-Oriented Programming (Summary)

This is a very quick introduction to object-oriented programming that focuses mainly on terminology and the syntax of defining and using objects. Let's quickly review the code that we looked at in the beginning of the chapter. At this point you should fully understand what is going on.

```
stuff = list()
stuff.append('python')
stuff.append('chuck')
stuff.sort()
print (stuff[0])

print (stuff.__getitem__(0))
print (list.__getitem__(stuff,0))

# Code: http://www.py4e.com/code3/party1.py
```

The first line constructs a `list` object. When Python creates the `list` object, it calls the *constructor* method (named `__init__`) to set up the internal data attributes that will be used to store the list data. Due to *encapsulation* we neither need to know, nor need to care about how these internal data attributes are arranged.

We are not passing any parameters to the *constructor* and when the constructor returns, we use the variable `stuff` to point to the returned instance of the `list` class.

The second and third lines are calling the `append` method with one parameter to add a new item at the end of the list by updating the attributes within `stuff`. Then in the fourth line, we call the `sort` method with no parameters to sort the data within the `stuff` object.

Then we print out the first item in the list using the square brackets which are a shortcut to calling the `__getitem__` method within the `stuff` object. And this is equivalent to calling the `__getitem__` method in the `list` class passing the `stuff` object in as the first parameter and the position we are looking for as the second parameter.

At the end of the program the `stuff` object is discarded but not before calling the *destructor* (named `__del__`) so the object can clean up any loose ends as necessary.

Those are the basics and terminology of object oriented programming. There are many additional details as to how to best use object oriented approaches when developing large applications and libraries that are beyond the scope of this chapter.³

This page titled [17.S: Object-Oriented Programming \(Summary\)](#) is shared under a [CC BY 4.0](#) license and was authored, remixed, and/or curated by via [source content](#) that was edited to the style and standards of the LibreTexts platform.

- [14.S: Object-Oriented Programming \(Summary\)](#) by [Chuck Severance](#) is licensed [CC BY-NC-SA 4.0](#). Original source: <https://www.py4e.com/html3>.

Detailed Licensing

Overview

Title: [Engineering Modeling and Analysis with Python](#)

Webpages: 191

All licenses found:

- [CC BY 4.0](#): 79.6% (152 pages)
- [CC BY-SA 4.0](#): 18.8% (36 pages)
- [Undeclared](#): 1.6% (3 pages)

By Page

- [Engineering Modeling and Analysis with Python - CC BY 4.0](#)
 - [Front Matter - CC BY-SA 4.0](#)
 - [TitlePage - CC BY-SA 4.0](#)
 - [InfoPage - CC BY-SA 4.0](#)
 - [Table of Contents - Undeclared](#)
 - [About the Authors - CC BY-SA 4.0](#)
 - [Licensing - Undeclared](#)
 - [About the Book - CC BY-SA 4.0](#)
 - [Licensing - CC BY-SA 4.0](#)
 - [1: Introduction to Engineering Modeling and Analysis with Python - CC BY 4.0](#)
 - [1.1: Introduction - CC BY 4.0](#)
 - [2: Variables, Expressions, and Statements - CC BY 4.0](#)
 - [2.1: Values and Types - CC BY 4.0](#)
 - [2.2: Variables - CC BY 4.0](#)
 - [2.3: Variable names and Keywords - CC BY 4.0](#)
 - [2.4: Statements - CC BY 4.0](#)
 - [2.5: Operators and Operands - CC BY 4.0](#)
 - [2.6: Expressions - CC BY 4.0](#)
 - [2.7: Order of Operations - CC BY 4.0](#)
 - [2.8: Modulus Operator - CC BY 4.0](#)
 - [2.9: String Operations - CC BY 4.0](#)
 - [2.10: Asking the user for input - CC BY 4.0](#)
 - [2.11: Comments - CC BY 4.0](#)
 - [2.12: Choosing Mnemonic Variable Names - CC BY 4.0](#)
 - [2.13: Debugging - CC BY 4.0](#)
 - [2.E: Variables, Expressions, and Statements \(Exercises\) - CC BY 4.0](#)
 - [2.G: Glossary - CC BY 4.0](#)
 - [3: Conditional Execution - CC BY 4.0](#)
 - [3.1: Boolean Expressions - CC BY 4.0](#)
 - [3.2: Logical Operators - CC BY 4.0](#)
 - [3.3: Conditional Execution - CC BY 4.0](#)
 - [3.4: Alternative Execution - CC BY 4.0](#)
 - [3.5: Chained Conditionals - CC BY 4.0](#)
 - [3.6: Nested Conditionals - CC BY 4.0](#)
 - [3.7: Catching exceptions Using Try and Except - CC BY 4.0](#)
 - [3.8: Short-Circuit Evaluation of Logical Expressions - CC BY 4.0](#)
 - [3.9: Debugging - CC BY 4.0](#)
 - [3.E: Conditional Execution \(Exercises\) - CC BY 4.0](#)
 - [3.G: Conditional Execution \(Glossary\) - CC BY 4.0](#)
 - [4: Functions - CC BY 4.0](#)
 - [4.1: Function Calls - CC BY 4.0](#)
 - [4.2: Built-in Functions - CC BY 4.0](#)
 - [4.3: Type Conversion Functions - CC BY 4.0](#)
 - [4.4: Random Numbers - CC BY 4.0](#)
 - [4.5: Math Functions - CC BY 4.0](#)
 - [4.6: Adding New Functions - CC BY 4.0](#)
 - [4.7: Definitions and Uses - CC BY 4.0](#)
 - [4.8: Flow of Execution - CC BY 4.0](#)
 - [4.9: Parameters and Arguments - CC BY 4.0](#)
 - [4.10: Fruitful functions and void functions - CC BY 4.0](#)
 - [4.11: Why functions? - CC BY 4.0](#)
 - [4.12: Debugging - CC BY 4.0](#)
 - [4.E: Functions \(Exercises\) - CC BY 4.0](#)
 - [4.G: Functions \(Glossary\) - CC BY 4.0](#)
 - [5: Iterations - CC BY 4.0](#)
 - [5.1: Updating Variables - CC BY 4.0](#)
 - [5.2: The while Statement - CC BY 4.0](#)
 - [5.3: Infinite Loops - CC BY 4.0](#)
 - [5.4: "Infinite loops" and break - CC BY 4.0](#)
 - [5.5: Finishing iterations with continue - CC BY 4.0](#)
 - [5.6: Definite loops using for - CC BY 4.0](#)
 - [5.7: Loop patterns - CC BY 4.0](#)
 - [5.8: Counting and Summing Loops - CC BY 4.0](#)
 - [5.9: Maximum and Minimum Loops - CC BY 4.0](#)
 - [5.10: Debugging - CC BY 4.0](#)
 - [5.E: Iterations \(Exercises\) - CC BY 4.0](#)
 - [5.G: Iterations \(Glossary\) - CC BY 4.0](#)
 - [6: Strings - CC BY 4.0](#)

- 6.1: A string is a sequence - CC BY 4.0
- 6.2: Getting the length of a string using len - CC BY 4.0
- 6.3: Traversal through a string with a loop - CC BY 4.0
- 6.4: String Slices - CC BY 4.0
- 6.5: Strings are immutable - CC BY 4.0
- 6.6: Looping and Counting - CC BY 4.0
- 6.7: The in operator - CC BY 4.0
- 6.8: String Comparison - CC BY 4.0
- 6.9: String Methods - CC BY 4.0
- 6.10: Parsing strings - CC BY 4.0
- 6.11: Format operator - CC BY 4.0
- 6.12: Debugging - CC BY 4.0
- 6.E: Strings (Exercises) - CC BY 4.0
- 6.G: Strings (Glossary) - CC BY 4.0
- 7: Files - CC BY 4.0
 - 7.1: Persistence - CC BY 4.0
 - 7.2: Opening Files - CC BY 4.0
 - 7.3: Text files and Lines - CC BY 4.0
 - 7.4: Reading Files - CC BY 4.0
 - 7.5: Searching through a File - CC BY 4.0
 - 7.6: Letting the user choose the file name - CC BY 4.0
 - 7.7: Using try, except, and open - CC BY 4.0
 - 7.8: Writing Files - CC BY 4.0
 - 7.9: Debugging - CC BY 4.0
 - 7.E: Files (Exercises) - CC BY 4.0
 - 7.G: Files (Glossary) - CC BY 4.0
- 8: Lists - CC BY 4.0
 - 8.1: A list is a sequence - CC BY 4.0
 - 8.2: Lists are mutable - CC BY 4.0
 - 8.3: Traversing a List - CC BY 4.0
 - 8.4: List operations - CC BY 4.0
 - 8.5: List Slices - CC BY 4.0
 - 8.6: List Methods - CC BY 4.0
 - 8.7: Deleting Elements - CC BY 4.0
 - 8.8: Lists and Functions - CC BY 4.0
 - 8.9: Lists and Strings - CC BY 4.0
 - 8.10: Parsing lines - CC BY 4.0
 - 8.11: Objects and Values - CC BY 4.0
 - 8.12: Aliasing - CC BY 4.0
 - 8.13: List arguments - CC BY 4.0
 - 8.14: Debugging - CC BY 4.0
 - 8.E: Lists (Exercises) - CC BY 4.0
 - 8.G: Lists (Glossary) - CC BY 4.0
- 9: Dictionaries - CC BY 4.0
 - 9.1: Dictionaries - CC BY 4.0
 - 9.2: Dictionary as a Set of Counters - CC BY 4.0
 - 9.3: Dictionaries and Files - CC BY 4.0
 - 9.4: Looping and Dictionaries - CC BY 4.0
 - 9.5: Advanced Text Parsing - CC BY 4.0
 - 9.6: Debugging - CC BY 4.0
 - 9.E: Dictionaries (Exercises) - CC BY 4.0
 - 9.G: Dictionaries (Glossary) - CC BY 4.0
- 10: Tuples - CC BY 4.0
 - 10.1: Tuples are Immutable - CC BY 4.0
 - 10.2: Comparing Tuples - CC BY 4.0
 - 10.3: Tuple Assignment - CC BY 4.0
 - 10.4: Dictionaries and Tuples - CC BY 4.0
 - 10.5: Multiple assignment with dictionaries - CC BY 4.0
 - 10.6: The most common words - CC BY 4.0
 - 10.7: Using Tuples as Keys in Dictionaries - CC BY 4.0
 - 10.8: Sequences- strings, lists, and tuples - Oh My! - CC BY 4.0
 - 10.9: Debugging - CC BY 4.0
 - 10.E: Tuples (Exercises) - CC BY 4.0
 - 10.G: Tuples (Glossary) - CC BY 4.0
- 11: Libraries - CC BY-SA 4.0
 - 11.1: NumPy - Numerical Python - CC BY-SA 4.0
 - 11.2: SciPy - Scientific Computing in Python - CC BY-SA 4.0
 - 11.3: Pandas - Data Manipulation and Analysis - CC BY-SA 4.0
 - 11.4: Matplotlib — Data Visualization in Python - CC BY-SA 4.0
 - 11.5: Scikit-learn (sklearn) - Supervised and Unsupervised Learning - CC BY-SA 4.0
- 12: Model Building and Regression - CC BY-SA 4.0
 - 12.1: Functions for Model Building and Regression - CC BY-SA 4.0
 - 12.2: Linear Regression - CC BY-SA 4.0
 - 12.3: Optimization - Parameter Estimation for Nonlinear Functions - CC BY-SA 4.0
 - 12.4: Summary - CC BY-SA 4.0
- 13: Statistics, Probability, and Interpolation - CC BY-SA 4.0
 - 13.1: Statistics and Histograms - CC BY-SA 4.0
 - 13.2: Normal Distribution - CC BY-SA 4.0
 - 13.3: Random Number Generation - CC BY-SA 4.0
 - 13.4: Interpolation - CC BY-SA 4.0
 - 13.5: Summary - CC BY-SA 4.0
- 14: Linear Algebra Equations - CC BY-SA 4.0
 - 14.1: Matrix Methods for Linear Equations - CC BY-SA 4.0
 - 14.2: Well-Determined Fully Specified System - CC BY-SA 4.0
 - 14.3: Underdetermined Systems - CC BY-SA 4.0
 - 14.4: Overdetermined Systems - CC BY-SA 4.0
 - 14.5: A General Solution - CC BY-SA 4.0

- 14.6: Summary - [CC BY-SA 4.0](#)
- 15: Numerical Methods for Calculus and Differential Equations - [CC BY-SA 4.0](#)
 - 15.1: Numerical Integration - [CC BY-SA 4.0](#)
 - 15.2: Numerical Differentiation - [CC BY-SA 4.0](#)
 - 15.3: First-Order Differential Equations - [CC BY-SA 4.0](#)
 - 15.4: Higher-Order Differential Equations - [CC BY-SA 4.0](#)
 - 15.5: Summary - [CC BY-SA 4.0](#)
- 16: Regular Expressions - [CC BY 4.0](#)
 - 16.1: Regular Expressions - [CC BY 4.0](#)
 - 16.2: Character matching in regular expressions - [CC BY 4.0](#)
 - 16.3: Extracting data using regular expressions - [CC BY 4.0](#)
 - 16.4: Combining searching and extracting - [CC BY 4.0](#)
 - 16.5: Escape Character - [CC BY 4.0](#)
 - 16.6: Bonus section for Unix / Linux users - [CC BY 4.0](#)
 - 16.7: Debugging - [CC BY 4.0](#)
 - 16.E: Regular Expressions (Exercises) - [CC BY 4.0](#)
 - 16.G: Regular Expressions (Glossary) - [CC BY 4.0](#)
- 16.S: Regular Expressions (Summary) - [CC BY 4.0](#)
- 17: Object-Oriented Programming - [CC BY 4.0](#)
 - 17.1: Managing Larger Programs - [CC BY 4.0](#)
 - 17.2: Getting Started - [CC BY 4.0](#)
 - 17.3: Using Objects - [CC BY 4.0](#)
 - 17.4: Starting with Programs - [CC BY 4.0](#)
 - 17.5: Subdividing a Problem - Encapsulation - [CC BY 4.0](#)
 - 17.6: Our First Python Object - [CC BY 4.0](#)
 - 17.7: Classes as Types - [CC BY 4.0](#)
 - 17.8: Many Instances - [CC BY 4.0](#)
 - 17.9: Object Lifecycle - [CC BY 4.0](#)
 - 17.10: Inheritance - [CC BY 4.0](#)
 - 17.G: Object-Oriented Programming (Glossary) - [CC BY 4.0](#)
 - 17.S: Object-Oriented Programming (Summary) - [CC BY 4.0](#)
- Back Matter - [CC BY 4.0](#)
 - Index - [CC BY 4.0](#)
 - Glossary - [CC BY 4.0](#)
 - Detailed Licensing - [CC BY 4.0](#)
 - Detailed Licensing - [Undeclared](#)

Detailed Licensing

Overview

Title: [Engineering Modeling and Analysis with Python](#)

Webpages: 191

All licenses found:

- [CC BY 4.0](#): 79.6% (152 pages)
- [CC BY-SA 4.0](#): 18.8% (36 pages)
- [Undeclared](#): 1.6% (3 pages)

By Page

- [Engineering Modeling and Analysis with Python - CC BY 4.0](#)
 - [Front Matter - CC BY-SA 4.0](#)
 - [TitlePage - CC BY-SA 4.0](#)
 - [InfoPage - CC BY-SA 4.0](#)
 - [Table of Contents - Undeclared](#)
 - [About the Authors - CC BY-SA 4.0](#)
 - [Licensing - Undeclared](#)
 - [About the Book - CC BY-SA 4.0](#)
 - [Licensing - CC BY-SA 4.0](#)
 - [1: Introduction to Engineering Modeling and Analysis with Python - CC BY 4.0](#)
 - [1.1: Introduction - CC BY 4.0](#)
 - [2: Variables, Expressions, and Statements - CC BY 4.0](#)
 - [2.1: Values and Types - CC BY 4.0](#)
 - [2.2: Variables - CC BY 4.0](#)
 - [2.3: Variable names and Keywords - CC BY 4.0](#)
 - [2.4: Statements - CC BY 4.0](#)
 - [2.5: Operators and Operands - CC BY 4.0](#)
 - [2.6: Expressions - CC BY 4.0](#)
 - [2.7: Order of Operations - CC BY 4.0](#)
 - [2.8: Modulus Operator - CC BY 4.0](#)
 - [2.9: String Operations - CC BY 4.0](#)
 - [2.10: Asking the user for input - CC BY 4.0](#)
 - [2.11: Comments - CC BY 4.0](#)
 - [2.12: Choosing Mnemonic Variable Names - CC BY 4.0](#)
 - [2.13: Debugging - CC BY 4.0](#)
 - [2.E: Variables, Expressions, and Statements \(Exercises\) - CC BY 4.0](#)
 - [2.G: Glossary - CC BY 4.0](#)
 - [3: Conditional Execution - CC BY 4.0](#)
 - [3.1: Boolean Expressions - CC BY 4.0](#)
 - [3.2: Logical Operators - CC BY 4.0](#)
 - [3.3: Conditional Execution - CC BY 4.0](#)
 - [3.4: Alternative Execution - CC BY 4.0](#)
 - [3.5: Chained Conditionals - CC BY 4.0](#)
 - [3.6: Nested Conditionals - CC BY 4.0](#)
 - [3.7: Catching exceptions Using Try and Except - CC BY 4.0](#)
 - [3.8: Short-Circuit Evaluation of Logical Expressions - CC BY 4.0](#)
 - [3.9: Debugging - CC BY 4.0](#)
 - [3.E: Conditional Execution \(Exercises\) - CC BY 4.0](#)
 - [3.G: Conditional Execution \(Glossary\) - CC BY 4.0](#)
 - [4: Functions - CC BY 4.0](#)
 - [4.1: Function Calls - CC BY 4.0](#)
 - [4.2: Built-in Functions - CC BY 4.0](#)
 - [4.3: Type Conversion Functions - CC BY 4.0](#)
 - [4.4: Random Numbers - CC BY 4.0](#)
 - [4.5: Math Functions - CC BY 4.0](#)
 - [4.6: Adding New Functions - CC BY 4.0](#)
 - [4.7: Definitions and Uses - CC BY 4.0](#)
 - [4.8: Flow of Execution - CC BY 4.0](#)
 - [4.9: Parameters and Arguments - CC BY 4.0](#)
 - [4.10: Fruitful functions and void functions - CC BY 4.0](#)
 - [4.11: Why functions? - CC BY 4.0](#)
 - [4.12: Debugging - CC BY 4.0](#)
 - [4.E: Functions \(Exercises\) - CC BY 4.0](#)
 - [4.G: Functions \(Glossary\) - CC BY 4.0](#)
 - [5: Iterations - CC BY 4.0](#)
 - [5.1: Updating Variables - CC BY 4.0](#)
 - [5.2: The while Statement - CC BY 4.0](#)
 - [5.3: Infinite Loops - CC BY 4.0](#)
 - [5.4: "Infinite loops" and break - CC BY 4.0](#)
 - [5.5: Finishing iterations with continue - CC BY 4.0](#)
 - [5.6: Definite loops using for - CC BY 4.0](#)
 - [5.7: Loop patterns - CC BY 4.0](#)
 - [5.8: Counting and Summing Loops - CC BY 4.0](#)
 - [5.9: Maximum and Minimum Loops - CC BY 4.0](#)
 - [5.10: Debugging - CC BY 4.0](#)
 - [5.E: Iterations \(Exercises\) - CC BY 4.0](#)
 - [5.G: Iterations \(Glossary\) - CC BY 4.0](#)
 - [6: Strings - CC BY 4.0](#)

- 6.1: A string is a sequence - CC BY 4.0
- 6.2: Getting the length of a string using len - CC BY 4.0
- 6.3: Traversal through a string with a loop - CC BY 4.0
- 6.4: String Slices - CC BY 4.0
- 6.5: Strings are immutable - CC BY 4.0
- 6.6: Looping and Counting - CC BY 4.0
- 6.7: The in operator - CC BY 4.0
- 6.8: String Comparison - CC BY 4.0
- 6.9: String Methods - CC BY 4.0
- 6.10: Parsing strings - CC BY 4.0
- 6.11: Format operator - CC BY 4.0
- 6.12: Debugging - CC BY 4.0
- 6.E: Strings (Exercises) - CC BY 4.0
- 6.G: Strings (Glossary) - CC BY 4.0
- 7: Files - CC BY 4.0
 - 7.1: Persistence - CC BY 4.0
 - 7.2: Opening Files - CC BY 4.0
 - 7.3: Text files and Lines - CC BY 4.0
 - 7.4: Reading Files - CC BY 4.0
 - 7.5: Searching through a File - CC BY 4.0
 - 7.6: Letting the user choose the file name - CC BY 4.0
 - 7.7: Using try, except, and open - CC BY 4.0
 - 7.8: Writing Files - CC BY 4.0
 - 7.9: Debugging - CC BY 4.0
 - 7.E: Files (Exercises) - CC BY 4.0
 - 7.G: Files (Glossary) - CC BY 4.0
- 8: Lists - CC BY 4.0
 - 8.1: A list is a sequence - CC BY 4.0
 - 8.2: Lists are mutable - CC BY 4.0
 - 8.3: Traversing a List - CC BY 4.0
 - 8.4: List operations - CC BY 4.0
 - 8.5: List Slices - CC BY 4.0
 - 8.6: List Methods - CC BY 4.0
 - 8.7: Deleting Elements - CC BY 4.0
 - 8.8: Lists and Functions - CC BY 4.0
 - 8.9: Lists and Strings - CC BY 4.0
 - 8.10: Parsing lines - CC BY 4.0
 - 8.11: Objects and Values - CC BY 4.0
 - 8.12: Aliasing - CC BY 4.0
 - 8.13: List arguments - CC BY 4.0
 - 8.14: Debugging - CC BY 4.0
 - 8.E: Lists (Exercises) - CC BY 4.0
 - 8.G: Lists (Glossary) - CC BY 4.0
- 9: Dictionaries - CC BY 4.0
 - 9.1: Dictionaries - CC BY 4.0
 - 9.2: Dictionary as a Set of Counters - CC BY 4.0
 - 9.3: Dictionaries and Files - CC BY 4.0
 - 9.4: Looping and Dictionaries - CC BY 4.0
 - 9.5: Advanced Text Parsing - CC BY 4.0
 - 9.6: Debugging - CC BY 4.0
 - 9.E: Dictionaries (Exercises) - CC BY 4.0
 - 9.G: Dictionaries (Glossary) - CC BY 4.0
- 10: Tuples - CC BY 4.0
 - 10.1: Tuples are Immutable - CC BY 4.0
 - 10.2: Comparing Tuples - CC BY 4.0
 - 10.3: Tuple Assignment - CC BY 4.0
 - 10.4: Dictionaries and Tuples - CC BY 4.0
 - 10.5: Multiple assignment with dictionaries - CC BY 4.0
 - 10.6: The most common words - CC BY 4.0
 - 10.7: Using Tuples as Keys in Dictionaries - CC BY 4.0
 - 10.8: Sequences- strings, lists, and tuples - Oh My! - CC BY 4.0
 - 10.9: Debugging - CC BY 4.0
 - 10.E: Tuples (Exercises) - CC BY 4.0
 - 10.G: Tuples (Glossary) - CC BY 4.0
- 11: Libraries - CC BY-SA 4.0
 - 11.1: NumPy - Numerical Python - CC BY-SA 4.0
 - 11.2: SciPy - Scientific Computing in Python - CC BY-SA 4.0
 - 11.3: Pandas - Data Manipulation and Analysis - CC BY-SA 4.0
 - 11.4: Matplotlib — Data Visualization in Python - CC BY-SA 4.0
 - 11.5: Scikit-learn (sklearn) - Supervised and Unsupervised Learning - CC BY-SA 4.0
- 12: Model Building and Regression - CC BY-SA 4.0
 - 12.1: Functions for Model Building and Regression - CC BY-SA 4.0
 - 12.2: Linear Regression - CC BY-SA 4.0
 - 12.3: Optimization - Parameter Estimation for Nonlinear Functions - CC BY-SA 4.0
 - 12.4: Summary - CC BY-SA 4.0
- 13: Statistics, Probability, and Interpolation - CC BY-SA 4.0
 - 13.1: Statistics and Histograms - CC BY-SA 4.0
 - 13.2: Normal Distribution - CC BY-SA 4.0
 - 13.3: Random Number Generation - CC BY-SA 4.0
 - 13.4: Interpolation - CC BY-SA 4.0
 - 13.5: Summary - CC BY-SA 4.0
- 14: Linear Algebra Equations - CC BY-SA 4.0
 - 14.1: Matrix Methods for Linear Equations - CC BY-SA 4.0
 - 14.2: Well-Determined Fully Specified System - CC BY-SA 4.0
 - 14.3: Underdetermined Systems - CC BY-SA 4.0
 - 14.4: Overdetermined Systems - CC BY-SA 4.0
 - 14.5: A General Solution - CC BY-SA 4.0

- 14.6: Summary - [CC BY-SA 4.0](#)
- 15: Numerical Methods for Calculus and Differential Equations - [CC BY-SA 4.0](#)
 - 15.1: Numerical Integration - [CC BY-SA 4.0](#)
 - 15.2: Numerical Differentiation - [CC BY-SA 4.0](#)
 - 15.3: First-Order Differential Equations - [CC BY-SA 4.0](#)
 - 15.4: Higher-Order Differential Equations - [CC BY-SA 4.0](#)
 - 15.5: Summary - [CC BY-SA 4.0](#)
- 16: Regular Expressions - [CC BY 4.0](#)
 - 16.1: Regular Expressions - [CC BY 4.0](#)
 - 16.2: Character matching in regular expressions - [CC BY 4.0](#)
 - 16.3: Extracting data using regular expressions - [CC BY 4.0](#)
 - 16.4: Combining searching and extracting - [CC BY 4.0](#)
 - 16.5: Escape Character - [CC BY 4.0](#)
 - 16.6: Bonus section for Unix / Linux users - [CC BY 4.0](#)
 - 16.7: Debugging - [CC BY 4.0](#)
 - 16.E: Regular Expressions (Exercises) - [CC BY 4.0](#)
 - 16.G: Regular Expressions (Glossary) - [CC BY 4.0](#)
- 16.S: Regular Expressions (Summary) - [CC BY 4.0](#)
- 17: Object-Oriented Programming - [CC BY 4.0](#)
 - 17.1: Managing Larger Programs - [CC BY 4.0](#)
 - 17.2: Getting Started - [CC BY 4.0](#)
 - 17.3: Using Objects - [CC BY 4.0](#)
 - 17.4: Starting with Programs - [CC BY 4.0](#)
 - 17.5: Subdividing a Problem - Encapsulation - [CC BY 4.0](#)
 - 17.6: Our First Python Object - [CC BY 4.0](#)
 - 17.7: Classes as Types - [CC BY 4.0](#)
 - 17.8: Many Instances - [CC BY 4.0](#)
 - 17.9: Object Lifecycle - [CC BY 4.0](#)
 - 17.10: Inheritance - [CC BY 4.0](#)
 - 17.G: Object-Oriented Programming (Glossary) - [CC BY 4.0](#)
 - 17.S: Object-Oriented Programming (Summary) - [CC BY 4.0](#)
- Back Matter - [CC BY 4.0](#)
 - Index - [CC BY 4.0](#)
 - Glossary - [CC BY 4.0](#)
 - Detailed Licensing - [CC BY 4.0](#)
 - Detailed Licensing - [Undeclared](#)